第一讲 概述



1 机器学习用来干什么

2 机器学习的基本概念

3 机器学习技术与系统的种类

4 机器学习的数据挑战

5 机器学习的发展和应用

基本概念

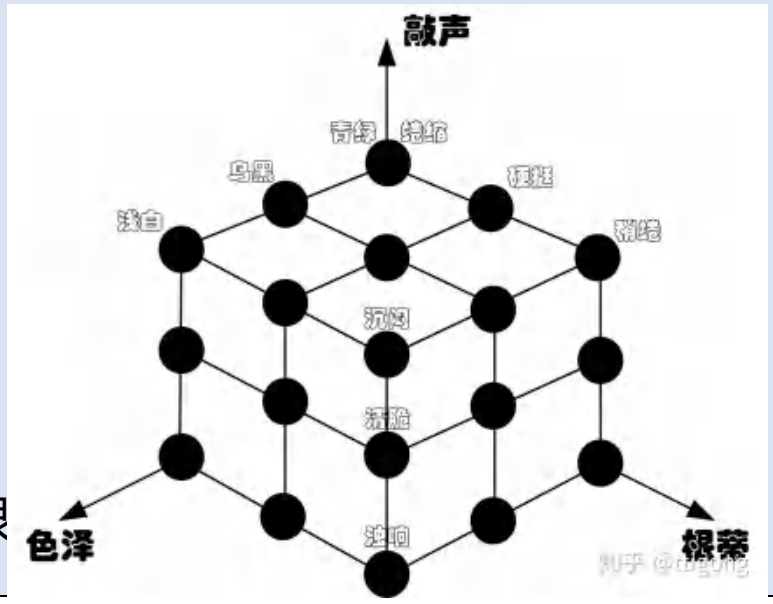
1 特征空间与特征向量

- 一组描述客体性质的变量，称为特征变量（feature）。变量的个数d称为特征维度，特征变量组成的向量称为特征向量，变量张成的空间称为特征空间（样本空间），变量的取值称为属性值。特征向量与特征向量（样本与样本）之间可以度量距离



如何描述一个西瓜？

$x^{(3)}$根



基本概念

- 每个对象实例可由一个特征向量表征，记为$\boldsymbol{x}_i$。**样本空间**是所有可能存在的、合理的样本的集合，记为X。每个$\boldsymbol{x}_i$均定义在d维特征空间中，其每个维度对应于一种特征，有：

$$\boldsymbol{x}_i = \begin{pmatrix} x_{i1} \\ \vdots \\ x_{id} \end{pmatrix} = (x_{i1}, \ldots, x_{id})^T$$

- 与样本空间对应，样本的分类标签或预测取值的集合称为**标记空间**，记为Y。比如一个经典的二分类问题，$Y=\{-1, 1\}$或$Y=\{0, 1\}$。在回归问题中，响应变量通常为1个实数值，则标记空间定义为响应变量的连续取值区间

注：多响应变量（多元输出回归）与多变量回归（多元回归）的区别

基本概念

- 对于$k(k\geq 2)$分类问题，也经常将标记值定义为一个k维向量，其中各分量下标与所在类别序号一致的置1，其余置0，如：

$$\boldsymbol{y_i} = \begin{pmatrix} y_{i1} \\ \vdots \\ y_{ik} \end{pmatrix} = (y_{i1}, \ldots, y_{ik})^T$$



如一个识别手写0-9数字的分类任务，样本和其标记值举例如下：

x_i	y_i	x_i	y_i
0	$(1,0,0,0,0,0,0,0,0,0)^T$	5	$(0,0,0,0,0,1,0,0,0,0)^T$
1	$(0,1,0,0,0,0,0,0,0,0)^T$	6	$(0,0,0,0,0,0,1,0,0,0)^T$
2	$(0,0,1,0,0,0,0,0,0,0)^T$	7	$(0,0,0,0,0,0,0,1,0,0)^T$
3	$(0,0,0,1,0,0,0,0,0,0)^T$	8	$(0,0,0,0,0,0,0,0,1,0)^T$
4	$(0,0,0,0,1,0,0,0,0,0)^T$	9	$(0,0,0,0,0,0,0,0,0,1)^T$

基本概念

2 数据集合 (Data set)

注：在本课程中，全部沿用以下符号——
训练样本个数m个，样本维数为d维

- 一个拥有m个样本的训练集可以记为：

$$D=\{(x_1、y_1),(x_2、y_2),\cdots,(x_m、y_m)\} \quad 其中，x_i \in X，y_i \in Y$$

- 在机器学习中，数据集合一般被划分为训练集（Training Set）、验证集（Validation Set）和测试集（Test Set），各自扮演着不同的角色，以确保模型具有良好的泛化能力和性能

训练集	验证集	测试集
用于模型训练 降低经验误差	验证模型正确性 调节超参数	评估泛化性能

→第二章：训练集、测试集、验证集

5 选择模型的策略

➢ **策略（strategy）**也称为学习准则

1. 经验风险最小化（empirical risk minimization）准则：

是机器学习里最常用的准则，即在训练样本上取得最小的经验误差，是模型有效的基础。经验误差最小化的求解思路：构造一个代价函数或损失函数（Loss function）来描述经验风险，优化参数使该函数值最小

损失函数$L(w)$：

给定独立同分布的学习样例集合$(x, y) \in X \times Y$和模型$\hat{y} = g(x, w)$，损失函数度量的是模型输出预测值和训练样本标记值之间的差异

$$L(w) = \text{distance}(y, \hat{y}) \text{ or } = \text{distance}\,[p(y|x), q(\hat{y}|x)]$$

数值差异 ↗　　概率分布差异 ↗

（有监督学习）

- 以下是机器学习中常用的损失函数：

损失函数	表达式
L1损失（L1 loss）	$\lvert\hat{y}_i - y_i\rvert$
L2损失（L2 loss） 最小二乘误差	$(\hat{y}_i - y_i)^2$
Huber损失（Huber loss）	$\begin{cases} \frac{1}{2}(\hat{y}_i - y_i)^2 & , \lvert\hat{y}_i - y_i\rvert < \delta \\ \delta\lvert\hat{y}_i - y_i\rvert - \frac{1}{2}\delta^2 & , \lvert\hat{y}_i - y_i\rvert \geq \delta \end{cases}$
交叉熵损失（cross-entropy loss）	$-y_i \cdot \ln\hat{y}_i - (1 - y_i) \cdot \ln(1 - \hat{y}_i)$ 二分类：$y_i \in \{1,0\}$，$\hat{y}_i$是预测样本属于正类的概率
铰链损失（hinge loss）	$\max(0, 1 - \hat{y}_i y_i)$ SVM：$y_i \in \{1, -1\}$
指数损失（exponential loss）	$\exp(\hat{y}_i y_i)$

- 损失函数通常针对单个训练样本，用于度量模型在该样本上的预测错误程度。**代价函数**（Cost function）通常针对整个训练集而言，是损失函数在所有训练样本上的平均值。用于度量模型在整个数据集上的平均误差，反映了模型对整体数据的适应能力



定义在m个训练样本上的最小二乘误差（平方误差），就是大家熟悉的均方误差（Mean Squared Error, MSE）

$$MSE = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2$$

注：本课程中不刻意区分损失函数和代价函数，统称为*Loss*

- 泛化能力（generalization）是指模型对新样本的适应能力，反映了一个模型在训练集数据之外的数据上的表现

2. 结构风险最小化（structural risk minimization）准则：

结构风险在经验风险的基础上增加对模型复杂度的正则化（regularization）约束，以降低模型复杂度，提升模型的泛化能力

选择假设（模型）的原则：

- 在选择机器学习方法的过程中对某种假设的偏好称为**归纳偏倚**（inducive bias）。归纳偏倚起到了“模型选择”的作用，使我们可以从假设空间中选择出更符合现实规则的模型
- 奥卡姆剃刀(Occam's Razor)：选择最简单的那个模型



5 选择模型的策略

- $E_{in}(g)$，学到的假设 g 在训练样本(in-of-sample)上的损失，称为经验误差。训练（training)的过程→使得经验误差$E_{in}(g)$ 尽可能小
- $E_{out}(g)$，假设 g 在除了训练样本外的其他所有样本(out-of-sample)上的损失，称为泛化误差。在测试（test)过程中希望$E_{out}(g)$接近$E_{in}(g)$

泛化能力(generalization)：

机器学习的任务是建立样本空间X到标记空间Y的映射 $g: X \rightarrow Y$，但这个映射要能够**适应新样本**（对没有见过的数据要有效！）。这种能力称为泛化能力，指学习的结果对新样本的适应能力，对样本空间的描述能力

第二讲 模型选择与评估

参考课件2025版

1 监督学习的计算理论

2 误差构成与过拟合

3 模型验证与测试

4 模型的性能度量



1 有效假设

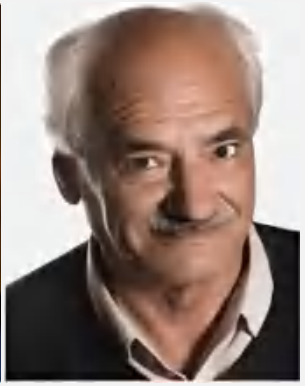
- **假设（hypothesis）**可以看作是对于真实模型的一种近似或猜测。机器学习中的“模型” 等同于一个具体的假设或假设集。比如“线性回归模型” 即等同于这样一个假设：解释变量与响应变量之间的关系可以用一条直线来描述。
- **有效假设**是指在给定数据集中，能够产生不同二分结果（又称对分，dichotomies）的假设。这些假设对于数据的描述或预测是有意义的。即使假设空间包含无限多个假设，但其中许多假设在给定数据集上产生的结果是相同的，则可视为等效的。

Baidu百科 百度百科——全球最大中文百科全书

- **VC维**（Vapnik-Chervonenkis Dimension）是前苏联数学家弗拉基米尔·瓦普尼克（Vladimir Vapnik）和亚历山大·切沃年科（Alexey Chervonenkis）在1968年共同提出的，是**衡量模型复杂度的关键指标**



Vapnik



Chervonenkis

2 VC维

- **增长函数$\Pi_H(m)$**：假设空间 H 对m个样本所能赋予二分类标记的最大的可能数，其上界为 2^m 。这里每种标记称为一个**对分**



- **H散列（shatter）**：H作用于大小为 m 的样本集 D 时，产生的对分数量等于 2^m，即 $\Pi_H(m) = 2^m$ 时，则称 D 被 H 散列了，H 中的假设可以区分 D 中的每一个可能的对分。



i.e 二维平面的任意3个数据可以被**直线假设**散列：$\Pi_H(3) = 2^3 = 8$。4个数据不能被直线假设散列：$\Pi_H(4) = 14 < 2^4 = 16$

散列能力：指的是假设集合能够将数据集划分为不同类别的能力。如果一个假设集合能够将数据集散列，则它具有很高的灵活性和表达能力。

提问：那么4个数据可以被什么样的假设散列呢？

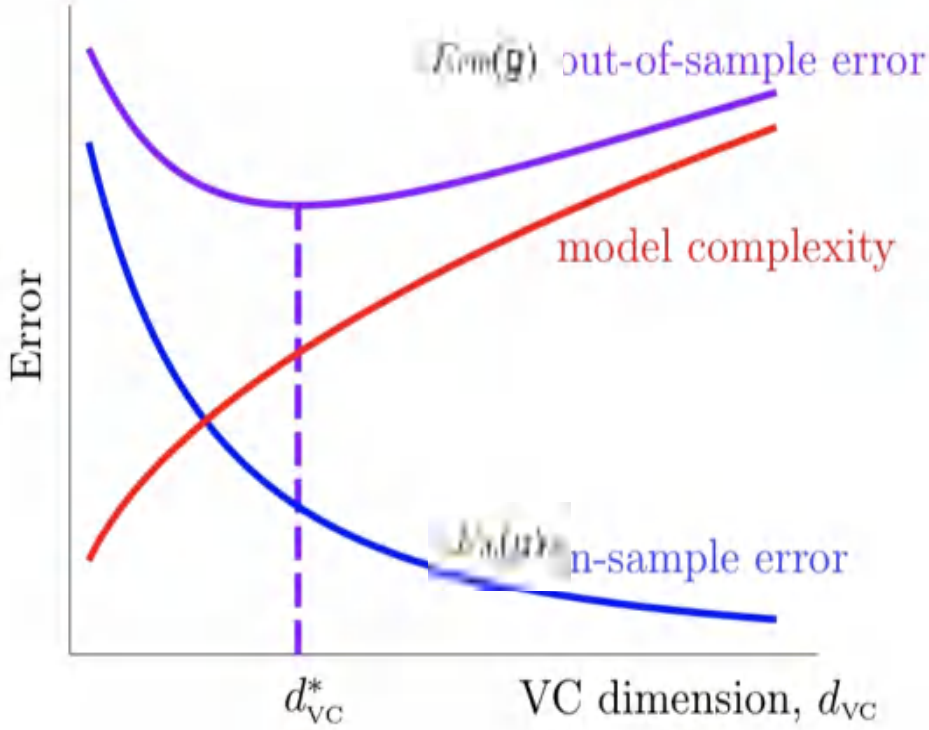


- **中断点（break point）**：对于假设空间H的增长函数$\Pi_H(m)$，当$m=1$逐渐增大到k过程中，出现$\Pi_H(k) < 2^k$时，则称k是该假设空间的中断点。任何大于中断点的数据集，H都没有办法散列它
- **VC维**：可以被H散列的点的最大数据集大小称为H的VC维。$\mathrm{VC}(H)=k-1$，其中k是H的中断点。VC维的大小与具体的学习策略和优化算法无关，与数据集分布无关，只与假设空间有关

4 VC维的意义

- **VC维用来度量假设H的复杂度和学习能力。**

当固定样本数m时，随着VC维的上升，$E_{\text{in}}(g)$会不断降低，同时模型复杂度 Ω 会不断上升。因此机器学习的模型选择要寻找一个合适的VC维使 $E_{\text{out}}(g)$最小



第二讲 模型选择与评估



参考课件2025版

1 监督学习的计算理论

2 误差构成与过拟合

3 模型验证与测试

4 模型的性能度量



1 偏差-方差分解

- 在机器学习中，误差泛指模型预测值与真实值之间的差异，是评估模型性能的重要依据，也是不可避免的。误差进一步可以分解为偏差、方差和噪声的组合

偏差 Bias	又称为表观误差，通常指的是模型预测的平均值（期望）与真实值之间的差异，反映模型精度
方差 Variance	指模型在不同训练集上学习到的结果之间的差异，反映模型的稳定性
噪声 Noise	通常指数据集中无法解释的人为疏忽、标记错误或随机变异的部分，是不可避免的

误差构成与过拟合

- 各学习器在x上的预测输出的数学期望记为: $\mathbb{E}_D[\hat{y}_D]$
- 各学习器预测输出的方差为: $\mathbb{E}_D[(\hat{y}_D - \mathbb{E}_D[\hat{y}_D])^2]$
- 训练数据存在噪声，即标记y_D与真值y有差异，则噪声ε有:

$$\varepsilon^2 = \mathbb{E}_D[(y_D - y)^2]$$

- 模型预测值的期望与真实值的差别为偏差，有:

$$bias^2 = (\mathbb{E}_D[\hat{y}_D] - y)^2$$

可观测误差期望:

$$\mathbb{E}_D[(\hat{y}_D - y_D)^2]$$

$$= \mathbb{E}_D[(\hat{y}_D - \mathbb{E}_D[\hat{y}_D])^2] + \mathbb{E}_D[(\mathbb{E}_D[\hat{y}_D] - y)^2] + \mathbb{E}_D[(y - y_D)^2] + 2(c - y)\underbrace{\mathbb{E}_D[y - y_D]}_{=0}$$

方差（$\hat{y}_D$ 与 $\mathbb{E}_D[\hat{y}_D]$ 的关系）

偏差（$\mathbb{E}_D[\hat{y}_D]$与y的关系）

噪声（y_D与y的关系）



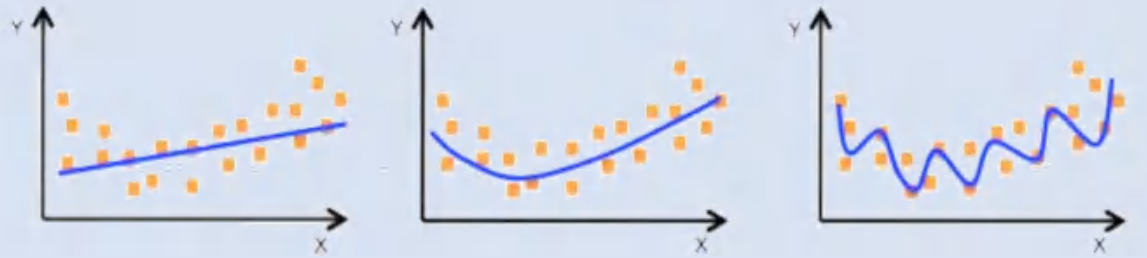
2 欠拟合与过拟合

- **欠拟合（Underfitting）：**是指模型在训练集、验证集和测试集上均表现不佳的情况。情况通常发生于模型（假设）过于简单，没有足够的能力去提取数据的复杂关系或特征

- **过拟合（Overfitting）：**是指模型在训练集上表现很好，而在验证和测试阶段误差很大，即模型的泛化能力很差。通常是因为模型没有学习到数据的普遍规律，而学习到了训练数据的噪声或个性特征

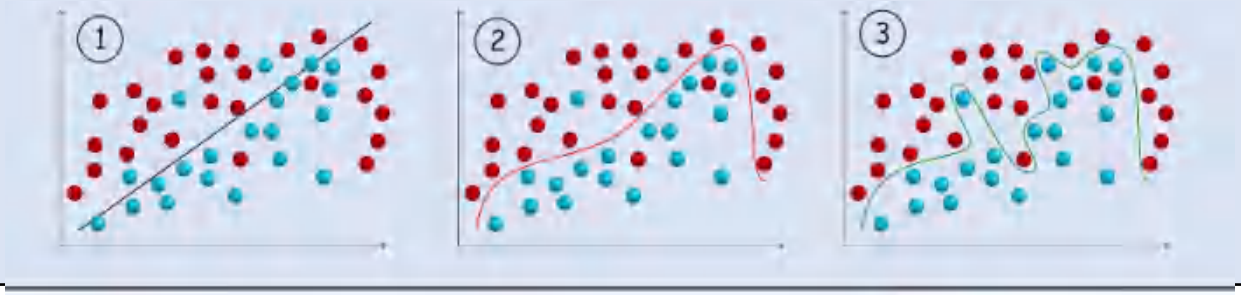
误差构成与过拟合

回归：



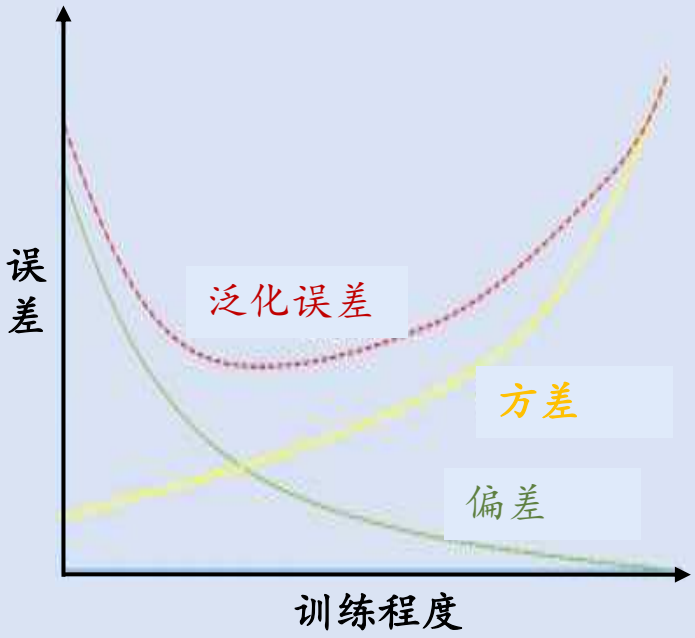
欠拟合 → 好拟合 → 过拟合

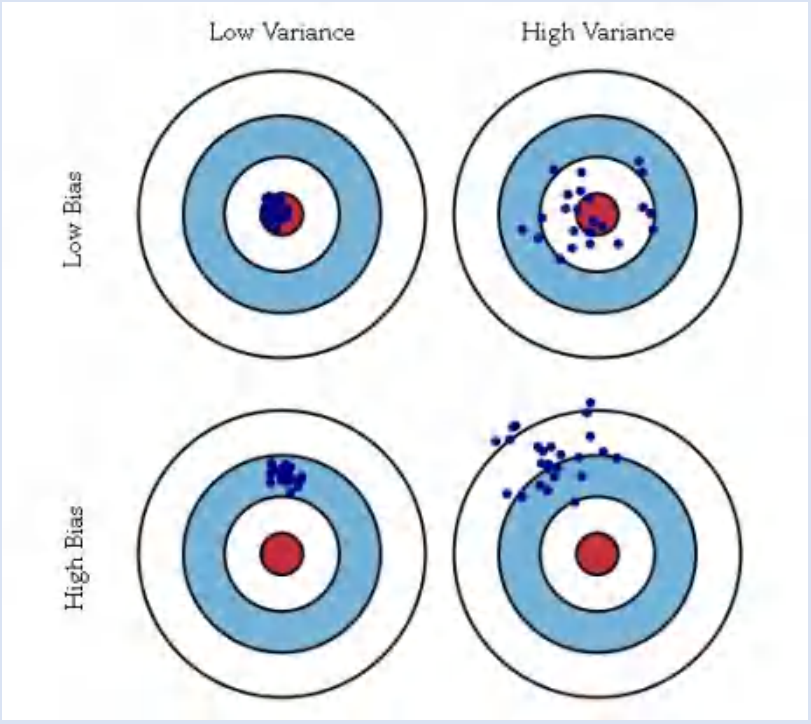
分类：



误差构成与过拟合

- 训练不足时，模型拟合能力不强，训练误差由偏差主导
- 随着训练程度加深，模型拟合能力增强，训练误差由方差主导



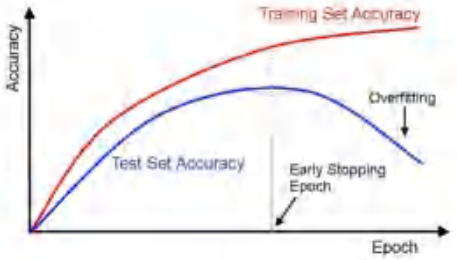


误差构成与过拟合

- “欠拟合”、“过拟合”是机器学习最需要解决的问题之一。可以从模型选择、数据样本和训练各个方面入手解决

模型	样本	训练
• 正则化 • Dropout • Stacking集成 • Bagging集成 • Boosting集成	• 清洗样本，减少/添加噪声 • 数据增强（旋转、平移、翻转、Mixup） • 分层采样 • 交叉验证 • 多项式特征	• Early Stopping





第二讲 模型选择与评估

参考课件2025版

1 监督学习的计算理论

2 误差构成与过拟合

3 模型验证与测试

4 模型的性能度量

模型评估的基本问题

- 如何获得测试样本？ → 验证测试
- 如何评估性能优劣？ → 性能度量
- 如何比较学习器性能？ → 比较检验



1 训练集、验证集和测试集

- 模型选择与评估：对候选模型的泛化误差进行评估，然后选择泛化误差最小的模型。但是泛化误差是定义在样本集之外的，无法直接获得。现实中的模型到底如何评估与选择呢?

关键：怎么获得“验证集” (validation set) 和“测试集” (test set)？

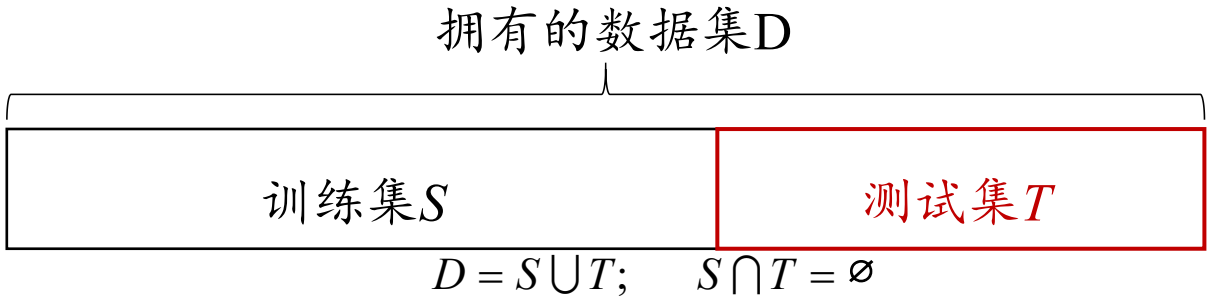
训练集	验证集	测试集
降低经验误差	验证模型正确性 调节超参数	评估泛化误差

测试集、验证集应该与训练集“互斥”



2 留出法

- 假设包含m个样例的数据集 $D=\{(\boldsymbol{x}_1,\boldsymbol{y}_1),(\boldsymbol{x}_2,\boldsymbol{y}_2),\ldots(\boldsymbol{x}_m,\boldsymbol{y}_m)\}$，从$D$中选择一部分作为训练集 S，留出（hold out）一部分作为测试集 T:



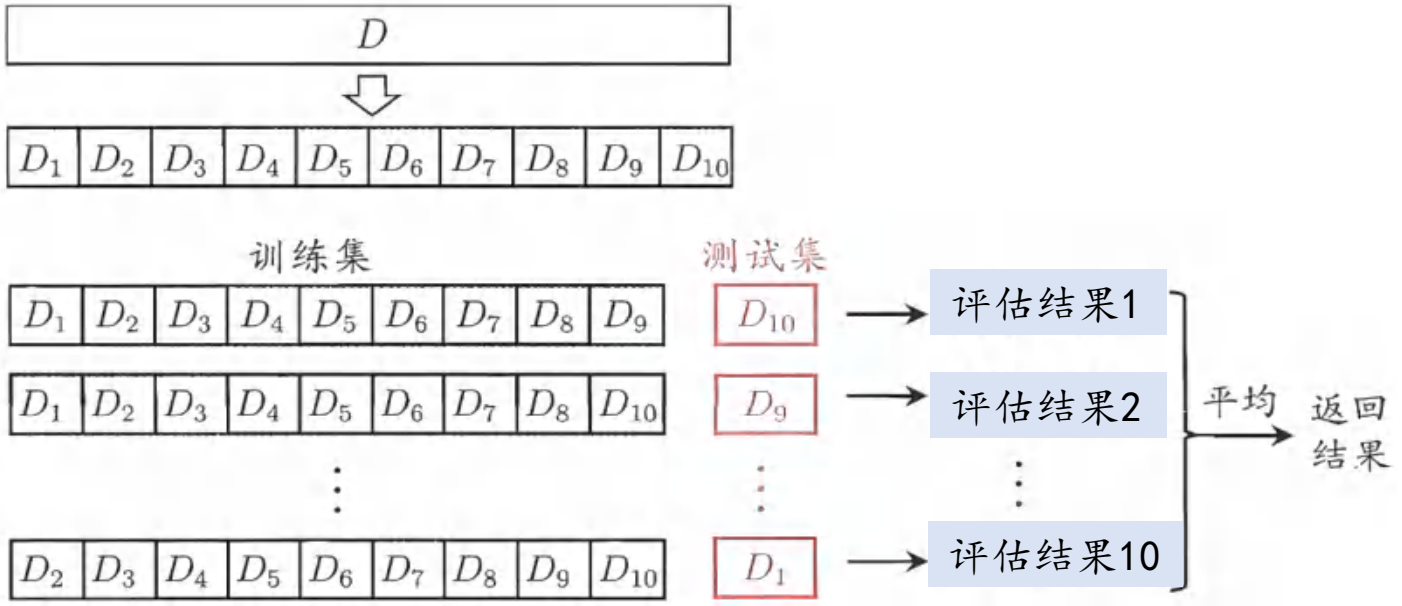
- 一般测试集不能太大也不能太小 （例如：70%的数据用于训练，30%的数据用于测试）
- 单次划分可能导致结果不稳定。通常需要多次重复划分 （例如：100次随机划分），取平均值作为留出法的评估结果



3 k 折交叉验证法

- 将数据集 D 划分为 k 个大小相同的互斥子集，满足 $D=D_1 \cup D_2 \cup \ldots \cup D_k$，$D_i \cap D_j = \varnothing$ $(i \neq j)$。采用分层采样的方法使得这些子集的数据分布一致。每次用$k-1$个子集的并集构成训练集，余下的那个子集作为测试集。共有k种训练集/测试集划分的情况，从而可进行k次交叉验证（k-fold Cross-Validation），最终返回k次评估结果的均值、标准差
- 如果模型性能在不同折之间存在较大差异，可能意味着数据集存在某种不平衡或模型对特定的样本过于敏感，则需要进一步的数据清洗或模型调整

模型验证与测试



10折交叉验证示意图

提问：k折交叉验证会得到k个参数不同的模型，那么最终使用哪一个呢？



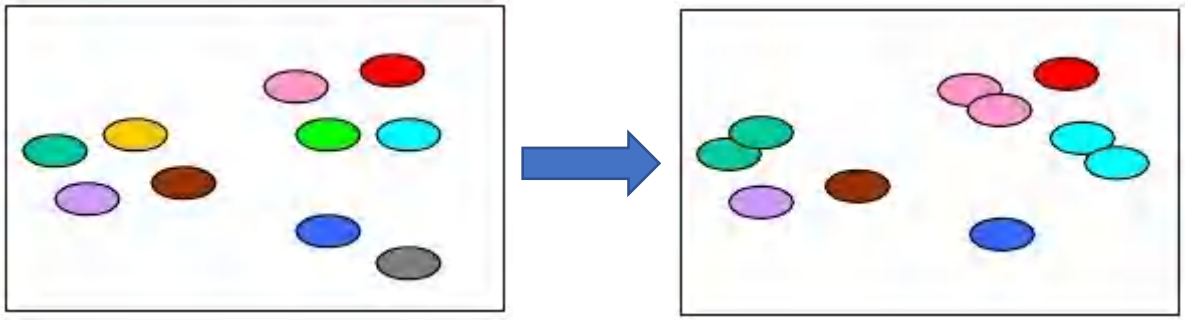
- k折交叉验证是一个用于**评估模型性能**的技术，而不是一个用于直接选择最终模型的技术：

交叉验证的一般步骤

- **评估所有折的性能**：计算每一折上的性能指标（如准确率、召回率、F1分数等），并计算k折的平均值和标准差；
- **选择最佳假设和模型超参**：基于平均结果，确定哪种模型配置（如不同的超参数、特征选择方法、假设类型等）的性能最好；
- **使用全部数据训练最终模型**：确定最佳模型配置后，使用全部可用的数据（即原始的训练集，有时也包括验证集，但不包括测试集）来训练一个最终的模型；
- **评估最终模型**：使用独立的测试集来评估最终模型的泛化性能

4 自助采样法

- 自助采样（bootstrap sampling）又称“有放回采样”：给定包含m个样本的数据集D，每次随机从D中挑选一个样本，将其副本放入D'（该样本自身仍在数据集D中，使得下次采样时仍有可能被选中）。重复执行m次，就得到了包含m个样本的数据集D'



训练集与原样本集同规模，数据分布有所改变

*在m次采样中，某样本始终不被选中的概率极限 $\lim_{m\to\infty}\left(1-\frac{1}{m}\right)^m \mapsto \frac{1}{e}\approx 0.368$



- 以D'作为训练集，$D \backslash D'$用作测试集。这样的测试结果，被称为“包外估计”（out-of-bag estimate）
  - 优点：对于数据集较小、难以有效划分训练/测试集时很有效。此外，自助法能从初始数据集中产生多个不同的训练集，有利于集成学习方法（→第6讲）
  - 缺点：自助法改变了初始数据集的分布，会引入估计偏差

第二讲 模型选择与评估



1 监督学习的计算理论

2 误差构成与过拟合

3 模型验证与测试

4 模型的性能度量



模型的性能度量

1 分类任务的性能度量指标

- 二分类任务存在正例和反例。基于真实情况和预测结果，可得到以下混淆矩阵（Confusion Matrix）：分为真正例（Ture Positive, TP）、假正例（False Positive, FP）、真反例（Ture Negative, TN)、假反例（ False Negative, FN ）四类结果

真实情况	预测结果	
	正例	反例
正例	TP (真正例)	FN (假反例)
反例	FP (假正例)	TN (真反例)

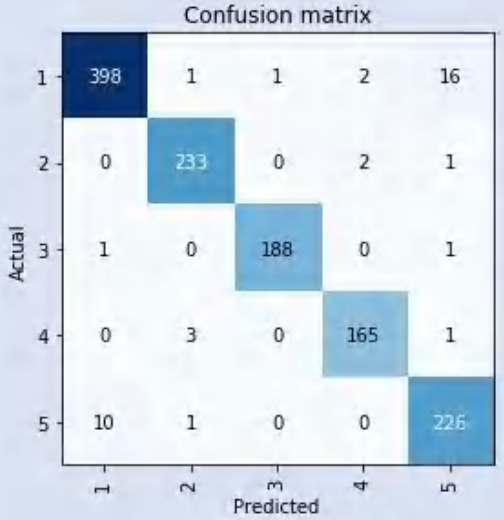
二分类的混淆矩阵

模型的性能度量

- 多分类的混淆矩阵



示例1



示例2

- 常见的二分类性能度量指标

正确率Accuracy：分类正确的样本数占样本总数的比例 $(TP+TN)/(P+N)$

错误率Error：分类错误的样本数占样本总数的比例 $(FP+FN)/(P+N)$

Accuracy =1 - error

模型的性能度量

精度/查准率Precision	分类为正例的样本中实际为正例的比例 $TP/(TP+FP)$
召回率/查全率Recall	正例中被分类为正例的比例 $TP/(TP+FN)$
灵敏度Sensitive	正例中被分类为正例的比例 $TP/(TP+FN)$
特异性Specificity	反例中被分类为反例的比例 $TN/(FP+TN)$
Fb-score	查准率和召回率的调和平均: $[(1+b)\cdot P \cdot R]/(b \cdot P+R)$

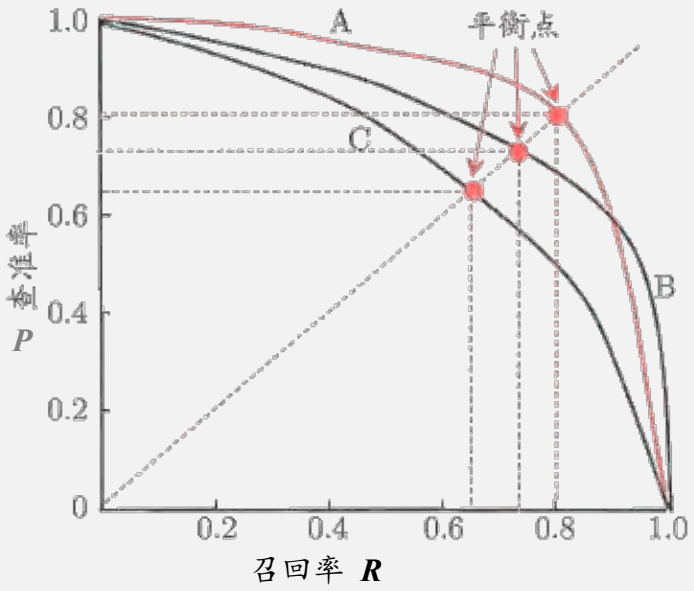
对正例的识别能力

对反例的识别能力

精确率和召回率是相互影响的，F1分数是二者平衡点的指标，b=1

- 精确率关注的是预测结果的准确性，反映了模型的准确性；
- 召回率关注的是对正类样本的识别能力，反映了模型的敏感性；
- F1-score 是精确率（Precision）和召回率（Recall）的调和平均数，值越高表示模型在分类任务中表现越好，能够更好地平衡精确率和召回率

模型的性能度量



- 若一个学习器A的P-R曲线被另一个学习器B的P-R曲线完全包住，则称：B的性能优于A。若A和B的曲线发生了交叉，则谁的曲线下的面积大，谁的性能更优。
- “平衡点”（Break-Event Point，简称BEP），即当P=R时的取值，平衡点的取值越高，性能更优

P：分类为正例的样本中实际为正例的比例 $TP/(TP+FP)$， FP 是假正例

R：正例中被分类为正例的比例 TP/(TP+FN)， FN 是假负例

在类别不平衡的情况下，简单的准确率P可能会误导。例如，在一个99%为负类和1%为正类的数据集中，模型只需预测所有样本为负类就能获得99%的准确率，但这并不能反映模型的实际性能。在医疗诊断、信用欺诈检测等领域，假阴性（未能识别出正类）可能带来严重后果，因此需要关注召回率R。

模型的性能度量

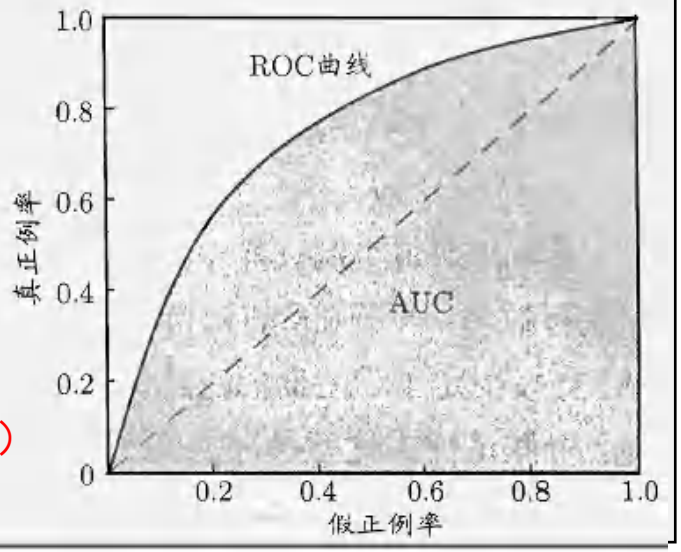
- **ROC（Receiver Operating Characteristic）曲线**：“接受者”操作特性曲线。与P-R曲线类似，逐次设置阈值把样本预测为“正例”和“反例”，计算当前的真正率和假正率。ROC曲线其与坐标轴围成的面积AUC越大性能越优

- 纵轴：是“真正例率”（True Positive Rate，简称TPR）；

$$TPR = \frac{TP}{TP + FN},$$ （灵敏度）

- 横轴为“假正例率”（False Positive Rate，简称FPR）；

$$FPR = \frac{FP}{TN + FP}.$$ （1-特异性）



ROC曲线主要关注真阳性率（召回率）和假阳性率，而不直接考虑精确率。这可能导致在某些应用中，模型的实际表现未能得到充分反映。

模型的性能度量

例：用已经训练好的学习器对8个测试样本（4个正例，4个反例，即 m+ = m− = 4）进行预测，假设预测结果为：

(s1, 0.77, +)、(s2, 0.65, −)、(s3, 0.56, +)、(s4, 0.56, −),

(s5, 0.50, +)、(s6, 0.30, +)、(s7, 0.20, −)、(s8, 0.15, −)

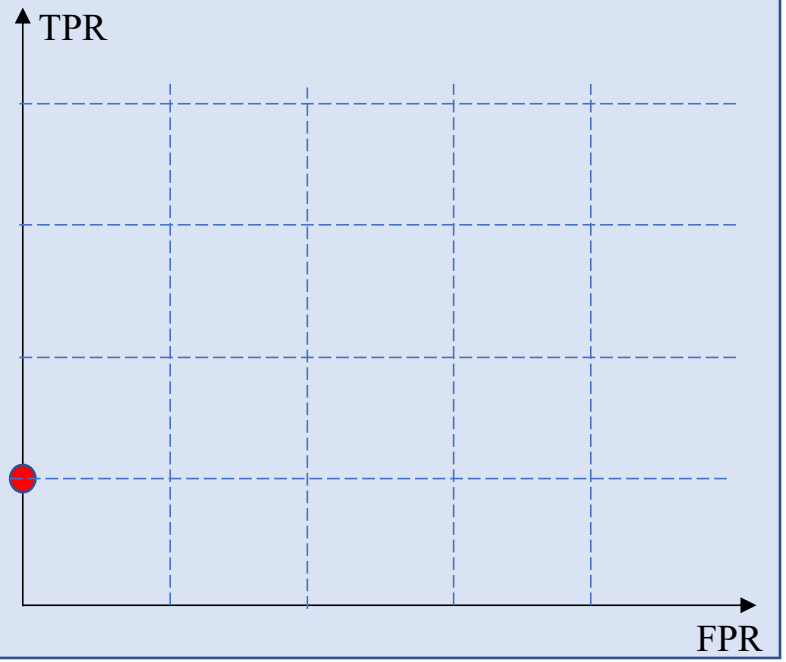
真实标签

预测概率

设将x轴（假正例率轴）的单位刻度定为1/m−，y轴（真正例率轴）的单位刻度定为1/m+，试绘制ROC曲线

阈值：0.77

真实情况	预测结果		
	+	-	
+	1 TP	3 FN	=1/4
-	0 FP	4 TN	=0



模型的性能度量

Eg 用已经训练好的学习器对8个测试样本（4个正例，4个反例，即 m+ = m− = 4）进行预测，假设预测结果为：

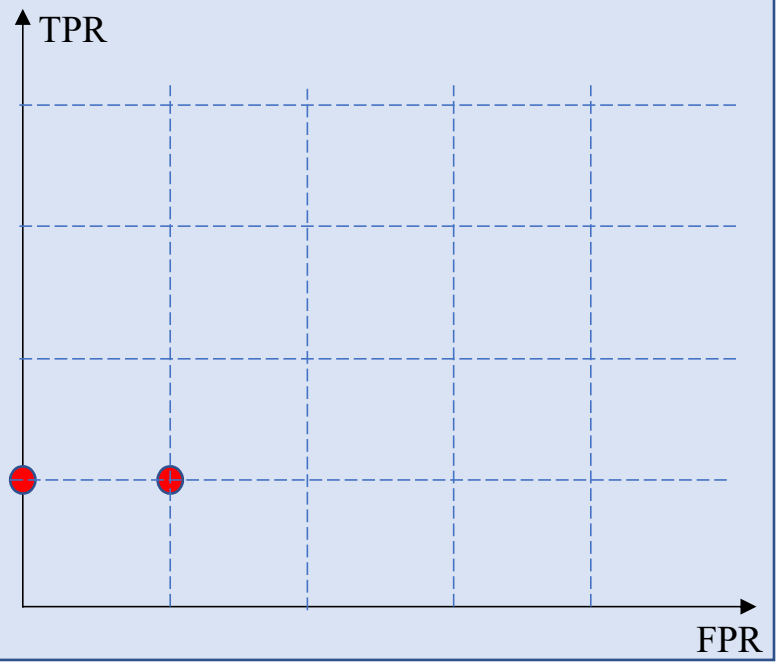
(s1, 0.77, +)、(s2, 0.65, −)、(s3, 0.56, +)、(s4, 0.56, −),

(s5, 0.50, +)、(s6, 0.30, +)、(s7, 0.20, −)、(s8, 0.15, −)

设将x轴（假正例率轴）的单位刻度定为1/m−，y轴（真正例率轴）的单位刻度定为1/m+，试绘制ROC曲线

阈值:
0.65

真实情况	预测结果		
	+	-	
+	1 TP	3 FN	=1/4
-	1 FP	3 TN	=1/4



模型的性能度量

例：用已经训练好的学习器对8个测试样本（4个正例，4个反例，即 m+ = m− = 4）进行预测，假设预测结果为：

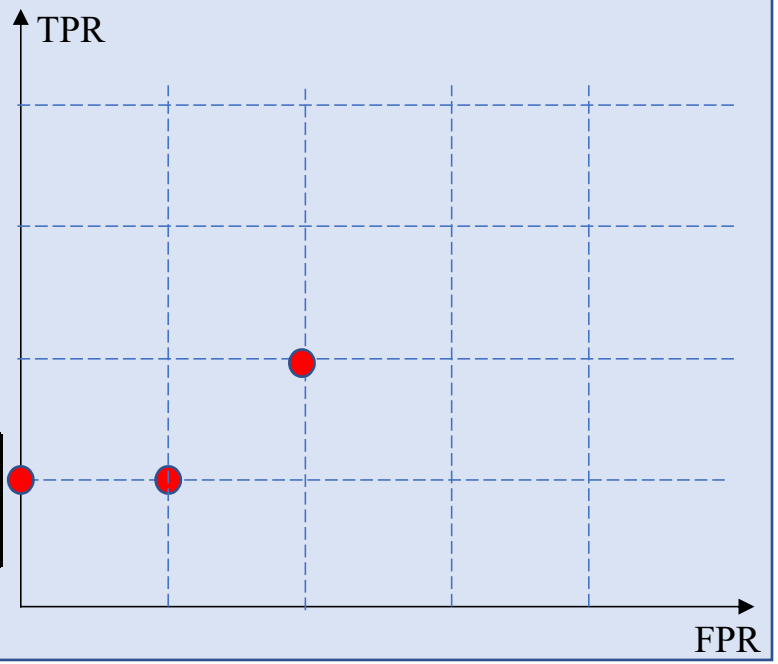
(s1, 0.77, +)、(s2, 0.65, −)、(s3, 0.56, +)、(s4, 0.56, −),

(s5, 0.50, +)、(s6, 0.30, +)、(s7, 0.20, −)、(s8, 0.15, −)

设将x轴（假正例率轴）的单位刻度定为1/m−，y轴（真正例率轴）的单位刻度定为1/m+，试绘制ROC曲线

阈值：0.56

真实情况	预测结果		
	+	-	
+	2 TP	2 FN	=1/2
-	2 FP	2 TN	=1/2



模型的性能度量

Eg 用已经训练好的学习器对8个测试样本（4个正例，4个反例，即 m+ = m− = 4）进行预测，假设预测结果为：

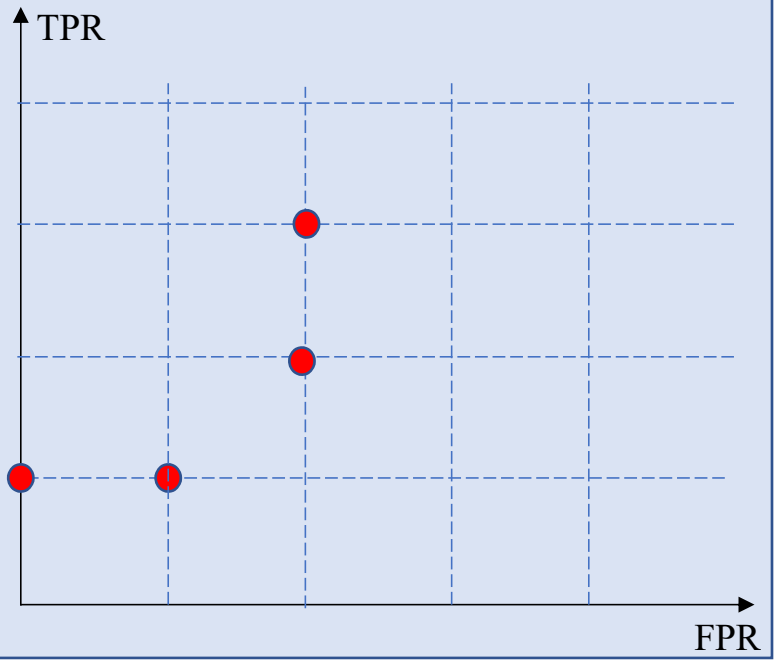
(s1, 0.77, +)、(s2, 0.65, −)、(s3, 0.56, +)、(s4, 0.56, −),

(s5, 0.50, +)、(s6, 0.30, +)、(s7, 0.20, −)、(s8, 0.15, −)

设将x轴（假正例率轴）的单位刻度定为1/m−，y轴（真正例率轴）的单位刻度定为1/m+，试绘制ROC曲线

阈值:
0.50

真实情况	预测结果		
	+	-	
+	3 TP	1 FN	=3/4
-	2 FP	2 TN	=1/2



模型的性能度量

例：用已经训练好的学习器对8个测试样本（4个正例，4个反例，即 m+ = m− = 4）进行预测，假设预测结果为：

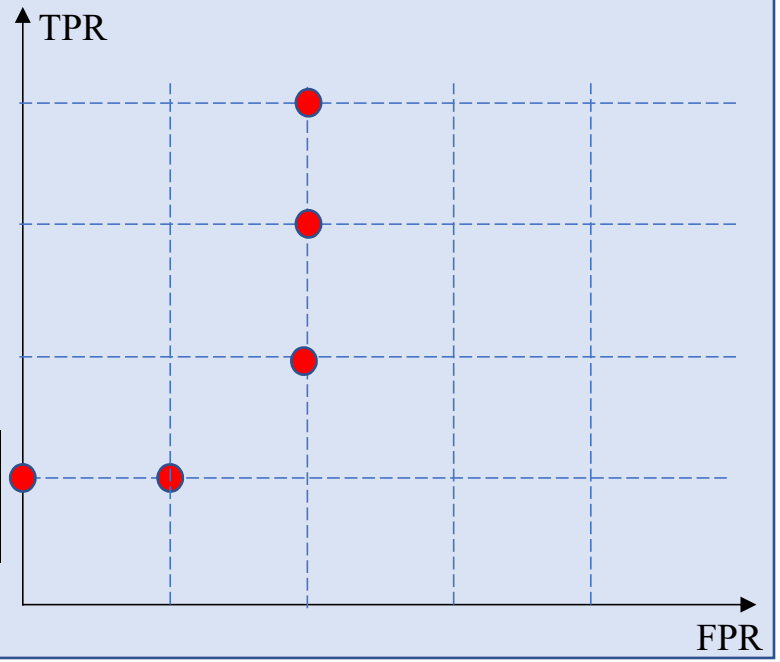
(s1, 0.77, +)、(s2, 0.65, −)、(s3, 0.56, +)、(s4, 0.56, −),

(s5, 0.50, +)、(s6, 0.30, +)、(s7, 0.20, −)、(s8, 0.15, −)

设将x轴（假正例率轴）的单位刻度定为1/m−，y轴（真正例率轴）的单位刻度定为1/m+，试绘制ROC曲线

阈值：0.30

真实情况	预测结果		
	+	-	
+	4 TP	0 FN	=1
-	2 FP	2 TN	=1/2



模型的性能度量

Eg 用已经训练好的学习器对8个测试样本（4个正例，4个反例，即 m+ = m− = 4）进行预测，假设预测结果为：

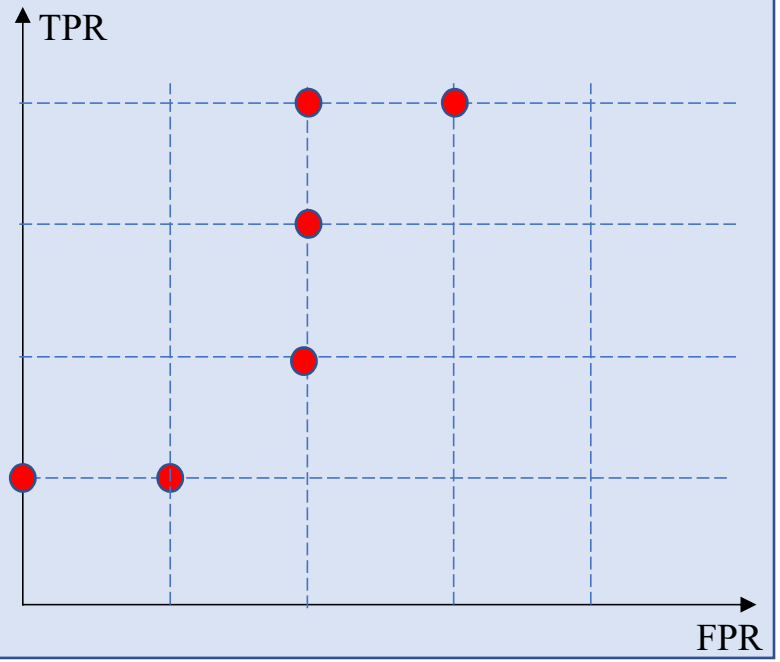
(s1, 0.77, +)、(s2, 0.65, −)、(s3, 0.56, +)、(s4, 0.56, −),

(s5, 0.50, +)、(s6, 0.30, +)、(s7, 0.20, −)、(s8, 0.15, −)

设将x轴（假正例率轴）的单位刻度定为1/m−，y轴（真正例率轴）的单位刻度定为1/m+，试绘制ROC曲线

阈值:
0.20

真实情况	预测结果		
	+	-	
+	4 TP	0 FN	=1
-	3 FP	1 TN	=3/4



模型的性能度量

Eg 用已经训练好的学习器对8个测试样本（4个正例，4个反例，即 m+ = m− = 4）进行预测，假设预测结果为：

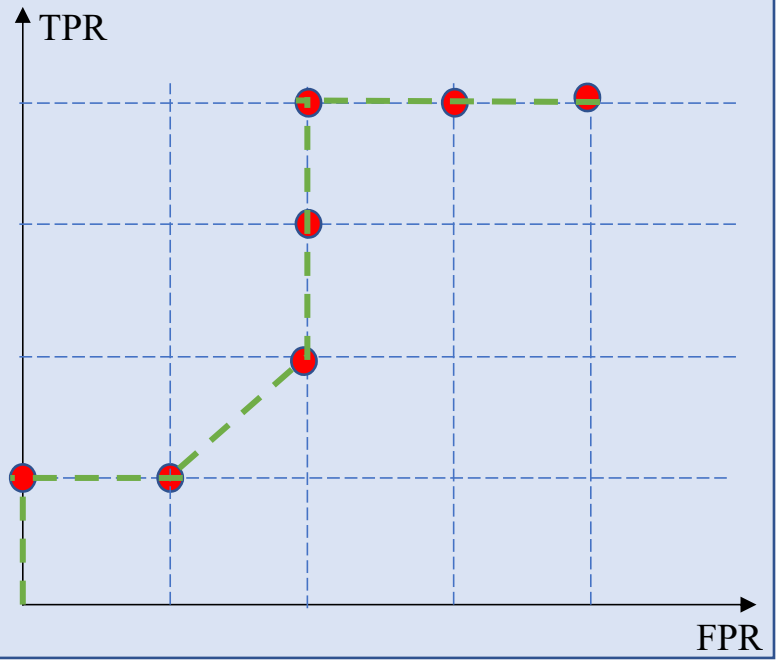
(s1, 0.77, +)、(s2, 0.65, −)、(s3, 0.56, +)、(s4, 0.56, −),

(s5, 0.50, +)、(s6, 0.30, +)、(s7, 0.20, −)、(s8, 0.15, −)

设将x轴（假正例率轴）的单位刻度定为1/m−，y轴（真正例率轴）的单位刻度定为1/m+，试绘制ROC曲线

阈值:
0.15

真实情况	预测结果		
	+	-	
+	4 TP	0 FN	=1
-	4 FP	0 TN	=1

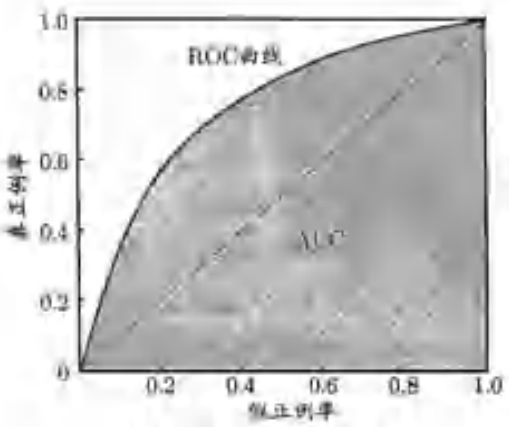


模型的性能度量

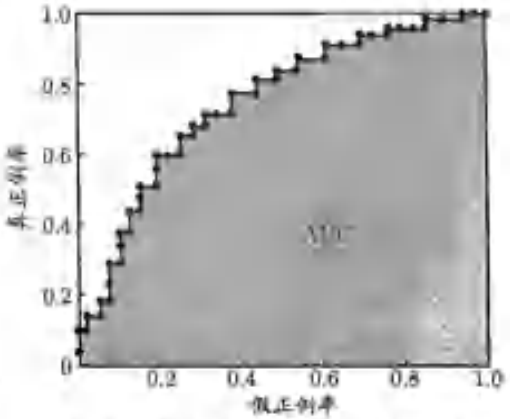
ROC与AUC

模型的性能比较：

- 若一个学习器A的ROC曲线被另一个学习器B的ROC曲线完全包住，则称B的性能优于A。
- 若A和B的曲线发生了交叉，则谁的曲线下的面积大，谁的性能更优。



(a) ROC 曲线与 AUC



(b) 基于有限样例绘制的 ROC 曲线与 AUC

$$\mathrm{AUC} = \frac{1}{2}\sum_{i=1}^{m-1}(x_{i+1} - x_i)\cdot(y_i + y_{i+1})\,.$$

公式推导可参考南瓜书P2

ROC曲线下的面积定义为AUC（Area Under ROC Curve），不同于P-R的是，这里的AUC是可估算的，即ROC曲线下每一个小矩形的面积之和。

- AUC越大，证明排序的质量越好，
- AUC为1时，证明所有正例排在了负例的前面，
- AUC为0时，所有的负例排在了正例的前面。



3 回归任务的性能度量指标

- 用以下指标度量n个预测值$\hat{y}$与测试样本标记值y的差异

平均绝对误差
Mean Absolute Error, MAE

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|\hat{y}_i - y_i|$$

均方误差
Mean-Square Error, MSE

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

均方根误差
Root MSE

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2}$$

提问1： 用于损失（代价）函数的MAE、 MSE与用于回归性能度量的MAE、 MSE有什么区别联系？

第三讲 贝叶斯分类器

1 贝叶斯决策理论基础

2 类条件概率的参数估计

3 朴素贝叶斯分类器

4 简单EM算法

Baidu百科 百度百科——全球最大中文百科全书

- 托马斯·贝叶斯（Thomas Bayes, 1702-1761），18世纪英国神学家、数学家、数理统计学家和哲学家，概率论理论创始人，被称为“归纳地运用数学概率”，“从特殊推论一般、从样本推论全体”的第一人。贝叶斯决策理论是贝叶斯派归纳理论的重要组成部分，也是机器学习中的一个基本方法



Thomas Bayes
(1702-1761)

1 贝叶斯公式

- **先验概率（prior probability）**：没有对样本进行任何观测情况下的概率。在一般分类问题中，常以c_i或$\omega_i(i=1,2,3\ldots k)$表示类型，以$P(c_i)$表示其先验概率，$\sum_i P(c_i)=1$

- **类条件概率（class-conditional probability）**：即属于类c_i的事件具有观测特征x的条件概率，记作$P(X=\boldsymbol{x}|Y=c_i)$（简记为$P(\boldsymbol{x}|c_i)$）。获得这些条件概率的分布情况，需知类条件概率分布律或概率密度函数
- **证据（evidence）**：观测到具有特征x的样本的概率

$$P(\boldsymbol{x})=\sum_i P(\boldsymbol{x},c_i)=\sum_i P(\boldsymbol{x}|c_i)P(c_i)$$

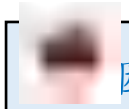
（全概率公式）

- **后验概率（posterior probability）**：系统在观测到某个具体样本x条件下，判断其属于某种类型的概率。需要组合先验知识和证据，利用贝叶斯公式计算获得，记作$P(c_i|\boldsymbol{x})$，显然$\sum_i P(c_i|\boldsymbol{x})=1$

后验概率　类条件概率　先验概率

样本$\boldsymbol{x}$属于c_k类的概率：$$P(c_k|\boldsymbol{x})=\frac{P(\boldsymbol{x}|c_k)P(c_k)}{\sum_i P(\boldsymbol{x}|c_i)P(c_i)}$$

证据



因拭子核酸检测的阴性$P(c_1)$与阳性$P(c_2)$先验概率分别是0.9和0.1。核酸检测的类条件概率密度函数已知，具有观测特征$\boldsymbol{x}$的样本经该函数查表可得$P(\boldsymbol{x}|c_1)=0.2$，$P(\boldsymbol{x}|c_2)=0.4$。利用贝叶斯公式计算样本$\boldsymbol{x}$属于$c_1$和$c_2$的后验概率

$$P(c_1|\boldsymbol{x})=\frac{P(\boldsymbol{x}|c_1)P(c_1)}{\sum_i P(\boldsymbol{x}|c_i)P(c_i)}=\frac{0.2\times0.9}{0.2\times0.9+0.4\times0.1}=0.818$$

$$P(c_2|\boldsymbol{x})=\frac{P(\boldsymbol{x}|c_2)P(c_2)}{\sum_i P(\boldsymbol{x}|c_i)P(c_i)}=\frac{0.4\times0.1}{0.2\times0.9+0.4\times0.1}=0.182$$

提问：因为$P(c_1|x)>P(c_2|x)$，那么能否据此确定观测样本x属于c_1类呢？

2 最小错误率分类决策

- 假设样本x的特征是一维时，t为x轴上的一点，形成两个决策区域:
  - $R_1 \sim (-\infty, t)$：决策为c_1，$P(error|x) = P(c_2|x)$
  - $R_2 \sim (t, +\infty)$：决策为c_2，$P(error|x) = P(c_1|x)$



e.g 刚才例子里根据后验概率0.818作出属于c_1决策时，实际就蕴含了0.182的错误率

- 则平均错误率有:

平均错误率（期望）

$$P(e) = \int_{-\infty}^{t} P(c_2|x)p(x)dx + \int_{t}^{\infty} P(c_1|x)p(x)dx$$ ← 期望的定义

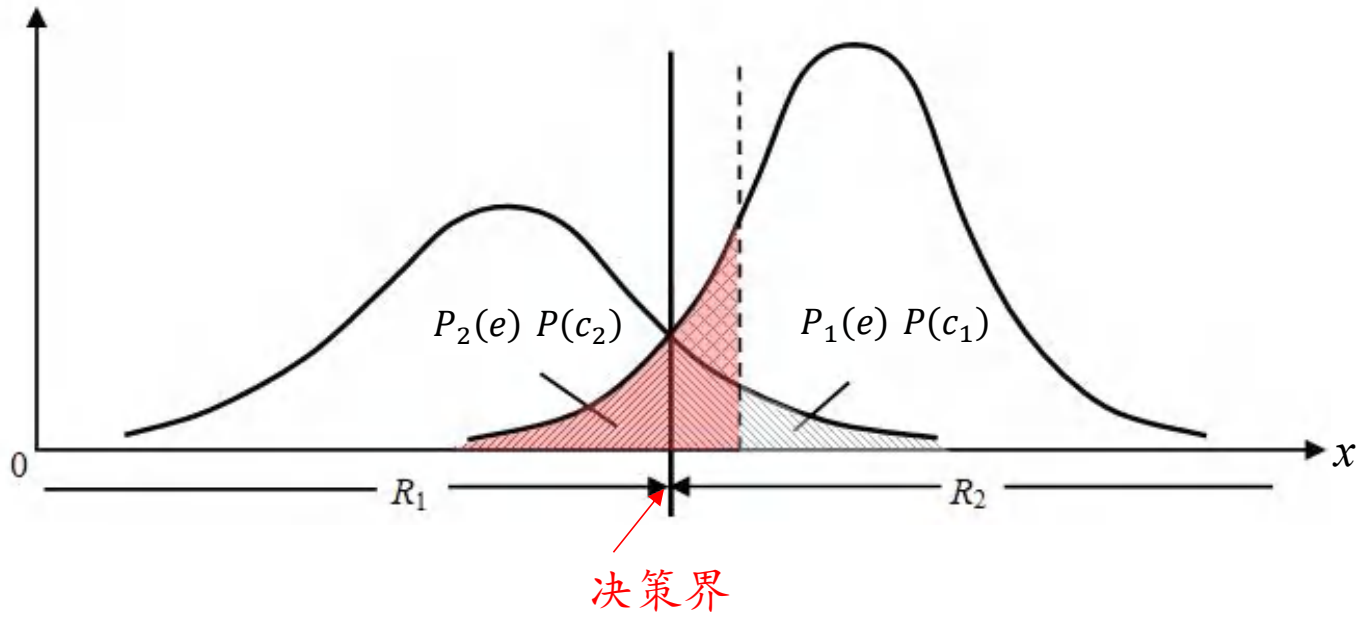
$$= \int_{R_1} p(x|c_2)P(c_2)dx + \int_{R_2} p(x|c_1)P(c_1)dx$$ ← 贝叶斯公式展开

$$P(e) = P_2(e)P(c_2) + P_1(e)P(c_1)$$

$P_2(e)$：第二类错误　　$P_1(e)$：第一类错误

$$P(e) = P_2(e)P(c_2) + P_1(e)P(c_1)$$

- 错误率$P(e)$可以用下图两个阴影部分面积表示
- 当t 满足$P(t|c_1)P(c_1) = P(t|c_2)P(c_2)$即$P(c_1|x=t) = P(c_2|x=t)$时，面积和最小，说明用后验概率进行决策可以使得分类错误率最小



- 贝叶斯分类器（Bayes'Classifier）选择具有最高后验概率的类作为决策结果。这种分类是一种基于**最小错误的分类决策**

选择c_i，如果$P(c_i|\boldsymbol{x})=max_i P(c_i|\boldsymbol{x})$

3 最小风险分类决策

- 错误率最小决策在很多情况下不是最佳选择。在贝叶斯决策理论中，当不同分类错误的风险（Risk）不相等时，为了最小化整体风险或预期损失，需要采用基于**最小风险的分类决策**
- 行动α_i：表示把 $\boldsymbol{x}$判决为c_i类的动作
- 给定k个类别，令λ_{ij}为将实属于c_j类样本，误分为c_i类所产生的损失。则采用行动α_i可能导致的**条件风险**（也称期望风险）为:

$$R(\alpha_i|\boldsymbol{x}) = \sum_{j=1}^{k} \lambda_{ij} P(c_j|\boldsymbol{x})$$

贝叶斯决策理论基础

- 贝叶斯分类器也可以选择具有最小风险行动作为决策依据

采取α_i，如果$R(\alpha_i|\boldsymbol{x})=min_i\ R(\alpha_i|\boldsymbol{x})$

- 针对不同的实际问题，如果希望尽可能避免将属于c_j的样本判断为c_i，则可将损失λ_{ij}设大。损失越大，期望风险也越大

损失 / 类别 行动	c_1	c_2	c_3		c_k
α_1	λ_{11}	λ_{12}	λ_{13}		λ_{1k}
α_2	λ_{21}	λ_{22}	λ_{23}		λ_{2k}
α_3					
				λ_{ij}	
α_k	λ_{k1}	λ_{k2}	λ_{k3}		λ_{kk}

（对角线元素为0）

第三讲 贝叶斯分类器

导航工程 2025修订版

1 贝叶斯决策理论基础

2 类条件概率的参数估计

3 朴素贝叶斯分类器

4 简单EM算法

类条件概率的参数估计

$$P(c_k|x)=\frac{P(X=x|Y=ck)P(Y=ck)}{\Sigma_i P(x|ci)P(ci)}$$

- 证据$P(\boldsymbol{x})$是具有特征 $\boldsymbol{x}$ 的样本的概率，可以从训练集中直接统计。采用诸如比例判决函数决策时，$P(\boldsymbol{x})$甚至可以不用计算
- 先验概率$P(c_k)$是样本空间中各类样本的占例，也可以从训练集中统计各类样本占总体的频率而获得
- 因此贝叶斯决策的核心是获得样本空间和标记空间的联合概率分布$P_{X,Y}$，即获得类条件概率$P(X|Y=c_k)$，以设计贝叶斯分类器。因此，**贝叶斯分类器的学习变成了对类条件概率的参数估计问题**

类条件概率的参数估计



现有成年人身高和小孩身高2类样本各400例：

	1.4	1.45	1.5	1.55	1.6	1.65	1.7	1.75	1.8	合计
成人	1	8	25	40	72	100	84	50	20	400
少年	59	82	137	73	34	12	3	0	0	400

- 现有测试样本x_1=1.55m，他属于哪个类？

  根据贝叶斯分类器原理，我们要计算$P(X=1.55|成人)$和$P(X=1.55|少年)$

- 测试样本x_2=1.58m，他属于哪个类？

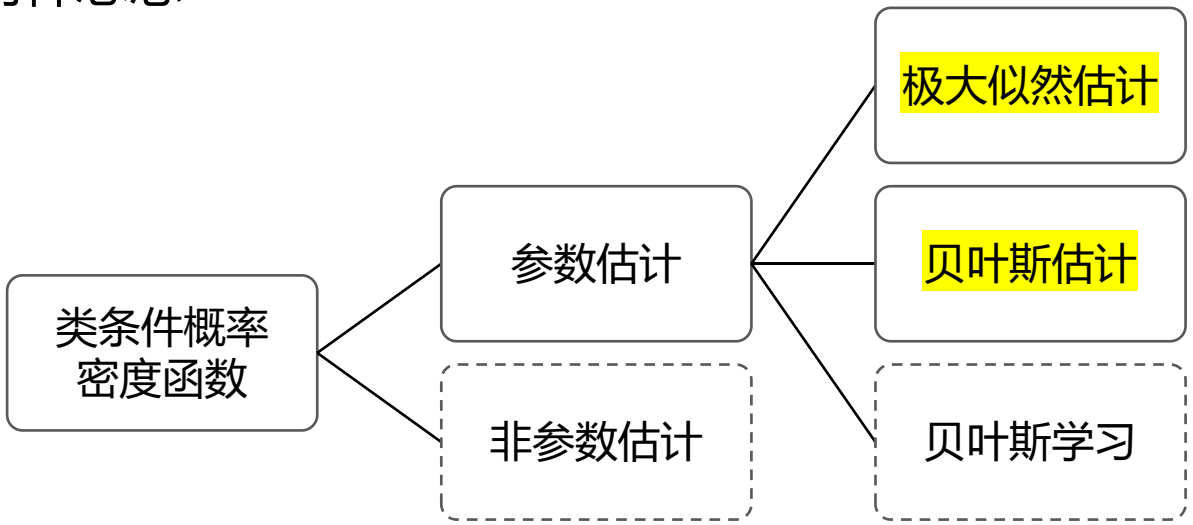
$P(X=1.58|Y=成人)$ 又怎么算呢？

$P(X=1.55|Y=成人)$ =40/400=0.1吗？

类条件概率的参数估计

类条件概率的参数估计

- 类条件概率的参数估计方法很多。比较常见的包括极大似然估计（Maximum Likelihood Estimation, MLE）和贝叶斯估计（Bayes Estimation），两类估计方法分别代表了频率主义学派和贝叶斯学派的两种思想



1 极大似然估计

- **极大似然原理：**假设造成某个随机事件A发生的原因有n个，分别记为$A_1, A_2, \ldots, A_n$。由原因A_i导致A发生的概率记为$P_i = P(A|A_i)$（i=1,2,..,n）。当观测到事件A已经发生时，若$P_i < P_k$ (i=1,..,n, $i \neq k$)，则事件A发生的原因是A_k
- $(\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_{m_k})$是样本空间$X$中$c_k$类的一组观测到的样本，看作随机事件发生的结果。$P(X|Y=c_k)$具有确定的概率分布形式，由一组参数$\{\theta_{c_k}\}$确定，看作原因。则可以构造**似然函数**：
- 似然函数是关于模型参数的函数，表示为 $L(\theta_{c_k}|x)$，其中θ_{c_k}是模型参数，x 是观测数据。它衡量了在特定参数值θ_{c_k}下，观测到当前数据x的概率。

类条件概率的参数估计

$$L(\theta_{c_k}) = L(\theta_{c_k}; \boldsymbol{x}_1, \ldots, \boldsymbol{x}_{m_k} | Y{=}c_k) = \begin{cases} \prod_{i=1}^{m_k} p(\theta_{c_k}; \boldsymbol{x}_i) & X\text{离散型，分布律为}p(\theta_{c_k}; \boldsymbol{x}|c_k) \\ \prod_{i=1}^{m_k} f(\theta_{c_k}; \boldsymbol{x}_i) & X\text{连续型，概率密度为}f(\theta_{c_k}; \boldsymbol{x}|c_k) \end{cases}$$

- $L(\theta_{c_k})$ 是类别c_k下的似然函数
- θ_{c_k}是与类别c_k相关的模型参数
- $x1, \ldots, x_{m_k}$ 是属于类别c_k的观测数据点
- m_k是类别c_k下的数据点数量

概率和似然的区别和极大似然估计的定义 - 一杯明月 - 博客园

➢ 以上似然函数又被称为**类似然**（class-likelihood）。要注意，类似然和该类样本总体X的联合分布（离散型为联合分布律，连续型为联合概率密度）形式一致，但含义不同：前者是固定$(\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_{m_k})$看作是参数$\{\theta_{c_k}\}$的函数；后者是给定类别参数$\{\theta_{c_k}\}$下 ，观测到特定样本数据$(\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots)$的概率

注意：后面内容不严格区分离散型和连续型，p（小写）和f均可代表概率密度

类条件概率的参数估计

- 根据极大似然原理，寻找到参数$\{\hat{\theta}_{c_k}\}$使得$(\boldsymbol{x}_1,\boldsymbol{x}_2,\ldots,\boldsymbol{x}_{m_k})$的出现具有最大可能，则称$\{\hat{\theta}_{c_k}\}$是概率分布参数$\{\theta_{c_k}\}$的**极大似然估计值**。

在实际中为避免概率连乘，通常用求取对数似然（log-likelihood）驻点的方式求解$\{\hat{\theta}_{c_k}\}$，这也是贝叶斯分类器的主要“学习”内容

极大似然估计

1. 根据c_k类的样本$(\boldsymbol{x}_1,\boldsymbol{x}_2,\ldots,\boldsymbol{x}_{m_k})$构造对数似然函数：

$$LL(\theta_{c_k})=\ln L(\theta_{c_k};\boldsymbol{x}_1,\ldots,\boldsymbol{x}_{m_k}|Y=c_k)=\begin{cases}\sum_{i=1}^{m_k}\ln p(\theta_{c_k};\boldsymbol{x}_i) & \text{离散型}\\ \sum_{i=1}^{m_k}\ln f(\theta_{c_k};\boldsymbol{x}_i) & \text{连续型}\end{cases}$$

2. 确定优化函数：$\hat{\theta}_{c_k}=\underset{\theta_{c_k}}{\operatorname{argmax}}\,LL(\theta_{c_k})$

3. 令$LL(\theta_{c_k})$对每一个$\hat{\theta}_{c_k}$参数的偏导为0（求驻点），计算$\hat{\theta}_{c_k}$

注：通常θ_{c_k}是一组参数

类条件概率的参数估计



Eg. 设总体的概率密度函数是：$f(x)=\begin{cases}3\lambda x^2\exp(-\lambda x^3) & x>0\\ 0 & else\end{cases}$

其中λ是未知参数，$x_1,x_2,\ldots,x_m$是一组样本值，求参数λ的极大似然估计

解：构造似然函数$L=\prod_{i=1}^{m}(3\lambda x_i^2\exp(-\lambda x_i^3))=(3^m\lambda^m\left(\prod_{i=1}^{m}x_i^2\right)\exp(-\lambda\sum_{i=1}^{m}x_i^3))$

取对数：$\mathrm{In}L=m\mathrm{In}(3\lambda)+\sum_{i=1}^{m}\mathrm{In}\,x_i^2-\lambda\sum_{i=1}^{m}x_i^3$

令$\frac{d\mathrm{In}L}{d\lambda}=\frac{m}{\lambda}-\sum_{i=1}^{m}x_i^3=0$

$\Rightarrow\hat{\lambda}=\frac{m}{\sum_{i=1}^{m}x_i^3}$

《概率论与数理统计》考研真题

类条件概率的参数估计

➢ 几类常见概率分布参数的MLE：

设c_k类数据集$D_k=\{x_i\}$，若其分布为：

- 伯努利分布$P(x)=p^x(1-p)^{1-x}$：事件发生的概率为p，不发生为$1-p$，

  该分布的参数θ_{c_k}就是p，其MLE：$\hat{p}=\frac{\sum_{x_i\in D_k} x_i}{|D_k|}$

- 均值为μ，方差为σ^2的一维正态分布~$N(\mu,\sigma^2)$：$f(x)=\frac{1}{\sqrt{2\pi}\sigma}\exp\left[-\frac{1}{2\sigma^2}(x-\mu)^2\right]$

  该分布的参数为μ和σ^2，其MLE：$\hat{\mu}_{c_k}=\frac{1}{|D_k|}\sum_{x_i\in D_k} x_i$

  $$\hat{\sigma}^2_{c_k}=\frac{1}{|D_k|}\sum_{x_i\in D_k}(x_i-\hat{\mu}_{c_k})^2$$

- 指数分布、泊松分布…*（请自行复习概率统计）*

第三讲 贝叶斯分类器

武汉大学 WUHAN UNIVERSITY

导航工程 2025修订版

1 贝叶斯决策理论基础

2 类条件概率的参数估计

3 朴素贝叶斯分类器

4 简单EM算法

1 条件独立性假设

- 在实际应用中，使用贝叶斯分类器经常碰到这种困难：类条件概率是所有特征上的联合概率，难以从有限的样本之间估计而得



帅？	性格好？	身高？	上进？	嫁与否
帅	不好	矮	不上进	不嫁
不帅	好	矮	上进	不嫁
帅	好	矮	上进	嫁
不帅	好	高	上进	嫁
帅	不好	矮	上进	不嫁
帅	不好	矮	上进	不嫁
帅	好	高	不上进	嫁
不帅	好	中	上进	嫁
帅	好	中	上进	嫁
不帅	不好	高	上进	嫁
帅	好	矮	不上进	不嫁
帅	好	矮	不上进	不嫁

根本就找不到这样的样本如何统计？

那么一个{不帅、性格不好、身高矮、不上进}的男生嫁还是不嫁？

朴素贝叶斯分类器

- **朴素（Naive）贝叶斯**推断是对条件概率分布做了**条件独立性**的假设。即认为**每个特征独立**地对分类结果发生影响

- 联合后验概率计算：

x为特征向量，表明是多维特征的样本，维数为d

$$P(c_k|\boldsymbol{x}) = \frac{P(X=\boldsymbol{x}|Y=c_k)P(Y=c_k)}{P(\boldsymbol{x})} = \frac{P(c_k)}{P(\boldsymbol{x})}\prod_{i=1}^{d} P(x^{(i)}|c_k)$$

- 朴素贝叶斯分类器的训练过程就是基于训练集D来计算先验概率$P(c_k)$，并为每一维特征$x^{(i)}$计算类条件概率$P(x^{(i)}|c_k)$，再求联合条件概率

第三讲 贝叶斯分类器

1 贝叶斯决策理论基础

2 类条件概率的参数估计

3 朴素贝叶斯分类器

4 简单EM算法

1 隐变量

- 在前面讨论中，我们一直假设训练样本所有的属性值是可以被观测到的。在实际工作中，有些特征属性因为客观原因无法观测，样本数据“不完整”



- e.g1 我们有200个核酸检测数据，但我们并不知道这些样本来自阴性群体c_1还是阳性群体c_2
- e.g2 我们有200个身高数据，但我们并不知道这些样本来自成人还是小孩

简单EM方法

- 如果概率分布的变量都是可观测的变量，那么可以使用MLE或MAP等方法估计分布参数。但如含有**隐变量（latent variale）**则无法直接求解。此时估计$P(X=x|Y=c_i)$的分布参数就需要使用**期望最大化EM算法（Expectation-Maximization）**



一些常见的存在隐变量的机器学习方法

模型	隐变量
高斯混合模型 (Gaussian Mixture Model, GMM)	每个观测数据点的类标签
隐马尔可夫模型 (Hidden Markov Model, HMM)	隐藏状态序列。这些隐藏状态之间存在一定的转移概率，但隐藏状态本身不可观测
k-Means聚类算法 →第8讲	每个样本点所属的类别（即聚类质心）

2 Q函数

- ***Q*函数**全称为“完全数据·对数似然函数·的期望”（Expectation of the Complete-data Log-likelihood Function），用于在E步中计算完全数据的对数似然函数在给定观测样本数据和当前参数估计下的**期望值**。*Q*函数的设计是EM方法的核心

简单EM方法

- 设未知类别样本$X=(\boldsymbol{x}_1,\boldsymbol{x}_2,\ldots,\boldsymbol{x}_m)$，对应的隐变量$z$为离散型随机变量 $z=(z_1, z_2,\ldots, z_n)$，称 (X, z) 为完全数据。$(\boldsymbol{x}_i, z_i)$的联合概率密度为$p(\boldsymbol{x}_i, z_i; \theta)$，有：

θ 的值决定了模型如何拟合数据

θ 的具体形式：取决于任务

例如：

在分类问题中，θ 可能是分类器的权重参数。

在聚类问题中，θ 可能是每个簇的中心和分布参数。

在隐马尔可夫模型（HMM）中，θ 包括状态转移概率、观测概率等。

θ 是模型的核心参数，描述了数据和隐变量的联合分布。通过 EM 算法，在隐变量 Z 不可观测的情况下，迭代优化 θ ，以最大化模型对观测数据的拟合程度

- 设未知类别样本$X=(\boldsymbol{x}_1,\boldsymbol{x}_2,\ldots,\boldsymbol{x}_m)$，对应的隐变量$z$为离散型随机变量 $z=(z_1, z_2,\ldots, z_n)$，称(X, z)为完全数据。$(\boldsymbol{x}_i, z_i)$的联合概率密度为$p(\boldsymbol{x}_i, z_i; \theta)$，有：

θ 的值决定了模型如何拟合数据

一般这个n是超参数，需预先设置

$$p(\boldsymbol{x}_i;\theta)=\sum_{j=1}^{n} p(\boldsymbol{x}_i, z_j;\theta)$$

其中θ为待估计的参数，则含隐变量z的极大似然估计写为：

$$\hat{\theta}, z=\underset{\theta}{\operatorname{argmax}} LL(\theta, z)=\underset{\theta,z}{\operatorname{argmax}} \sum_{i=1}^{m} \ln \sum_{j=1}^{n} p(\boldsymbol{x}_i, z_j;\theta)$$

因为有隐变量，不便直接求

简单EM方法

- 利用对数函数凹性，构造m个新的随机变量，其分布依次为$Q_1(z), Q_2(z),\ldots,$ $Q_m(z)$（这里$Q_i(z_j)$ 可理解为隐变量Z的后验分布），满足：

$$0 \le Q_i(z_j) \le 1, \quad \sum_{j=1}^{n} Q_i(z_j) = 1$$

说明$Q_i(z_j)$是一个合法的概率分布

- 由Jensen不等式可知：$f(E(X)) \ge E(f(X))$

$$LL(\theta, z) = \sum_{i=1}^{m} \ln \sum_{j=1}^{n} Q_i(z_j) \frac{p(\boldsymbol{x}_i, z_j; \theta)}{Q_i(z_j)} \ge \sum_{i=1}^{m} \sum_{j=1}^{n} Q_i(z_j) \ln \frac{p(\boldsymbol{x}_i, z_j; \theta)}{Q_i(z_j)}$$

等号成立当且仅当对于每个$1 \le i \le n$都存在常数c_i，使得：

$$\frac{p(\boldsymbol{x}_i, z_j; \theta)}{Q_i(z_j)} = c_i$$

$$p(x_i, z_j; \theta) = p(z_j | x_i; \theta) \cdot p(x_i; \theta)$$

Jensen不等式：对于一个凸函数，都有函数值的期望大于等于期望的函数值，对于一个凹函数则相反。对数函数是凹函数，则其期望小于等于期望的对数

简单EM方法

- 设第k轮迭代时估计的参数为$\hat{\theta}^{(k)}$，可以计算求$Q_i(z_j)$和$p(\boldsymbol{x}_i, z_j; \hat{\theta}^{(k)})$使得等号成立，有：

$$p(\boldsymbol{x}_i, z_j; \hat{\theta}^{(k)}) = c_i Q_i(z_j)$$

$$p(\boldsymbol{x}_i; \hat{\theta}^{(k)}) = \sum_{j=1}^{n} p(\boldsymbol{x}_i, z_j; \hat{\theta}^{(k)}) = c_i \sum_{j=1}^{n} Q_i(z_j) = c_i$$

- 利用条件概率公式：

$$Q_i(z_j) = \frac{p(\boldsymbol{x}_i, z_j; \hat{\theta}^{(k)})}{c_i} = \frac{p(\boldsymbol{x}_i, z_j; \hat{\theta}^{(k)})}{p(\boldsymbol{x}_i; \hat{\theta}^{(k)})} = p(z_j|\boldsymbol{x}_i; \hat{\theta}^{(k)})$$

- 则似然函数$LL(\theta, z)$有：

$$LL(\theta, z) = \sum_{i=1}^{m} \sum_{j=1}^{n} p(z_j|\boldsymbol{x}_i; \hat{\theta}^{(k)}) \ln \frac{p(\boldsymbol{x}_i, z_j; \theta)}{p(z_j|\boldsymbol{x}_i; \hat{\theta}^{(k)})}$$

- 丢掉常数项后，优化目标等价于：

$$\hat{\theta}, z = \underset{\theta, z}{\operatorname{argmax}} \sum_{i=1}^{m} \sum_{j=1}^{n} p(z_j|\boldsymbol{x}_i; \hat{\theta}^{(k)}) \ln p(\boldsymbol{x}_i, z_j; \theta)$$

- 则构建的Q函数是确定$\hat{\theta}^{(k)}$下的关于参数θ的函数，而且具有数学期望的形式。令其最大化，即可求得最优参数$\hat{\theta}$

$$Q(\theta, \hat{\theta}^{(k)}) = \sum_{i=1}^{m} \sum_{j=1}^{n} p(z_j|\boldsymbol{x}_i; \hat{\theta}^{(k)}) \ln p(\boldsymbol{x}_i, z_j; \theta)$$

观测样本条件下隐参数的分布　　完全数据的对数似然

Q函数设计的一般步骤

- 明确观测数据X和隐变量z
- 定义完全数据的对数似然函数$\ln p(\boldsymbol{x}_i, z_j; \theta)$
- 计算隐变量在当前参数$\hat{\theta}^{(k)}$的后验概率分布$p(z_j|\boldsymbol{x}_i; \hat{\theta}^{(k)})$
- 使用隐变量后验概率分布，对完全数据的对数似然求期望，即得到Q函数

3 EM算法流程

- EM方法通过迭代的方式，估计模型参数。由两个主要步骤组成：E步（Expectation步）和M步（Maximization步）

EM算法

输入$X=(\boldsymbol{x}_1,\boldsymbol{x}_2,\ldots,\boldsymbol{x}_m)$，以初始值$\hat{\theta}^{(0)}$为起点，迭代执行以下步骤：

- 对于第k轮迭代，参数为$\hat{\theta}^{(k)}$：

（1）E步：构造Q函数，描述完全数据对数似然的期望

$$Q(\theta,\hat{\theta}^{(k)})=\sum_{i=1}^{m}\sum_{j=1}^{n}p(z_j|\boldsymbol{x}_i;\hat{\theta}^{(k)})\ln p(\boldsymbol{x}_i,z_j;\theta)$$

（2）M步：极大化Q函数求取最优参数$\hat{\theta}$，并令其为$\hat{\theta}=\hat{\theta}^{(k+1)}$

- 当$\left|\hat{\theta}^{(k+1)}-\hat{\theta}^{(k)}\right|\leq\varepsilon$时终止迭代

简单EM方法

- EM算法一定可以收敛。但是否收敛到最优解以及收敛速度，与初始值有关。
- EM算法对初值的选择很敏感，不同的初值可能导致算法收敛到不同的局部最优解。
- 因此，在实际应用中，可能需要尝试多个不同的初值。

- EM算法的核心是优化含隐变量的对数似然函数 $\underset{\theta}{\operatorname{argmax}} LL(\theta, z) = \underset{\theta,z}{\operatorname{argmax}} \sum_{i=1}^{m} \ln \sum_{j=1}^{n} p(\boldsymbol{x}_i, z_j; \theta)$，但由于隐变量$z_j$存在，目标函数通常是非凸的；
- 非凸函数的特点是：存在多个“山谷”（局部最优），算法容易被初始值“带偏”到附近的局部最优，而非全局最优。

第四讲 线性模型

1 线性回归及其推广

2 对数几率回归分类器LR

3 多分类学习

4 线性判别分析LDA

5 支持向量机SVM

1 线性回归的基本模型

何为线性模型?

- **线性回归（linear regression）**是机器学习中预测数值型数据的最基础的方法。其目的是从样本中学习一个线性模型$g(\boldsymbol{x})$去表征自变量（解释变量）和因变量（响应变量）之间的关系

  - 简单线性回归：仅有一个自变量（样本特征）和一个响应变量（样本标记），二者之间的关系可以用一条直线近似表示 $\hat{y} = g(x) = w_0 + w_1 x + \epsilon$ ，ϵ 误差项
  - 多元线性回归：有两个或两个以上的特征，且特征和标记之间是线性关系。当特征维数为2时，线性模型是一个平面。当特征维数为3时，线性模型是一个立方体。以此类推，在高维空间中是一个超平面

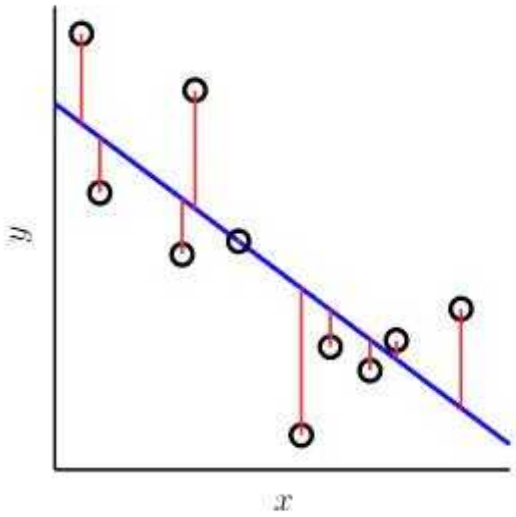
$$g(x) = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + \epsilon$$

  - 多响应变量回归：用于分析多个响应变量与特征之间的关系，可以看作是单响应变量回归的扩展

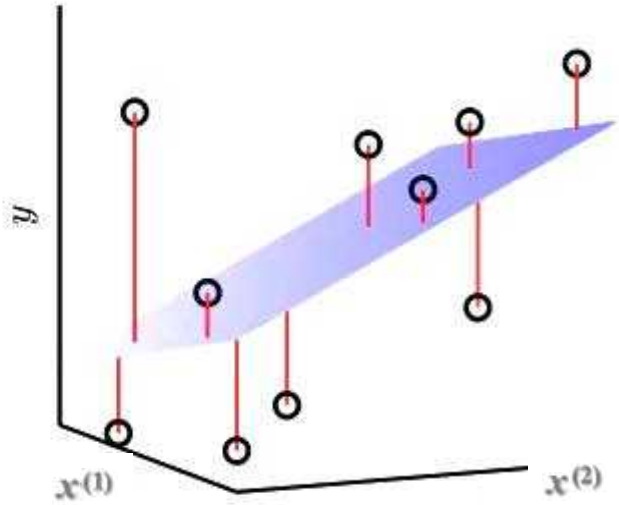
$$\hat{y}_j = g_j(x) = w_{j0} + w_{j1} x_1 + w_{j2} x_2 + \cdots + w_{jd} x_d + \epsilon_j$$

$j = 1,2,\ldots,n$ 表示共有 n 个响应变量

线性回归及其推广



一元线性回归



二元线性回归

$$\hat{y} = g(x) = w_0 + w_1 x + \epsilon$$

$$\hat{y} = g(x) = w_0 + w_1 x_1 + w_2 x_2 + \epsilon$$

线性回归及其推广

- 给定数据集 $D = \{(\boldsymbol{x_1}, y_1), (\boldsymbol{x_2}, y_2), \ldots (\boldsymbol{x_m}, y_m)\}$，样本空间由$d$维向量张成，$\boldsymbol{x_i} = (x_{i1}, x_{i2}, \ldots x_{id})^T$，$y_i \in \mathrm{R}$，则：

- 线性回归方程为：

$$\hat{y}_i = g(\boldsymbol{x_i}) = w_1 x_{i1} + w_2 x_{i2} + \cdots + w_d x_{id} + w_0$$

写为向量形式：

$$\hat{y}_i = g(\boldsymbol{x_i}) = \boldsymbol{x_i^T w} + w_0,$$

其中$\hat{y}_i$是回归预测值，$\boldsymbol{w} = (w_1, \ldots, w_d)^T$和$w_0$是模型的参数



*m*是样本个数，*d*是样本维数。一定要区分开！

2 线性回归的学习准则

- 损失函数：定义在单样本上
- 代价函数：定义在全体样本上

➢ 一般采用MSE作为代价函数（Cost）。当回归预测值与样本标记值具有最小均方误差时获得最优参数

均方误差（Mean-Square Error, MSE）：$MSE = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2$

$$\boldsymbol{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \\ w_0 \end{bmatrix} \quad \boldsymbol{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} & 1 \\ x_{21} & x_{22} & \cdots & x_{2d} & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{md} & 1 \end{bmatrix} \quad \boldsymbol{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}$$

增广1维

特征权重（参数） 样本特征属性值 样本标记值

要注意此处样本矩阵X与第七讲样本矩阵X的区别！

➢ 最小化MSE，求解最优参数

- 最小化代价函数：

$$\boldsymbol{w}^* = \underset{\boldsymbol{w}}{argmin} \sum_{i=1}^{m} (\hat{y}_i - y_i)^2$$

- 写成向量形式：

$$\boldsymbol{w}^* = \underset{\boldsymbol{w}}{argmin} (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w})^T (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w})$$

3 线性回归的优化算法

- 用**常最小二乘法（Ordinary least squares）**求解最优参数：

最小二乘法

令 $L(\boldsymbol{w}) = (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w})^T(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w})$ ← 平方误差代价函数

$$\frac{\partial L(\boldsymbol{w})}{\partial \boldsymbol{w}} = 2\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}) = 0$$ ← 求驻点

$$\boldsymbol{w}^* = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{y}$$

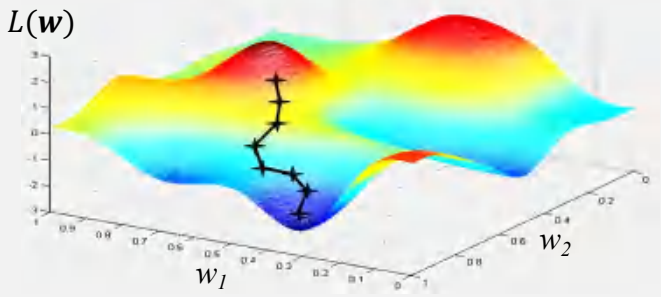
可求得： $\hat{y}_i = g(\boldsymbol{x}_i) = \boldsymbol{x}_i^T(\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{y}$

➢ 若X^TX不可逆，可采用**梯度下降法（Gradient descent）**训练参数：

梯度下降法

- 确定代价函数：$L(\boldsymbol{w}) = \frac{1}{2m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2$ 为了求导方便
- 计算当前参数下代价函数的梯度。梯度是一个向量，其各分量表示代价函数对各个参数w_i的偏导数。梯度的方向代表函数值增加最快的方向，因此**负梯度方向**就是代价函数减少最快的方向
- 确定学习率η（步长），根据步长沿着代价函数负梯度的方向迭代搜索，找到一个函数极值

$$\boldsymbol{w}^{(i+1)} \leftarrow \boldsymbol{w}^{(i)} - \eta \cdot \frac{\partial L(\boldsymbol{w}^{(i)})}{\partial \boldsymbol{w}^{(i)}}$$



$$\boldsymbol{w}^{(i+1)} \leftarrow \boldsymbol{w}^{(i)} - \eta \cdot \frac{\partial L(\boldsymbol{w}^{(i)})}{\partial \boldsymbol{w}^{(i)}}$$

上标(i)表示第i次迭代，每一次迭代都会更新一组参数$\boldsymbol{w}$，从而得到一个新的损失$L(\boldsymbol{w})$

$$\nabla L(\boldsymbol{w}) = \begin{pmatrix} \frac{\partial L}{\partial w_1} \\ \vdots \\ \frac{\partial L}{\partial w_d} \\ \frac{\partial L}{\partial w_0} \end{pmatrix}:$$

梯度：是一个$d+1$维向量(含对偏置参数w_0的偏导)，是函数变化最快的方向（负梯度：比喻为下山最快的方向）

$$\eta:$$

学习率：每次更新参数的幅度。比喻为下山的步长。注意，学习率不一定是常数，是可以根据梯度下降情况而动态变化的

→第九讲

4 线性回归的正则化

- 正则化是通过对代价函数加以限制，**防止模型过拟合的重要方法，**是结构风险最小化思想的一种具体体现，在机器学习中非常有用

$$L(\boldsymbol{w}) = E_{struct}(g) = E_{in}(g) + \lambda \boldsymbol{\Omega}(g)$$

- 结构风险最小化的核心思想是通过在经验风险的基础上加入模型复杂度的惩罚项，找到经验风险和模型复杂度之间的最佳平衡点，从而提高模型的泛化能力。这一思想广泛应用于机器学习中的正则化方法、模型选择和超参数优化等领域

→ 第一讲

4 线性回归的正则化

- 岭回归（Ridge regression）和套索回归（ Lasso regression ）是带有**正则化（regularization）**的线性回归
  - L2 正则化（岭回归）：惩罚参数的平方和，鼓励小参数值
  - L1 正则化（Lasso 回归）：惩罚参数的绝对值和，鼓励稀疏解

- 在线性回归中一般会对参数和（体现为L1范数或L2范数）加以限制，使得最优化时既让代价函数最小，又满足参数限制条件。在多项式回归中，则一般会对高次项的参数加以限制

- 岭回归在线性回归代价函数上添加了L2正则化项：

$$L(\boldsymbol{w}) = \frac{1}{2m}\left[\sum_{i=1}^{m}(\hat{y}_i - y_i)^2 + \lambda \sum_{j=1}^{d} w_j^2\right] \qquad (w_0\text{不惩罚})$$

正则化系数　　$\boldsymbol{w}$的L2范数的平方：$\|\boldsymbol{w}\|_2^2$

- 岭回归代价函数最小二乘优化的闭式解：

$$\boldsymbol{w}^* = (\boldsymbol{X}^T\boldsymbol{X} + \lambda \boldsymbol{I})^{-1}\boldsymbol{X}^T\boldsymbol{y}$$

$I = \begin{bmatrix} 1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & 1 \end{bmatrix}$ 是一个单位矩阵，保证了$(\boldsymbol{X}^T\boldsymbol{X} + \lambda \boldsymbol{I})$可逆，$\boldsymbol{w}$具有闭式解

- 岭回归代价函数的梯度下降迭代：

对第k个特征的参数w_k：

$$w_k^{(i+1)} \leftarrow w_k^{(i)} - \eta \cdot \left(\frac{\partial L(\boldsymbol{w})}{\partial w_k^{(i)}} + \frac{\lambda}{m} w_k^{(i)} \right)$$

- 岭回归正则化系数λ是一个对参数$\boldsymbol{w}$的惩罚系数：

- 从$L(\boldsymbol{w})$的角度看：若λ系数很大且要使得$L(\boldsymbol{w})$最小，必须减小$\boldsymbol{w}$。即惩罚$\boldsymbol{w}$，降低各特征的权重系数。λ过大会使得$\boldsymbol{w} \rightarrow [0,0,..0]$，模型会欠拟合
- 通过引入λ来限制了所有w_k之和，通过惩罚降低不重要的那些特征的权重，从而实现模型的结构风险最小化

- 套索回归在线性回归代价函数上添加了L1正则化项：

$$L(\boldsymbol{w}) = \frac{1}{2m}\left[\sum_{i=1}^{m}(\hat{y}_i - y_i)^2 + \lambda \sum_{j=1}^{d}|w_j|\right] \quad (w_0\text{不惩罚})$$

正则化系数　　$\boldsymbol{w}$的L1范数

- L2正则化只是使模型参数值趋近于0，而无法丢弃任何一个特征，因此无法做到稀疏化。L1正则化的作用是使得大部分特征权重等于0，去掉了那些不重要的模型参数，节省参数存储空间，优化模型结构

5 线性回归的性能度量

→ 第二讲

- 用以下指标度量n个预测值$\hat{y}$与测试样本标记值y的差异

平均绝对误差
Mean Absolute Error, MAE

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|\hat{y}_i - y_i|$$

均方误差
Mean-Square Error, MSE

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

均方根误差
Root MSE

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2}$$

提问1： 用于代价（损失）函数的MAE、 MSE与用于回归性能度量的MAE、 MSE有什么区别联系？

线性回归及其推广

残差平方和 *SSE*
Sum of Squares due to Error

$$SSE = \sum_{i=1}^{n} (\hat{y}_i - y_i)^2$$

回归平方和 *SSR*
Sum of Squares of Regression

$$SSR = \sum_{i=1}^{n} (\hat{y}_i - \bar{y})^2$$

总离平方和 *SST*
Total Sum of Squares

$$SST = \sum_{i=1}^{n} (y_i - \bar{y})^2$$

R-平方系数
（决定系数）
R-square

$$R^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}$$

SST是方差，可解释
SSE是残差，不可解释

分子是残差平方和，分母是总方差，R^2 是（总方差 - 残差平方和）/ 总方差 ”，即能够描述的“变化程度”占真实数据“变化程度”的比例

- 数据的总方差*SST*=*SSR*+*SSE*。R^2系数越接近1表明拟合优度越高

第四讲 线性模型

武汉大学 WUHAN UNIVERSITY

导航方向 2025版

1 线性回归及其推广

2 对数几率回归分类器LR

3 多分类学习

4 线性判别分析LDA

5 支持向量机SVM

对数几率回归分类器

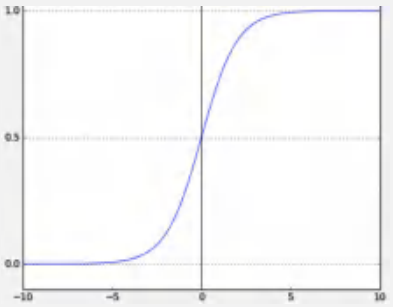
- 对数几率回归（Logistic Regression, LR）又称为“逻辑回归(音译)”，是一种**基于广义线性回归的数据分类方法**。对数几率回归虽然被称为“回归”，但实际上是一种经典的**线性二分类模型**

1 对数几率回归的模型

- 给定数据集 $D = \{(\boldsymbol{x_1}, y_1), (\boldsymbol{x_2}, y_2), \ldots (\boldsymbol{x_m}, y_m)\}$ ，样本空间由d维向量张成，$\boldsymbol{x_i} = (x_{i1}, x_{i2}, \ldots x_{id})^T$，$y_i \in \{c_1 = 1, \ c_0 = 0\}$
- 找一个单调可微的联系函数，将样本$\boldsymbol{x_i}$的线性回归预测值z_i与样本的分类后验概率$\hat{y}_i = P(c_1|\boldsymbol{x_i})$联系起来，构成一个线性分类器：即对样本$\boldsymbol{x_i}$与其分类后验概率$\hat{y}_i = P(c_1|\boldsymbol{x_i})$做广义线性回归，选择Logistic函数作为联系函数

- Logistic函数是一种形为S（Sigmoid）的函数，由皮埃尔·弗朗索瓦·韦吕勒（Pierre-François Verhulst）在研究人口增长时提出：

$$f(x) = \frac{1}{1 + e^{-x}}$$



对数几率回归分类器

- LR模型的设计思想：

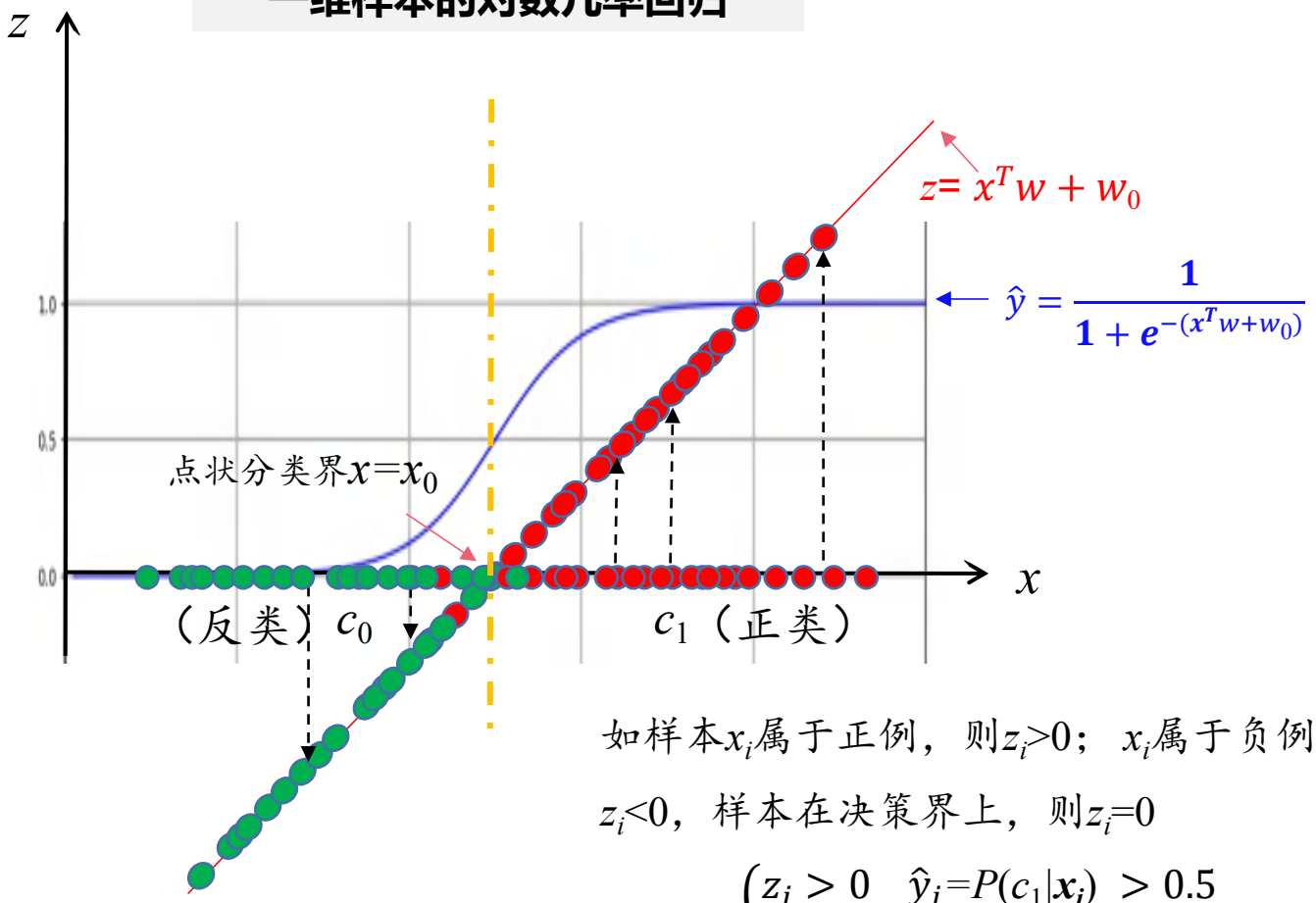
$$\boldsymbol{x}_i=\begin{bmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{id} \end{bmatrix} \xrightarrow{\text{①线性回归}} z_i = \boldsymbol{x}_i^T\boldsymbol{w} + w_0 \xrightarrow{\text{②Logistic映射}} \hat{y}_i = \frac{1}{1+e^{-z_i}} = \frac{1}{1+e^{-(\boldsymbol{x}_i^T\boldsymbol{w}+w_0)}} = P(c_1|X=\boldsymbol{x}_i)$$

满足：
$$\begin{cases} z_i > 0 & \hat{y}_i = P(c_1|\boldsymbol{x}_i) > 0.5 \\ z_i = 0 & \hat{y}_i = P(c_1|\boldsymbol{x}_i) = 0.5 \\ z_i < 0 & \hat{y}_i = P(c_1|\boldsymbol{x}_i) < 0.5 \end{cases}$$

如样本x_i属于正例，则$z_i>0$；x_i属于负例则$z_i<0$，样本在决策界上，则$z_i=0$

对数几率回归分类器

一维样本的对数几率回归

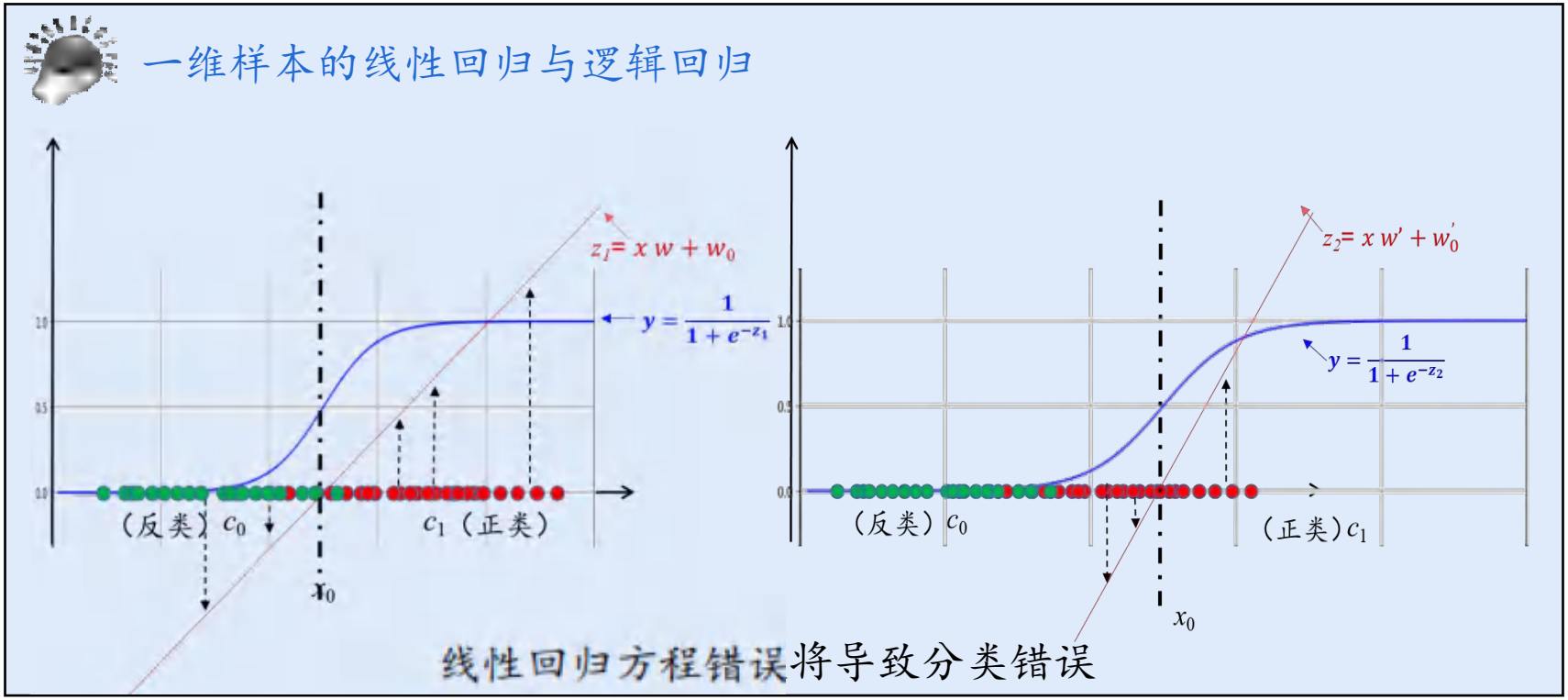


如样本x_i属于正例，则$z_i>0$；x_i属于负例则$z_i<0$，样本在决策界上，则$z_i=0$

满足：
$$\begin{cases} z_i > 0 & \hat{y}_i = P(c_1|\boldsymbol{x}_i) > 0.5 \\ z_i = 0 & \hat{y}_i = P(c_1|\boldsymbol{x}_i) = 0.5 \\ z_i < 0 & \hat{y}_i = P(c_1|\boldsymbol{x}_i) < 0.5 \end{cases}$$

对数几率回归分类器

- 在对数几率回归模型中，$z = \boldsymbol{x}^{\mathrm{T}}\boldsymbol{w} + w_0$ 是线性回归方程。线性回归的好坏，直接决定了分类模型的正确率



对数几率回归分类器

- 在贝叶斯分类器一讲中已经学习到，$P(c_1|\boldsymbol{x})=P(c_0|\boldsymbol{x})$是二分类的分类决策界，则：

- 决策界满足：

$$\ln\frac{P(c_1|\boldsymbol{x})}{P(c_0|\boldsymbol{x})}=\ln\frac{\hat{y}}{1-\hat{y}}=0$$

则：$\ln\frac{\hat{y}}{1-\hat{y}}=\ln\frac{\frac{1}{1+e^{-z}}}{\frac{e^{-z}}{1+e^{-z}}}=\ln e^{z}=z=0$　　即 $z=\boldsymbol{x}^{\mathrm{T}}\boldsymbol{w}+w_0=0$ 是决策界方程

- 同时$z=\ln\frac{\hat{y}}{1-\hat{y}}$ 是 $\boldsymbol{x}$ 作为正例与反例的概率的比值的对数，这个比值又被称为“几率”，这也就是模型被称为“对数几率回归”的由来

2 对数几率回归的学习准则

➢ LR模型的学习准则可以从最大似然估计导出，用于衡量模型预测值与真实值之间的差异

- 考虑让样本$D=\{(\boldsymbol{x_1},y_1),(\boldsymbol{x_2},y_2),\ldots(\boldsymbol{x_m},y_m)\}$ $y_i\in\{0,1\}$中每个样本$(\boldsymbol{x_i},y_i)$属于其真实标记的概率越大越好，构造似然函数：

$$l(\boldsymbol{w})=\prod_{i=1}^{m}[P(\boldsymbol{x}_i)]^{y_i}[1-P(\boldsymbol{x}_i)]^{1-y_i}$$ ← 伯努利型的似然估计

其中$P(\boldsymbol{x})=P(c_1|\boldsymbol{x})=\frac{1}{1+e^{-(x^T\boldsymbol{w}+w_0)}}=\frac{e^{(x^T\boldsymbol{w}+w_0)}}{1+e^{(x^T\boldsymbol{w}+w_0)}}$

- 则其对数似然函数有：

$$ll(\boldsymbol{w})=\ln l(\boldsymbol{w})=\sum_{i=1}^{m}y_i\cdot\ln P(\boldsymbol{x_i})+(1-y_i)\cdot\ln(1-P(\boldsymbol{x}_i))$$

对数几率回归分类器

- LR模型的优化目标函数可以写为：

$$\boldsymbol{w}^* = \underset{\boldsymbol{w}}{\operatorname{argmax}}\, ll(\boldsymbol{w}) = \underset{\boldsymbol{w}}{\operatorname{argmax}} \left[\sum_{i=1}^{m} y_i \cdot \ln P(\boldsymbol{x_i}) + (1 - y_i) \cdot \ln(1 - P(\boldsymbol{x_i})) \right]$$

- 对上式取负号，可以得到等价的优化目标函数：

$$L(\boldsymbol{w}) = - ll(\boldsymbol{w}) = \sum_{i=1}^{m} -y_i \cdot \ln P(\boldsymbol{x_i}) - (1 - y_i) \cdot \ln(1 - P(\boldsymbol{x_i}))$$

$$\boldsymbol{w}^* = \underset{\boldsymbol{w}}{\operatorname{argmin}}\, L(\boldsymbol{w}) = \underset{\boldsymbol{w}}{\operatorname{argmin}} \left[\sum_{i=1}^{m} -yi \cdot \ln P(\boldsymbol{x_i}) - (1 - y_i) \cdot \ln(1 - P(\boldsymbol{x_i})) \right]$$

- 其中$L(\boldsymbol{w})$函数即LR模型的代价（损失）函数，又叫“**二分类交叉熵（corss-entropy）**”损失函数

3 交叉熵损失

- 用交叉熵（互熵）定义损失函数经常用于分类问题中，在对数几率回归以及分类在神经网络中会大量使用到

→第1讲

Baidu百科 百度百科——全球最大中文百科全书

- 克劳德·艾尔伍德·香农（Claude Elwood Shannon，1916—2001），出生于美国密歇根州佩托斯基，美国数学家、发明家、密码学家，信息论创始人，美国国家工程院院士、美国国家科学院院士、美国艺术与科学院院士，麻省理工学院名誉教授。香农提出了信息熵的概念，为信息论和数字通信奠定了基础



克劳德·艾尔伍德·香农

Claude Elwood Shannon

(1916.4—2001.2)

对数几率回归分类器

➢ 对于样本$\boldsymbol{x_i}$，其标记值为y_i，预测值为$\hat{y}_i$，其二分类交叉熵：

- 其二分类交叉熵损失：

$$-y_i \cdot \ln P(\boldsymbol{x_i}) - (1-y_i) \cdot \ln(1-P(\boldsymbol{x}_i))$$

$$= -y_i \cdot \ln \hat{y}_i - (1-y_i) \cdot \ln(1-\hat{y}_i)$$

二分类，写为2项和

- 对于m个样本，其交叉熵损失的代价函数$L(\boldsymbol{w})$是所有样本损失的平均值（或总和）：

$$L(\boldsymbol{w}) = -\sum_{i=1}^{m} [y_i \cdot \ln P(\boldsymbol{x}_i) + (1-y_i) \cdot \ln(1-P(\boldsymbol{x}_i))]$$

$$= -\sum_{i=1}^{m} \left[y_i \cdot \ln \frac{P(\boldsymbol{x}_i)}{1-P(\boldsymbol{x}_i)} + \ln(1-P(\boldsymbol{x}_i)) \right] \quad \leftarrow P(\boldsymbol{x}) = \frac{1}{1+e^{-(\boldsymbol{x}^T\boldsymbol{w}+w_0)}}$$

$$= -\sum_{i=1}^{m} [y_i \cdot (\boldsymbol{x}^T\boldsymbol{w} + w_0) - \ln(1+e^{\boldsymbol{x}^T\boldsymbol{w}+w_0})]$$

对数几率回归分类器

- 交叉熵损失的正则化与线性回归中MSE正则化类似

$$代价函数L(\boldsymbol{w})= -\sum_{i=1}^{m}\left[y_i \cdot (\boldsymbol{x}^T\boldsymbol{w} + w_0) - \ln(1 + e^{\boldsymbol{x}^T\boldsymbol{w}+w_0}) + \frac{\lambda}{2}\sum_{j=1}^{d} w_j^2\right]$$

$\boldsymbol{w}$的L2范数的平方

λ为正则化系数

其二分类交叉熵损失：$-y_i \cdot \ln P(\boldsymbol{x_i}) - (1 - y_i) \cdot \ln(1 - P(\boldsymbol{x_i}))$

交叉熵损失衡量了模型预测的概率分布与真实分布之间的差异。

交叉熵函数的目标是最小化预测分布与真实分布之间的距离，从而提高模型的预测准确性。

对数几率回归分类器



e.g. 假设一组训练样本的标记值为{1，0，1，0，1}，分类器A的预测结果为{0.9，0.01，0.95，0.03，0.99}，分类器B的预测结果为{0.85，0.45，0.75，0.55，0.7}，两个分类器分别计算交叉熵：

A: $-[(1\ln 0.9+0\ln 0.1)+(0\ln 0.01+1\ln 0.99)+(1\ln 0.95+0\ln 0.05)+(0\ln 0.03+1\ln 0.97)+(1\ln 0.99+0\ln 0.01)]=0.16$

B: $-[(1\ln 0.85+0\ln 0.15)+(0\ln 0.45+1\ln 0.55)+(1\ln 0.75+0\ln 0.25)+(0\ln 0.55+1\ln 0.45)+(1\ln 0.7+0\ln 0.3)]=1.83$

上述两种情况对比，A的**损失**明显**低于**B的损失，说明分类器A更优

注：按照信息熵的定义，log应该取2为底。但机器学习中一般需要对交叉熵损失求导，故log用ln代替

第四讲 线性模型

1 线性回归及其推广

2 对数几率回归分类器LR

3 多分类学习

4 线性判别分析LDA

5 支持向量机SVM

1 用多次二分类实现多分类

- 现实中我们常遇到多分类学习任务。通常采用将一个多分类任务拆分为若干个二分类任务，将二分类学习推广到多分类学习。在测试时，对这些分类器的预测结果进行集成以获得最终结果

经典拆分策略

- 一对一，One vs.One，OvO
- 一对其余，One vs.Rest，OvR
- 多对多，Many vs. Many，MvM *

1 用多次二分类实现多分类

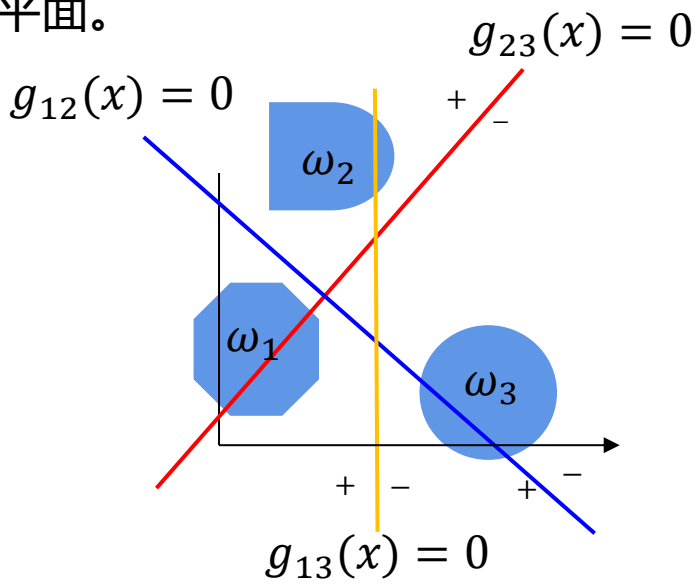
第一种情况：一对一，One vs.One，OvO

任意两类间可分别用判别平面分开（成对可分）。

这样 有 $M(M-1)/2$个判别平面。

对于两类问题，$M=2$，则有一个判别平面。

同理，三类问题则有三个判别平面。



判别函数： $g_{ij}(x)=W_{ij}{}^{T}X$

判别边界： $g_{ij}(x)=0$

性质：

$$g_{ij}(x)=w_{ij}^{T}x\begin{cases}>0, \text{若} x\in\omega_i\\ <0, \text{若} x\in\omega_j\end{cases}$$

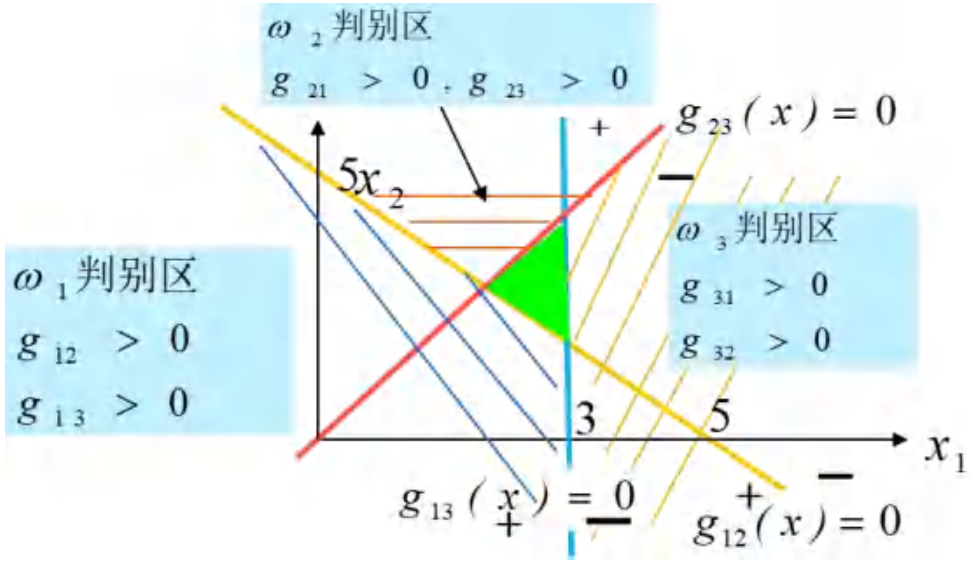
1 用多次二分类实现多分类

假设判别函数为：

$$\begin{cases} g_{12}(x) = -x_1 - x_2 + 5 \\ g_{13}(x) = -x_1 + 3 \\ g_{23}(x) = -x_1 + x_2 \end{cases}$$

判别边界为：

$$\begin{cases} g_{12}(x) = -x_1 - x_2 + 5 = 0 \\ g_{13}(x) = -x_1 + 3 = 0 \\ g_{23}(x) = -x_1 + x_2 = 0 \end{cases}$$



$$g_{12}(x) = -2, g_{13}(x) = -1, g_{23}(x) = -1$$

$$g_{21}(x) = 2, g_{31}(x) = 1, g_{32}(x) = 1$$

$$g_{3j}(x) > 0$$ 属于ω_3类

未知模式 $X=(x_1, x_2)^T=(4,3)^T$属于那一类？

1 用多次二分类实现多分类

第二种情况：一对其余，One vs.Rest，OvR

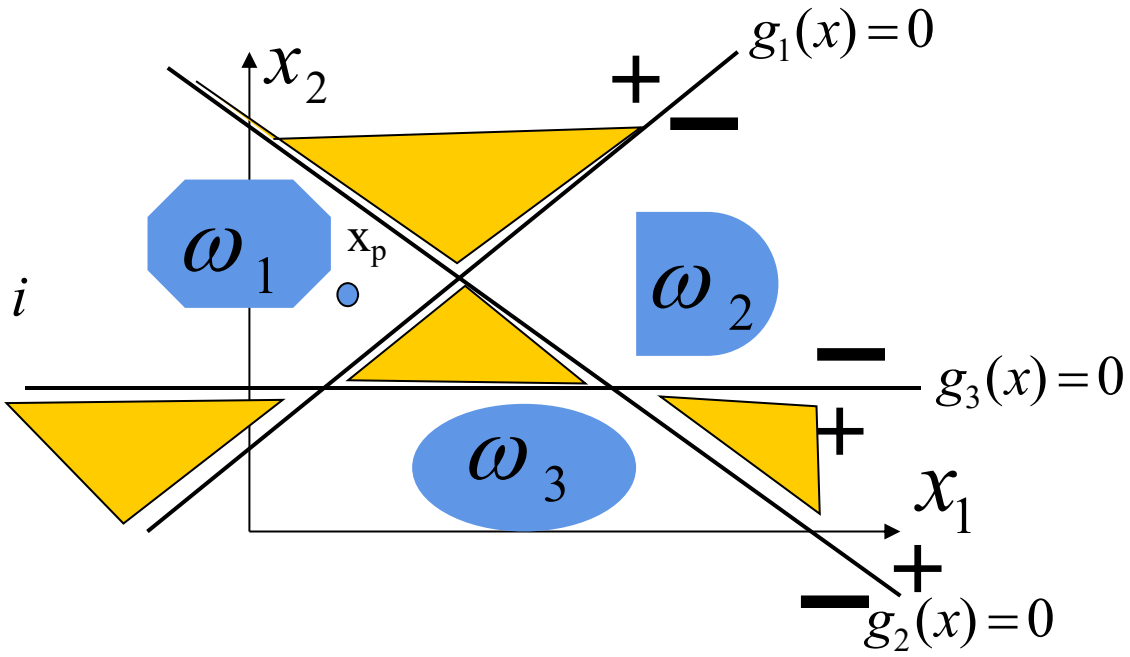
如图所示，每一类别可用单个判别边界与其它类别相分开，如果一模式X_p属于ω_1，则由图可清楚看出：这时$g_1(x_p)>0$、而$g_2(x_p)<0$、$g_3(x_p)<0$，ω_1 类与其它类之间的边界由 $g_1(x)=0$ 确定。

判别规则：

如果

$$\begin{cases} g_i(x) > 0 \\ g_j(x) \le 0, \forall j \ne i \end{cases}$$

则 $x \in \omega_i$



1 用多次二分类实现多分类

第二种情况：一对其余，One vs.Rest，OvR



e.g 已知三类c_1, c_2, c_3的决策界方程分别为：

$$\begin{cases} g_1(\boldsymbol{x_i}) = -x_{i1} + x_{i2} = 0 \\ g_2(\boldsymbol{x_i}) = x_{i1} + x_{i2} - 5 = 0 \\ g_3(\boldsymbol{x_i}) = -x_{i2} + 1 = 0 \end{cases}$$

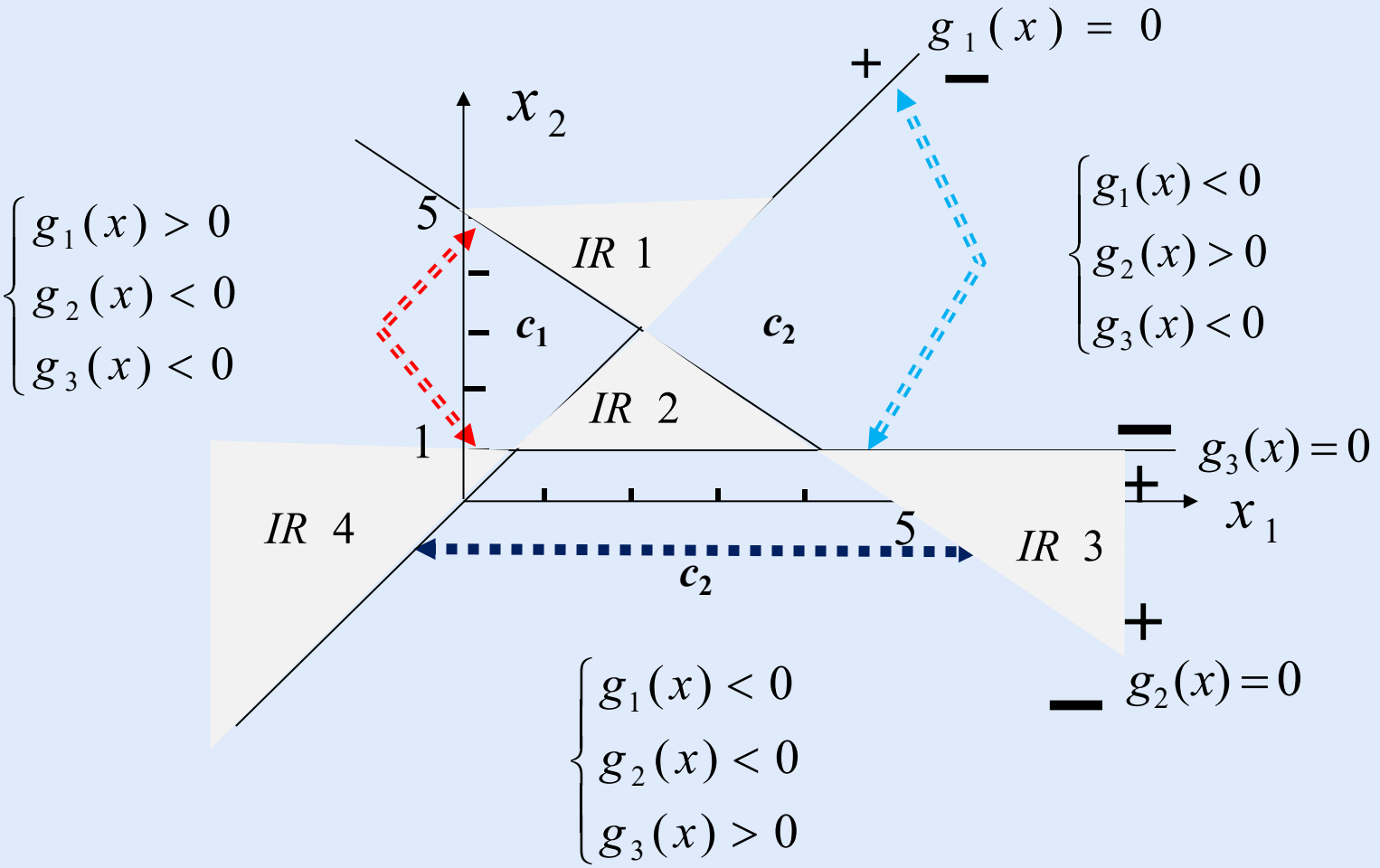
问： 当$\boldsymbol{x}=(6,5)^T$时属于那一类

解：将x分别代入三个决策界方程得

$$g_1(\boldsymbol{x}) = -1, g_2(\boldsymbol{x}) = 6, g_3(\boldsymbol{x}) = -4$$

由$g_1(\boldsymbol{x}) < 0$， $g_2(\boldsymbol{x}) > 0$， $g_3(\boldsymbol{x}) < 0$所以它属于c_2类

作图分析如下：


$g_1(x)=0$
x_2
5
1
c_1
c_2
IR 1
IR 2
IR 3
IR 4
5
c_2
$g_3(x)=0$
x_1
$g_2(x)=0$
$\begin{cases} g_1(x)>0 \\ g_2(x)<0 \\ g_3(x)<0 \end{cases}$
$\begin{cases} g_1(x)<0 \\ g_2(x)>0 \\ g_3(x)<0 \end{cases}$
$\begin{cases} g_1(x)<0 \\ g_2(x)<0 \\ g_3(x)>0 \end{cases}$

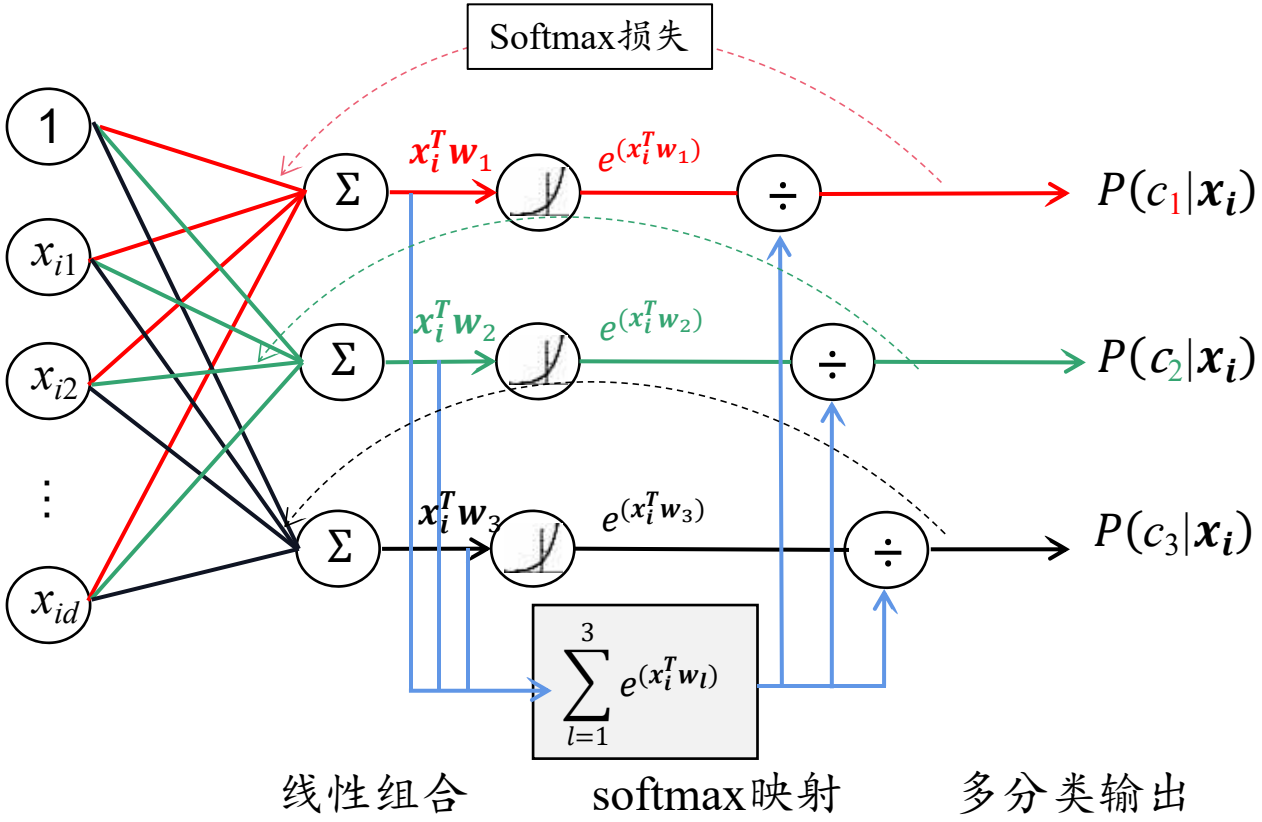
2 引入Softmax函数

- 对数几率回归使用的Logistic函数是一个典型的Sigmoid函数（S型函数），能够将样本的线性回归预测值映射到(0,1)之间，代表其属于正类的概率，因此适用于二分类问题
- 对于k分类系统，引入名为“[illegible]大（Softmax）[illegible]”：考虑样本 $D=\{(\boldsymbol{x_1},\boldsymbol{y_1}),(\boldsymbol{x_2},\boldsymbol{y_2}),\ldots(\boldsymbol{x_m},\boldsymbol{y_m})$，标记 $\boldsymbol{y_i}$ 是形如 $(0,0,0,1,0,0\ldots,0)^T$ 的$k\times 1$向量。输出$\hat{\boldsymbol{y}}_i$为$(\hat{c}_1,\hat{c}_2,\hat{c}_3,\ldots,\hat{c}_k)^T$ 的$k\times 1$向量，其中$\hat{c}_j$为样本$\boldsymbol{x_i}$属于c_j类的概率，满足：

$$\hat{c}_j=P(c_j|\boldsymbol{x_i})=\frac{e^{(x_i^T w_j)}}{\sum_{l=1}^{k}e^{(x_i^T w_l)}}$$

其中w_l是第l类（第l个广义线性回归）使用的参数向量$[w_{l1},w_{l1},\ldots,w_{ld},\ w_{l0}]$

增广偏置参数

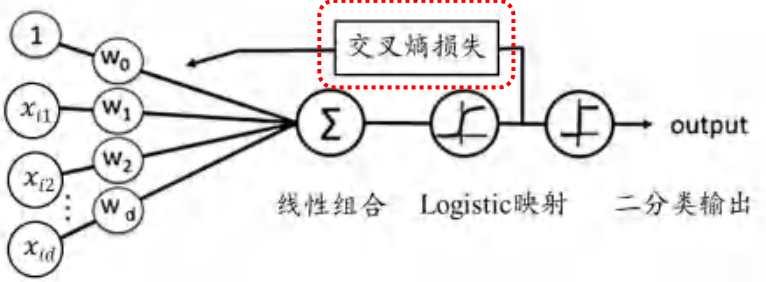


Softmax函数的计算流程

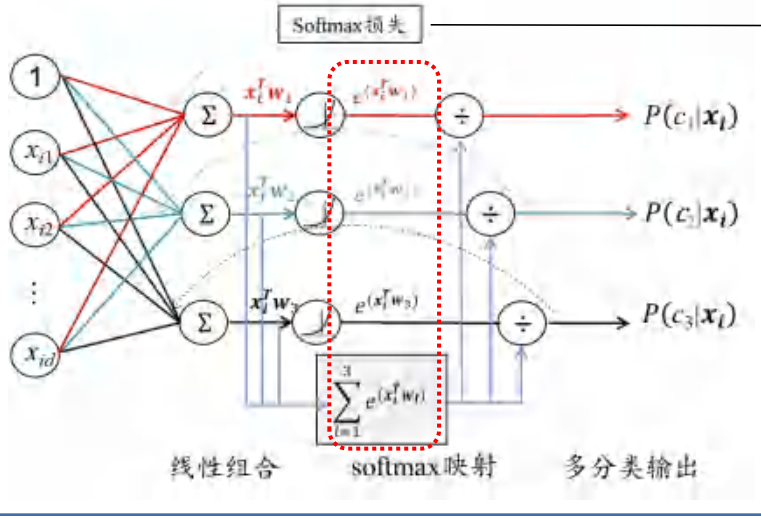
- Sigmoid函数与Softmax函数对比

Logistic (Sigmoid)

$$-y_i \cdot \ln P(\boldsymbol{x}_i) - (1-y_i) \cdot \ln(1-P(\boldsymbol{x}_i))$$



Softmax



$$-\sum_{j=1}^{k} y_{ij} \cdot \ln\left(\frac{e^{(\boldsymbol{x}_i^T \boldsymbol{w}_j)}}{\sum_{l=1}^{k} e^{(\boldsymbol{x}_i^T \boldsymbol{w}_l)}}\right)$$

- 关于Softmax多分类的损失函数（多分类交叉熵损失）以及损失函数可导性分析在神经网络部分进一步讲解

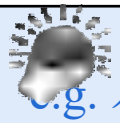
- 二分类样本标记$y_i \in \{1, 0\}$，二分类交叉熵：

$$-y_i \cdot \ln\hat{y}_i - (1 - y_i) \cdot \ln(1 - \hat{y}_i)$$

- k分类样本标记$\boldsymbol{y_i}$是形如$(0,0,0,1,0,0\ldots,0)^T$的$k\times 1$向量。输出为$(\hat{c}_1, \hat{c}_2, \hat{c}_3,\ldots, \hat{c}_k)^T$的$k\times 1$向量，$k$分类交叉熵：

$$-\sum_{j=1}^{k} y_{ij} \cdot \ln\hat{c}_j$$

→第九讲



e.g. 在手写数字识别任务中，如果样本是数字“5”，那么真实分布应该为：[0, 0, 0, 0, 0, 1, 0, 0, 0, 0]，如果输出的分布为：[0.1, 0.1, 0, 0, 0, 0.7, 0, 0.1, 0, 0]，则k=10，该样本的**10分类交叉熵**损失为：

10分类，写为10项和

$$Loss = -0 \times \log 0.1 - 0 \times \log 0.1 - 0 \times \log 0 - 0 \times \log 0 - 0 \times \log 0 - 1 \times \log 0.7 - 0 \times \log 0 - 0 \times \log 0.1 - 0 \times \log 0 - 0 \times \log 0 \approx 0.3567$$

0log0=0

如果输出的分布为：[0.2, 0.3, 0.1, 0, 0, 0.3, 0.1, 0, 0, 0]，那么损失函数得：

$$Loss = -0 \times \log 0.2 - 0 \times \log 0.3 - 0 \times \log 0.1 - 0 \times \log 0 - 0 \times \log 0 - 1 \times \log 0.3 - 0 \times \log 0.1 - 0 \times \log 0 - 0 \times \log 0 - 0 \times \log 0 \approx 1.2040$$

上述两种情况对比，第一个分布的损失明显低于第二个分布的损失，说明第一个分布更接近于真实分布

注：同样使用ln计算

线性判别分析

Baidu百科 百度百科——全球最大中文百科全书

- 罗纳德·费希尔（1890～1962），英国统计与遗传学家，现代统计科学的奠基人之一，并对达尔文进化论作了基础澄清的工作。线性判别分析（Linear Discriminant Analysis, LDA）是一种经典的线性模型，在两分类问题上最早由费希尔在1936年提出，也被称为Fisher判别分析



罗纳德·费希尔
Ronald Fisher
(1890—1962)

- 线性判别分析（Linear Discriminant Analysis, LDA）模型是一种线性分类器模型。同时该模型也是一种有监督的降维模型

注意区分：在机器学习领域还有另外一个LDA，被称为隐迪利克雷分配（Latent Dirichlet Allocation），主要用于文档主题分析，现在也被广泛用于时空数据分析。它和线性判别分析都是机器学习的经典模型，应用领域完全不同

1 线性判别分析的模型

- LDA的思想：对于二分类问题，在给定的训练样集上，设法将样例投影到一条“直线”上：使得同类样例的投影点尽可能的近，异类样例的投影点尽可能的远离

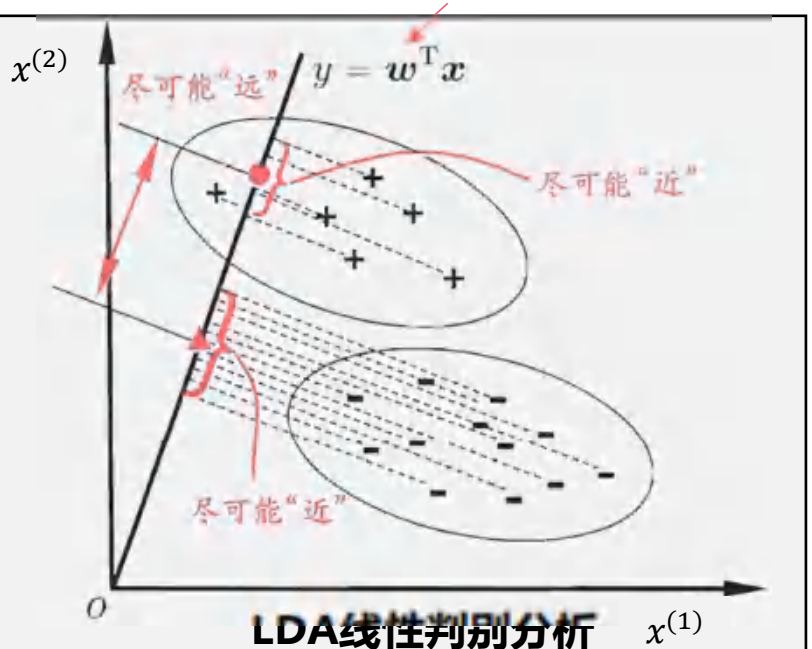
注意：这不是直线方程！

- **作为特征提取模型：**

  将d维度数据投影到k-1维上，k为样本的类别数

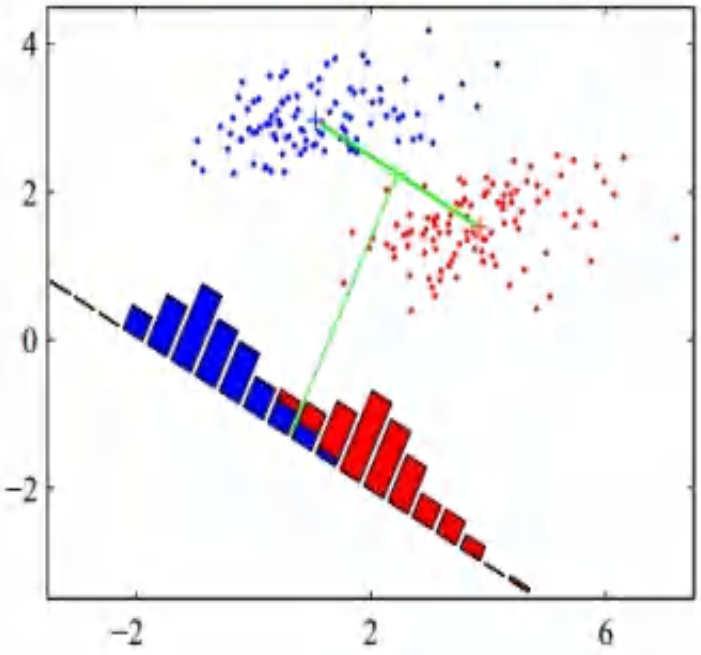
- **作为分类器：**

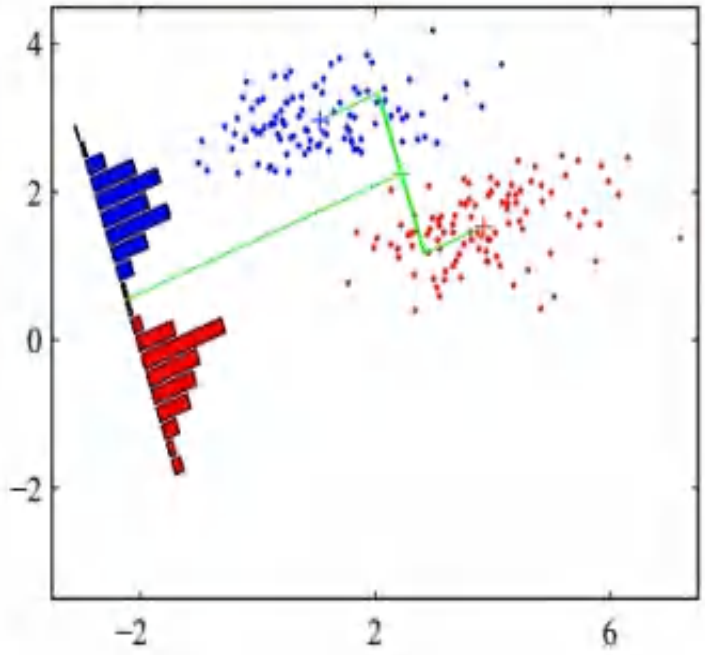
  根据样本训练好这条“直线”，在预测阶段将新样本投影到该直线上，根据投影点的位置确定新样本的类别



线性判别分析

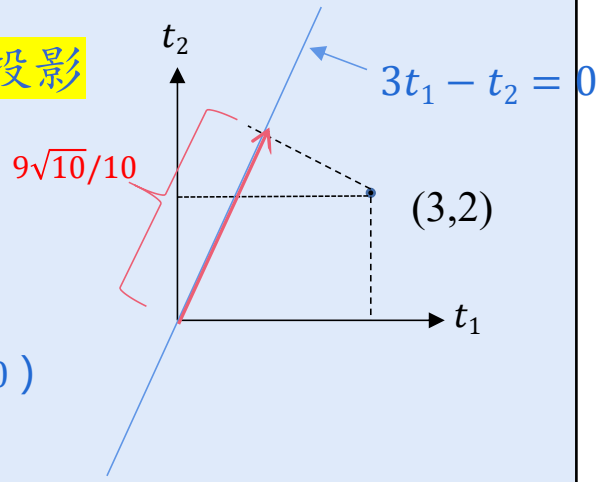
提问：以下哪种投影直线更好用于分类？





线性判别分析

e.g. 计算向量（3, 2）在直线 $3t_1 - t_2 = 0$ 下的投影



解：由直线方程可知直线的方向向量$w=(1,3)^T$，化为单位向量$w^T=(\sqrt{10}/10, 3\sqrt{10}/10)$，则其在直线下的投影为

$$\begin{pmatrix}\sqrt{10}/10\\ 3\sqrt{10}/10\end{pmatrix} \cdot \begin{pmatrix}3\\2\end{pmatrix} = (\sqrt{10}/10, 3\sqrt{10}/10)\begin{pmatrix}3\\2\end{pmatrix} = (9\sqrt{10}/10)$$

点乘

*注意：$9\sqrt{10}/10$是t_1—t_2坐标系下向量(3,2)在新空间w下的投影坐标。因新空间（直线）只有一个基向量，所以投影点为一个数（2维投影到1维）。这个数字其实也是t_1—t_2坐标系下**投影向量**（9/10，27/10）的长度

- 平面直线$At_1+Bt_2+C=0$的法向量为（A,B），方向向量为（-B,A）
- 在同一坐标系下，向量$\vec{u}$在**单位向量**$\vec{v}$的投影向量为$(\vec{u}\cdot\vec{v})\times\vec{v}$

线性判别分析

➢ 考虑二分类，样本$D=\{(\boldsymbol{x_1},y_1),(\boldsymbol{x_2},y_2),...(\boldsymbol{x_m},y_m)\}$，$y_i\in\{0,1\}$，每个样本含有$d$维特征，$X_i=(x_{i1},x_{i2},...x_{id})^T$

- 第 i 类样本的集合：X_i
- 第 i 类样本的均值向量：$\boldsymbol{\mu_i}$
- 第 i 类样本的散度矩阵：$\boldsymbol{\Sigma_i}$
- 两类样本的中心向量在直线$\boldsymbol{w}$上的投影：$\boldsymbol{w^T\mu_0}$ 和 $\boldsymbol{w^T\mu_1}$（直线w下的1维坐标）
- 两类样本在直线$\boldsymbol{w}$上投影点的方差：$\boldsymbol{w^T\Sigma_0 w}$ 和 $\boldsymbol{w^T\Sigma_1 w}$

散度矩阵和协方差矩阵很像，少除一个样本数量$N_i=m$

w为直线的方向向量

$$\Sigma_i=\sum_{x\in C_i}(x-\mu_i)(x-\mu_i)^T \qquad \mathrm{Cov}_i=\frac{1}{N_i}\sum_{x\in C_i}(x-\mu_i)(x-\mu_i)^T$$



再次强调！m是样本个数，d是样本维数

2 线性判别分析的学习准则

➢ 考虑优化目标：

- 同类样例的投影点尽可能的近：两个投影的协方差和尽可能小

$$\boldsymbol{w}^T\boldsymbol{\Sigma}_0\boldsymbol{w} + \boldsymbol{w}^T\boldsymbol{\Sigma}_1\boldsymbol{w}$$

- 异类样例投影点尽可能的远离：各均值向量投影点距离尽可能大

$$\|\boldsymbol{w}^T\boldsymbol{\mu}_0 - \boldsymbol{w}^T\boldsymbol{\mu}_1\|_2^2$$

距离度量选用2范数平方

➢ 构造目标函数：

$$J = \frac{\|\boldsymbol{w}^T\boldsymbol{\mu}_0 - \boldsymbol{w}^T\boldsymbol{\mu}_1\|_2^2}{\boldsymbol{w}^T\boldsymbol{\Sigma}_0\boldsymbol{w} + \boldsymbol{w}^T\boldsymbol{\Sigma}_1\boldsymbol{w}} = \frac{\boldsymbol{w}^T(\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)^T\boldsymbol{w}}{\boldsymbol{w}^T(\boldsymbol{\Sigma}_0 + \boldsymbol{\Sigma}_1)\boldsymbol{w}}$$

- 定义类内散度（within-class scatter）矩阵 S_w：

$$S_w = \boldsymbol{\Sigma}_0 + \boldsymbol{\Sigma}_1 = \sum_{x \in X_0} (\boldsymbol{x} - \boldsymbol{\mu}_0)(\boldsymbol{x} - \boldsymbol{\mu}_0)^T + \sum_{x \in X_1} (\boldsymbol{x} - \boldsymbol{\mu}_1)(\boldsymbol{x} - \boldsymbol{\mu}_1)^T$$

参考协方差定义

- 定义类间散度（between-class scatter）矩阵 S_b：

$$S_b = (\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)^T$$

- 目标函数可以重写为一种称作广义瑞利商（generalized Rayleigh quotient）的形式：

$$J = \frac{\boldsymbol{w}^T(\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)^T\boldsymbol{w}}{\boldsymbol{w}^T(\Sigma_0 + \Sigma_1)\boldsymbol{w}} = \frac{\boldsymbol{w}^T S_b \boldsymbol{w}}{\boldsymbol{w}^T S_w \boldsymbol{w}}$$

- LDA 的目标是找到一个投影方向 $\boldsymbol{w}^*$，使得类间散布最大化，同时类内散布最小化。优化目标可以表示为：

$$\boldsymbol{w}^* = \underset{\boldsymbol{w}}{argmax}\left(\frac{\boldsymbol{w}^T S_b \boldsymbol{w}}{\boldsymbol{w}^T S_w \boldsymbol{w}}\right.$$

3 线性判别分析的优化算法

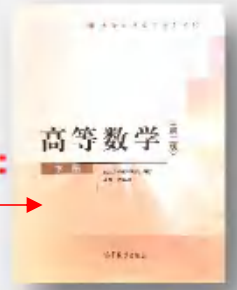
- 广义瑞利商分子分母都是关于$\boldsymbol{w}$的二次项，因此最优参数$\boldsymbol{w}^*$的长度不重要，只要确定其方向，可令$\boldsymbol{w}^T S_{\mathrm{w}} \boldsymbol{w}=1$，优化目标等价为：

$$\boldsymbol{w}^* = \underset{w}{argmin}(-\boldsymbol{w}^T S_{\mathrm{b}} \boldsymbol{w})$$

$$s.t. \quad \boldsymbol{w}^T S_{\mathrm{w}} \boldsymbol{w} = 1$$

求解有约束的极值：

Lagrange乘数法 →



- 构造Lagrange函数：

$$L(\boldsymbol{w}, \lambda) = \boldsymbol{w}^T S_{\mathrm{b}} \boldsymbol{w} - \lambda(\boldsymbol{w}^T S_{\mathrm{w}} \boldsymbol{w} - 1)$$

$$\frac{\partial L}{\partial w} = S_{\mathrm{b}} \boldsymbol{w} - \lambda S_{\mathrm{w}} \boldsymbol{w} = 0$$

极值解$\boldsymbol{w}^*$ 应满足 $\Rightarrow$ $S_{\mathrm{w}}^{-1} S_{\mathrm{b}} \boldsymbol{w}^* = \lambda \boldsymbol{w}^*$

$S_w^{-1} S_b$矩阵的秩？

求目标函数极值问题变成了求矩阵特征值问题，最优投影方向就是这个 $S_w^{-1} S_b$ 矩阵的特征向量

线性判别分析

- 可知$\boldsymbol{w}^*$是$S_w^{-1}S_b$的特征向量

$$\lambda \boldsymbol{w}^* = S_w^{-1}(\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)^T \boldsymbol{w}^*$$

因为$(\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)^T \boldsymbol{w}^*$是标量，不影响$\boldsymbol{w}^*$的方向，可以省去特征值分解的过程

即$\boldsymbol{w}^*$与$S_w^{-1}(\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)$共线：

$$\boldsymbol{w}^* = S_w^{-1}(\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)$$

- 最优投影直线的方向向量：$w^* = S_w^{-1}(\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1) = (\Sigma_0 + \Sigma_1)^{-1}(\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1$

  是一个与样本均值和类内方差有关量



扩展阅读：LDA为什么长这样？

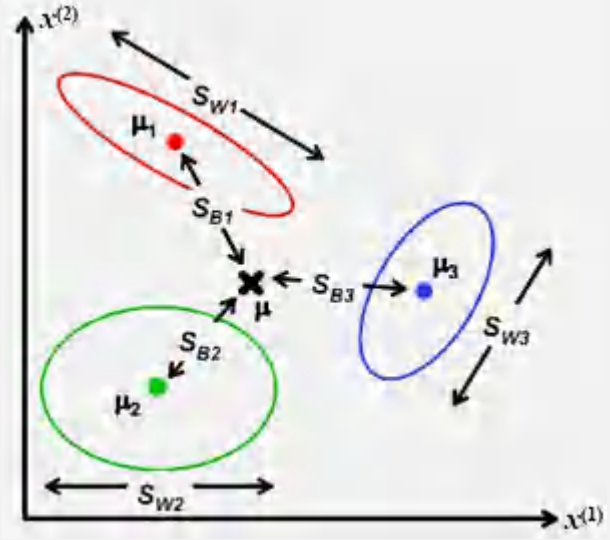
探讨：LDA分类器是属于判别式方法，还是生成式方法？

➢ 定义类间散度（between-class scatter）矩阵S_b:

$$S_B = S_t - S_w = \sum_{i=1}^{k} |X_i|(\boldsymbol{\mu}_i - \bar{\boldsymbol{\mu}})(\boldsymbol{\mu}_i - \bar{\boldsymbol{\mu}})^T$$

$|X_i|$: 第i类样本数

LDA有监督降维



输入：数据集$D = \{x^{(1)}, x^{(2)}, ..., x^{(m)}\}$

输出：降维后的样本集D';

- 计算类内散度矩阵S_w和类间散度矩阵S_B;
- 计算矩阵$S_w^{-1} S_B$的特征值;
- 找出矩阵$S_w^{-1} S_B$最大的$k-1$个特征值和其对应的$k-1$个特征向量$(w_1, w_2, \ldots, w_{k-1})$;
- 将原始样本集投影到以$(w_1, w_2, \ldots, w_{k-1})$为基向量生成的低维空间中($k-1$维)，得到新样本集$D'$

→第七讲

- 所以二分类LDA是将样本投影到一根直线上，k分类LDA至多投影到k-1维上。作为降维工具，LDA降维是一种有监督降维（需要知道类标记），且将d 维样本最多降至k-1维

第四讲 线性模型

1 线性回归及其推广

2 对数几率回归分类器LR

3 多分类学习

4 线性判别分析LDA

5 支持向量机SVM

支持向量机

1 支持向量机的模型

类比LR里面的线性回归预测值z

- 对于线性分类器而言，其判决函数：$f(x)=\begin{cases} \boldsymbol{w}^{\mathrm{T}}\boldsymbol{x_i}+w_0 \geq 1, \ y_i=1 & \text{（正类）} \\ \boldsymbol{w}^{\mathrm{T}}\boldsymbol{x_i}+w_0=0 & \text{（决策界）} \\ \boldsymbol{w}^{\mathrm{T}}\boldsymbol{x_i}+w_0 \leq -1, \ y_i=-1 & \text{（负类）} \end{cases}$

- 用 $y(w^Tx+w_0)$ 的正负性来判定或表示分类的正确性，定义分类的函数间隔（Functional margin）和几何间隔（Geometrical margin）

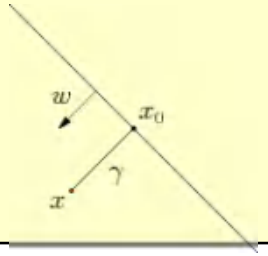
函数间隔： $\hat{\gamma}=y\cdot f(x)$

几何间隔： $\dot{\gamma}=\dfrac{f(x)}{\|\boldsymbol{w}\|}$ ， $|\dot{\gamma}|=\dfrac{y\cdot f(x)}{\|\boldsymbol{w}\|}=\dfrac{\hat{\gamma}}{\|\boldsymbol{w}\|}$

这里的w是直线的法向量，和LDA里方向向量不一样！

- 点x到平面的距离： $\dfrac{f(x)}{\|\boldsymbol{w}\|}=\dfrac{\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}+w_0}{\|\boldsymbol{w}\|}$

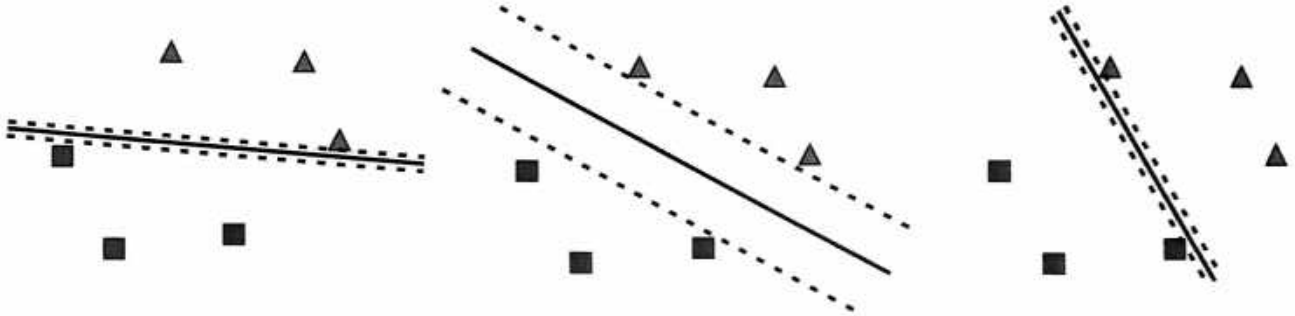
点与平面内任意一点的连线，在平面的法向量上的投影长



1 支持向量机的模型

类比LR里面的线性回归预测值z

- 对于线性分类器而言，其判决函数：$f(x)=\begin{cases} \boldsymbol{w}^{\mathrm{T}}\boldsymbol{x_i}+w_0 \geq 1, \ y_i=1 & （正类） \\ \boldsymbol{w}^{\mathrm{T}}\boldsymbol{x_i}+w_0=0 & （决策界） \\ \boldsymbol{w}^{\mathrm{T}}\boldsymbol{x_i}+w_0 \leq -1, \ y_i=-1 & （负类） \end{cases}$



分类器 1　　分类器 2　　分类器 3

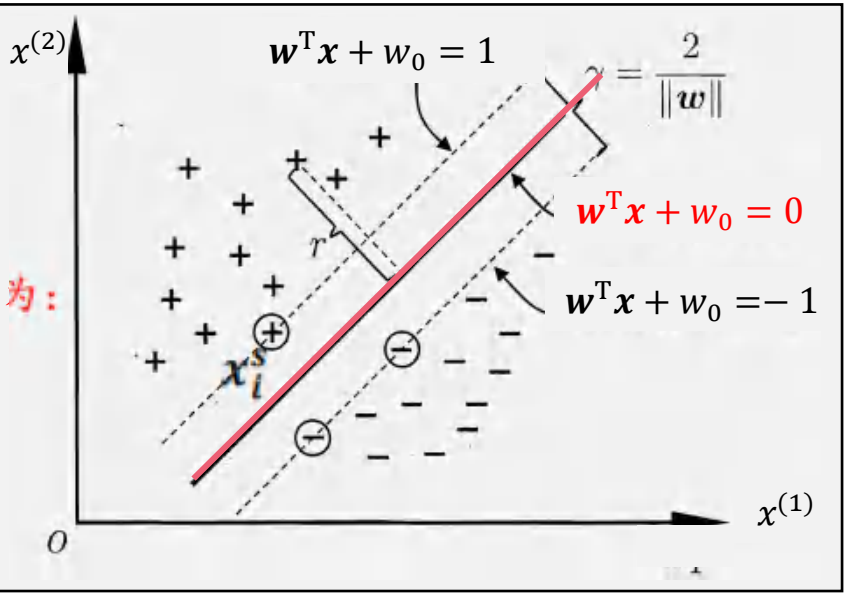
支持向量机

- 根据点到分类平面的几何间隔，定义支持向量 （support vector）为：

$$f(x_i^s)=\begin{cases} w^{\mathrm{T}}x_i^{s+}+w_0=1, & y_i=1 \\ w^{\mathrm{T}}x_i^{s-}+w_0=-1, & y_i=-1 \end{cases}$$

- 正反两类的支持向量到分类平面的距离和为：

$$\gamma=\frac{y_i\cdot f(x_i^{s+})}{\|w\|}+\frac{y_i\cdot f(x_i^{s-})}{\|w\|}=\frac{2}{\|w\|}$$



- 称 γ 为支持向量机模型里的间隔（margin）。分类决策界是最适合分开两类数据的一条“直线”。“最适合”的优化标准就是让这条直线离两边支持向量的间隔最大（$\gamma -> max$）

2 支持向量机的学习准则

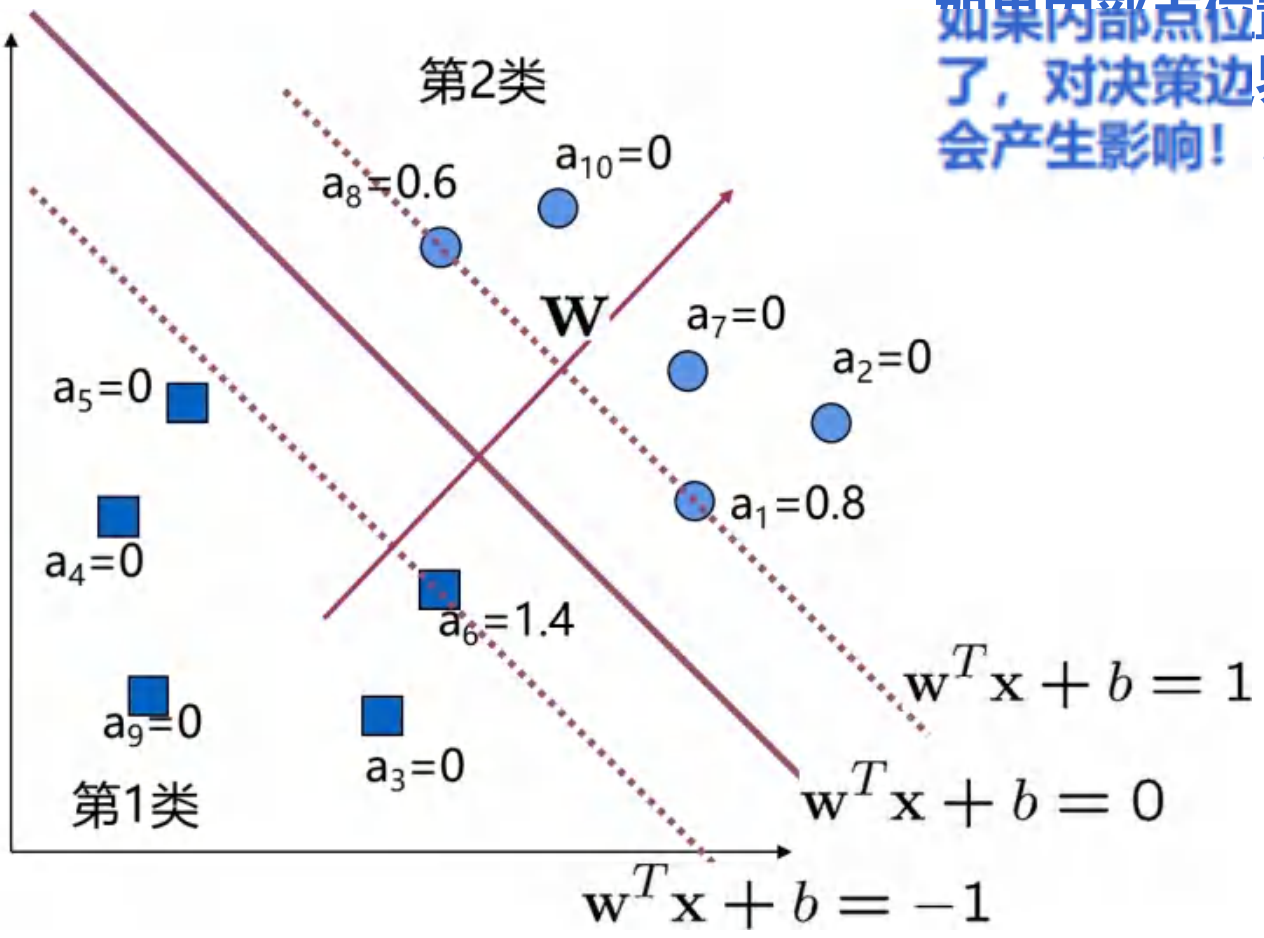
- 最大化间隔的优化准则表示为：

$$(\boldsymbol{w}, w_0)^* = \underset{\boldsymbol{w},w_0}{argmin} \frac{1}{2}\|\boldsymbol{w}\|^2 \quad , \quad s.t. \quad y_i(\boldsymbol{w}^T\boldsymbol{x_i}+w_0) \geq 1 \quad i=1,2,\ldots,m$$

- 最大化间隔的过程也是一个不断找寻支持向量的过程

支持向量机

几何解释



如果内部点位置改变了，对决策边界并不会产生影响！！！

第五讲 决策树

1 决策树的基本模型

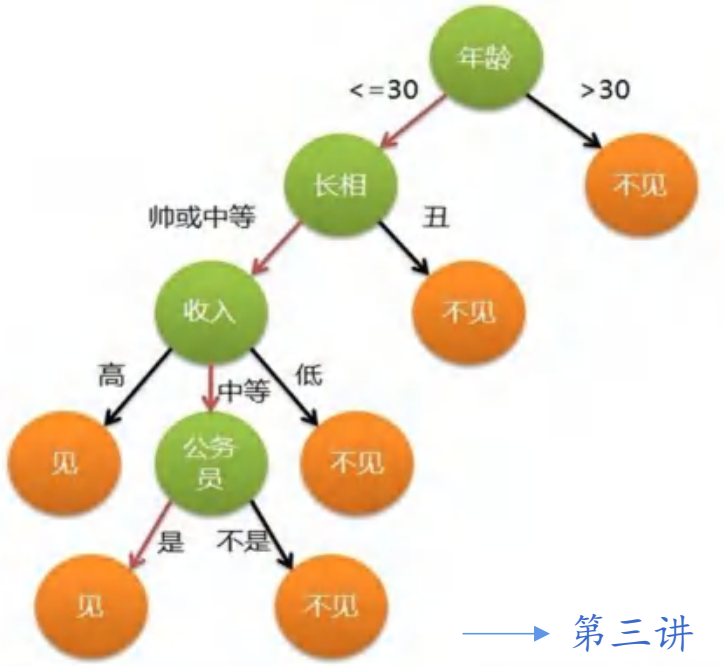
2 最佳属性划分

3 决策树剪枝

4 决策树用于回归问题

决策树的基本模型

- 很多时候，规则是潜在的。我们只能根据样本的属性取值和既往判断结果来学习规则（有监督的学习）。研究如何从训练数据中提取潜在判断规则的机器学习方法，就是“**决策树（Decision tree）**”



→ 第三讲

“是否同意相亲”的决策示意图

第五讲 决策树

导航方向 2025版

1 决策树的基本模型

2 最佳属性划分

3 决策树剪枝

4 决策树用于回归问题

最佳属性划分

- 上一节例子中，是在“年龄”、“收入”、“学生”和“信誉”4个特征中每次选择一个进行数据集的划分，从而构建决策树。如果划分的数据集是“最优”的，则被称为“最佳特征划分”
- 更多的时候决策树的生成会基于“最佳属性划分”。比如当一个特征的离散属性不止2个，或者具有连续属性时，而需要生成二叉树时，就需要找出最佳的属性进行划分
- “最佳属性划分”和“最佳特征划分” 是同一个概念的不同表述。在决策树的构建过程中，关键就是以**结点纯度**作为指标选择最佳的特征（或属性）进行数据划分

1 信息熵纯度

- 在决策树中，一般用不纯性度量（impurity measure）评估分类树划分的优劣。如果样本D的熵越小，则可认为其纯度（purity）越高。“熵”是度量样本纯度的代表性指标之一
- 样本集D中第i类样本占比记为$p_i(i = 1,2,\ldots,k)$，则样本D的熵为：

$$\mathrm{Ent}(D) = -\sum_{i=1}^{k} p_i \log_2 p_i$$

→第四讲

熵越小，则样本集D的纯度越高。从样本数据中估计概率（一般都是极大似然估计）而计算出来的熵又叫经验熵

2 ID3决策树

- ID3（Iterative Dichotomiser 3, 迭代二分器3）算法是一种经典的决策树学习算法，由Quinlan于上世纪70年代提出。ID3 算法的核心是在决策树各个结点上使用**信息增益（Information Gain）**进行不纯性度量，递归地构建决策树

最佳属性划分

- 假定特征a有V个可能的离散属性$\{a^1, a^2, \ldots, a^V\}$，若使用$a$来对样本集$D$进行划分，则会产生$V$个分支结点：其中第$v$个分支结点包含了$D$中所有特征$a$上属性为$a^v$的样本，记为$D^v$
- 特征a对样本集D的信息增益定义为：样本集D的信息熵与给定特征a条件下D的条件熵之差，记为：

所含样本个数

$$\mathbf{Gain}(D,a) = \mathrm{Ent}(D) - \mathrm{Ent}(D|a) = \mathrm{Ent}(D) - \sum_{v=1}^{V} \frac{|D^v|}{|D|} \mathrm{Ent}(D^v)$$

$Ent(D|a)$：在特征 a 的条件下，数据集D的条件熵，表示划分后的不确定性。
信息增益：表示划分前后不确定性减少的程度

- ID3模型最优划分属性选择依据：

$$a^* = \underset{a}{argmax}\,\mathbf{Gain}(D,a)$$

最佳属性划分



前面决策树生成中，为什么首先选择“年龄”属性进行划分

计数	年龄	收入	学生	信誉	归类：买计算机？
64	青	高	否	良	不买
64	青	高	否	优	不买
128	中	高	否	良	买
60	老	中	否	良	买
64	老	低	是	良	买
64	老	低	是	优	不买
64	中	低	是	优	买
128	青	中	否	良	不买
64	青	低	是	良	买
132	老	中	是	良	买
64	青	中	是	优	买
32	中	中	否	优	买
32	中	高	是	良	买
63	老	中	否	优	不买
1	老	中	否	优	买

$$\text{Ent}(D)=-\frac{641}{1024}log_2\frac{641}{1024}-\frac{383}{1024}log_2\frac{383}{1024}=0.9537$$

$$\text{Gain}(D,\text{年龄})=0.9537-\left(\frac{384}{1024}\times 0.9182+0+\frac{384}{1024}\times 0.9172\right)=0.2654$$

$D^v_{\text{年龄}1}=384$

计数	年龄	收入	学生	信誉	归类：买计算机？
64	青	高	否	良	不买
64	青	高	否	优	不买
128	青	中	否	良	不买
64	青	低	是	良	买
64	青	中	是	优	买

$$\text{Ent}(D_{\text{年龄}1})=-\frac{256}{384}log_2\left(\frac{256}{384}\right)-\frac{128}{384}log_2\left(\frac{128}{384}\right)=0.9182$$

计数	年龄	收入	学生	信誉	归类：买计算机？
128	中	高	否	良	买
64	中	低	是	优	买
32	中	中	否	优	买
32	中	高	是	良	买

$D^v_{\text{年龄}2}=256$　　$\text{Ent}(D_{\text{年龄}2})=0$

计数	年龄	收入	学生	信誉	归类：买计算机？
60	老	中	否	良	买
64	老	低	是	良	买
64	老	低	是	优	不买
132	老	中	是	良	买
63	老	中	否	优	不买
1	老	中	否	优	买

$D^v_{\text{年龄}3}=384$　　$\text{Ent}(D_{\text{年龄}3})=0.9172$

最佳属性划分

前面决策树生成中，为什么首先选择“年龄”属性进行划分

计数	年龄	收入	学生	信誉	归类：买计算机？
64	青	高	否	良	不买
64	青	高	否	优	不买
128	中	高	否	良	买
60	老	中	否	良	买
64	老	低	是	良	买
64	老	低	是	优	不买
64	中	低	是	优	买
128	青	中	否	良	不买
64	青	低	是	良	买
132	老	中	是	良	买
64	青	中	是	优	买
32	中	中	否	优	买
32	中	高	是	良	买
63	老	中	否	优	不买
1	老	中	否	优	买

$$\text{Ent}(D)=-\frac{641}{1024}log_2\frac{641}{1024}-\frac{383}{1024}log_2\frac{383}{1024}=0.9537$$

$$\text{Gain}(D,\text{学生})=0.9537-(\frac{484}{1024}\times 0.5635+\frac{540}{1024}\times 0.9761)$$
$$=0.1726$$

计数	年龄	收入	学生	信誉	归类：买计算机？
64	老	低	是	良	买
64	老	低	是	优	不买
64	中	低	是	优	买
64	青	低	是	良	买
132	老	中	是	良	买
64	青	中	是	优	买
32	中	高	是	良	买

$D^v_{\text{学生}1}=484$　$\text{Ent}(D_{\text{学生}1})=0.5635$

计数	年龄	收入	学生	信誉	归类：买计算机？
64	青	高	否	良	不买
64	青	高	否	优	不买
128	中	高	否	良	买
60	老	中	否	良	买
128	青	中	否	良	不买
32	中	中	否	优	买
63	老	中	否	优	不买
1	老	中	否	优	买

$D^v_{\text{学生}2}=540$　$\text{Ent}(D_{\text{学生}2})=0.9761$

3 C4.5决策树

- 信息增益准则对可取值数目较多（分支较多）的属性有所偏好，为减少这种偏好可能带来的不利影响（易过拟合），Quinlan在1993年提出了著名的C4.5决策树，使用**增益率（Gain ratio）**来选择最优划分属性：

$$\text{Gain_ratio}(D,a)=\frac{\text{Gain}(D,a)}{\text{IV}(a)} \quad , \quad \text{IV}(a) =-\sum_{v=1}^{V}\frac{|D^v|}{|D|}log_2\frac{|D^v|}{|D|}$$

- 其中IV(a)称为特征a的固有值（intrinsic value），本质上就是一种熵，它反映了根据特征a对数据集 D 进行划分时，各个子集的分布情况带来的不确定性。特征a可能取值数目越多，一般IV(a)也会越大，可作为分母平衡一下信息增益

最佳属性划分

- C4.5算法最优划分属性选择依据：

$$a^* = \underset{a}{argmax}\,\textbf{Gain_ratio}(D,a)$$

$$\text{Gain_ratio}(D,a)=\frac{\text{Gain}(D,a)}{\text{IV}(a)} \quad , \quad \text{IV}(a) = -\sum_{v=1}^{V}\frac{|D^v|}{|D|}log_2\frac{|D^v|}{|D|}$$

$$\text{Gain}(D,\text{年龄})=0.9537-\left(\frac{384}{1024}\times 0.9182+0+\frac{384}{1024}\times 0.9172\right)=0.2654$$

$$\text{IV}(\text{年龄}) = -\sum_{v=1}^{V}\frac{|D^v|}{|D|}log_2\frac{|D^v|}{|D|} = -\left(2*\frac{384}{1024}log_2\frac{384}{1024}+\frac{256}{1024}log_2\frac{256}{1024}\right)=1.56128$$

$$\text{Gain_ratio}(\text{D},\text{年龄})=\frac{0.2654}{1.56128}\approx 0.1700$$

$$\text{Gain}(D,\text{学生})=0.9537-\left(\frac{484}{1024}\times 0.5635+\frac{540}{1024}\times 0.9761\right)=0.1726$$

$$\text{IV}(\text{学生}) = -\sum_{v=1}^{V}\frac{|D^v|}{|D|}log_2\frac{|D^v|}{|D|} = -\left(\frac{484}{1024}log_2\frac{484}{1024}+\frac{540}{1024}log_2\frac{540}{1024}\right)=0.99784$$

$$\text{Gain_ratio}(\text{D},\text{学生})=\frac{0.1726}{0.99784}\approx 0.1730$$

4 CART决策树

- 分类与回归树 (Classification And Regression Tree, CART) 模型由Breiman等人于1984年提出，是一种应用广泛的决策树模型。CART既可以用于处理分类问题，也可以用于处理回归问题
- CART 决策树是二叉树。内部结点特征的取值为“是”和“否”，左分支是取值为“是”的分支，右分支是“否”分支。这样的决策树等价于递归地二分每个特征，将样本空间划分为有限个单元，并在这些单元上确定预测的概率分布

4 CART决策树

- 分类与回归树 (Classification And Regression Tree, CART) 模型由Breiman等人于1984年提出，是一种应用广泛的决策树模型。CART既可以用于处理分类问题，也可以用于处理回归问题

Baidu百科 百度百科——全球最大中文百科全书

- 里奥·布莱曼（Leo Breiman），世界著名应用统计学家，先后在UCLA和UC Berkeley任教，是机器学习领域的主要领导者之一。布莱曼系统提出了分类与回归树（CART）方法，并对集成学习做出重大贡献，在决策树基础上提出了随机森林（Random Forests）方法。



Leo Breiman
(1928-2005)

最佳属性划分

- CART决策树使用**基尼系数（Gini index）**度量数据纯度
- 样本集D中第i类样本占比记为$p_i(i=1,2,\ldots,k)$。样本的基尼系数越小，则样本集D的纯度越高，表示为：

$$\mathbf{Gini}(D)=\sum_{i=1}^{k}\sum_{i\neq i'}p_ip_{i'}=1-\sum_{i=1}^{k}p_i^2$$

对于两分类，有 $\mathrm{Gini}(D)=2p(1-p)$

- 采用属性a划分的基尼系数表示为：

$$\mathrm{Gini_index}(D,a)=\sum_{v=1}^{V}\frac{|D^v|}{|D|}\mathrm{Gini}(D^v)$$

最佳属性划分

- CART算法最优划分属性选择的依据是最小化基尼系数：

$$a^* = \underset{a}{argmin}\,\textbf{Gini_index}(D, a)$$

e.g. 试计算这两种划分的基尼系数，看哪一种划分更优

计数	年龄	收入	学生	信誉	归类：买计算机？
64	青	高	否	良	不买
64	青	高	否	优	不买
128	青	中	否	良	不买
64	青	低	是	良	买
64	青	中	是	优	买

计数	年龄	收入	学生	信誉	归类：买计算机？
128	中	高	否	良	买
64	中	低	是	优	买
32	中	中	否	优	买
32	中	高	是	良	买

计数	年龄	收入	学生	信誉	归类：买计算机？
60	老	中	否	良	买
64	老	低	是	良	买
64	老	低	是	优	不买
132	老	中	是	良	买
63	老	中	否	优	不买
1	老	中	否	优	买

计数	年龄	收入	学生	信誉	归类：买计算机？
64	老	低	是	良	买
64	老	低	是	优	不买
64	中	低	是	优	买
64	青	低	是	良	买
132	老	中	是	良	买
64	青	中	是	优	买
32	中	高	是	良	买

计数	年龄	收入	学生	信誉	归类：买计算机？
64	青	高	否	良	不买
64	青	高	否	优	不买
128	中	高	否	良	买
60	老	中	否	良	买
128	青	中	否	良	不买
32	中	中	否	优	买
63	老	中	否	优	不买
1	老	中	否	优	买

最佳划分的选择

4 CART决策树

$$\text{Gini_index}(D,a)=\sum_{v=1}^{V}\frac{|D^v|}{|D|}\text{Gini}(D^v)$$

计数	年龄	收入	学生	信誉	归类：买计算机？
64	青	高	否	良	不买
64	青	高	否	优	不买
128	青	中	否	良	不买
64	青	低	是	良	买
64	青	中	是	优	买

是“青年”

计数	年龄	收入	学生	信誉	归类：买计算机？
128	中	高	否	良	买
64	中	低	是	优	买
32	中	中	否	优	买
32	中	高	是	良	买

不是“青年”

计数	年龄	收入	学生	信誉	归类：买计算机？
60	老	中	否	良	买
64	老	低	是	良	买
64	老	低	是	优	不买
132	老	中	是	良	买
63	老	中	否	优	不买
1	老	中	否	优	买

年龄	总数	买	不买
老	384	257	127
青	384	128	256
中	256	256	0

$$a^*=\underset{a}{argmin}\,\textbf{Gini_index}(D,a)$$

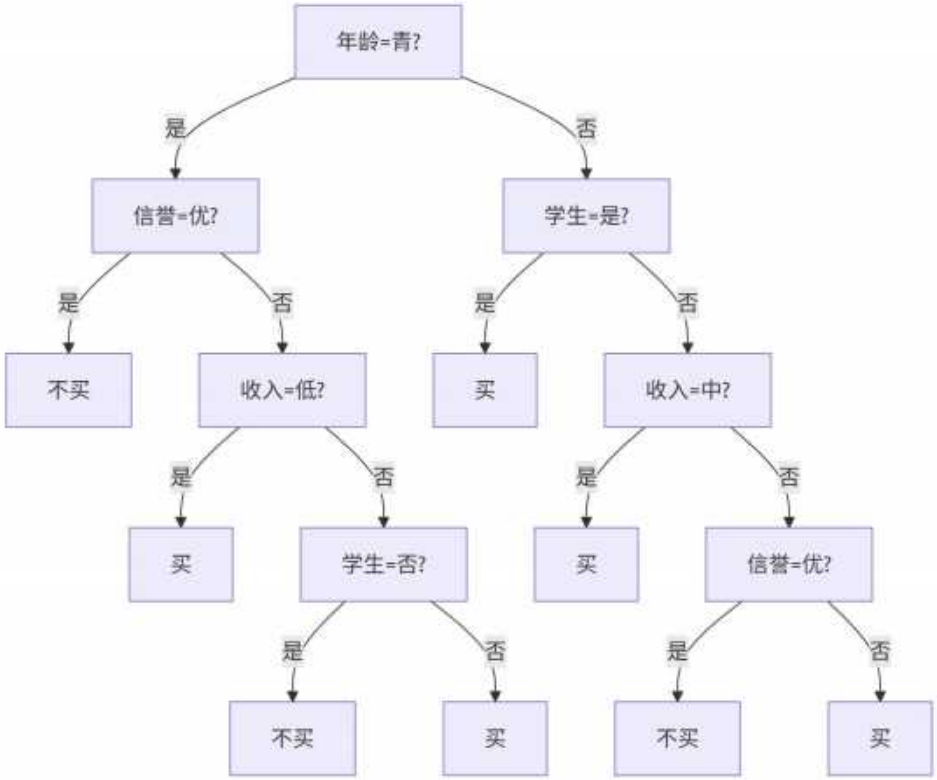
$$Gini(D,\text{青})=\frac{384}{1024}\left(2\times\frac{128}{384}\times\frac{256}{384}\right)+\frac{384+256}{1024}\left(2\times\frac{257+256}{384+256}\times\frac{127}{384+256}\right)=0.365$$

课堂计算$Gini(D,\text{老})$ ； $Gini(D,\text{中})$

最佳划分的选择

4 CART决策树

计数	年龄	收入	学生	信誉	归类：买计算机？
64	青	高	否	良	不买
64	青	高	否	优	不买
128	中	高	否	良	买
60	老	中	否	良	买
64	老	低	是	良	买
64	老	低	是	优	不买
64	中	低	是	优	买
128	青	中	否	良	不买
64	青	低	是	良	买
132	老	中	是	良	买
64	青	中	是	优	买
32	中	中	否	优	买
32	中	高	是	良	买
63	老	中	否	优	不买
1	老	中	否	优	买



5 连续属性与缺失属性的处理

- 若属性取值是连续值，不能直接根据数值进行结点划分。需要使用**连续属性离散化技术**。比如C4.5决策树采用二分法(bi-partition)对连续属性进行处理

计数	年龄	收入	学生	信誉	归类：买计算机?
64	青	高	否	良	不买
64	青	高	否	优	不买
128	中	高	否	良	买
60	老	中	否	良	买
64	老	低	是	良	买
64	老	低	是	优	不买
64	中	低	是	优	买
128	青	中	否	良	不买
64	青	低	是	良	买
132	老	中	是	良	买
64	青	中	是	优	买
32	中	中	否	优	买
32	中	高	是	良	买
63	老	中	否	优	不买
1	老	中	否	优	买



计数	年龄	收入	学生	信誉	归类：买计算机?
64	18	高	否	良	不买
64	19	高	否	优	不买
128	25	高	否	良	买
60	45	中	否	良	买
64	50	低	是	良	买
64	55	低	是	优	不买
64	28	低	是	优	买
128	20	中	否	良	不买
64	21	低	是	良	买
132	55	中	是	良	买
64	22	中	是	优	买
32	32	中	否	优	买
32	35	高	是	良	买
63	60	中	否	优	不买
1	65	中	否	优	买

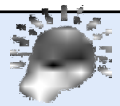
最佳属性划分

- 可以根据样本中属性a的连续取值$\{a^1,a^2,a^3,\ldots,a^n\}$，按照中位点$t_i=\frac{a^i+a^{i+1}}{2}$的方式设置一批候选划分点$T_a=\{t_1,t_2,\ldots\}$，再通过计算每一个划分点二分后的信息增益，找到使信息增益最大的候选点作为划分点，将属性 a 以划分点为界离散化

$$\boldsymbol{Gain}(D,a) = \mathrm{Ent}(D) - \sum_{v=1}^{V}\frac{|D^v|}{|D|}\mathrm{Ent}(D^v)$$

- 若当前结点划分属性为连续属性，该属性还可以作为其后代结点的划分属性继续使用

最佳属性划分



“年龄”属性的候选划分点集合包含14个候选值：T年龄={18.5, 19.5, 20.5, 21.5, 23.5, 26.5, 30.0, 33.5, 40.0, 47.5, 52.5, 55.0, 57.5, 62.5}，分别对应的信息增益为{0.0937，0.1997，0.4684，0.2599，0.1601，0.0496, 0.0199, 0.0101, 0.0035, 0.0004, 0.0134, 0.0855, 0.0855, 0.0006}，由此可以计算出属性“年龄”的信息增益为0.4684，对应于划分点20.5

计数	年龄	收入	学生	信誉	归类：买计算机?
64	18	高	否	良	不买
64	19	高	否	优	不买
128	25	高	否	良	买
60	45	中	否	良	买
64	50	低	是	良	买
64	55	低	是	优	不买
64	28	低	是	优	买
128	20	中	否	良	不买
64	21	低	是	良	买
132	55	中	是	良	买
64	22	中	是	优	买
32	32	中	否	优	买
32	35	高	是	良	买
63	60	中	否	优	不买
1	65	中	否	优	买



计数	年龄	收入	学生	信誉	归类：买计算机?
64		高	否	良	不买
64		高	否	优	不买
128		高	否	良	买
60		中	否	良	买
64		低	是	良	买
64		低	是	优	不买
64		低	是	优	买
128		中	否	良	不买
64		低	是	良	买
132		中	是	良	买
64		中	是	优	买
32		中	否	优	买
32		高	是	良	买
63		中	否	优	不买
1		中	否	优	买

以划分点为界离散化

最佳属性划分

- 如何在属性值缺失的情况下计算样本纯度?
- 以属性不缺失的样本计算信息增益，乘以无缺失样本占该属性所有样本的比例作为权重系数

计数	年龄	收入	学生	信誉	归类：买计算机?
64	青	高	否	良	不买
64	青	高	否	优	不买
128	中	高	否	良	买
60	老	中	否	良	买
64	老	低	是	良	买
64	老	低	是	优	不买
64	中	低	是	优	买
128	青	中	否	良	不买
64	青	低	是	良	买
132	老	中	是	良	买
64	青	中	是	优	买
32	中	中	否	优	买
32	中	高	是	良	买
63	老	中	否	优	不买
1	老	中	否	优	买



计数	年龄	收入	学生	信誉	归类：买计算机?
64	■	高	否	良	不买
64	青	高	否	■	不买
128	中	高	否	良	买
60	■	中	否	良	买
64	老	低	■	良	买
64	老	低	是	优	不买
64	■	低	是	优	买
128	青	■	否	良	不买
64	青	低	是	■	买
132	老	中	是	良	买
64	青	中	是	优	买
32	中	■	否	优	买
32	中	高	是	良	买
63	老	中	■	优	不买
1	老	中	否	优	买

武汉大学 WUHAN UNIVERSITY

最佳属性划分

计数	年龄	收入	学生	信誉	归类：买计算机?
64		高	否	良	不买
64	青	高	否		不买
128	中	高	否	良	买
60		中	否	良	买
64	老	低		良	买
64	老	低	是	优	不买
64		低	是	优	买
128	青		否	良	不买
64	青	低	是		买
132	老	中	是	良	买
64	青	中	是	优	买
32	中		否	优	买
32	中	高	是	良	买
63	老	中		优	不买
1	老	中	否	优	买

$\tilde{D}^v_{年龄1}=320$

$\text{Ent}(\tilde{D}_{年龄1})=-\frac{128}{320}log_2\frac{128}{320}-\frac{192}{320}log_2\frac{192}{320}=0.9706$

$\tilde{D}^v_{年龄2}=192$

$\text{Ent}(\tilde{D}_{年龄2})=0$

注意这些系数和前面例子的差别

$\tilde{D}^v_{年龄3}=324$

$\text{Ent}(\tilde{D}_{年龄3})=-\frac{197}{324}log_2\frac{197}{324}-\frac{127}{324}log_2\frac{127}{324}=0.9661$

$\text{Ent}(\tilde{D})=-\frac{517}{836}log_2\frac{517}{836}-\frac{319}{836}log_2\frac{319}{836}=0.9591$

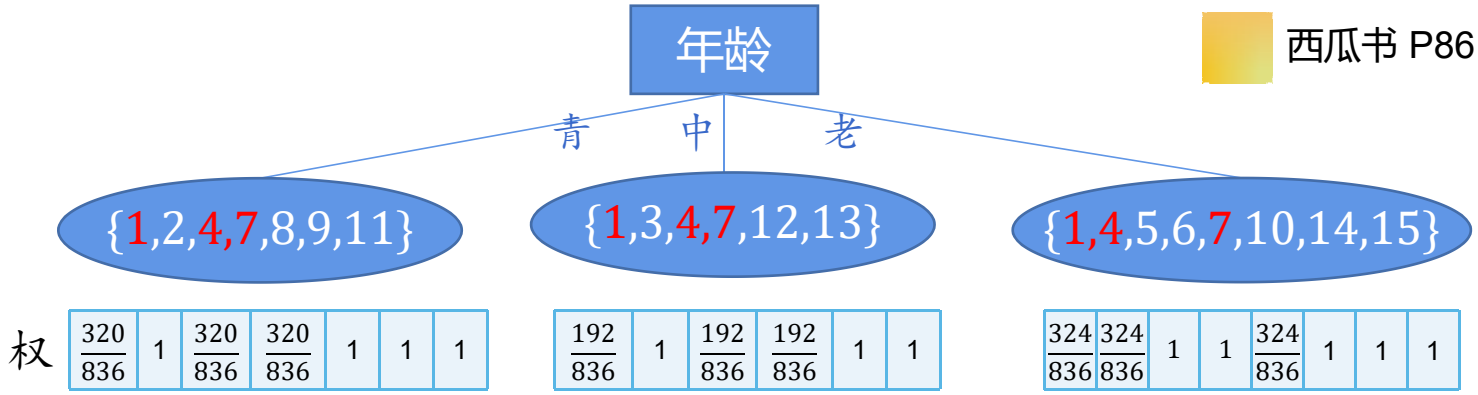
$\text{Gain}(\tilde{D},年龄)=0.9591-(\frac{320}{836}\times 0.9706+0+\frac{324}{836}\times 0.9661)=0.2248$

$\text{Gain}(D,年龄)=\rho * \text{Gain}(\tilde{D},年龄)=\frac{836}{1024}\times 0.2248=0.1835$

5 连续属性与缺失属性的处理

- 给定划分属性，若样本该特征属性缺失，如何对样本进行划分？

- 将该样本以一定的概率（权重系数）划分到所有子结点中。权重系数可以用各个分枝上属性值不缺失的样本之间的比例确定



西瓜书 P86

第五讲 决策树

1 决策树的基本模型

2 最佳属性划分

3 决策树剪枝

4 决策树用于回归问题

2 最小二乘回归树

- CART模型可以处理回归任务，其构建的回归决策树被称为“**最小二乘回归树**”。其思想是：对特征空间的划分采用启发式方法，每次划分时逐一考察当前集合中所有特征的所有取值，根据**平方误差最小化准则**选择最优划分点
- 给定数据集 $D = \{(\boldsymbol{x_1}, y_1), (\boldsymbol{x_2}, y_2), \ldots (\boldsymbol{x_m}, y_m)\}$ ，样本空间由d 维向量张成，$\boldsymbol{x_i} = (x_{i1}, x_{i2}, \ldots x_{id})^T$, $y_i \in \mathrm{R}$ ，则：

决策树用于回归问题

- 将样本第a维特征$x^{(a)}$和它的取值s，作为划分特征和划分属性点，可定义出两个区域$R_1(a,s)=\{x|x^{(a)}\le s\}$和$R_2(a,s)=\{x|x^{(a)}>s\}$，最优划分应使得两个区域平方误差和最小

真实值　　预测值

$$a^*,s^*=\underset{a,s}{argmin}\ [\sum_{x_i\in R_1(a,s)}(y_i-\hat{c}_1)^2+\sum_{x_i\in R_2(a,s)}(y_i-\hat{c}_2)^2]=argmin\ \mathrm{L(s)}$$

- 其中 $\hat{c}_1=\frac{1}{N_1}\Sigma_{x_i\in R_1(a,s)}y_i$，$\hat{c}_2=\frac{1}{N_2}\Sigma_{x_i\in R_2(a,s)}y_i$

  分别为R_1区域和R_2区域样本的回归预测值

  该区域的样本总数

- $\hat{c}_1$和$\hat{c}_2$就是各自对应区域内样本标记y_i的均值。以此为回归预测值可使得各自区域内平方误差最小，是该区域的最优输出值（预测值）

决策树用于回归问题

一维决策树回归

x	1	2	3	4	5	6	7	8	9	10
y	5.56	5.7	5.91	6.4	6.8	7.05	8.9	8.7	9	9.05

- 一维数据回归，最优划分属性就是x。计算9个划分点的代价函数：

s	1.5	2.5	3.5	4.5	5.5	6.5	7.5	8.5	9.5
c_1	5.56	5.63	5.72	5.89	6.07	6.24	6.62	6.88	7.11
c_2	7.5	7.73	7.99	8.25	8.54	8.91	8.92	9.03	9.05
L(s)	15.72	12.07	8.36	5.78	3.91	1.93	8.01	11.73	15.74

- 划分区域为：$R_1=\{1,2,3,4,5,6\}, R_2=\{7,8,9,10\}$
- 对应输出值：$\hat{c}_1=6.24$，$\hat{c}_2=8.91$

代价函数最小，最优划分

决策树用于回归问题

- 下面继续对R_1区域划分：

s	1.5	2.5	3.5	4.5	5.5
c_1	5.56	5.63	5.72	5.89	6.07
c_2	6.37	6.54	6.75	6.93	7.05

s	1.5	2.5	3.5	4.5	5.5
L(s)	1.3087	0.754	0.2771	0.4368	1.0644

- 对应输出值： $\hat{c}_1$ =6.75， $\hat{c}_2$ =5.72

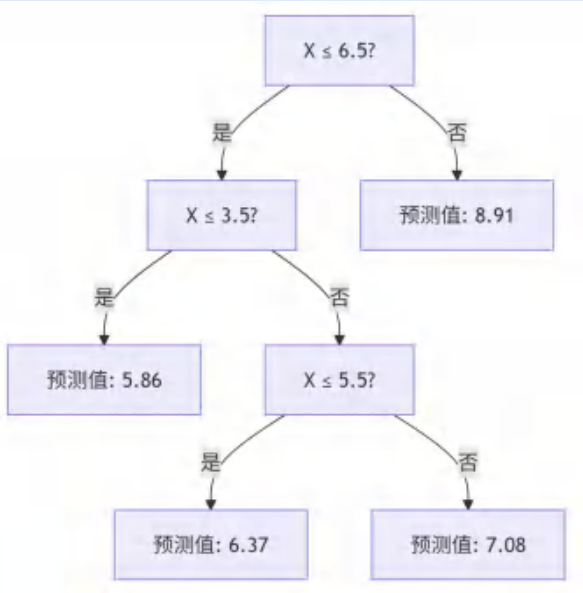
- 得到的两次划分回归树：

$$T=\begin{cases}5.72, & x \le 3.5\\ 6.75, & 3.5 < x \le 6.5\\ 8.91, & x > 6.5\end{cases}$$

决策树用于回归问题

e.g 一维决策树回归

x	1	2	3	4	5	6	7	8	9	10
y	5.56	5.7	5.91	6.4	6.8	7.05	8.9	8.7	9	9.05



决策树用于回归问题

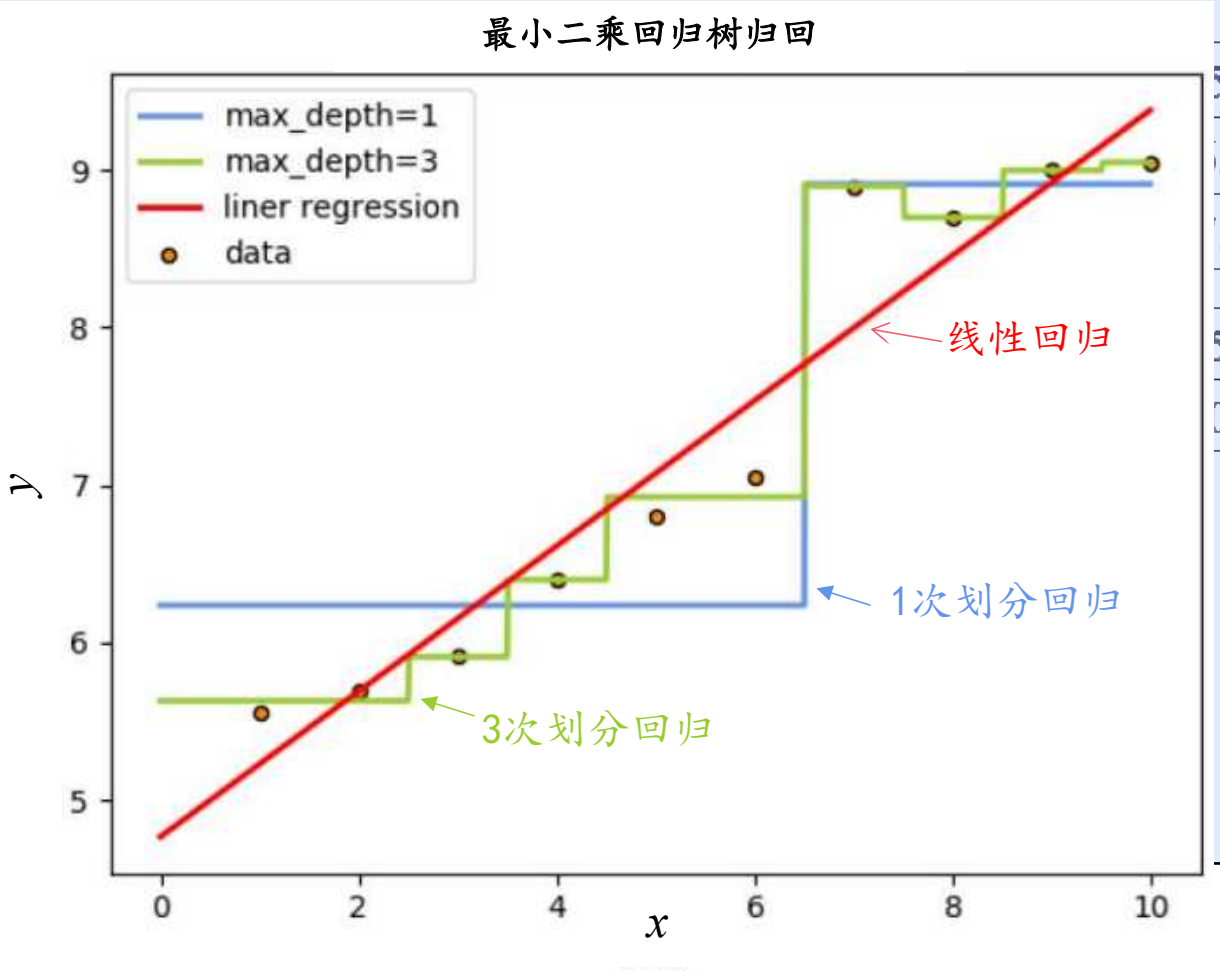
- 下面继续对R_1区域划分：

s									
c_1									
c_2									

s									
L(s)									

- 对应输

- 得到



第六讲 集成学习

1 个体与集成

2 装袋（Bagging）与随机森林

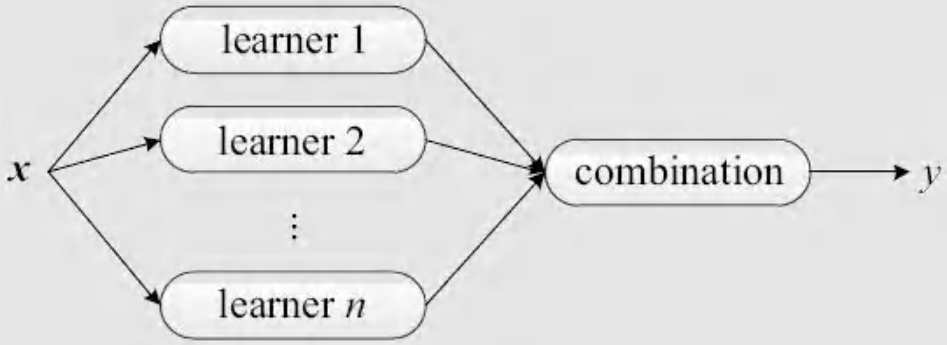
3 堆叠（Stacking）

4 提升（Boosting）与提升树

个体与集成

- **集成学习（Ensemble Learning）**通常是指利用**多个同种类**的学习器来对一个问题进行学习，其核心是**提高学习器的泛化能力**。在处理分类问题时也被称为多分类器系统（multi-classifier）

集成学习：一个集成由多个**个体学习器**构成，个体学习器分别在训练数据上训练获得，它们可以是决策树、神经网络或其他算法



同种类学习器的例子：

- 决策树：多个决策树（如在随机森林中）
- 神经网络：多个相同结构的神经网络（如在集成神经网络中）
- 支持向量机：多个支持向量机模型

1 基学习器

- 同质集成 (Homogeneous Ensemble)：集成中只包含同种类型的个体学习器，此时个体学习器也被称为“**基学习器（base learner）**”，相应的学习算法称为“基学习方法”
- 异质集成 (Heterogeneous Ensemble)：只要是使用多个学习器来解决问题，就可以认为符合集成学习的广义形式。个体学习器由不同的学习方法生成，不存在“基学习方法”。相应的，个体学习器一般不称为“基学习器”

个体与集成

- 集成学习的思想来自于群体智慧 (Crowd intelligence)
- 聚合一组学习器 (预测器/估计器：分类器或回归器) 的预测结果，得到比最好的单个学习器结果还要好，则集成是成功的

	测试例1	测试例2	测试例3
h_1	√	√	×
h_2	×	√	√
h_3	√	×	√
集成	√	√	√

(a) 集成提升性能

	测试例1	测试例2	测试例3
h_1	√	√	×
h_2	√	√	×
h_3	√	√	×
集成	√	√	×

(b) 集成不起作用

	测试例1	测试例2	测试例3
h_1	√	×	×
h_2	×	√	×
h_3	×	×	√
集成	×	×	×

(c) 集成起负作用

注意：并不是集成就会得到好结果 ⟹ **“好而不同”是关键！**

第六讲 集成学习

1 个体与集成

2 装袋（Bagging）与随机森林

3 堆叠（Stacking）

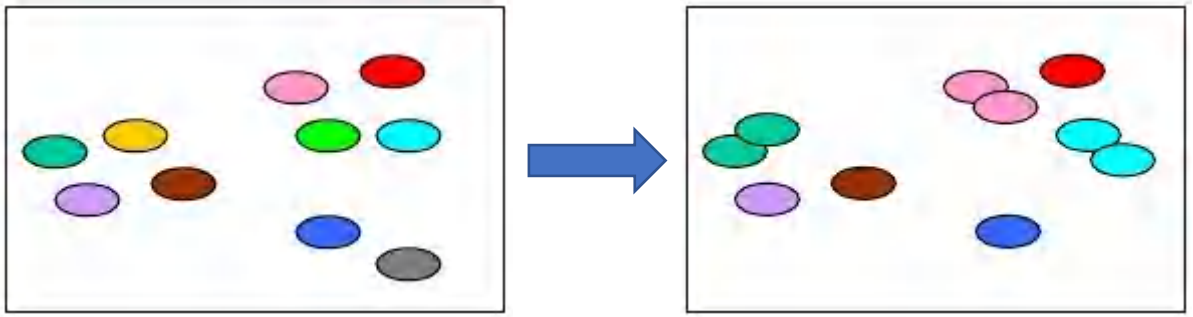
4 提升（Boosting）与提升树

装袋与随机森林

- Bagging（Bootstrap Aggregating）是一类并行式集成学习方法
- Bagging旨在通过组合多个弱学习器来提高整体模型的预测性能和鲁棒性，其核心思想是通过结合多个独立训练的模型来降低预测方差，提高模型的稳定性

1 样本扰动与平衡

- 其基于**自助采样法（bootstrap sampling）**给定包含m个样本的数据集D，每次随机从D 中挑选一个样本，将其副本放入D' （该样本仍在数据集D中，使得下次采样时仍有可能被选中）。重复执行m 次，就可以得到了包含m个样本的数据集D'

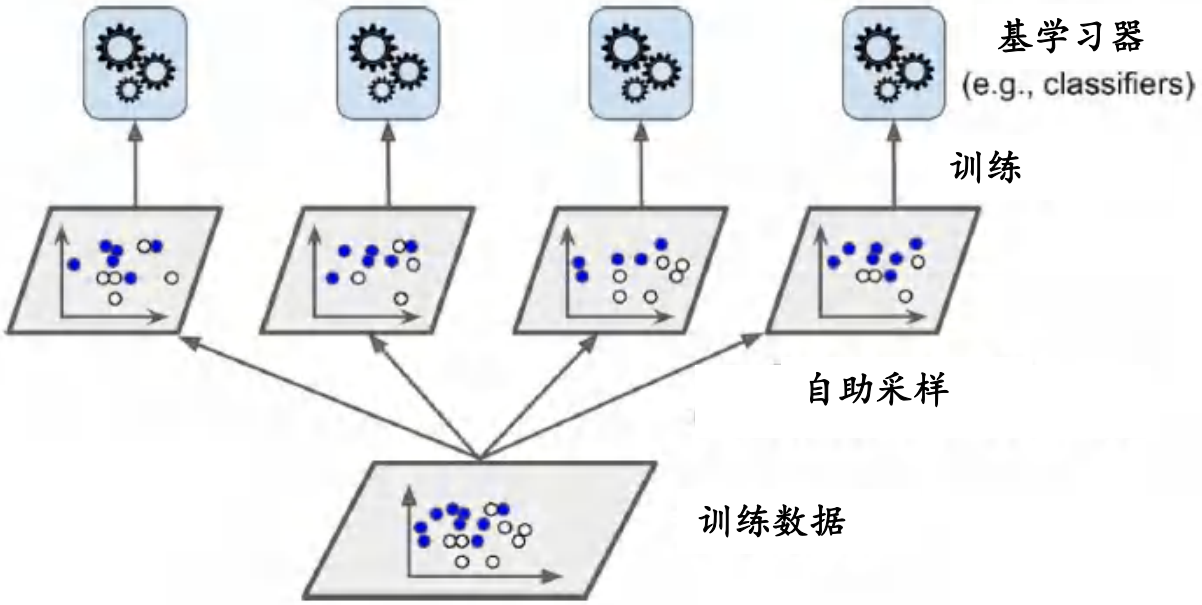


训练集与原样本集同规模　　数据分布有所改变

→第2讲

*在m次采样中，某样本始终不被选中的概率极限 $\lim_{m\to\infty}\left(1-\frac{1}{m}\right)^m \mapsto \frac{1}{e}\approx 0.368$

- 照此来得到T个采样集，然后基于采样集来训练基学习器，最终使用投票或平均策略将这些学习器组合

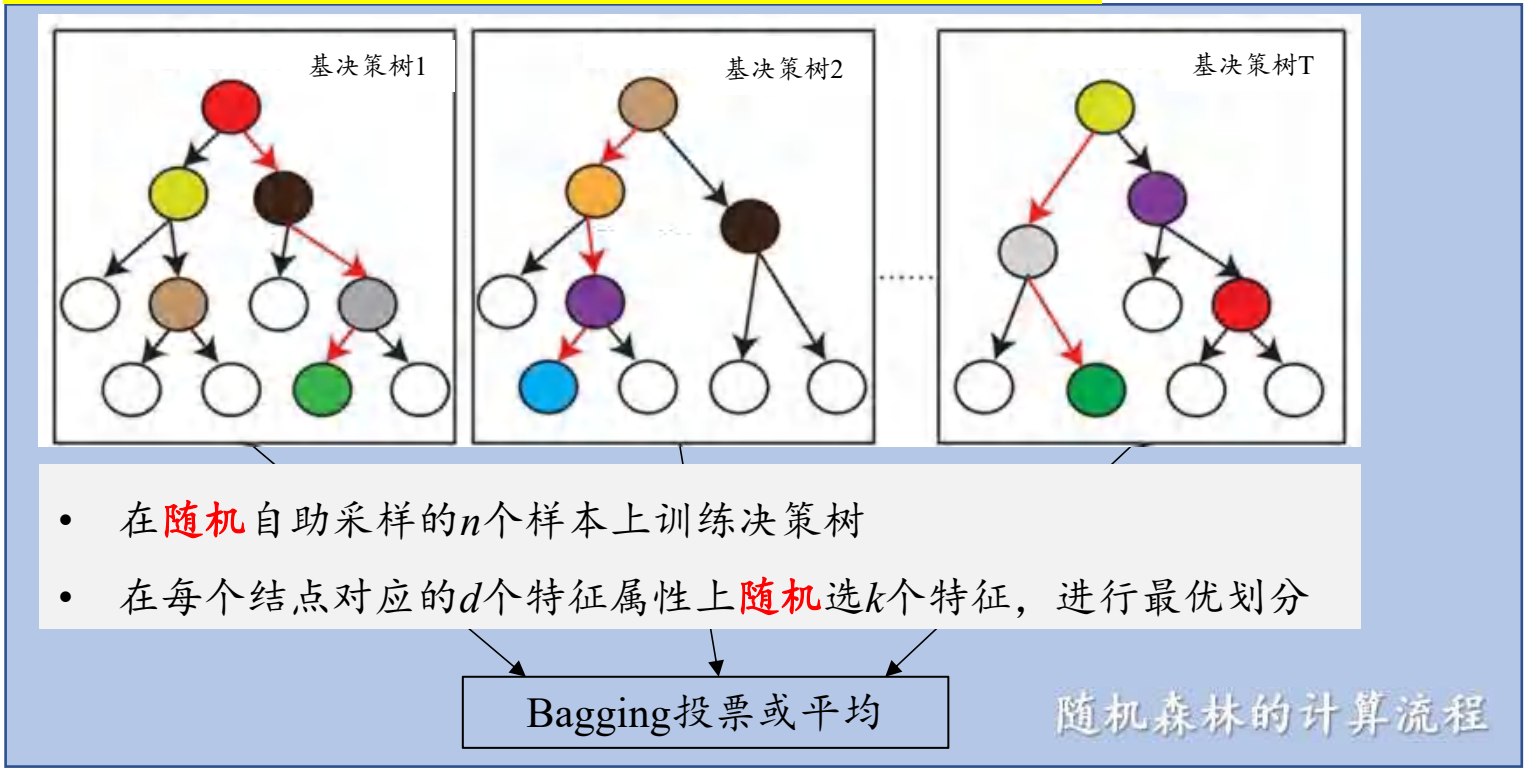


Bagging集成学习流程

Bagging算法高效且能直接应用与分类（简单投票）和回归任务（简单平均）。自主采样法剩下大约36.8%的样本来对泛化性能进行“**包外估计**”（out-of-bag estimate）

4 随机森林

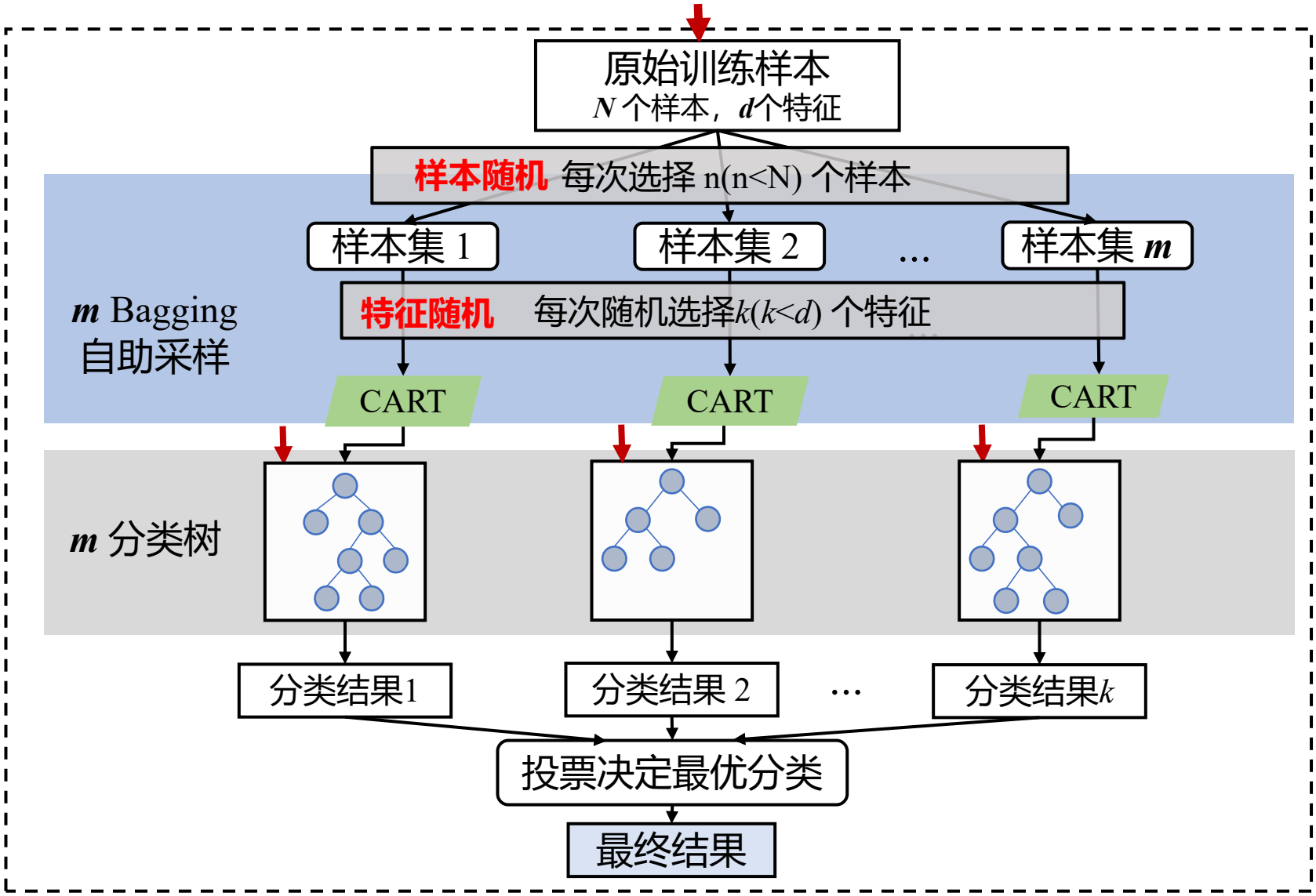
- 以决策树方法构建基学习器的Bagging称为随机森林（Random Forrest, RF），是一种非常经典的机器学习方法



装袋与随机森林

- 传统决策树在选择划分属性时是在当前结点的特征（属性）集合（假设d个）中选择一个。而在随机森林方法中，在决策树的训练过程中引入了**随机属性选择**
- 对基决策树的每个结点，先从该结点的特征集合中随机选择一个包含k个属性的子集，然后再从这个子集中选择一个最优属性用于划分。这里的参数k控制了随机性的引入程度：若令$k = d$，则基决策树的构建与传统决策树相同；若令$k = 1$，则是随机选择一个属性用于划分；一般情况下，$k = \log_2 d + 1$

装袋与随机森林


原始训练样本
N 个样本，d个特征
样本随机 每次选择 n(n<N) 个样本
样本集 1
样本集 2
…
样本集 m
m Bagging
自助采样
特征随机 每次随机选择k(k<d) 个特征
CART
CART
CART
m 分类树
分类结果1
分类结果 2
…
分类结果k
投票决定最优分类
最终结果

装袋与随机森林

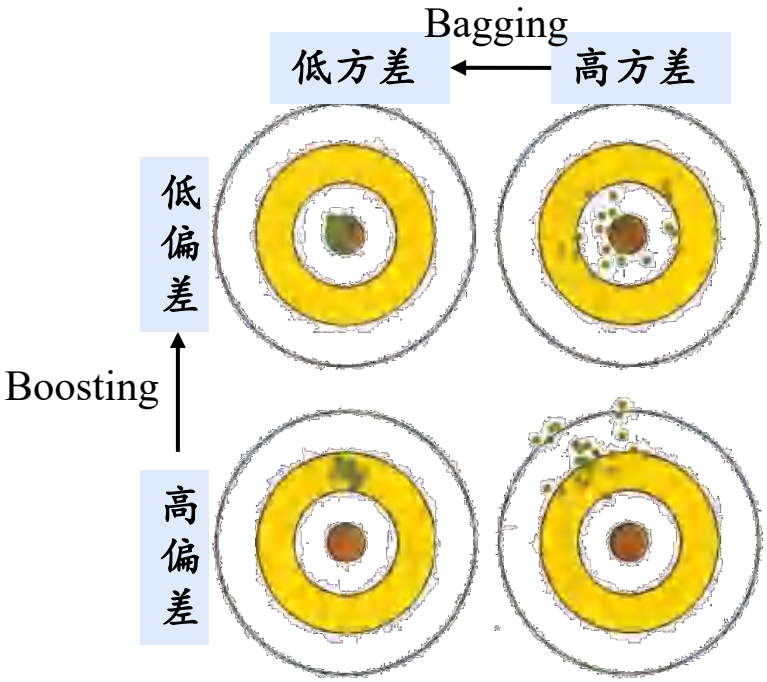
- RF与一般Bagging方法中基学习器的多样性主要来自样本扰动（自助采样）不同，RF中基学习器的多样性不仅来自**样本扰动**，还来自**特征扰动**，这就使得个体学习器之间差异性得以增加

- RF的解释性不如决策树，但不必为如何选择好的超参数而费心，且通常不需要剪枝。因为集成后的模型对来自于单个决策树的噪声抵抗力强。唯一需要关心的参数是随机森林中决策树的个数 。通常情况下，树越多，随机森林分类器的性能就越好，伴随着计算成本就越大

5 Bagging方法与方差

→第2讲

- 从偏差-方差角度来看，Bagging主要关注降低方差，因此它在不剪枝决策、神经网络等易受样本扰动的学习器上效用更为明显



- **降低方差**：Bagging通过集成多个模型（如决策树），利用自助采样生成不同的训练集，从而减少模型的方差。每个模型在不同的训练集上训练，最终的预测结果通过投票或平均来获得，这样可以平滑个别模型的波动。

1 个体与集成

2 装袋（Bagging）与随机森林

3 堆叠（Stacking）

4 提升（Boosting）与提升树

堆叠

1 堆叠集成举例

➢ Stacking首先在原始数据中训练初级基学习器，然后生成一个“新数据集”训练元学习器。一般在这个“新数据集”中，基学习器的输出被当作样例特征，原始样本的标记仍被当作样例标记

e.g 用基学习器在训练集上进行5折交叉验证，最终输出5个验证集的预测，把它们按行拼接起来；

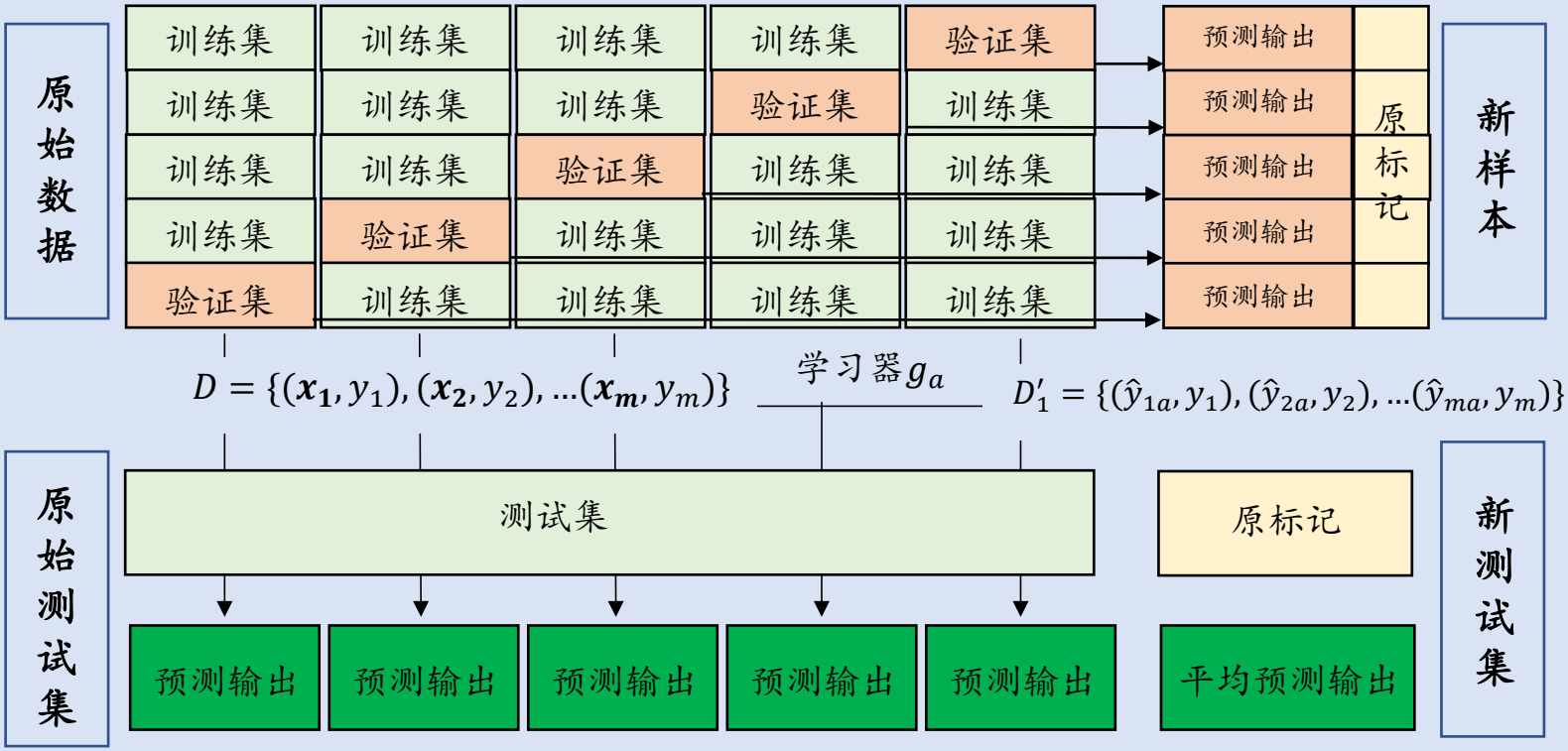
4份训练，1份验证



$D = \{(\boldsymbol{x_1}, y_1), (\boldsymbol{x_2}, y_2), \ldots (\boldsymbol{x_m}, y_m)\}$ 学习器g_a $D'_a = \{(\hat{y}_{1a}, y_1), (\hat{y}_{2a}, y_2), \ldots (\hat{y}_{ma}, y_m)\}$

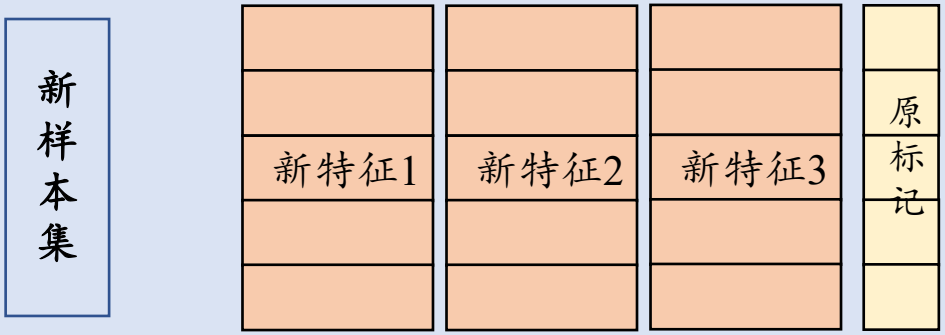
堆叠

- 用训练好的模型对测试集数据进行预测，输出的5个预测值取平均最为测试集的预测输出

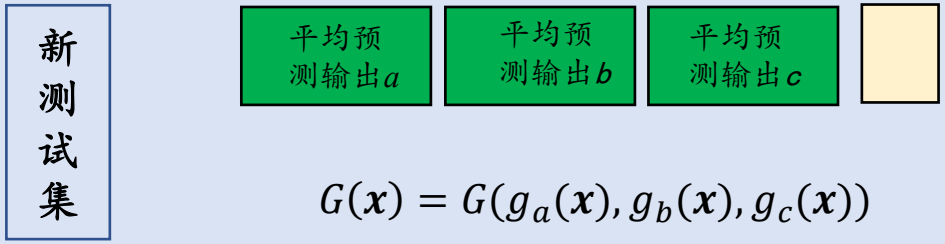


堆叠

- 假设有a、b、c三个基学习器，那么经过上面的操作后，得到每个学习器对应的新样本（包含训练集、标记值和测试集）
- 将三个基学习器得到的新样例特征和原标记构成新样本。一般使用LR模型（逻辑回归）作为元学习器进行训练



$$D' = \{([\hat{y}_{1a}, \hat{y}_{1b}, \hat{y}_{1c}], y_1), ([\hat{y}_{2a}, \hat{y}_{2b}, \hat{y}_{2c}], y_2), \ldots([\hat{y}_{ma}, \hat{y}_{mb}, \hat{y}_{mc}], y_m)\}$$



$$G(\boldsymbol{x}) = G(g_a(\boldsymbol{x}), g_b(\boldsymbol{x}), g_c(\boldsymbol{x}))$$

元学习器训练了什么？

学习基学习器预测结果（新特征）与真实标签之间的映射关系，从而学会如何最优地融合基学习器的预测，输出更准确的最终预测

第六讲 集成学习

导航方向 2025版

1 个体与集成

2 装袋（Bagging）与随机森林

3 堆叠（Stacking）

4 提升（Boosting）与提升树

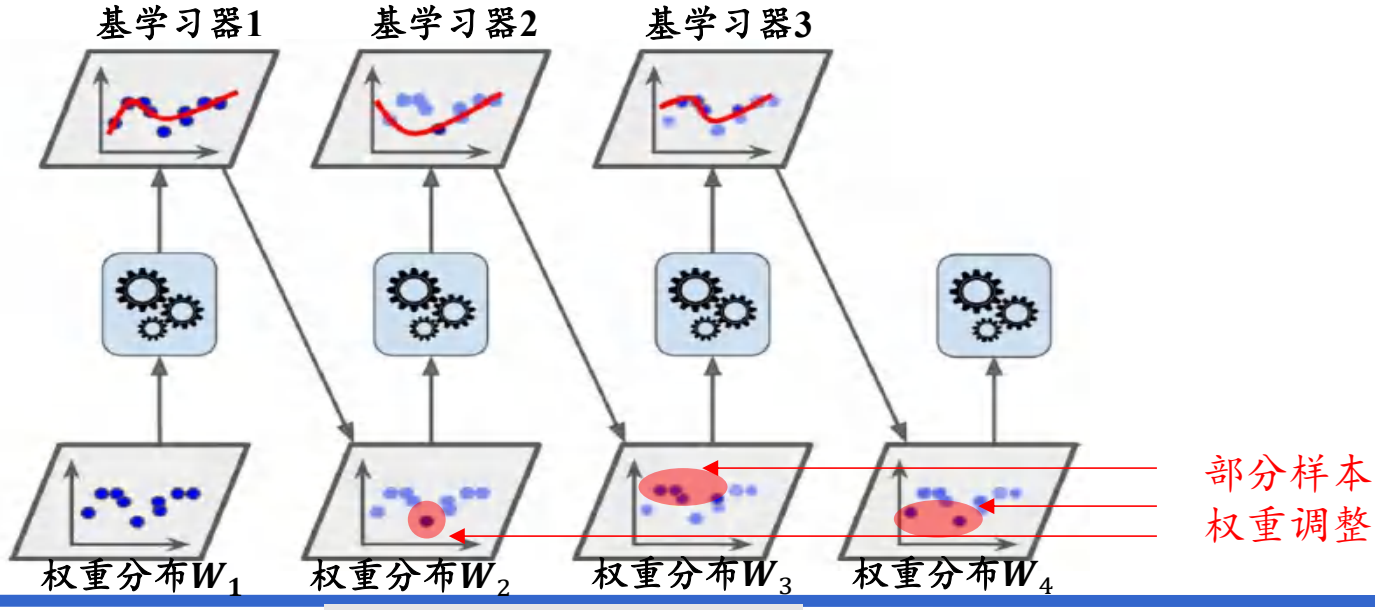
提升与提升树

- Boosting是一类**串行式集成**学习方法。
- 它的基学习器之间存在强依赖关系。要求基学习器能对特定的数据分布进行学习，可通过**“重采样法”**和**“重赋权法”**来实施
- 代表方法为自适应提升法（Adaptive Boosting，AdaBoost）、梯度提升法（Gradient Boosting）等

- **重赋权方法** (re-weighting)。在训练过程的每一轮中，根据样本分布为每个训练样本重新赋予一个权重
- **重采样方法** (re-sampling)。对无法接受带权样本的基学习算法，可通过在每一轮学习中，根据样本分布对训练集重新采样，再用重采样获得的样本集对基学习器进行训练

1 AdaBoost方法

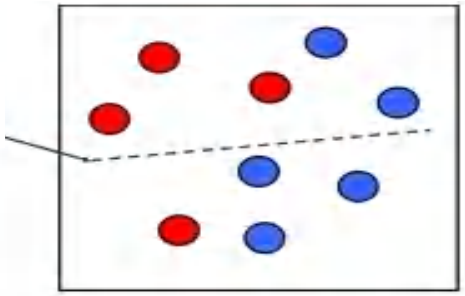
- 先从初始训练集训练出一个基学习器，再根据学习器的表现**对训练样本分布进行调整**，使得先前基学习器做错的训练样本在后续获得更多关注，并训练下一个基学习器；如此重复进行，直到基学习器的数目达到预先指定值T，最终将在T个基学习器进行加权求和



AdaBoost集成学习流程

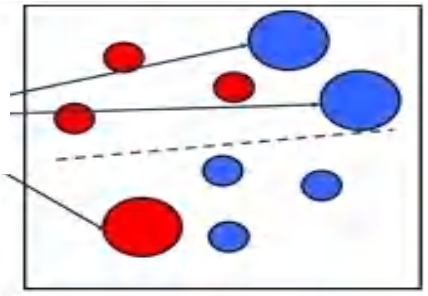
提升与提升树

基分类器1



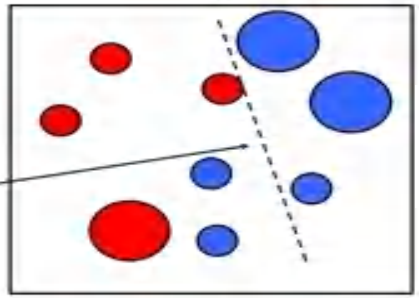
a

分类错误的样本权重增加



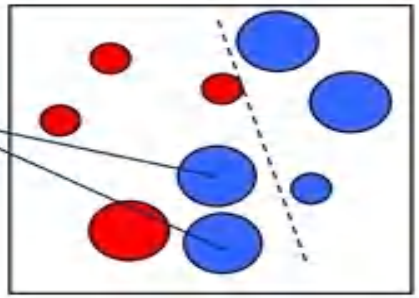
b

基分类器2



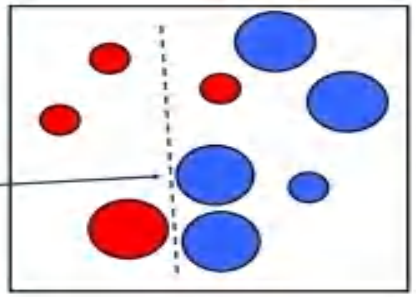
c

分类错误的样本权重增加



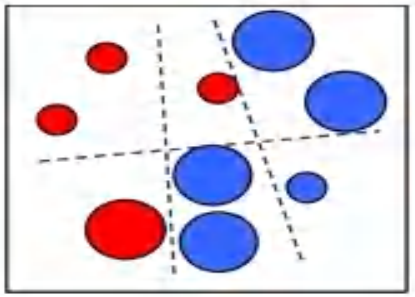
d

基分类器3



e

最终分类器是各基分类器的线性组合



f

AdaBoost集成学习改变每一轮数据权重分布

（图片来自于网络）

- 给定数据集 $D=\{(\boldsymbol{x_1},y_1),(\boldsymbol{x_2},y_2),...(\boldsymbol{x_m},y_m)\}$ $(x_i \in X \ ， \ y_i \in \{-1,1\})$

  和弱学习算法用AdaBoost构造最终的分类器$G(x)$

注意正负类标签

AdaBoost流程

（1）初始化训练数据的权值分布

$$W_1 = (w_{11},...,w_{1i},...,w_{1m}), \quad w_{1i}=\frac{1}{m}, \quad i=1,2,\cdots,m$$

初始等权重

（2）对$t=1,2,\cdots,\mathrm{T}$

（a）使用具有权值分布W_t的训练数据集学习，得到基分类器

$$g_t(\boldsymbol{x}): X \to \{-1,+1\}$$

（b）计算$g_t(\boldsymbol{x})$在训练数据集上的分类误差率e_t

$$e_t=\sum_{i=1}^{m} P(g_t(x_i)\neq y_i)=\sum_{i=1}^{m} w_{ti} I(g_t(x_i)\neq y_i)$$

即分类错误的样本的权重和

《统计学习方法》P156

提升与提升树

（c）计算$g_t(x)$的系数

$$\alpha_t = \frac{1}{2}\ln\frac{1-e_t}{e_t}$$

分类错误率越小的基学习器，权重越大

分类错误率>0.5的基学习器权重为负

（d）更新训练集的权值分布

$$W_{t+1} = (w_{t+1,1}, \ldots, w_{t+1,i}, \ldots, w_{t+1,m})$$

$$w_{t+1,i} = \frac{w_{ti}}{Z_t}\exp(-\alpha_t y_i g_t(x_i)), \quad i = 1,2,\cdots,m$$

分类正确：1

分类错误：-1

其中Z_t是一个规范化因子,使W_{t+1}成为一个概率分布

$$Z_t = \sum_{i=1}^{m} w_{ti}\exp(-\alpha_t y_i g_t(x_i))$$

提升与提升树

（3）构建基分类器的线性组合

$$f(x) = \sum_{t=1}^{T} \alpha_t g_t(x)$$

得到最终分类器：

$$G(x) = sign\big(f(x)\big) = sign\left(\sum_{t=1}^{T} \alpha_t g_t(x)\right)$$

组合学习器的符号决定最终分类

例题：根据下列样本，获得一个强分类器

x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1

因为x取值离散值，所以θ可以依次取0.5，1.5，2.5，...，8.5来对x进行分类，这里可以有两种分类方法：

- $x < \theta$时分为正类，$x > \theta$时分为负类，分类错误率对应ϵ_i^1
- $x > \theta$时分为正类，$x < \theta$时分为负类，分类错误率对应ϵ_i^2

第一轮迭代

1、选择最优弱分类器

所有样本的权重相同：

$$D_1(x) = \{w_{10}, w_{11}, w_{12}, w_{13}, w_{14}, w_{15}, w_{16}, w_{17}, w_{18}, w_{19}\}$$

$$= \left\{\frac{1}{10}, \frac{1}{10}, \frac{1}{10}, \frac{1}{10}, \frac{1}{10}, \frac{1}{10}, \frac{1}{10}, \frac{1}{10}, \frac{1}{10}, \frac{1}{10}\right\}$$

提升与提升树

例题：根据下列样本，获得一个强分类器

第一轮迭代 1、选择最优弱分类器

x	0	1	2	3	4	5	6	7	8	9	
y	1	1	1	-1	-1	-1	1	1	1	-1	
x<θ 时分为正类， x>θ 时分为负类											错误率ϵ_1^1
θ=0.5	1	-1	-1	-1	-1	-1	-1	-1	-1	-1	0.5
θ=1.5	1	1	-1	-1	-1	-1	-1	-1	-1	-1	0.4
θ=2.5	1	1	1	-1	-1	-1	-1	-1	-1	-1	0.3
θ=3.5	1	1	1	1	-1	-1	-1	-1	-1	-1	0.4
θ=4.5	1	1	1	1	1	-1	-1	-1	-1	-1	0.5
θ=5.5	1	1	1	1	1	1	-1	-1	-1	-1	0.6
θ=6.5	1	1	1	1	1	1	1	-1	-1	-1	0.5
θ=7.5	1	1	1	1	1	1	1	1	-1	-1	0.4
θ=8.5	1	1	1	1	1	1	1	1	1	-1	0.3

x>θ 时分为正类， x<θ 时分为负类											错误率ϵ_1^2
θ=0.5	-1	1	1	1	1	1	1	1	1	1	0.5
θ=1.5	-1	-1	1	1	1	1	1	1	1	1	0.6
θ=2.5	-1	-1	-1	1	1	1	1	1	1	1	0.7
θ=3.5	-1	-1	-1	-1	1	1	1	1	1	1	0.6
θ=4.5	-1	-1	-1	-1	-1	1	1	1	1	1	0.5
θ=5.5	-1	-1	-1	-1	-1	-1	1	1	1	1	0.4
θ=6.5	-1	-1	-1	-1	-1	-1	-1	1	1	1	0.5
θ=7.5	-1	-1	-1	-1	-1	-1	-1	-1	1	1	0.6
θ=8.5	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	0.7

提升与提升树

例题：根据下列样本，获得一个强分类器

第一轮迭代 1、选择最优弱分类器

x	0	1	2	3	4	5	6	7	8	9	
y	1	1	1	-1	-1	-1	1	1	1	-1	
x<θ 时分为正类， x>θ 时分为负类											错误率ϵ_1^1
θ=0.5	1	-1	-1	-1	-1	-1	-1	-1	-1	-1	0.5
θ=1.5	1	1	-1	-1	-1	-1	-1	-1	-1	-1	0.4
θ=2.5	1	1	1	-1	-1	-1	-1	-1	-1	-1	0.3
θ=3.5	1	1	1	1	-1	-1	-1	-1	-1	-1	0.4
θ=4.5	1	1	1	1	1	-1	-1	-1	-1	-1	0.5
θ=5.5	1	1	1	1	1	1	-1	-1	-1	-1	0.6
θ=6.5	1	1	1	1	1	1	1	-1	-1	-1	0.5
θ=7.5	1	1	1	1	1	1	1	1	-1	-1	0.4
θ=8.5	1	1	1	1	1	1	1	1	1	-1	0.3

最优弱分类器

x>θ 时分为正类， x<θ 时分为负类											错误率ϵ_1^2
θ=0.5	-1	1	1	1	1	1	1	1	1	1	0.5
θ=1.5	-1	-1	1	1	1	1	1	1	1	1	0.6
θ=2.5	-1	-1	-1	1	1	1	1	1	1	1	0.7
θ=3.5	-1	-1	-1	-1	1	1	1	1	1	1	0.6
θ=4.5	-1	-1	-1	-1	-1	1	1	1	1	1	0.5
θ=5.5	-1	-1	-1	-1	-1	-1	1	1	1	1	0.4
θ=6.5	-1	-1	-1	-1	-1	-1	-1	1	1	1	0.5
θ=7.5	-1	-1	-1	-1	-1	-1	-1	-1	1	1	0.6
θ=8.5	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	0.7

例题：根据下列样本，获得一个强分类器

第一轮迭代 1、选择最优弱分类器 按2.5切分数据

2、计算最优弱分类器的权重 $\alpha_1 = 0.5 \times \ln((1-0.3)/0.3) = 0.4236$

3、更新样本权重

x = 0，1，2，3，4，5，9时，$G_1(x_i)$分类正确，则样本权重为：$w_{2i} = \frac{w_{1i}}{Z_1} e^{(-\alpha_1 y_i G_1(x_i))} = \frac{0.1 * e^{-0.4236}}{Z_1}$

$Z_1 = 0.9165$

x = 6，7，8时，$G_1(x_i)$分类错误，则样本权重为：$w_{2i} = 0.1 * e^{-0.4236}/Z_1$

x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1
$G_1(x_i)$	1	1	1	-1	-1	-1	-1	-1	-1	-1
$w_{1i} \cdot e^{(-\alpha_1 y_i G_1(x_i))}$	0.06547	0.06547	0.06547	0.06547	0.06547	0.06547	0.15275	0.15275	0.15275	0.06547
w_{2i}	0.0714	0.0714	0.0714	0.0714	0.0714	0.0714	0.1667	0.1667	0.1667	0.0714
$\alpha_1 * G_1(x_i)$	0.4236	0.4236	0.4236	-0.4236	-0.4236	-0.4236	-0.4236	-0.4236	-0.4236	-0.4236
$fx_1 = sign(\alpha_1 * G_1(x_i))$	1	1	1	-1	-1	-1	-1	-1	-1	-1

准确率为0.7

例题：根据下列样本，获得一个强分类器

第一轮迭代 1、选择最优弱分类器 按2.5切分数据

2、计算最优弱分类器的权重 $\alpha_1 = 0.5 \times \ln((1-0.3)/0.3) = 0.4236$

3、更新样本权重

x = 0，1，2，3，4，5，9时，y分类正确，则样本权重为：$w_{2i} = \frac{w_{1i}}{Z_1} e^{(-\alpha_1 y_i G_1(x_i))} = \frac{0.1 * e^{-0.4236}}{Z_1}$

$Z_1 = 0.9165$

x = 6，7，8时，y分类错误，则样本权重为： $w_{2i} = 0.1 * e^{-0.4236}/Z_1$

x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1
g_1	1	1	1	-1	-1	-1	-1	-1	-1	-1
$w_{1i} \cdot e^{(-\alpha_1 y_i G_1(x_i))}$	0.06547	0.06547	0.06547	0.06547	0.06547	0.06547	0.15275	0.15275	0.15275	0.06547
w_{2i}	0.0714	0.0714	0.0714	规范化后权重		0.0714	0.1667	0.1667	0.1667	0.0714
$\alpha_1 * g_1$	0.4236	0.4236	0.4236	-0.4236	-0.4236	-0.4236	-0.4236	-0.4236	-0.4236	-0.4236
$fx_1 = sign(\alpha_1 * g_1)$	1	1	1	-1	-1	-1	-1	-1	-1	-1

准确率为0.7

第二轮迭代

1、选择最优弱分类器

$$e_t = \sum_{i=1}^{m} w_{ti} I(G_t(x_i) \neq y_i)$$

x<θ 时分为正类， x>θ 时分为负类T											错误率ϵ_2^1
θ=0.5	1	-1	-1	-1	-1	-1	-1	-1	-1	-1	0.6429
θ=1.5	1	1	-1	-1	-1	-1	-1	-1	-1	-1	0.5715
θ=2.5	1	1	1	-1	-1	-1	-1	-1	-1	-1	0.5001
θ=3.5	1	1	1	1	-1	-1	-1	-1	-1	-1	0.5715
θ=4.5	1	1	1	1	1	-1	-1	-1	-1	-1	0.6429
θ=5.5	1	1	1	1	1	1	-1	-1	-1	-1	0.7143
θ=6.5	1	1	1	1	1	1	1	-1	-1	-1	0.5476
θ=7.5	1	1	1	1	1	1	1	1	-1	-1	0.3809
θ=8.5	1	1	1	1	1	1	1	1	1	-1	0.2142
w_{2i}	0.0714	0.0714	0.0714	0.0714	0.0714	0.0714	0.1667	0.1667	0.1667	0.0714	
x>θ 时分为正类， x<θ 时分为负类											错误率ϵ_2^2
θ=0.5	-1	1	1	1	1	1	1	1	1	1	0.357
θ=1.5	-1	-1	1	1	1	1	1	1	1	1	0.4284
θ=2.5	-1	-1	-1	1	1	1	1	1	1	1	0.4998
θ=3.5	-1	-1	-1	-1	1	1	1	1	1	1	0.4284
θ=4.5	-1	-1	-1	-1	-1	1	1	1	1	1	0.357
θ=5.5	-1	-1	-1	-1	-1	-1	1	1	1	1	0.286
θ=6.5	-1	-1	-1	-1	-1	-1	-1	1	1	1	0.4523
θ=7.5	-1	-1	-1	-1	-1	-1	-1	-1	1	1	0.619
θ=8.5	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	0.7857

第二轮迭代

$$e_t = \sum_{i=1}^{m} w_{ti} I(g_t(x_i) \neq y_i)$$

1、选择最优弱分类器

x<θ 时分为正类， x>θ 时分为负类T											错误率ϵ_2^1
θ=0.5	1	-1	-1	-1	-1	-1	-1	-1	-1	-1	0.6429
θ=1.5	1	1	-1	-1	-1	-1	-1	-1	-1	-1	0.5715
θ=2.5	1	1	1	-1	-1	-1	-1	-1	-1	-1	0.5001
θ=3.5	1	1	1	1	-1	-1	-1	-1	-1	-1	0.5715
θ=4.5	1	1	1	1	1	-1	-1	-1	-1	-1	0.6429
θ=5.5	1	1	1	1	1	1	-1	-1	-1	-1	0.7143
θ=6.5	1	1	1	1	1	1	1	-1	-1	-1	0.5476
θ=7.5	1	1	1	1	1	1	1	1	-1	-1	0.3809
θ=8.5	1	1	1	1	1	1	1	1	1	-1	0.2142
w_{2i}	0.0714	0.0714	0.0714	0.0714	0.0714	0.0714	0.1667	0.1667	0.1667	0.0714	
x>θ 时分为正类， x<θ 时分为负类											错误率ϵ_2^2
θ=0.5	-1	1	1	1	1	1	1	1	1	1	0.357
θ=1.5	-1	-1	1	1	1	1	1	1	1	1	0.4284
θ=2.5	-1	-1	-1	1	1	1	1	1	1	1	0.4998
θ=3.5	-1	-1	-1	-1	1	1	1	1	1	1	0.4284
θ=4.5	-1	-1	-1	-1	-1	1	1	1	1	1	0.357
θ=5.5	-1	-1	-1	-1	-1	-1	1	1	1	1	0.286
θ=6.5	-1	-1	-1	-1	-1	-1	-1	1	1	1	0.4523
θ=7.5	-1	-1	-1	-1	-1	-1	-1	-1	1	1	0.619
θ=8.5	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	0.7857

最优弱分类器

例题：根据下列样本，获得一个强分类器

第二轮迭代 1、选择最优弱分类器 按8.5切分数据

2、计算最优弱分类器的权重 $a_2 = 0.5 \times \ln((1 - 0.2142) / 0.2142) = 0.6496$

3、更新样本权重

x = 0, 1, 2, 6, 7, 8, 9时，y分类正确，则样本权重为： $w_{3i} = w_{2i} \times \exp(-0.6496) / Z_2$

x = 3, 4, 5时，y分类错误，则样本权重为： $w_{3i} = w_{2i} \times \exp(0.6496) / Z_2$

x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1
$G_2(x_i)$	1	1	1	1	1	1	1	1	1	-1
w3i	0.0455	0.0455	0.0455	0.1667	0.1667	0.1667	0.1061	0.1061	0.1061	0.0455
$\alpha_2 * G_2(x_i)$	0.6496	0.6496	0.6496	0.6496	0.6496	0.6496	0.6496	0.6496	0.6496	-0.6496
$\alpha_1 * G_1(x_i)$	0.4236	0.4236	0.4236	-0.4236	-0.4236	-0.4236	-0.4236	-0.4236	-0.4236	-0.4236
$\alpha_1 * G_1 + \alpha_2 * G_2$	1.0732	1.0732	1.0732	0.226	0.226	0.226	0.226	0.226	0.226	-1.0732
$fx_2 = sign(\alpha_1 * G_1 + \alpha_2 * G_2)$	1	1	1	1	1	1	1	1	1	-1

第二轮迭代后构造的分类器

准确率为0.7

例题：根据下列样本，获得一个强分类器

第二轮迭代 1、选择最优弱分类器 按8.5切分数据

2、计算最优弱分类器的权重 $a_2 = 0.5 \times \ln((1 - 0.2142) / 0.2142) = 0.6496$

3、更新样本权重

x = 0, 1, 2, 6, 7, 8, 9时，y分类正确，则样本权重为： $w_{3i} = w_{2i} \times \exp(-0.6496) / Z_2$

x = 3, 4, 5时，y分类错误，则样本权重为： $w_{3i} = w_{2i} \times \exp(0.6496) / Z_2$

x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1
$G_2(x_i)$	1	1	1	1	1	1	1	1	1	-1
w_{3i}	0.0455	0.0455	0.0455	0.1667	0.1667	0.1667	0.1061	0.1061	0.1061	0.0455
$\alpha_2 * G_2$	0.6496	0.6496	0.6496	0.6496	0.6496	0.6496	0.6496	0.6496	0.6496	-0.6496
$\alpha_1 * G_1$	0.4236	0.4236	0.4236	-0.4236	-0.4236	-0.4236	-0.4236	-0.4236	-0.4236	-0.4236
$\alpha_1 * G_1 + \alpha_2 * G_2$	1.0732	1.0732	1.0732	0.226	0.226	0.226	0.226	0.226	0.226	-1.0732
$fx_2 = sign(\alpha_1 * G_1 + \alpha_2 * G_2)$	1	1	1	1	1	1	1	1	1	-1

规范化后权重

第二轮迭代后构造的分类器

准确率为0.7

第三轮迭代

1、选择最优弱分类器

x<θ 时分为正类， x>θ 时分为负类											错误率ϵ_3^1
θ=0.5	1	-1	-1	-1	-1	-1	-1	-1	-1	-1	0.4093
θ=1.5	1	1	-1	-1	-1	-1	-1	-1	-1	-1	0.3638
θ=2.5	1	1	1	-1	-1	-1	-1	-1	-1	-1	0.3183
θ=3.5	1	1	1	1	-1	-1	-1	-1	-1	-1	0.485
θ=4.5	1	1	1	1	1	-1	-1	-1	-1	-1	0.6517
θ=5.5	1	1	1	1	1	1	-1	-1	-1	-1	0.8184
θ=6.5	1	1	1	1	1	1	1	-1	-1	-1	0.7123
θ=7.5	1	1	1	1	1	1	1	1	-1	-1	0.6062
θ=8.5	1	1	1	1	1	1	1	1	1	-1	0.5001
w3i	0.0455	0.0455	0.0455	0.1667	0.1667	0.1667	0.1061	0.1061	0.1061	0.0455	
x>θ 时分为正类， x<θ 时分为负类											错误率ϵ_3^2
θ=0.5	-1	1	1	1	1	1	1	1	1	1	0.5911
θ=1.5	-1	-1	1	1	1	1	1	1	1	1	0.6366
θ=2.5	-1	-1	-1	1	1	1	1	1	1	1	0.6821
θ=3.5	-1	-1	-1	-1	1	1	1	1	1	1	0.5154
θ=4.5	-1	-1	-1	-1	-1	1	1	1	1	1	0.3487
θ=5.5	-1	-1	-1	-1	-1	-1	1	1	1	1	0.182
θ=6.5	-1	-1	-1	-1	-1	-1	-1	1	1	1	0.2881
θ=7.5	-1	-1	-1	-1	-1	-1	-1	-1	1	1	0.3942
θ=8.5	-1	-1	-1	-1	-1	-1	-1	-1	-1	1	0.5003

最优弱分类器

例题：根据下列样本，获得一个强分类器

第三轮迭代 1、选择最优弱分类器 按5.5切分数据

2、计算最优弱分类器的权重 $a_3 = 0.5 \times \ln((1-0.182)/0.182) = 0.752$

3、更新样本权重

x = 3, 4, 5, 6, 7, 8时，G_3分类正确，则样本权重为：$w_{4i} = w_{3i} \times \exp(-0.752)/Z_3$

x = 0, 1, 2, 9时，G_3分类错误，则样本权重为：$w_{4i} = w_{4i} \times \exp(0.752)/Z_3$

x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1
G_3	-1	-1	-1	-1	-1	-1	1	1	1	1
w_{4i}	0.1249	0.1249	0.1249	0.1019	0.1019	0.1019	0.0649	0.0649	0.0649	0.1249
α_3*G_3	-0.7514	-0.7514	-0.7514	-0.7514	-0.7514	-0.7514	0.7514	0.7514	0.7514	0.7514
α_2*G_2	0.6496	0.6496	0.6496	0.6496	0.6496	0.6496	0.6496	0.6496	0.6496	-0.6496
α_1*G_1	0.4236	0.4236	0.4236	-0.4236	-0.4236	-0.4236	-0.4236	-0.4236	-0.4236	-0.4236
$\alpha_1*G_1+\alpha 2*G2+\alpha 3*G3$	0.3218	0.3218	0.3218	-0.5254	-0.5254	-0.5254	0.9774	0.9774	0.9774	-0.3218
$fx3=sign(\alpha_1*G_1+\alpha_2*G_2+\alpha_3*G_3)$	1	1	1	-1	-1	-1	1	1	1	-1

第三轮迭代后构造的分类器

准确率为1

提问：AdaBoost算法中的Adaptive自适应体现在哪？

例题：根据下列样本，获得一个强分类器

第三轮迭代 1、选择最优弱分类器 按5.5切分数据

2、计算最优弱分类器的权重 $a_3 = 0.5 \times \ln((1-0.182)/0.182) = 0.752$

3、更新样本权重

x = 3, 4, 5, 6, 7, 8时，G_3分类正确，则样本权重为：$w_{4i} = w_{3i} \times \exp(-0.752)/Z_3$

x = 0, 1, 2, 9时，G_3分类错误，则样本权重为：$w_{4i} = w_{4i} \times \exp(0.752)/Z_3$

x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1
G_3	-1	-1	-1	-1	-1	-1	1	1	1	1
w_{4i}	0.1249	0.1249	0.1249	0.1019	0.1019	0.1019	0.0649	0.0649	0.0649	0.1249
α_3*G_3	-0.7514	-0.7514	-0.7514	-0.7514	-0.7514	-0.7514	0.7514	0.7514	0.7514	0.7514
α_2*G_2	0.6496	0.6496	0.6496	0.6496	0.6496	0.6496	0.6496	0.6496	0.6496	-0.6496
α_1*G_1	0.4236	0.4236	0.4236	-0.4236	-0.4236	-0.4236	-0.4236	-0.4236	-0.4236	-0.4236
$\alpha_1*G_1+\alpha 2*G2+\alpha 3*G3$	0.3218	0.3218	0.3218	-0.5254	-0.5254	-0.5254	0.9774	0.9774	0.9774	-0.3218
$fx3=sign(\alpha_1*G_1+\alpha_2*G_2+\alpha_3*G_3)$	1	1	1	-1	-1	-1	1	1	1	-1

规范化后权重

第三轮迭代后构造的分类器

准确率为1

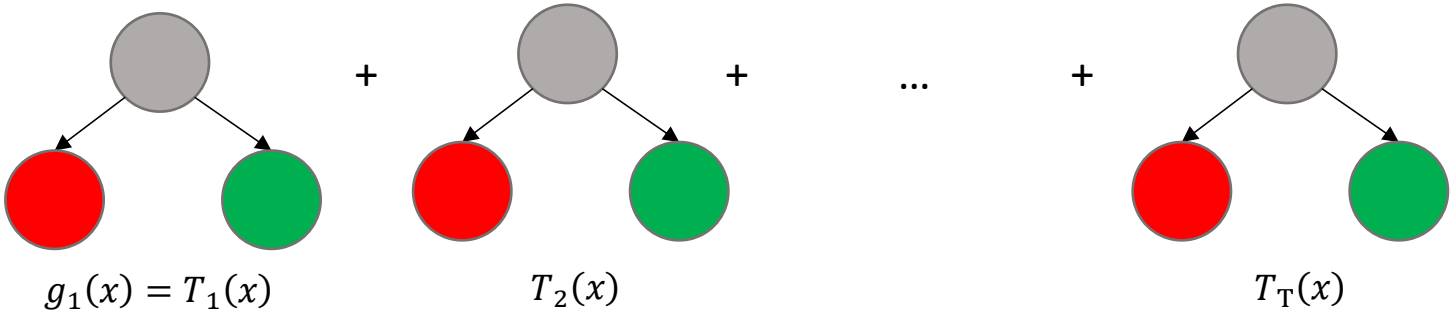
提问：AdaBoost算法中的Adaptive自适应体现在哪？

2 梯度提升树

- 以决策树方法构建基学习器的Boosting方法称为“提升树”
- 以CART为基学习器，可构造用于分类/回归问题的梯度提升树

- 将Adaboost方法中α_t均设为1，$g_t(x)$视为一棵棵分类二叉树：

$$G(x) = sign\left(\sum_{t=1}^{T} \alpha_t g_t(x)\right)$$

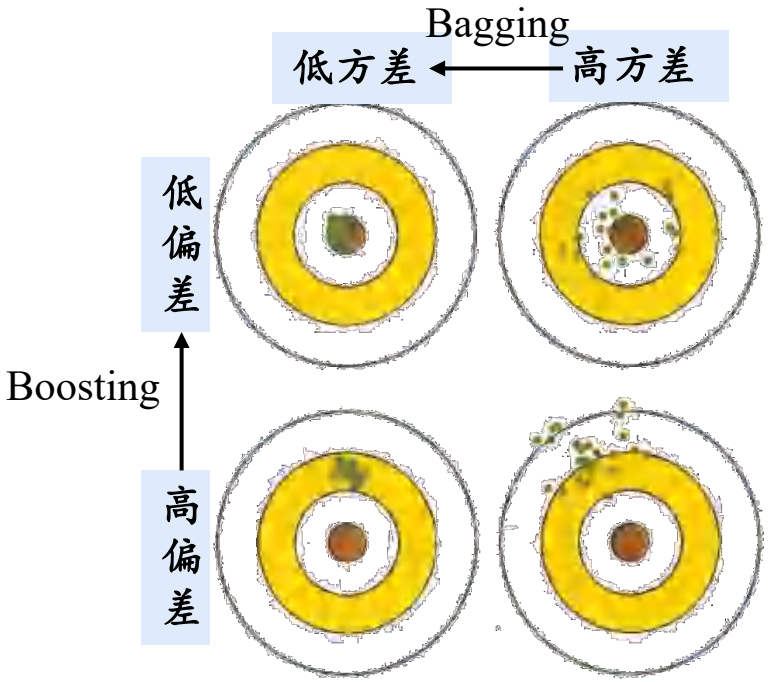


提升树模型

4 Boosting方法与偏差

→第2讲

- 从偏差-方差角度来看，Boosting主要关注降低偏差。因此Boosting能基于泛化性能弱的学习器构建出很强的集成



第七讲 维度归约

1 主成分分析（PCA）

2 自编码器（Autoencoder）

3 多维标定（MDS）

4 流形学习简介

主成分分析

- 现实应用中样本特征维数经常成千上万。当涉及到距离计算、内积计算以及样本矩阵、协方差矩阵计算时，高维数是所有机器学习器共同面临的严重障碍，被称为“维数灾难”（curse of dimensionality）https://www.douyin.com/video/7227000507963821371
- 控制和避免维数灾难的思路包括：

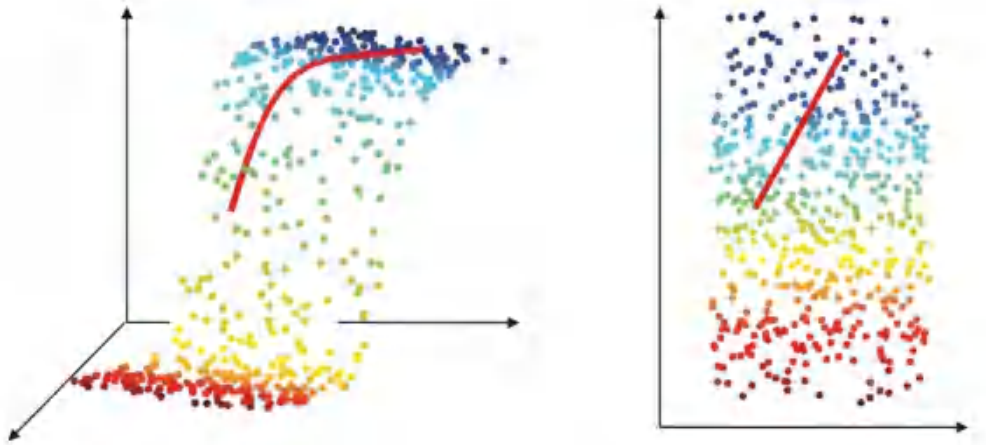
- **特征选择（feature selection）**：从d维特征中找出提供最多信息的k个，丢弃剩下的d-k个特征（归纳偏倚）

还记得岭回归和套索回归吗？

- **特征提取（feature extraction）**：找出d'个维的新特征集合，这些维是原来d个维的组合（$d'<<d$），又称为**维度归约(dimension reduction)或降维**。即通过数学变换，将原高维属性空间转变为低维“子空间”，在子空间中样本密度大幅度提高，计算也变得更为容易

主成分分析

- 样本虽然是高维的，但与学习任务密切相关的也许仅是某个低维分布，即高维空间中的一个“**低维嵌入 (embedding)**”



(a) 三维空间中观察到的样本点　　(b) 二维空间中的曲面

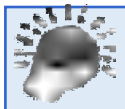
- 比较降维前后学习器的性能，若性能有所提高则认为降维起到了积极作用；若维数降至2维或者3维，还可通过可视化的技术来直观地判断降维效果

（图片来自《西瓜书》）

1 内积和基变换

也写作 $\langle \xi, \eta \rangle$

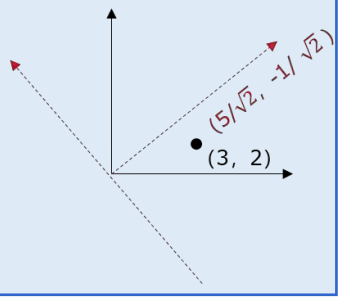
- 向量 $\eta = \begin{pmatrix} \eta_1 \\ \vdots \\ \eta_d \end{pmatrix}$ 和 $\xi = \begin{pmatrix} \xi_1 \\ \vdots \\ \xi_d \end{pmatrix}$ 的内积 $\sum_{i=1}^{d} \eta_i \xi_i$ 表示为 η 到 ξ 的投影长度乘以 ξ 的模。设 $|\xi| = 1$，则 $\xi \cdot \eta$ 表示为 η 向 ξ 所在直线投影的标量大小
- 基（也称为基底）是描述向量空间的基本工具。**向量空间中任意一个元素，都可以唯一地表示成基向量的线性组合。**要准确描述向量，首先要确定一组基，然后给出在基向量上的投影值。比如向量(3,2) = 3(1,0) +2(0,1)



e.g. 计算向量（3, 2）在 $\left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right), \left(-\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right)$ 这组基下的坐标

解：基矩阵和向量求内积

$$\begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} 3 \\ 2 \end{pmatrix} = \begin{pmatrix} \frac{5}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{pmatrix}$$



- 将样本特征矩阵变换到一个新的空间中的通用方法：

$$\begin{pmatrix} \boldsymbol{w}_1^T \\ \boldsymbol{w}_2^T \\ \vdots \\ \boldsymbol{w}_{d'}^T \end{pmatrix} (\boldsymbol{x}_1 \quad \boldsymbol{x}_2 \quad \cdots \quad \boldsymbol{x}_m) = \begin{pmatrix} \boldsymbol{w}_1 \cdot \boldsymbol{x}_1 & \boldsymbol{w}_1 \cdot \boldsymbol{x}_2 & \cdots & \boldsymbol{w}_1 \cdot \boldsymbol{x}_m \\ \boldsymbol{z}_1 & \boldsymbol{z}_2 & \cdots & \boldsymbol{z}_m \\ \vdots & \vdots & \vdots & \vdots \\ \boldsymbol{w}_{d'} \cdot \boldsymbol{x}_1 & \boldsymbol{w}_{d'} \cdot \boldsymbol{x}_2 & \cdots & \boldsymbol{w}_{d'} \cdot \boldsymbol{x}_m \end{pmatrix}$$

基向量1
基向量2
…
基向量R

特征向量1（样本1）　特征向量m（样本m）　新特征向量1（新样本1）　新特征向量m（新样本m）

提问：原样本和新样本各有多少维特征？

$\boldsymbol{w}_i$是个几维向量？

2 线性投影与降维

- **主成分分析（Principal Component Analysis, PCA）** 是指通过**正交变换**将一组高维的、可能存在相关性的样本变换为一组低维的、线性不相关的样本。转换后的新样本特征称为“主成分”，是能最大程度代表原样本特征的特性

主成分分析

- 一般来说欲获得低维子空间，最简单的是对原始高维空间进行线性变换。给定d维空间中的样本$X=(\boldsymbol{x}_1,\boldsymbol{x}_2,...,\boldsymbol{x}_m)\in\Re^{d\times m}$，变换之后得到$d'\leq d$维空间中的样本$Z=W^TX$，$W$是变换矩阵，$Z=(\boldsymbol{z}_1,\boldsymbol{z}_2,...,\boldsymbol{z}_m)\in\Re^{d'\times m}$是样本$X$在新空间中的表达，且是在一个**线性超平面**上的表达

$$\boldsymbol{Z}=\boldsymbol{W}^T\boldsymbol{X}$$

变换矩阵$\boldsymbol{W}$由新空间的d'个d维向量作为基向量。若$\{\boldsymbol{w}_1,\boldsymbol{w}_2,...,\boldsymbol{w}_{d'}\}$这组基向量两两正交，则实现了PCA方法中将“高维的、可能存在相关性的变量转换为一组低维的、线性不相关的变量”的变换目的（正交变换）

提问：1. 什么叫向量正交？

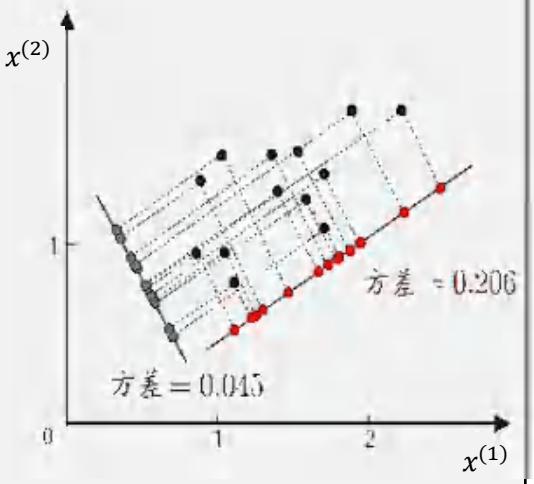
2. 想一想正交投影相较于非正交投影用于降维的优点

3 最大可分性

- PCA方法可以从多个角度解释，最常见的推导角度有：
  - 最大可分性：样本点在一个线性超平面上的投影能尽可能分开
  - 最近重构性：样本点到一个线性超平面的投影距离都足够近；

PCA（1）

- 假定有m个样本，目标是获得样本在$d'(d'<<d)$维空间中的表示$Z \in \Re^{d' \times m}$，使样本投影尽可能分开
- 样本点在一个线性超平面上的投影能尽可能分开，也就避免了样本在降维后“重叠”，保持了其原本最大的信息熵。最大可分性可以理解为：投影后样本点的方差最大化，且各新特征不相关



PCA最大可分性

主成分分析

- 样本矩阵X投影后的各特征维度的方差就是协方差矩阵$\frac{1}{m}XX^T$的特征值。因此PCA找到最大方差的投影即找样本协方差矩阵最大的特征值。最佳投影方向就是最大特征值所对应的特征向量方向，次佳投影方向就是第二大特征值对应的特征向量方向…

PCA的计算流程

输入：样本集$D=\{\boldsymbol{x_1},\boldsymbol{x_2},\cdots,\boldsymbol{x_m}\}$和低维空间维数$d'$

1. 对所有样本进行中心化：$\boldsymbol{x_i}\leftarrow\boldsymbol{x_i}-\frac{1}{m}\sum_{i=1}^{m}\boldsymbol{x_i}$；

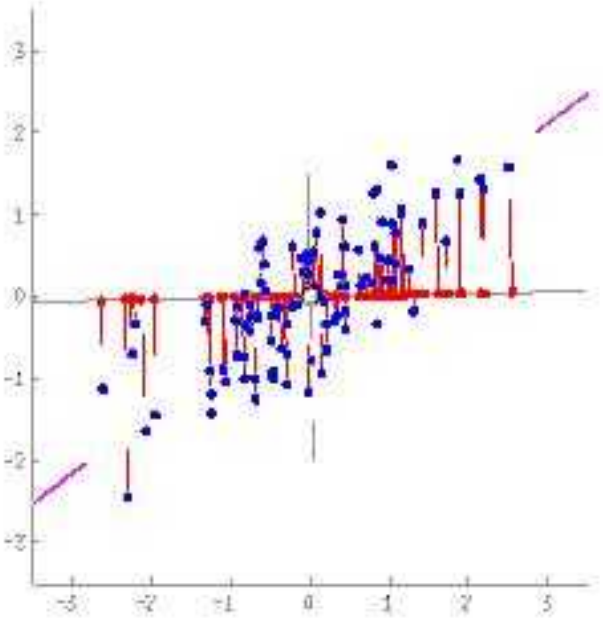
2. 计算样本的散度（或协方差）矩阵XX^T；

3. 对散度（或协方差）矩阵XX^T做特征分解；

4. 取最大的d'个特征值所对应的特征向量$\boldsymbol{w_1},\boldsymbol{w_2},\cdots,\boldsymbol{w_{d'}}$

输出：投影矩阵$W=\{\boldsymbol{w_1},\boldsymbol{w_2},\cdots,\boldsymbol{w_{d'}}\}$ （标准正交基还需要单位化）

4 最近重构性



投影距离越长，损失的信息也会越多

！一定要分清楚投影距离和回归残差的区别

（图片来自互联网）

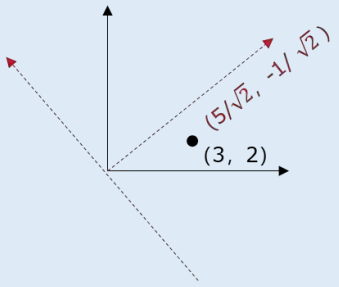
主成分分析

- **数据样本与基向量：** 设原数据样本是d维的，共有m个数据$(\boldsymbol{x_1}, \boldsymbol{x_2}, \cdots, \boldsymbol{x_m})$，对应的基向量$\{\boldsymbol{\alpha_1}, \boldsymbol{\alpha_2}, \cdots, \boldsymbol{\alpha_d}\}$，其中$\boldsymbol{\alpha}_i$是标准正交基即$||\boldsymbol{\alpha_i}||_2 = 1$，$\boldsymbol{\alpha}_i^T \boldsymbol{\alpha}_j = 0$。
- **降维过程：** 若将数据从d维降到d'维，则标准正交投影空间$W = (\boldsymbol{w_1}, \boldsymbol{w_2}, \cdots, \boldsymbol{w_{d'}})$，样本点$\boldsymbol{x_i}$在$d'$维坐标中的投影为：$\boldsymbol{z_i} = (z_{i1}, z_{i2}, \cdots, z_{id'})^T$。$\boldsymbol{z_i} = W^T \boldsymbol{x_i}$，其中$z_{ij} = \boldsymbol{w}_j^T \boldsymbol{x_i}$是$\boldsymbol{x_i}$降维后的第$j$维的坐标
- **数据恢复：** 如果用$\boldsymbol{z_i}$来恢复原始数据$\boldsymbol{x_i}$，则得到的恢复数据

$$\hat{\boldsymbol{x}}_{\boldsymbol{i}} = \sum_{j=1}^{d'} z_{ij} \boldsymbol{w}_j = W \boldsymbol{z_i}$$

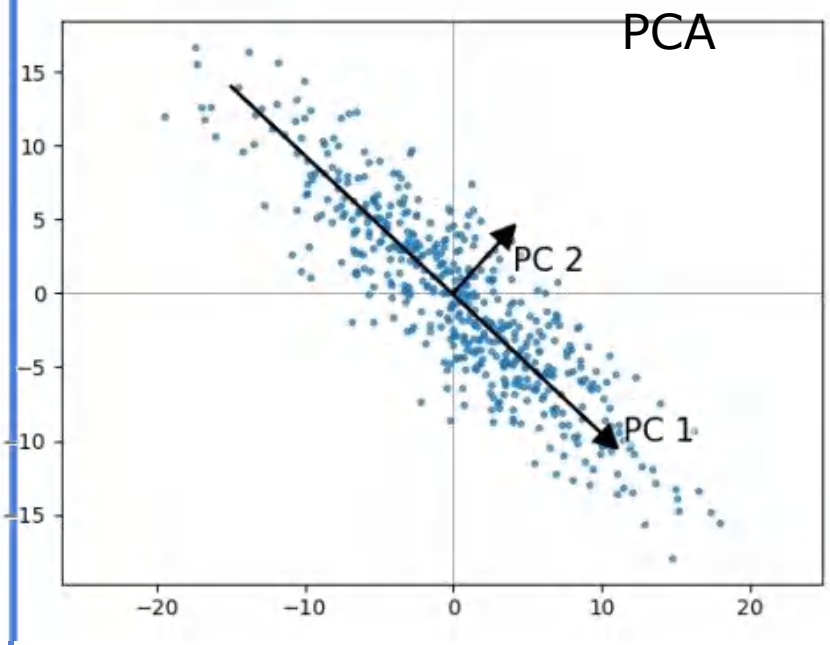


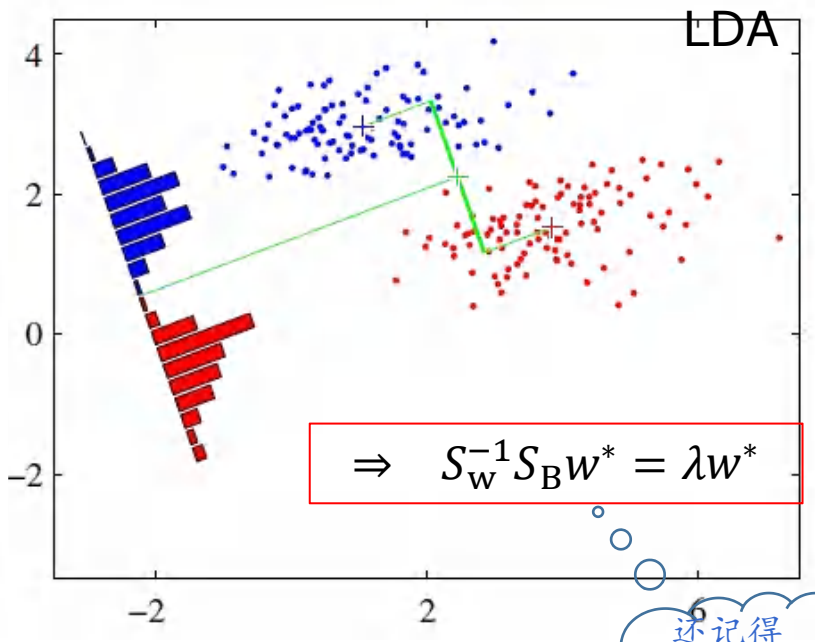
e.g. 将向量z_i=$\begin{pmatrix} 5/\sqrt{2} \\ -1/\sqrt{2} \end{pmatrix}$恢复回（1,0）、（0,1）这组基下



解：$5/\sqrt{2} \cdot \begin{pmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix} + （-1/\sqrt{2}） \cdot \begin{pmatrix} -1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix} = \begin{pmatrix} 3 \\ 2 \end{pmatrix}$

7 PCA与LDA的比较



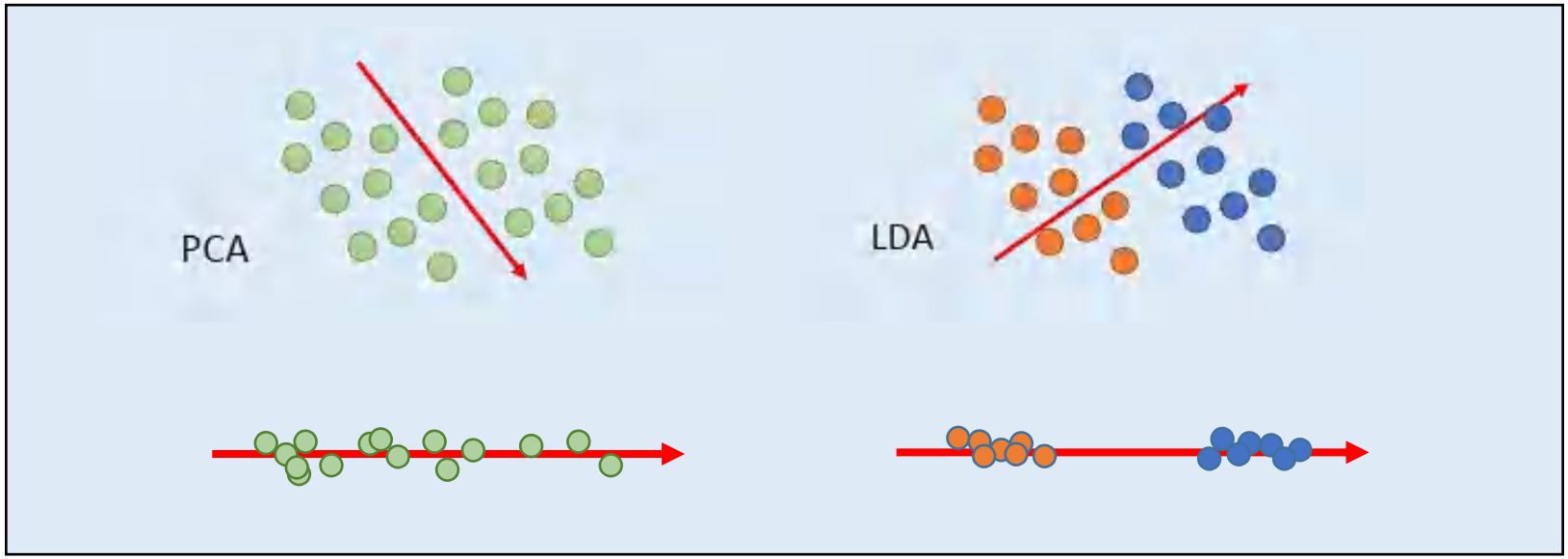


还记得这个式子吗？

- LDA是有监督的降维方法，保持了类信息。PCA是无监督的
- LDA降维最多降到类别数k-1的维数，PCA没有这个限制
- LDA更依赖均值，如果样本信息更依赖方差的话，效果将没有PCA好

主成分分析

提问：绘制一个二维数据集示意图，让PCA和LDA投影效果都好，或者都不好



多维标定

- **多维标定（Multiple Dimensional Scaling, MDS）**是一种经典的数据降维方法。该方法降维的核心思想是：寻找一个低维子空间，样本在此空间的距离与原始空间中样本间距离保持不变

假定有m个样本，在原始空间中的距离矩阵为$D \in \Re^{m \times m}$，其第i行j列的元素$dist_{ij}$为样本x_i到x_j的距离。目标是获得样本在$d'(d' \leq d)$维空间中的表示$Z \in \Re^{d' \times m}$，新样本间的欧氏距离与原始空间中的距离相等或相近，即

$$\left\| z_i - z_j \right\| = dist_{ij}$$

- 设降维后新样本距离与原样本距离尽可能接近（不严格相等）可取新样本内积矩阵最大的d'个特征值构成对角阵 $\tilde{\Lambda} = diag(\lambda_1, \lambda_2, \ldots \lambda_{d'})$，令$\tilde{V}$表示对应的特征向量矩阵，则$Z$可表达为：

$$Z = \tilde{\Lambda}^{\frac{1}{2}} \tilde{V}^T \in \Re^{d' \times m}$$

MDS的计算流程

输入：样本集$\{x_1, x_2, \cdots, x_m\}$和低维空间维数$d'$

1. 计算样本的距离矩阵D；

2. 由距离矩阵D构造降维后新样本的内积矩阵$B = Z^T Z$；

3. 对B矩阵做特征分解；

4. 取最大的d'个特征值构成对角矩阵$\tilde{\Lambda}$，对应的特征向量构成矩阵$\tilde{V}$

输出：新样本矩阵 $Z = \tilde{\Lambda}^{\frac{1}{2}} \tilde{V}^T \in \Re^{d' \times m}$

第七讲 维度归约

1 主成分分析（PCA）

2 自编码器（Autoencoder）

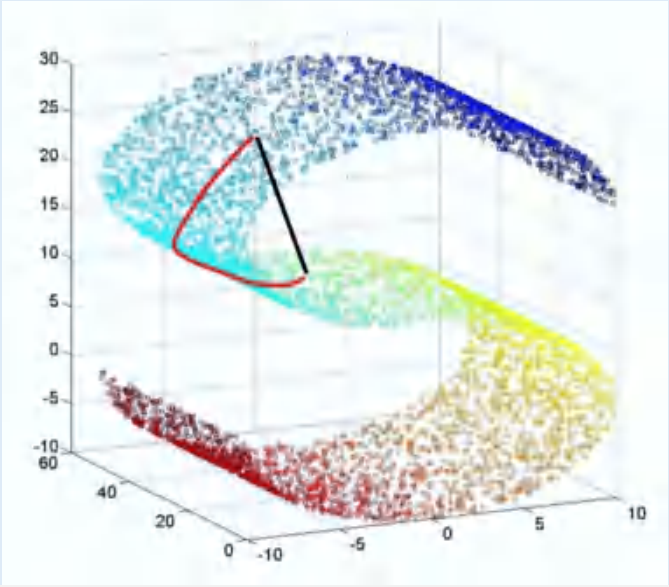
3 多维标定（MDS）

4 流形学习简介

流形学习

- 流形学习 (Manifold Learning)：假设数据是均匀采样于一个高维欧氏空间中的低维流形，流形学习就是从高维采样数据中恢复低维流形结构，即找到高维空间中的低维流形，并求出相应的嵌入映射，以实现维数约简或者数据可视化

- “流形”是局部与欧氏空间同胚的空间。它在**局部具有欧氏空间的性质**，能用欧氏距离来进行距离计算。若低维流形嵌入到高维空间中，则数据样本在高维空间的分布虽然复杂，但在局部上仍具有欧氏空间的性质。因此，可以**在局部建立**降维映射，然后再设法将局部映射关系推广到全局



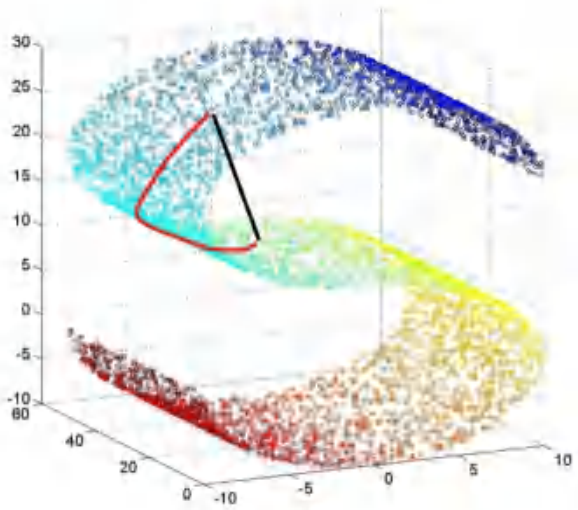
流形上的测地线距离与高维空间直线距离

（图片来自《西瓜书》）

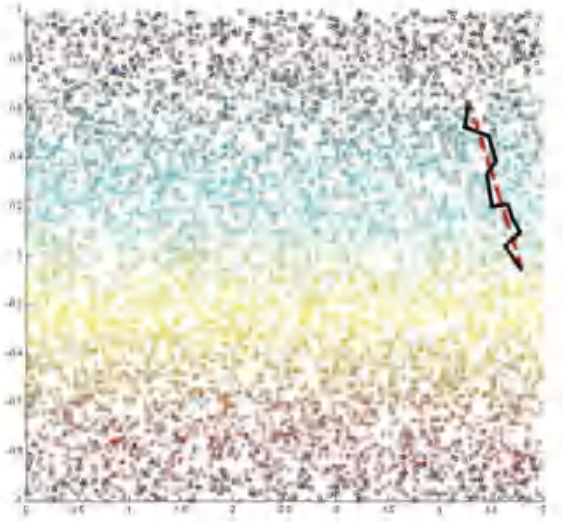
1 等度量映射

- 低维流形嵌入到高维空间之后，直接在高维空间中计算直线距离是不准确的，因为高维空间中直线距离在低维嵌入流形上不可达。因而采用“测地线”（geodesic）距离作为低维嵌入流形上两点间的本真距离
- 保持测地线距离不变的降维称为等度量映射（Isometric Mapping，Isomap）降维，是一种经典的流形学习方法

- 因为是低维嵌入流形，故而邻近的样本间可以计算欧式距离。则基于最短路径算法可计算任意样本间的测地线距离



(a) 测地线距离与高维直线距离



(b) 测地线距离与近邻距离

- 常用的两点间最短路径算法：Dijkstra算法 → 《数据结构》

（图片来自《西瓜书》）

- 对每个点基于欧氏距离找出其近邻点，然后建立一个邻接矩阵图。图中近邻点之间存在距离，而非近邻点之间距离为∞。计算两点之间测地线距离的问题，就转变为计算近邻连接图上两点之间的最短路径问题
- 对于**邻接矩阵**的构建，常用的有两种方法：
  - **指定近邻点个数**，如kNN一样选取k个最近的邻居；
  - **指定邻域半径**，距离小于该阈值的被认为是它的近邻点；
- 获得样本两点间距离矩阵D，其第i行j列的元素$dist_{ij}$为样本x_i到x_j的距离，则可调用MDS方法进行降维

ISOMAP的计算流程

输入: 样本集 $D = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_m\}$;
近邻参数 k;
低维空间维数 d'.

过程:

1: **for** $i = 1, 2, \ldots, m$ **do**
2: 确定 $\boldsymbol{x}_i$ 的 k 近邻;
3: $\boldsymbol{x}_i$ 与 k 近邻点之间的距离设置为欧氏距离, 与其他点的距离设置为无穷大;
4: **end for**
5: 调用最短路径算法计算任意两样本点之间的距离 $\text{dist}(\boldsymbol{x}_i, \boldsymbol{x}_j)$;
6: 将 $\text{dist}(\boldsymbol{x}_i, \boldsymbol{x}_j)$ 作为 MDS 算法的输入;
7: **return** MDS 算法的输出

输出: 样本集 D 在低维空间的投影 $Z = \{\boldsymbol{z}_1, \boldsymbol{z}_2, \ldots, \boldsymbol{z}_m\}$.

第八讲 聚类

1 聚类任务概述

2 划分式聚类方法

3 基于密度的聚类

4 层次化聚类

聚类任务概述

- 聚类（Clustering）是按照某个特定标准（如距离）把一个数据集分割成不同的类或簇，使得同一个簇内的数据对象的相似性尽可能大，同时不在同一个簇中的数据对象的差异性也尽可能地大。聚类后同一类的数据尽可能聚集到一起，不同类数据尽量分离

聚类和分类：

- 聚类(Clustering)：把相似的数据划分到一起，具体划分的时候并不关心这一类的标签，目标就是把相似的数据聚合到一起，是一种无监督学习(Unsupervised Learning)方法；
- 分类(Classification)：把不同的数据划分开，其过程是通过训练数据集获得一个分类器，再通过分类器去预测未知数据，是一种有监督学习(Supervised Learning)方法；

第八讲 聚类

1 聚类任务概述

2 划分式聚类方法

3 基于密度的聚类

4 层次化聚类

划分式聚类

- 划分式聚类（也叫原型聚类）方法需要事先指定簇类的数目或者聚类中心，通过反复迭代，直至最后达到“簇内的点足够近，簇间的点足够远”的目标
- ***k*均值（*k*-means）**方法是一种经典的划分式聚类方法

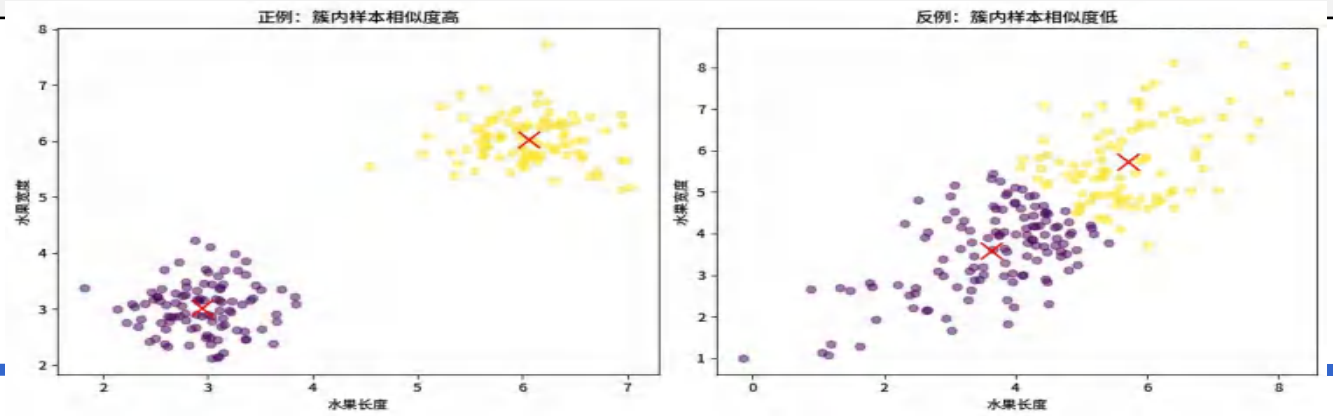
划分式聚类

1 k-means聚类方法

➢ 给定样本集$D = \{\boldsymbol{x_1}, \boldsymbol{x_2}, \cdots, \boldsymbol{x_m}\}$。定义$k$-means算法中针对所有聚类所得簇$C = \{C_1, C_2, \cdots C_k\}$的聚类紧密程度：

$$E = \sum_{i=1}^{k} \sum_{\boldsymbol{x} \in C_i} ||\boldsymbol{x} - \boldsymbol{\mu_i}||_2^2$$

其中$\boldsymbol{\mu_i} = \frac{1}{|C_i|} \sum_{\boldsymbol{x} \in C_i} \boldsymbol{x}$是簇$C_i$的均值向量（又称为参考向量、质心）。$E$刻画了簇内样本围绕簇均值向量的紧密程度，$E$越小簇内样本的相似度越高



1 k-means聚类方法

- 给定样本集$D = \{\boldsymbol{x_1}, \boldsymbol{x_2}, \cdots, \boldsymbol{x_m}\}$。定义$k$-means算法中针对所有聚类所得簇$C = \{C_1, C_2, \cdots C_k\}$的聚类紧密程度：

$$E = \sum_{i=1}^{k} \sum_{\boldsymbol{x} \in C_i} ||\boldsymbol{x} - \boldsymbol{\mu_i}||_2^2$$

其中$\boldsymbol{\mu_i} = \frac{1}{|C_i|} \sum_{\boldsymbol{x} \in C_i} \boldsymbol{x}$是簇$C_i$的均值向量（又称为参考向量、质心）。$E$刻画了簇内样本围绕簇均值向量的紧密程度，$E$越小簇内样本的相似度越高

- k-means的优化目标：

$$C^* = \{C_1, C_2, \cdots C_k\}^* = \underset{C_1, C_2, \cdots C_k}{argmin}\, E$$

- k-means采用了贪心策略，迭代近似求解上述优化问题。其思想与EM方法的思想近似，也被看做是一种广义的EM方法

k-means流程

- 首先随机选出k个样本作为k个簇的初始均值向量，根据距离将m个样本划分到k个簇（样本离哪个质心近就划归哪个簇）

输入：样本集 $D = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_m\}$；
聚类簇数 k.

过程：
1: 从 D 中随机选择 k 个样本作为初始均值向量 $\{\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \ldots, \boldsymbol{\mu}_k\}$
2: **repeat**
3: 令 $C_i = \varnothing \ (1 \leqslant i \leqslant k)$
4: **for** $j = 1, 2, \ldots, m$ **do**
5: 计算样本 $\boldsymbol{x}_j$ 与各均值向量 $\boldsymbol{\mu}_i \ (1 \leqslant i \leqslant k)$ 的距离：$d_{ji} = \|\boldsymbol{x}_j - \boldsymbol{\mu}_i\|_2$；
6: 根据距离最近的均值向量确定 $\boldsymbol{x}_j$ 的簇标记：$\lambda_j = \arg\min_{i \in \{1,2,\ldots,k\}} d_{ji}$；
7: 将样本 $\boldsymbol{x}_j$ 划入相应的簇：$C_{\lambda_j} = C_{\lambda_j} \bigcup \{\boldsymbol{x}_j\}$；
8: **end for**

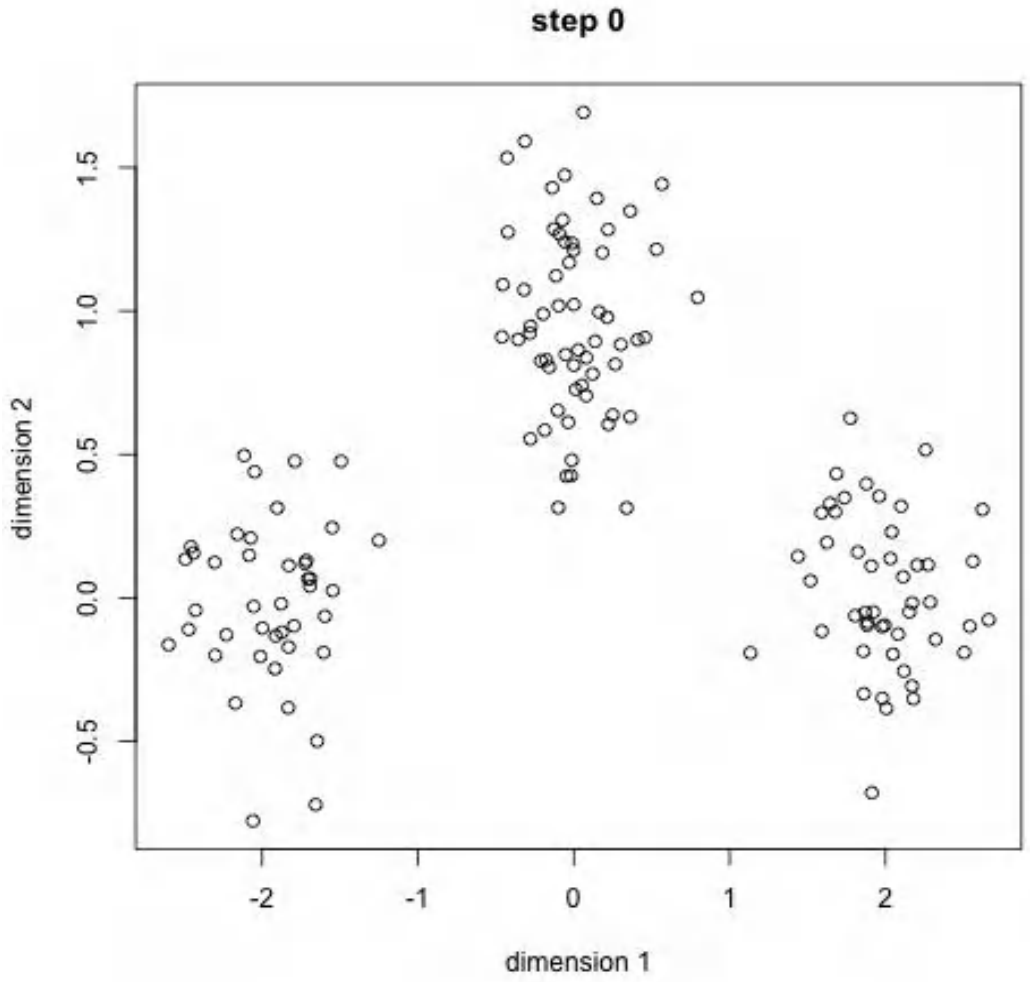
《西瓜书》P203

划分式聚类

- 然后计算k个簇新的均值向量，并计算m个样本到这k个新质心的距离，重新将m个样本划分到k个簇
- 重复上述步骤，直到k个簇新的均值向量不变化

```
    for i = 1, 2, ..., k do
      计算新均值向量: μ'_i = (1/|C_i|) Σ_{x∈C_i} x;
      if μ'_i ≠ μ_i then
        将当前均值向量 μ_i 更新为 μ'_i
      else
        保持当前均值向量不变
      end if
    end for
  until 当前均值向量均未更新
  输出: 簇划分 C = {C_1, C_2, ..., C_k}
```

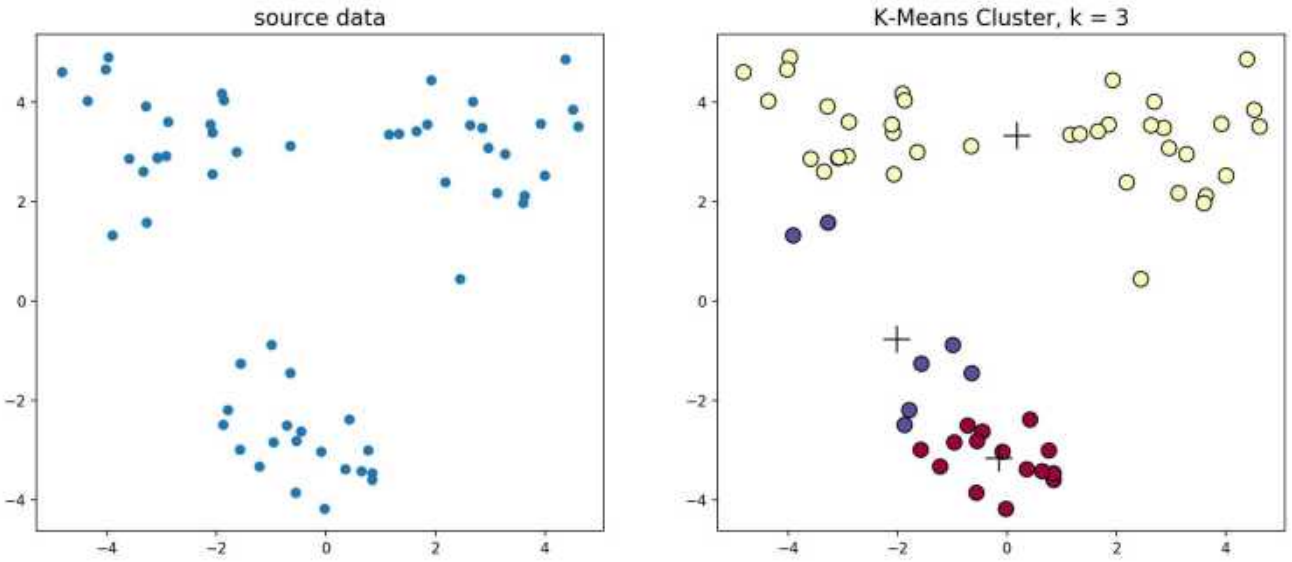
划分式聚类



k均值算法的执行过程（动画）

（图片来自于网络）

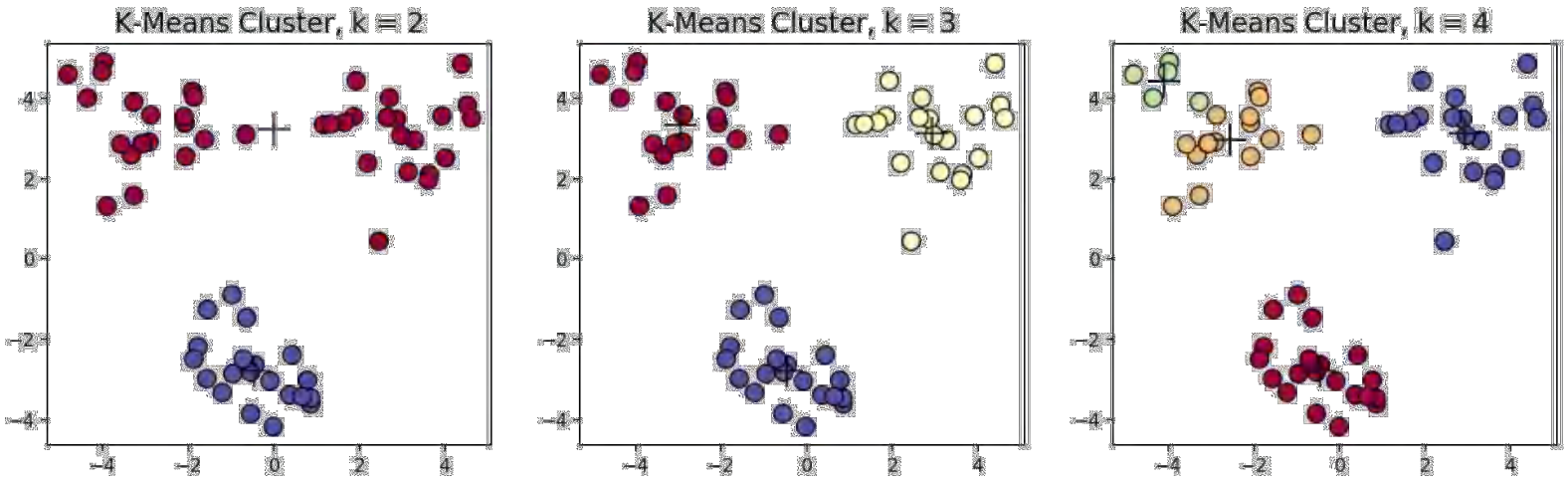
- 经典k-means算法有几个缺点：需要提前确定k值，对初始质心点敏感，对异常数据敏感



初始质心选取不当

（图片来自于网络）

- 经典k-means算法有几个缺点：需要提前确定k值，对初始质心点敏感，对异常数据敏感



k选取不当

（图片来自于网络）

划分式聚类



例：给定含有5个样本的集合 $X=\begin{bmatrix}0 & 0 & 1 & 5 & 5\\ 2 & 0 & 0 & 0 & 2\end{bmatrix}$

试用k均值聚类算法将样本聚到两个类中（k=2）

（1）选择$\boldsymbol{\mu_1^{(0)}}=\boldsymbol{x_1}=(0,2)^T$，$\boldsymbol{\mu_2^{(0)}}=\boldsymbol{x_2}=(0,0)^T$作为初始质心

（2）以$\boldsymbol{\mu_1^{(0)}}$，$\boldsymbol{\mu_2^{(0)}}$为簇$C_1^{(0)}$，$C_2^{(0)}$的中心，计算$\boldsymbol{x_3}=(1,0)^T$，$\boldsymbol{x_4}=(5,0)^T$，$\boldsymbol{x_5}=(5,2)^T$与$\boldsymbol{\mu_1^{(0)}}=(0,2)^T$，$\boldsymbol{\mu_2^{(0)}}=(0,0)^T$的欧氏距离平方

- 对$\boldsymbol{x_3}=(1,0)^T$，$\mathrm{d}\left(\boldsymbol{x_3},\boldsymbol{\mu_1^{(0)}}\right)=5$，$\mathrm{d}\left(\boldsymbol{x_3},\boldsymbol{\mu_2^{(0)}}\right)=1$，将$\boldsymbol{x_3}$分到簇$C_2^{(0)}$
- 对$\boldsymbol{x_4}=(5,0)^T$，$\mathrm{d}\left(\boldsymbol{x_4},\boldsymbol{\mu_1^{(0)}}\right)=29$，$\mathrm{d}\left(\boldsymbol{x_4},\boldsymbol{\mu_2^{(0)}}\right)=25$，将$\boldsymbol{x_4}$分到簇$C_2^{(0)}$
- 对$\boldsymbol{x_5}=(5,2)^T$，$\mathrm{d}\left(\boldsymbol{x_5},\boldsymbol{\mu_1^{(0)}}\right)=25$，$\mathrm{d}\left(\boldsymbol{x_5},\boldsymbol{\mu_2^{(0)}}\right)=29$，将$\boldsymbol{x_5}$分到簇$C_1^{(0)}$

划分式聚类

（3）对簇$C_1^{(1)}=\{\boldsymbol{x_1},\boldsymbol{x_5}\}$，$C_2^{(1)}=\{\boldsymbol{x_2},\boldsymbol{x_3},\boldsymbol{x_4}\}$，计算新质心$\boldsymbol{\mu_1^{(1)}}$，$\boldsymbol{\mu_2^{(1)}}$：

$\boldsymbol{\mu_1^{(1)}}=(2.5,2.0)^T$，$\boldsymbol{\mu_2^{(1)}}=(2,0)^T$

（4）重复步骤（2）和步骤（3）。将$\boldsymbol{x_1}$分到簇$C_1^{(1)}$，将$\boldsymbol{x_2}$分到簇$C_2^{(1)}$，$\boldsymbol{x_3}$分到簇$C_2^{(1)}$，$\boldsymbol{x_4}$分到簇$C_2^{(1)}$，$\boldsymbol{x_5}$分到簇$C_1^{(1)}$，得到新的簇$C_1^{(2)}=\{\boldsymbol{x_1},\boldsymbol{x_5}\}$，$C_2^{(2)}=\{\boldsymbol{x_2},\boldsymbol{x_3},\boldsymbol{x_4}\}$。

（5）由于得到的新的簇没有变换，聚类停止。得到聚类结果：

$$C_1^*=\{x_1,x_5\},\ C_2^*=\{x_2,x_3,x_4\}$$

第八讲 聚类

1 聚类任务概述

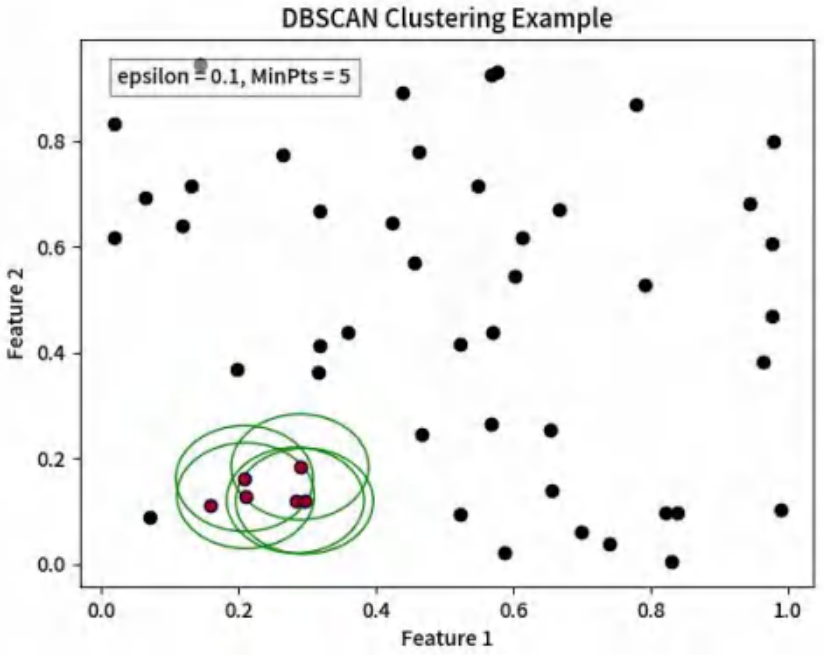
2 划分式聚类方法

3 基于密度的聚类

4 层次化聚类

1 DBSCAN聚类密度

- DBSCAN是一种基于密度的聚类算法（density-based clustering），这类算法假设聚类结构能通过样本分布的紧密程度确定。DBSCAN基于一组**“邻域”参数（ϵ，$MinPts$）**来刻画样本分布的紧密程度

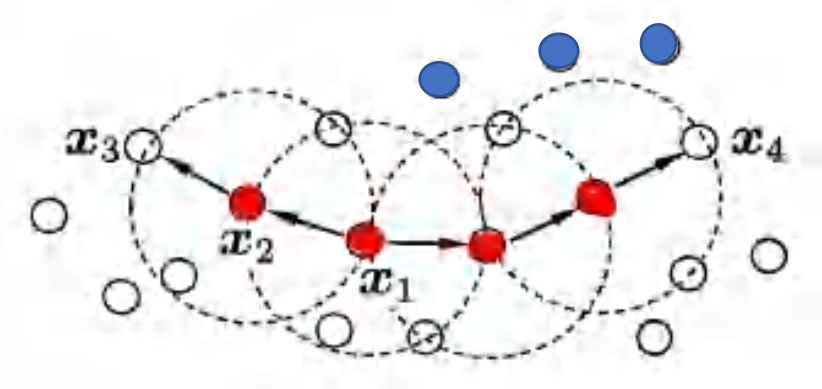


基于密度的聚类

➢ 给定样本集$D = \{\boldsymbol{x_1}, \boldsymbol{x_2}, \cdots, \boldsymbol{x_m}\}$，定义如下几个概念：

- ϵ-邻域：对$\boldsymbol{x_j} \in D$，其ϵ-邻域包含样本集D中与$\boldsymbol{x_j}$的距离不大于ϵ的样本
- $MinPts$：当一个点的邻域（以该点为圆心、ϵ-邻域半径）内包含的点数（包含该点自身）不少于 MinPts
- 核心对象：若$\boldsymbol{x_j}$的ϵ-邻域至少包含$MinPts$个样本，则$\boldsymbol{x_j}$是一个核心对象
- 密度直达：若$\boldsymbol{x_j}$位于$\boldsymbol{x_i}$的ϵ-邻域中，且$\boldsymbol{x_i}$是核心对象，则$\boldsymbol{x_j}$由 $\boldsymbol{x_i}$密度直达
- 密度可达：对$\boldsymbol{x_i}$与$\boldsymbol{x_j}$，若存在样本序列$\boldsymbol{p_1}, \boldsymbol{p_2}, \cdots, \boldsymbol{p_n}$，其中$\boldsymbol{p_1} = \boldsymbol{x_i}$，$\boldsymbol{p_n} = \boldsymbol{x_j}$且$\boldsymbol{p_{i+1}}$由$\boldsymbol{p_i}$密度直达，则称$\boldsymbol{x_j}$由$\boldsymbol{x_i}$密度可达，密度可达有方向
- 密度相连：对$\boldsymbol{x}_i$与$\boldsymbol{x}_j$，若存在$\boldsymbol{x}_k$使得$\boldsymbol{x}_i$与$\boldsymbol{x}_j$均由$\boldsymbol{x}_k$密度可达，则称$\boldsymbol{x}_i$与$\boldsymbol{x}_j$密度相连

基于密度的聚类



- x_1到x_2是密度直达； x_4 由x_1 密度直达； x_3 由x_2密度直达
- x_3由x_1密度可达；
- x_4由x_2密度可达（通过 x_1）
- x_3与x_4是密度相连

ϵ描述了某一样本的邻域距离阈值，$MinPts$描述了某一样本的距离为ϵ的邻域中样本个数的阈值

红色点均为核心对象，$MinPts = 3$

蓝色点均为异常点（噪音点），既不是核心点也不是边界点

- 基于上述概念，DBSCAN将“簇”定义为：由密度可达关系导出的最大的密度相连样本集合。给定邻域参数（ϵ，$MinPts$），簇$C \subseteq D$是满足以下性质的非空样本子集：

  - 连续性：$\boldsymbol{x_i} \in C$，$\boldsymbol{x_j} \in C \Rightarrow \boldsymbol{x_i}$与$\boldsymbol{x_j}$密度相连
  - 最大性：$\boldsymbol{x_i} \in C$，$\boldsymbol{x_j}$由$\boldsymbol{x_i}$密度可达$\Rightarrow \boldsymbol{x_j} \in C$

- 一个DBSCAN的簇里面可以有一个或者多个核心对象:

  - 如果只有一个核心对象，则簇里其他的非核心对象样本都在这个核心对象的ϵ –邻域里；
  - 如果有多个核心对象，则簇里的任意一个核心对象的ϵ –邻域中一定有一个其他的核心对象，否则这两个核心对象无法密度可达。这些核心对象的ϵ –邻域里所有的样本的集合组成的一个DBSCAN聚类簇

2 DBSCAN聚类步骤

- DBSCAN算法初始任意选择一个核心对象，然后**找到所有这个核心对象能够密度可达的样本集合，即为一个聚类簇。**接着继续选择另一个没有类别的核心对象去寻找密度可达的样本集合，这样就得到另一个聚类簇。一直运行到所有核心对象都有类别为止

- 从任意核心对象出发，基于密度可达性扩展聚类。完成一簇后再挑选未被选中的核心点重复过程，直至遍历完所有核心点

DBSCAN流程

- 根据给定的邻域参数(ϵ , $MinPts$)找出所有的核心对象

输入：样本集 $D = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_m\}$;
　　邻域参数 $(\epsilon, MinPts)$.

过程：

```
初始化核心对象集合: Ω = ∅
for j = 1, 2, ..., m do
    确定样本 x_j 的 ε-邻域 N_ε(x_j);
    if |N_ε(x_j)| ≥ MinPts then
        将样本 x_j 加入核心对象集合: Ω = Ω ∪ {x_j}
    end if
end for
```

先确定核心对象

《西瓜书》P213

基于密度的聚类

- 以任意核心点为出发点，找出由其**密度可达**的样本生成聚类簇，直到所有核心对象均被访问过为止

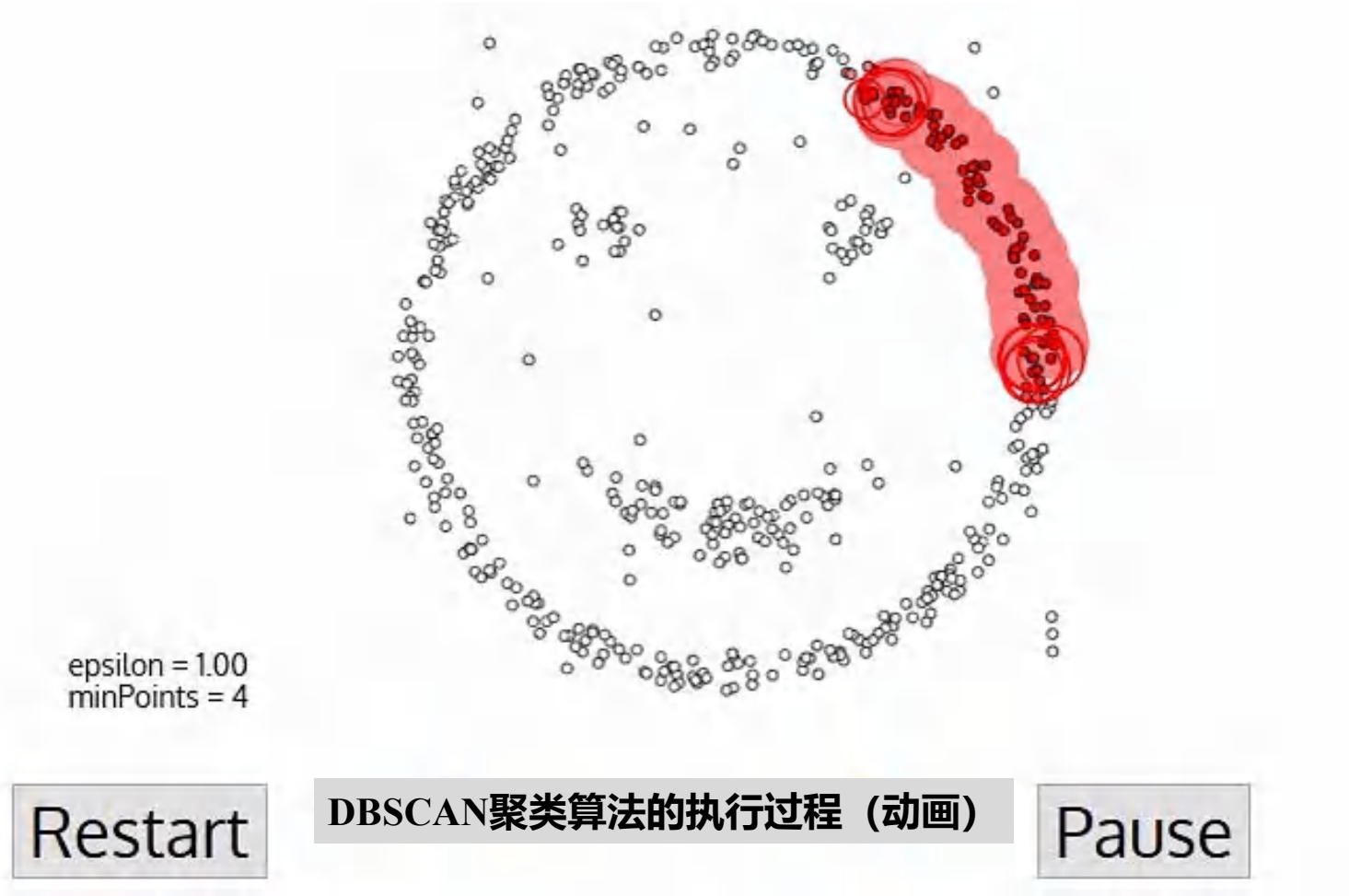
```
初始化聚类簇数: k = 0
初始化未访问样本集合: Γ = D
while Ω ≠ ∅ do
    记录当前未访问样本集合: Γ_old = Γ;
    随机选取一个核心对象 o ∈ Ω, 初始化队列 Q =< o >;
    Γ = Γ \ {o};
    while Q ≠ ∅ do
        取出队列 Q 中的首个样本 q;
        if |N_ε(q)| ⩾ MinPts then
            令 Δ = N_ε(q) ⋂ Γ;
            将 Δ 中的样本加入队列 Q;
            Γ = Γ \ Δ;
        end if
    end while
    k = k + 1, 生成聚类簇 C_k = Γ_old \ Γ;
    Ω = Ω \ C_k
end while
输出: 簇划分 C = {C_1, C_2, ..., C_k}
```

判断是否为核心对象

若是，则将该核心对象领域内的密度直达样本加入簇

在未被聚类的核心对象中选一个为种子，继续执行算法

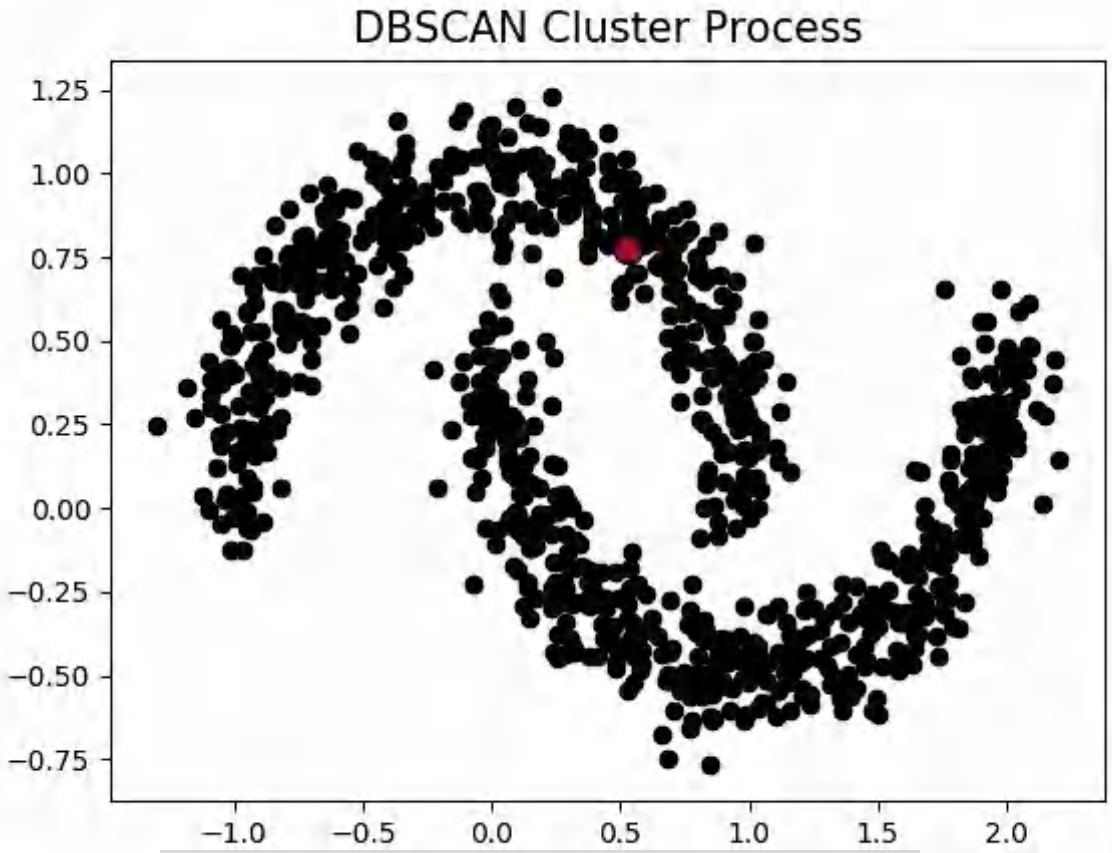
基于密度的聚类



DBSCAN聚类算法的执行过程（动画）

（图片来自于网络）

基于密度的聚类

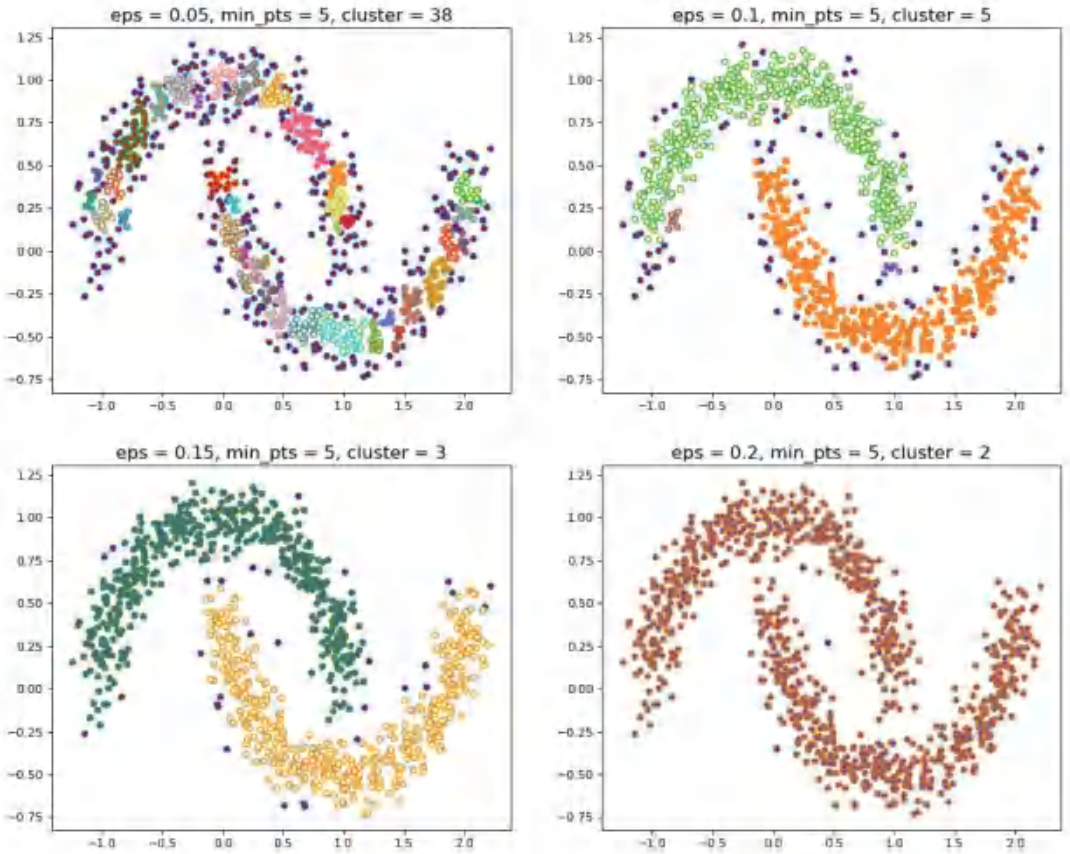


DBSCAN聚类算法的执行过程（动画）

（图片来自于网络）

3 DBSCAN聚类分析

提问：不同参数对聚类结果造成什么影响？

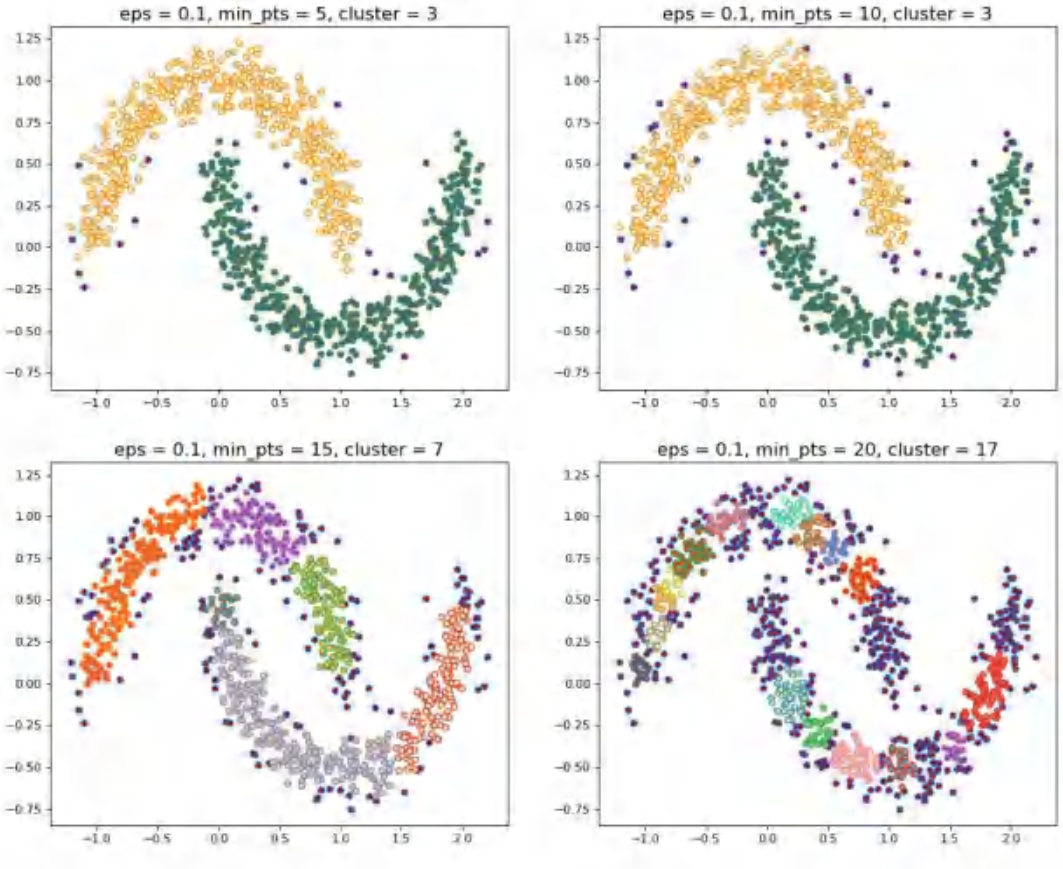


不同ϵ 值的聚类结果

（图片来自于网络）

基于密度的聚类

提问：不同参数对聚类结果造成什么影响？



不同MinPts值的聚类结果

（图片来自于网络）

- 一般来说，DBSCAN算法有以下几个特点：
  - 需要提前确定ϵ 和MinPts值
  - 不需要提前设置聚类类别的个数
  - 对初值选取不敏感，对噪声不敏感
  - 对密度不均的数据聚合效果不好

第九讲 神经网络

1 神经网络基本概念

2 多层前馈神经网络

3 BP神经网络

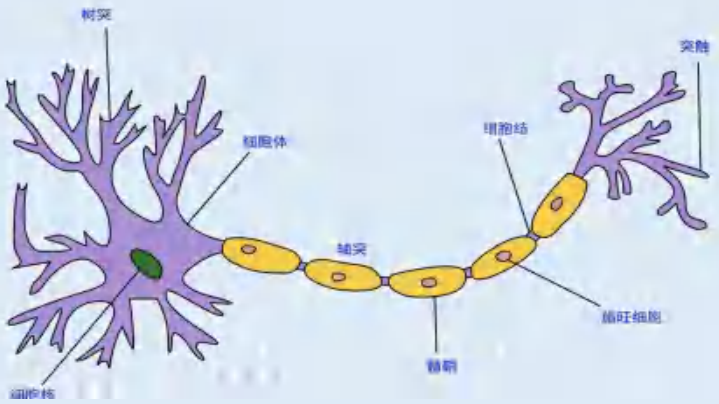
1 神经元模型

- 生物神经元有**兴奋**和**抑制**两种状态，抑制状态的神经元由树突和细胞体接收传来的兴奋电位，当输入兴奋总量超过阈值，神经元被激发进入兴奋状态，产生输出脉冲，由突触传递给其它神经元

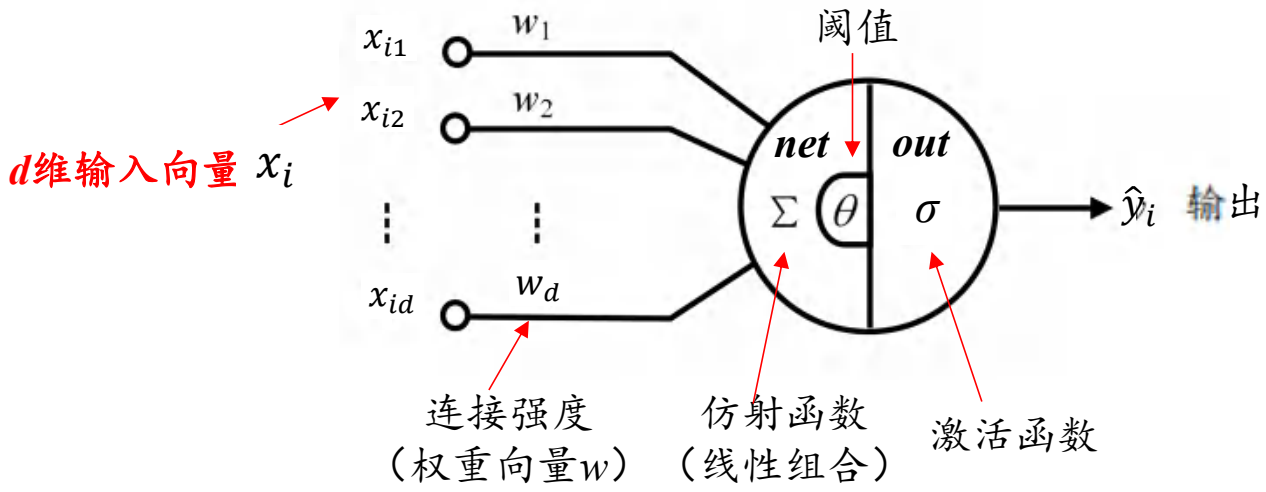
- 科学研究发现：人类大脑皮质包含大约140-160亿神经元， 小脑包含大约550-700亿神经元，加起来大约是860亿，并且一个立方毫米的神经元可以产生超过1000TB的数据

生物神经元的基本组成:

1. 细胞体
2. 树突
3. 轴突
4. 突触



➢ 通过对生物神经元模拟，得到人工神经元（artificial neuron）结构：

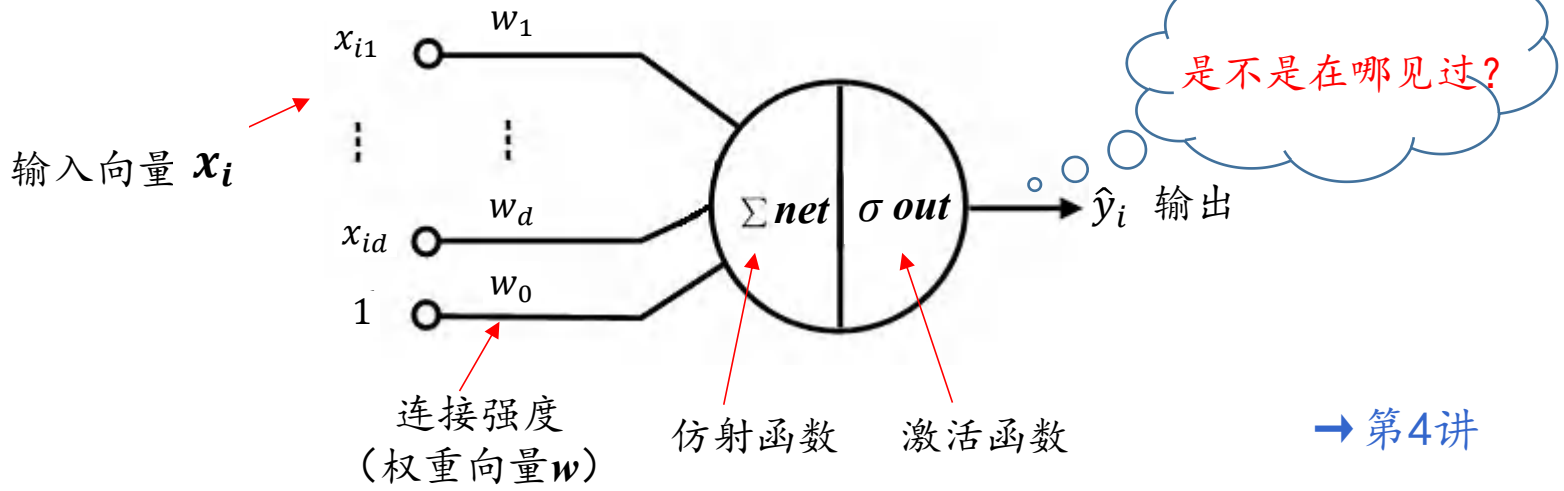


人工神经网络的基本神经元

➢ 对于样本x_i，单个神经元的输出$\hat{y}_i$为

$$\hat{y}_i = \sigma\left(\sum_{l=1}^{d} w_l x_{il} - \theta\right) \longrightarrow \hat{y}_i = \sigma(x^T w + w_0)$$

➢ 因此神经元模型又可表示为：



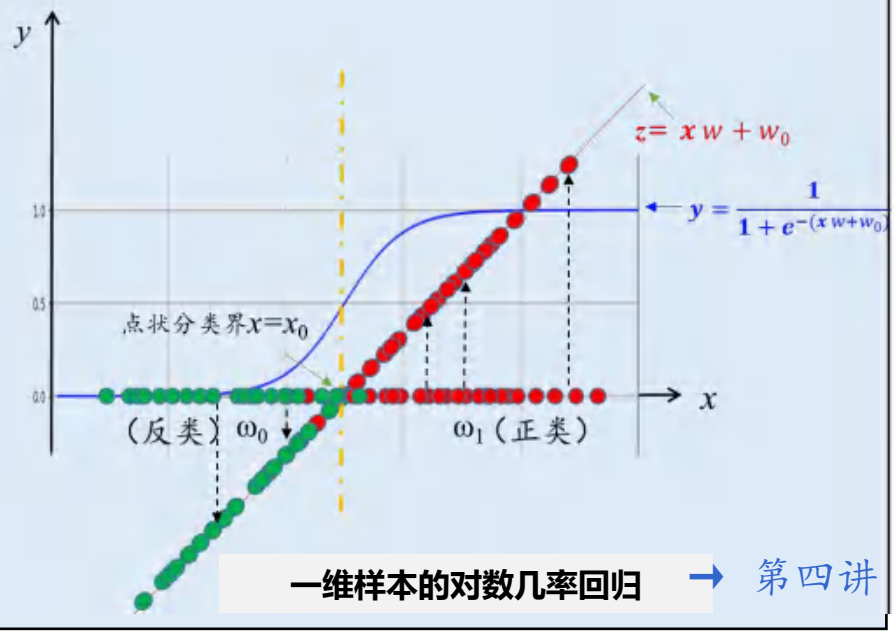
→ 第4讲

2 激活函数

- **激活函数（activation function）**是人工神经元里体现**非线性**的重要函数，实现输入信号到输出信号的非线性转换
- 激活函数都是**非线性函数**

- 对数几率回归中使用的logistic函数就是一个典型的激活函数

$$y = \frac{1}{1+e^{-z}} = \frac{1}{1+e^{-(x^T w+w_0)}}$$



一维样本的对数几率回归 → 第四讲

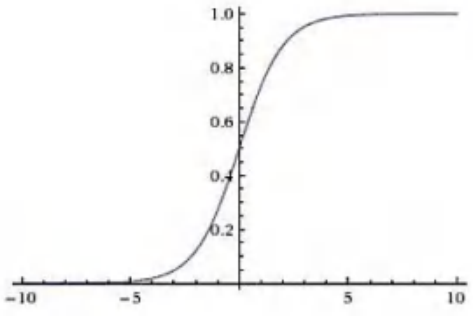
- 常见的激活函数及其导函数：

激活函数一般要求是**连续可导**的（允许少数点上不可导），这与神经网络反向传播训练的方法有关

激活函数	形式	导数形式
Sigmoid	$f(x)=\frac{1}{1+e^{-x}}$	$f'(x)=f(x)(1-f(x))$
Tanh	$f(x)=tanh(x)=\frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$	$f'(x)=1-(f(x))^2$
ReLU	$f(x)=max(0,x)=\begin{cases}0 & x\leq 0\\ x & x>0\end{cases}$	$f'(x)=\begin{cases}0 & x\leq 0\\ 1 & x>0\end{cases}$
Leaky ReLU	$f(x)=max(0.001x,x)=\begin{cases}0.001x & x\leq 0\\ x & x>0\end{cases}a$	$f(x)=max(0.001x,x)=\begin{cases}0.001 & x\leq 0\\ 1 & x>0\end{cases}$
PReLU	$f(x)=max(\alpha x,x)=\begin{cases}\alpha x & x\leq 0\\ x & x>0\end{cases}$	$f(x)=max(\alpha x,x)=\begin{cases}\alpha & x\leq 0\\ 1 & x>0\end{cases}$
RReLU	PReLU中的 α 随机取值	
ELU	$f(x)=\begin{cases}x & x\geq 0\\ \alpha(e^{x}-1) & x<0\end{cases}$	$f(x)=\begin{cases}1 & x\geq 0\\ \alpha e^{x} & x<0\end{cases}$
Maxout	$f(x)=max(w_1^Tx+b_1,w_2^Tx+b_2)$	$f(x)=max(w_1,w_2)$

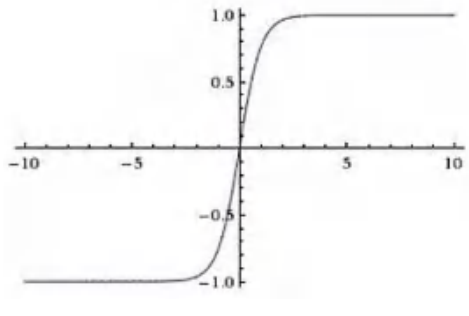
Sigmoid函数
（Logistic函数）

Sigmoid: $y = \frac{1}{1+e^{-x}}$



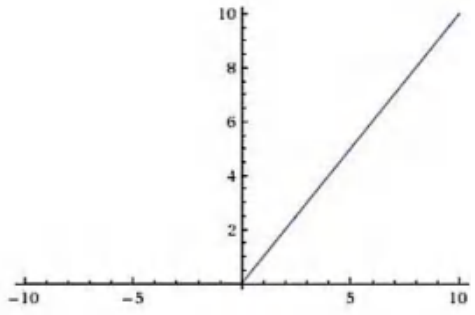
Tanh函数
（双曲正切函数）

Tanh: $y = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$



ReLU函数
（修正线性单元函数）

$Relu: y = max(0, x)$



修正线性单元
（Rectified Linear Unit，ReLU）

饱和激活函数

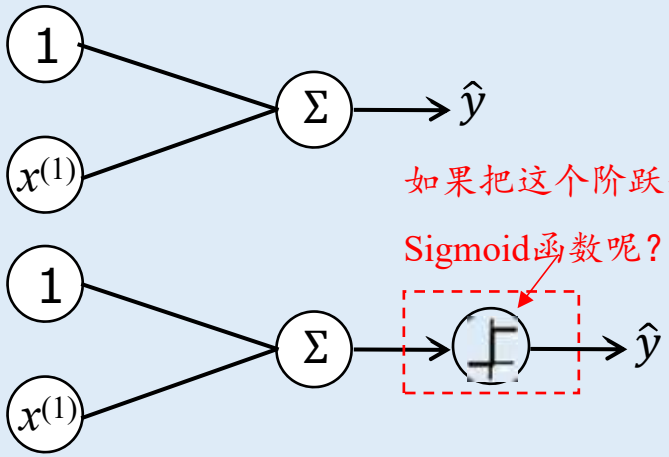
非饱和激活函数

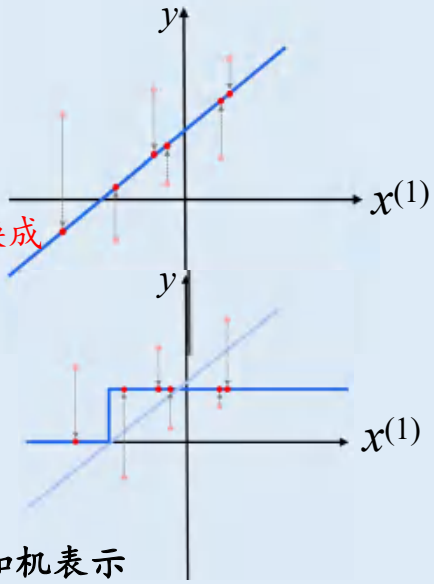
对于函数$f(x)$，若$x \to -\infty$时，其导数$f'(x) \to 0$称为左饱和；若$x \to +\infty$时，其导数$f'(x) \to 0$称为右饱和，当同时满足时称为两端饱和

3 神经网络基础结构

- 形如$\hat{y}_i = \sigma(\boldsymbol{x}^T w + w_0)$的网络也称为感知机（perceptron），是神经网络的基础结构

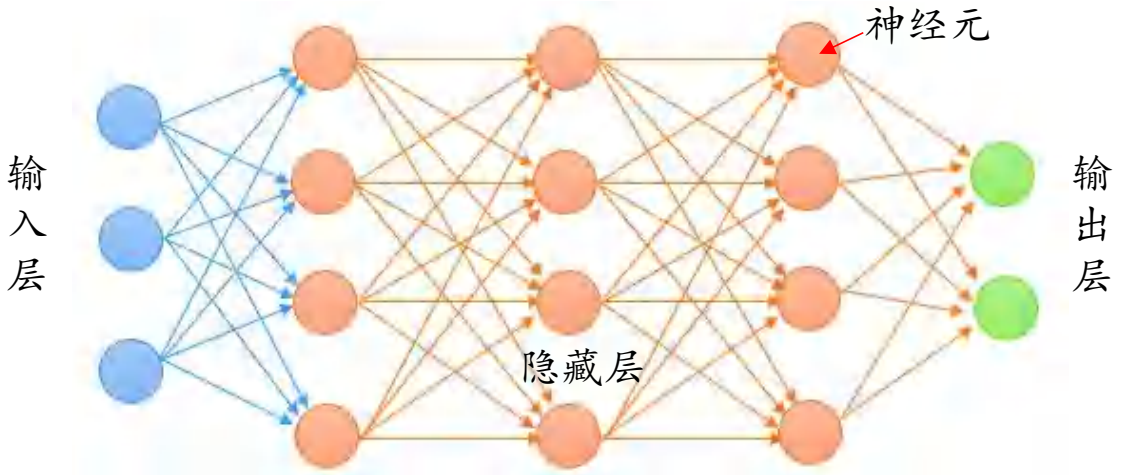
感知机由***Rosenblatt***于1957年提出，学习的目标是通过改变权值使神经网络由给定的输入得到给定的输出





一维回归和分类的感知机表示

- ANNs是一种模仿生物神经网络行为特征，进行分布式并行信息处理的机器学习模型。这种网络依靠模型内部的复杂程度，通过调整人工神经元之间的连接关系和强度，达到处理信息的目的



人工神经网络的典型结构

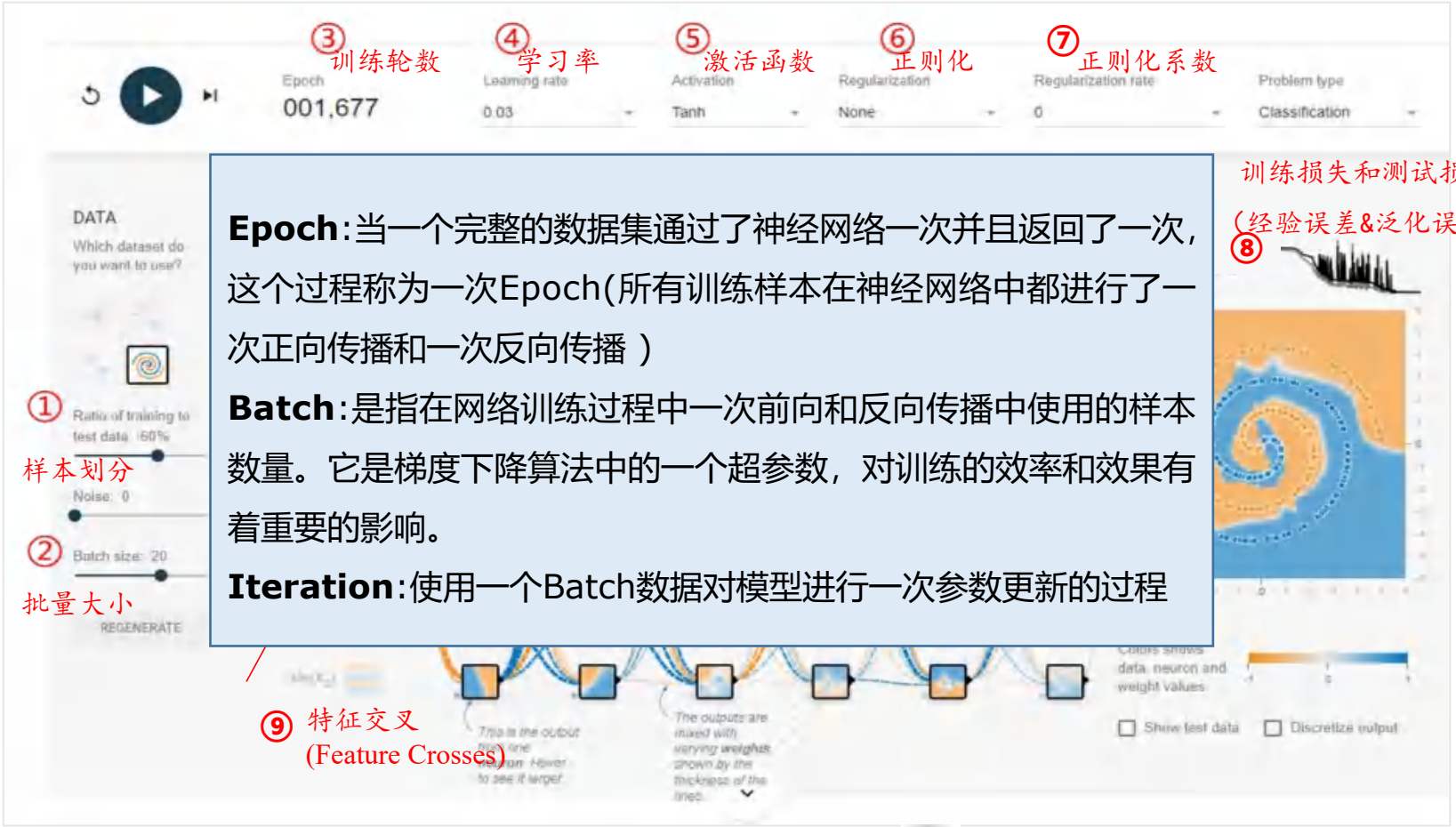
提问：ANNs模型的参数是什么？

- 如果不用激活函数，每一层节点的输入都是上层输出的线性函数。网络的逼近能力也相当有限。

  试推导：无论神经网络有多少隐藏层，若无非线性激活函数，网络都可以合并为单层

多层前馈神经网络

➢ 利用前馈神经网络构处理分类和回归问题



Epoch:当一个完整的数据集通过了神经网络一次并且返回了一次，这个过程称为一次Epoch(所有训练样本在神经网络中都进行了一次正向传播和一次反向传播)

Batch:是指在网络训练过程中一次前向和反向传播中使用的样本数量。它是梯度下降算法中的一个超参数，对训练的效率和效果有着重要的影响。

Iteration:使用一个Batch数据对模型进行一次参数更新的过程

多层神经网络 By ®Tensorflow

第九讲 神经网络

导航工程 2025

1 神经网络基本概念

2 多层前馈神经网络

3 BP神经网络

WUHAN UNIVERSITY

BP神经网络

- **反向传播（Back-Propagating）**思想在70年代就由芬兰学者Linnainmaa提出。Werbos在1974年的哈佛大学博士论文中明确提出通过反向传播算法来训练人工神经网络。人们更为熟知的反向传播领域先驱是Hinton、Rumelhart和Williams。他们于1986年在Nature发表论文，详细介绍了名为BP神经网络技术。一直以来，反向传播被认为是神经网联和深度学习的根基，也是人工智能最重要的算法思想之一。



D. Rumelhart



G. Hinton



S.Linnainmaa



Paul J. Werbos

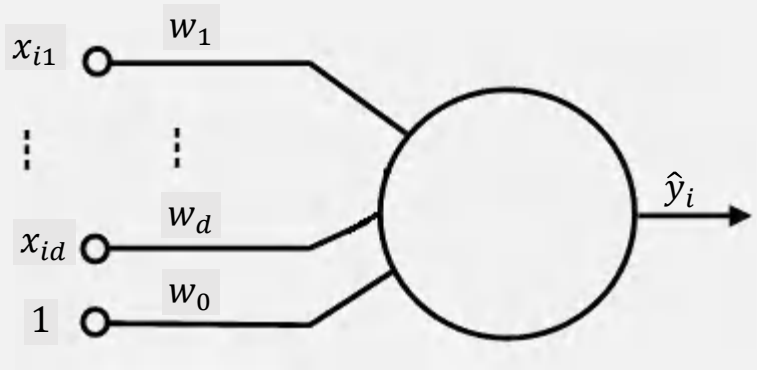
1 学习规则

- 1957年，美国学者*Rosenblatt* 首次提出感知机。**感知机规则**根据网络的输出误差对神经元连接权重进行修正：
  - 若神经元实际输出比期望输出大，则减少输入为正的连接的权重，增大所有输入为负的连接的权重；
  - 反之，则增大所有输入为正的连接权的权重，减少所有输入为负的连接权的权重

$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} + \eta(y_i - \hat{y}_i)\ \boldsymbol{x_i}$$

1 学习规则

- 设$y_i \in \{-1,1\}$为神经元i的期望输出（监督标记），$\hat{y}_i$为神经元的实际输出，$y_i - \hat{y}_i$为误差信号或学习信号。神经元i到神经元j的连接权为w_{ij}。则感知器（*Perceptron*）学习规则可以表示为：



➢ 对于单层感知机(线性分类器)：

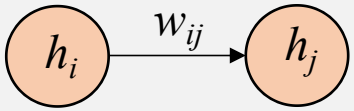
$$\hat{y}_i = \text{sign}(\boldsymbol{w}^T \boldsymbol{x_i} + w_0)$$

➢ 权重更新规则：

$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} + \eta(y_i - \hat{y}_i)\ \boldsymbol{x_i}$$

BP神经网络

- 1986年，认知心理学家*McClelland*和*Rumelhart*在神经网络训练中引入了**δ规则**，也称为连续感知机学习规则，以**目标（损失函数）的负梯度方向对参数进行调整**



$$w_{ij}^{(t+1)} = w_{ij}^{(t)} - \eta \cdot \frac{\partial Loss}{\partial w_{ij}^{(t)}}$$

$$= w_{ij}^{(t)} - \eta \cdot \delta_{h_j} \cdot out_{h_i}$$

损失的梯度在w_{ij}方向的分量

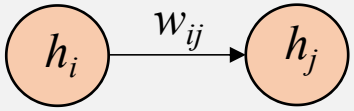
神经元h_j的误差信号

神经元h_i的输出

- 把δ规则写成神经元h_i的输出与神经元h_j的误差信号乘积的形式，方便通过误差的反向传播实现梯度下降法运算

接下来重点推导为什么：$\frac{\partial Loss}{\partial w_{ij}^{(t)}} = \delta_{h_j} \cdot out_{h_i}$以及这样做的好处↓

BP神经网络



损失的梯度在 w_{ij} 方向的分量

$$w_{ij}^{(t+1)} = w_{ij}^{(t)} - \eta \cdot \frac{\partial Loss}{\partial w_{ij}^{(t)}}$$

$$= w_{ij}^{(t)} - \eta \cdot \delta_{h_j} \cdot out_{h_i}$$

δ_{h_j}：神经元h_j的误差信号　　out_{h_i}：神经元h_i的输出

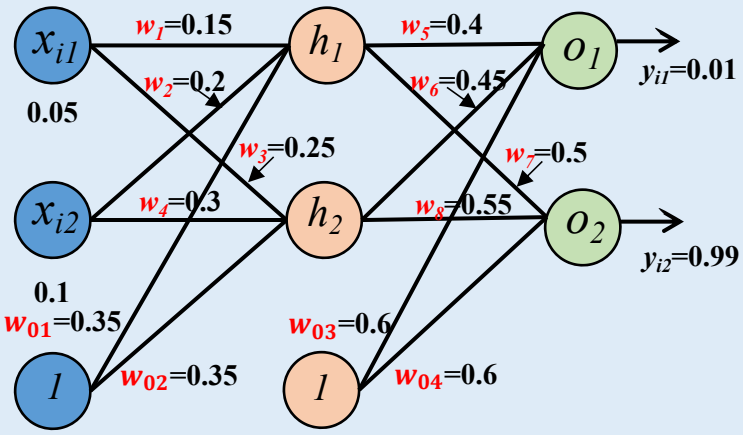
根据链式求导法则 $\frac{\partial Loss}{\partial w_{ij}^{(t)}} = \frac{\partial Loss}{\partial net_{h_j}} \cdot \frac{\partial net_{h_j}}{\partial w_{ij}^{(t)}}$，其中 $net_{h_j} = \sum_i w_{ij} \cdot out_{h_i}$

- 计算 $\frac{\partial net_{h_j}}{\partial w_{ij}^{(t)}} = \frac{\partial \sum_i w_{ij} \cdot out_{h_i}}{\partial w_{ij}^{(t)}} = out_{h_i}$
- 定义δ_{h_j}，将$\frac{\partial Loss}{\partial net_{h_j}}$定义为神经元 h_j的误差信号δ_{h_j}，它反映了损失函数对神经元 h_j 输入的敏感程度；
- 若$|\delta_{h_j}|$大，说明 net_{h_j} 一个小改变会引起 $Loss$较大改变，损失函数对 h_j 输入敏感；若 $|\delta_{h_j}|$小，net_{h_j} 改变对$Loss$ 影响小，损失函数对 h_j 输入不敏感

2 反向误差传播算法

- BP算法（Back-Propagation Training Algorithm）是神经网络更新权重参数的重要方法

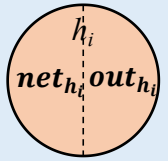
e.g. 设神经网络结构即初始权重如下，用BP算法进行各边权重的更新



- 激活函数：Sigmoid函数 $y = \frac{1}{1+e^{-x}}$
- 损失函数：平方误差函数（1/2作系数）

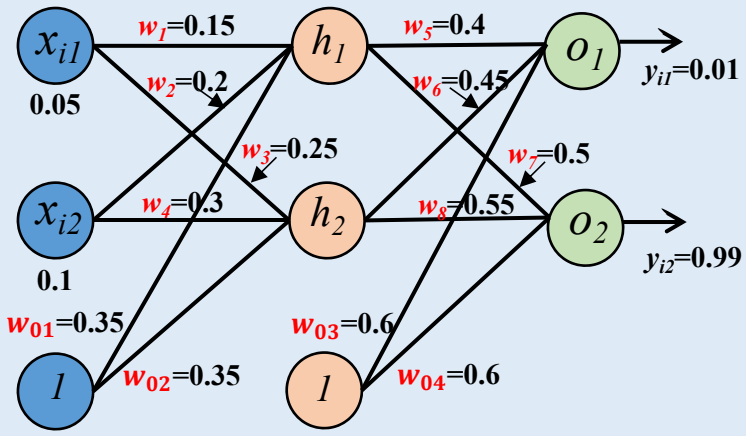
$$Loss = \frac{1}{2}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- 样本$\boldsymbol{x_i} = \begin{pmatrix}0.05\\0.1\end{pmatrix}$，标记值$\boldsymbol{y_i} = \begin{pmatrix}0.01\\0.99\end{pmatrix}$
- 学习率η=0.5
- 对于结点h_i:
  - 仿射函数：$\boldsymbol{net_{h_i}}$
  - 激活函数：$\boldsymbol{out_{h_i}}$



[1].例题来自于Matt Mazur的“A Step by Step Backpropagation Example”

- 网络前向传播1次：



$$net_{h_1} = w_1 x_{i1} + w_2 x_{i2} + w_{01}$$
$$= 0.15\times0.05+0.2\times0.1+0.35=0.3775$$

$$out_{h_1} = \frac{1}{1+e^{-net_{h_1}}} = \frac{1}{1+e^{-0.3775}} = 0.5933$$

$$net_{h_2} = w_3 x_{i1} + w_4 x_{i2} + w_{02}$$
$$= 0.25\times0.05+0.3\times0.1+0.35=0.3925$$

$$out_{h_2} = \frac{1}{1+e^{-net_{h_2}}} = 0.5967$$

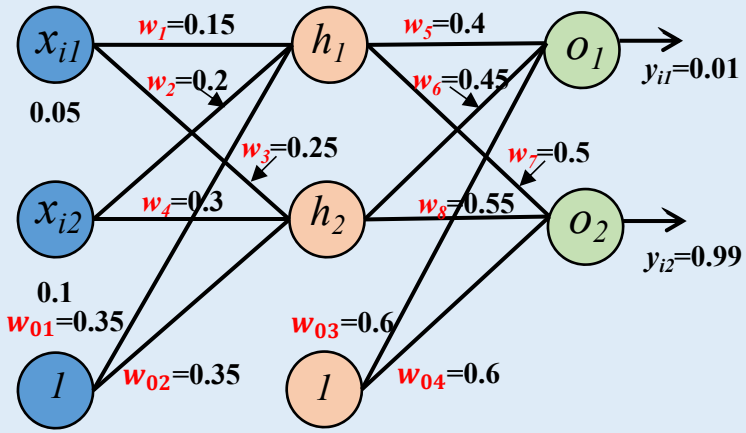
$$net_{o_1} = w_5 \cdot out_{h_1} + w_6 \cdot out_{h_2} + w_{03}$$
$$=0.4\times0.5933+0.45\times0.5967+0.6=1.1058$$

$$out_{o_1} = \frac{1}{1+e^{-net_{o_1}}} = \frac{1}{1+e^{-1.1058}} = 0.7514$$

$$out_{o_2} = 0.7729$$

BP神经网络

- 计算损失函数：*Loss是参数W={w_1, w_2, …, w_8, w_{01}…, w_{04}}的函数*



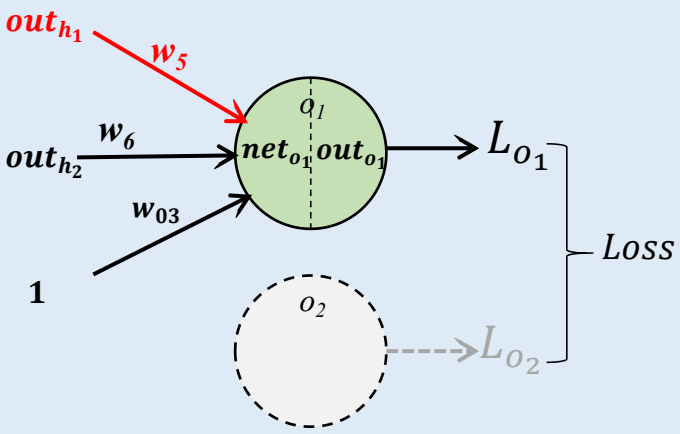
方便后续求导后消除系数

$$L_{o_1} = \frac{1}{2}(y_{i1} - out_{o_1})^2 = \frac{1}{2}(0.01 - 0.7514)^2 = 0.2748$$

$$L_{o_2} = \frac{1}{2}(y_{i2} - out_{o_2})^2 = \frac{1}{2}(0.99 - 0.7729)^2 = 0.0236$$

$$Loss = L_{o_1} + L_{o_2} = 0.2984$$

- 梯度下降法更新参数w_5:



w_5不会对L_{o_2}有任何影响

首先观察一下$Loss$与参数w_5的关系：

$$L_{o_1} = \frac{1}{2}(y_{i1} - out_{o_1})^2$$

$$= \frac{1}{2}\left(y_{i1} - \frac{1}{1+e^{-net_{o_1}}}\right)^2$$

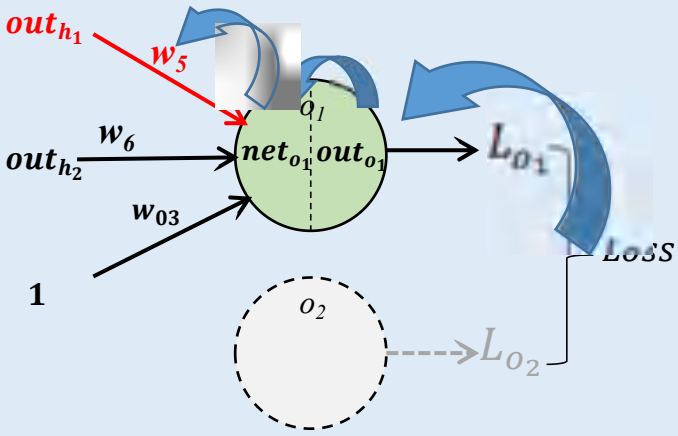
$$= \frac{1}{2}\left(y_{i1} - \frac{1}{1+e^{-(w_5 \cdot out_{h_1} + w_6 \cdot out_{h_2} + w_{03})}}\right)^2$$

即$Loss$是关于参数w_5的复合函数

复合函数求偏导的链式法则：$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$

BP神经网络

- 梯度下降法更新参数w_5:



$Loss$在参数w_5上的梯度分量：$\frac{\partial Loss}{\partial w_5}$

$$\frac{\partial Loss}{\partial w_5}=\frac{\partial Loss}{\partial out_{o_1}}\times\frac{\partial out_{o_1}}{\partial net_{o_1}}\times\frac{\partial net_{o_1}}{\partial w_5}$$

复合函数求偏导的链式法则

① $$\frac{\partial Loss}{\partial out_{o_1}}=\frac{\partial\left[\frac{1}{2}\left(y_{i1}-out_{o_1}\right)^2\right]}{\partial out_{o_1}}=out_{o_1}-y_{i1}$$

$$=0.7514-0.01=0.7414$$

② $$\frac{\partial out_{o_1}}{\partial net_{o_1}}=\frac{\partial\left[\frac{1}{1+e^{-net_{o_1}}}\right]}{\partial net_{o_1}}$$

out_{o_1}是神经元o_1的激活函数

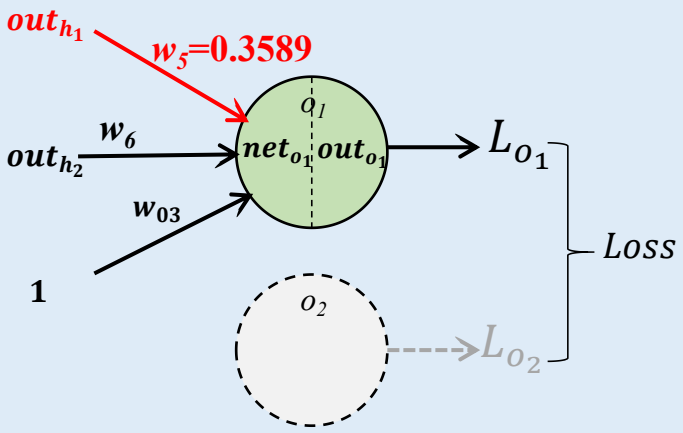
$$=out_{o_1}\left(1-out_{o_1}\right)=0.7514\times(1-0.7514)=0.1868$$

③ $$\frac{\partial net_{o_1}}{\partial w_5}=\frac{\partial\left[w_5\cdot out_{h_1}+w_6\cdot out_{h_2}+w_{03}\right]}{\partial w_5}=out_{h_1}$$

$$=0.5933$$

sigmoid函数具有以下性质：$f'(x)=f(x)\,(1-f(x))$

- 梯度下降法更新参数w_5:



就是神经元o_1的误差信号δ_{o_1}

$$\frac{\partial Loss}{\partial w_5}=\frac{\partial Loss}{\partial out_{o_1}}\times\frac{\partial out_{o_1}}{\partial net_{o_1}}\times out_{h_1}$$

$$=0.7414\times0.1868\times0.5933=0.08217$$

$$w_5 \leftarrow w_5-\eta\cdot\frac{\partial Loss}{\partial w_5}=0.4-0.5\times0.08217=0.3589$$

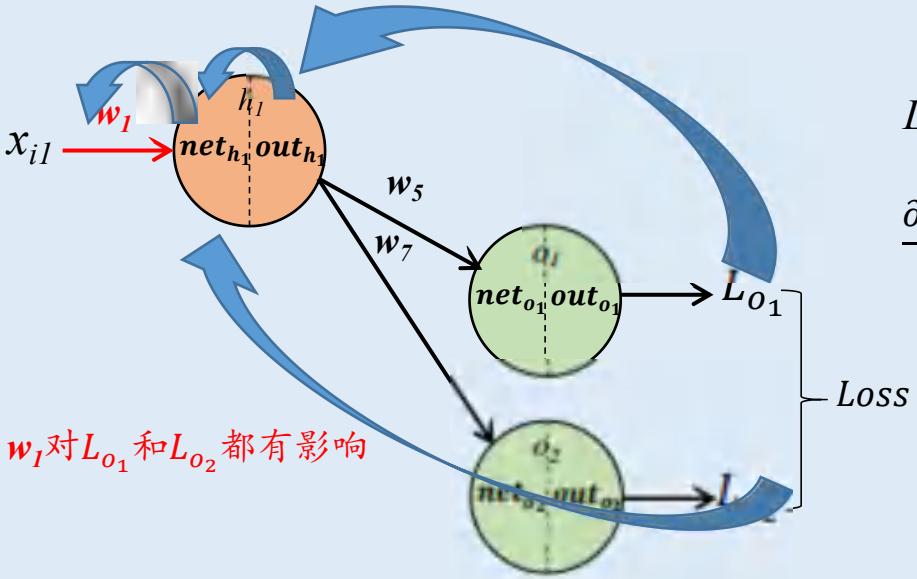
➢ 根据δ学习规则，参数w_5的更新规则是:

$$w_5 \leftarrow w_5-\eta\cdot\frac{\partial Loss}{\partial w_5}=w_5-\eta\cdot\delta_{o_1}\cdot out_{h_1}$$

out_{h_1} —w_5→ o_1

BP神经网络

- 梯度下降法更新参数w_1：

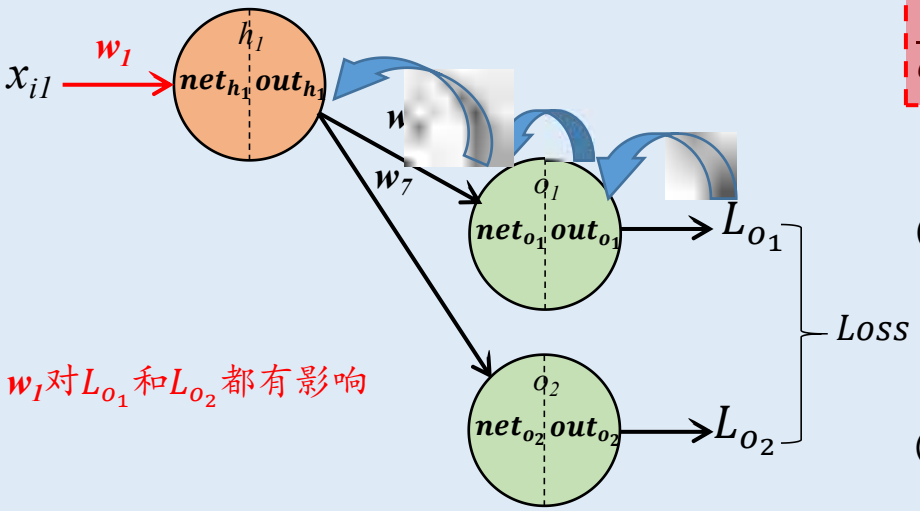


$Loss$在参数w_1上的梯度分量：$\frac{\partial Loss}{\partial w_1}$

$$\frac{\partial Loss}{\partial w_1} = \frac{\partial L_{o_1}}{\partial out_{h_1}} \times \frac{\partial out_{h_1}}{\partial net_{h_1}} \times \frac{\partial net_{h_1}}{\partial w_1} + \frac{\partial L_{o_2}}{\partial out_{h_1}} \times \frac{\partial out_{h_1}}{\partial net_{h_1}} \times \frac{\partial net_{h_1}}{\partial w_1}$$

和o_1结点计算方法一样

- 梯度下降法更新参数w_1:



w_1对L_{o_1}和L_{o_2}都有影响

$$\frac{\partial L_{o_1}}{\partial out_{h_1}} = \frac{\partial L_{o_1}}{\partial out_{o_1}} \times \frac{\partial out_{o_1}}{\partial net_{o_1}} \times \frac{\partial net_{o_1}}{\partial out_{h_1}}$$

① $$\frac{\partial L_{o_1}}{\partial out_{o_1}} = \frac{\partial \left[\frac{1}{2}\left(y_{i1} - out_{o_1}\right)^2\right]}{\partial out_{o_1}} = out_{o_1} - y_{i1}$$

$$= 0.01 - 0.7514 = 0.7414$$

② $$\frac{\partial out_{o_1}}{\partial net_{o_1}} = \frac{\partial \left[\frac{1}{1+e^{-net_{o_1}}}\right]}{\partial net_{o_1}}$$ ①②和前面一样

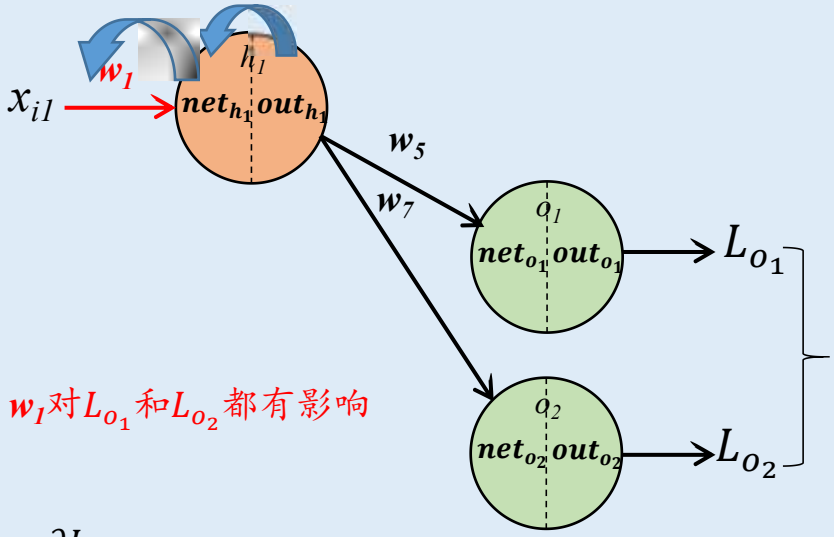
$$= out_{o_1}\left(1 - out_{o_1}\right) = 0.1868$$

③ $$\frac{\partial net_{o_1}}{\partial out_{h_1}} = \frac{\partial\left[w_5 \cdot out_{h_1} + w_6 \cdot out_{h_2} + w_{03}\right]}{\partial out_{h_1}} = w_5$$

$$= 0.4$$

BP神经网络

- 梯度下降法更新参数w_1:



w_1对L_{o_1}和L_{o_2}都有影响

④ $$\frac{\partial out_{h_1}}{\partial net_{h_1}} = \frac{\partial\left[\frac{1}{1+e^{-net_{h_1}}}\right]}{\partial net_{h_1}} = out_{h_1}(1-out_{h_1})$$

$$= 0.5933\times(1-0.5933) = 0.2413$$

⑤ $$\frac{\partial net_{h_1}}{\partial w_1} = \frac{\partial[w_1 x_{i1} + w_2 x_{i2} + w_{01}]}{\partial w_1} = x_{i1}$$

$$= 0.05$$

$$\frac{\partial L_{o_1}}{\partial w_1} = ① \times ② \times ③ \times ④ \times ⑤ = 0.7414\times 0.1868 \times 0.4 \times 0.2413 \times 0.05 = 0.0006684$$

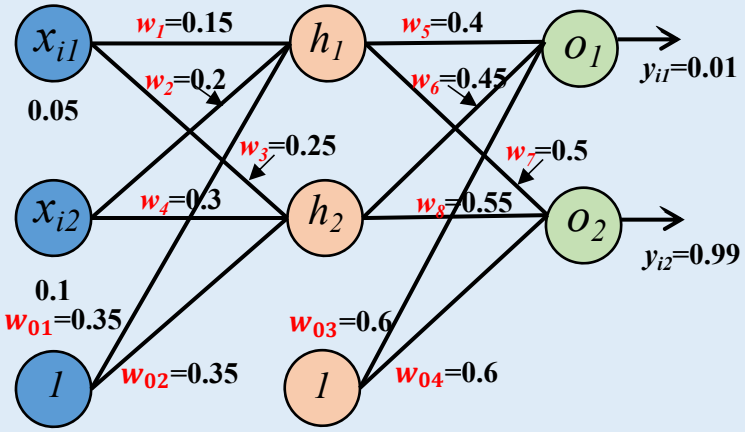
$$\frac{\partial L_{o_2}}{\partial w_1} = \frac{\partial L_{o_2}}{\partial out_{o_2}} \times \frac{\partial out_{o_2}}{\partial net_{o_2}} \times \frac{\partial net_{o_2}}{\partial out_{h_1}} \times \frac{\partial out_{h_1}}{\partial net_{h_1}} \times \frac{\partial net_{h_1}}{\partial w_1} = -0.2171\times 0.1755 \times 0.5 \times 0.2413 \times 0.05 = -0.0002298$$

$$\frac{\partial Loss}{\partial w_1} = 0.0006684 - 0.0002298 = 0.0004386$$

$$w_1 \leftarrow w_1 - \eta \cdot \frac{\partial Loss}{\partial w_1} = 0.15 - 0.5\times 0.0004386 = 0.1498$$

BP神经网络

- 梯度下降法更新参数w_1：



$$\frac{\partial Loss}{\partial w_1} = \frac{\partial L_{o_1}}{\partial out_{o_1}} \times \frac{\partial out_{o_1}}{\partial net_{o_1}} \times w_5 \times \frac{\partial out_{h_1}}{\partial net_{h_1}} \times x_{i1} + \frac{\partial L_{o_2}}{\partial out_{o_2}} \times \frac{\partial out_{o_2}}{\partial net_{o_2}} \times w_7 \times \frac{\partial out_{h_1}}{\partial net_{h_1}} \times x_{i1}$$

$$= \left(\delta_{o_1} \times w_5 + \delta_{o_2} \times w_7\right) \times \frac{\partial out_{h_1}}{\partial net_{h_1}} \times x_{i1}$$

δ_{o_1} δ_{o_2}

就是神经元h_1的误差信号δ_{h_1}

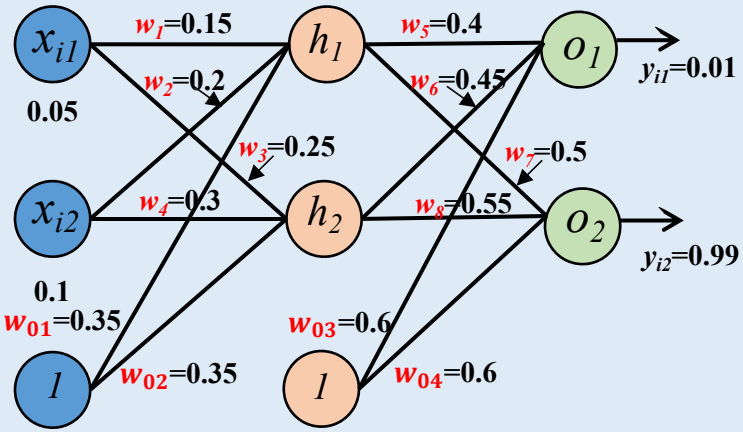
- 根据δ学习规则，参数w_1的更新规则是：

$$w_1 \leftarrow w_1 - \eta \cdot \frac{\partial Loss}{\partial w_1} = w_1 - \eta \cdot \delta_{h_1} \cdot x_{i1}$$

$x_{i1} \xrightarrow{w_1} h_1$

BP神经网络

- 梯度下降法更新参数w_1：



- 根据δ学习规则，与输出o_i相连接的参数w_{o_i}的更新规则是：

$$w_{o_i} \leftarrow w_{o_i} - \eta \cdot \frac{\partial Loss}{\partial w_{o_i}} = w_{o_i} - \eta \cdot \delta_{o_i} \cdot out_{h_j}$$

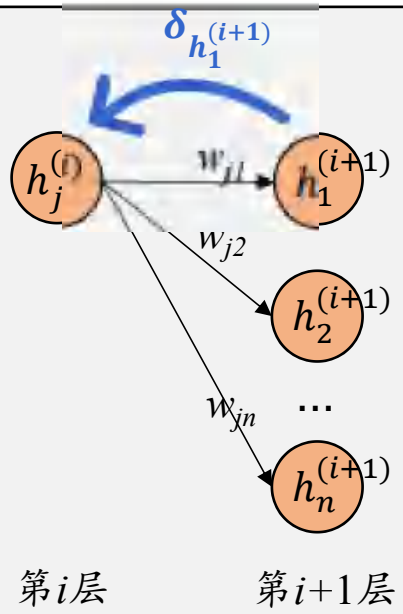
$out_{h_1} \xrightarrow{w_5} o_1$

- 根据δ学习规则，隐层单元参数w_1的更新规则是：

$$w_{h_i} \leftarrow w_{h_i} - \eta \cdot \frac{\partial Loss}{\partial w_{h_i}} = w_{h_i} - \eta \cdot \delta_{h_i} \cdot x_{ij}$$

$x_{i1} \xrightarrow{w_1} h_1$

➢ 对于第i层的隐层节点$h_j^{(i)}$，其误差信号$\delta_{h_j^{(i)}}$有：



$$\delta_{h_j^{(i)}} = \left(\sum_{l=1}^{n} \delta_{h_l^{(i+1)}} \cdot w_{jl}\right) \cdot \frac{\partial out_{h_j^{(i)}}}{\partial net_{h_j^{(i)}}}$$

后一层神经元的误差信号乘以连接权重的线性组合

自身神经元激活函数的导函数

➢ 由上可以看到，**BP神经网络可以反向逐层计算各神经元的误差信号，即实现了误差的反向传播**

- **误差信号的物理含义**：**损失函数对净输入（仿射函数*net*）的梯度。**

  由于**特殊的激活函数和损失函数**使用，使得$\delta^{(i)}$表示为该层预测值与真实值之差，故而被称为“误差”

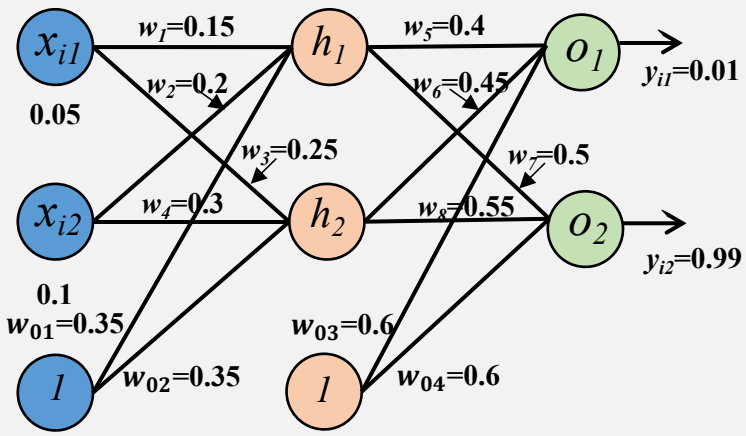
- 损失函数对单个神经元净输入（仿射函数*net*）的导数，是该层误差梯度的一个分量，称为该神经元的误差
- 故而第i层误差与第$i+1$层误差满足：

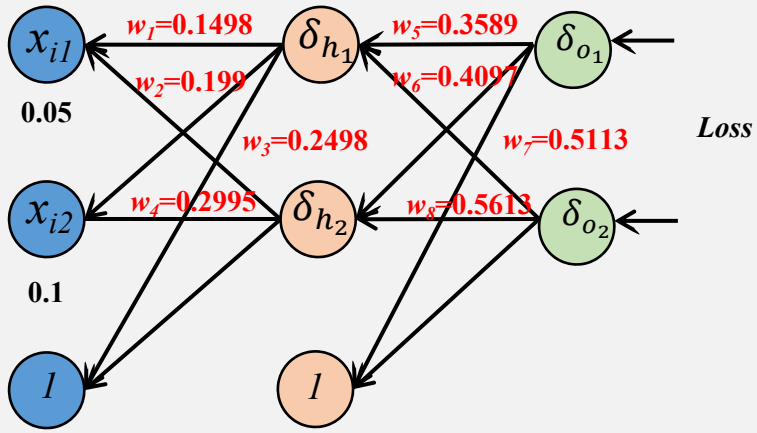
$$\delta^{(i)} = \left(W^{i+1}\right)^T \delta^{(i+1)} \odot \sigma'^{(i)}$$

⊙为向量逐个元素积　第i层各神经元激活函数导数

BP神经网络的训练流程

1. 初始各边权重；
2. 网络前向传播，计算损失函数值；
3. 网络反向传播，计算各结点的误差信号；
4. 按δ 学习规则更新各边的权重（参数）；
5. 重复2-4，直到训练误差满足要求；



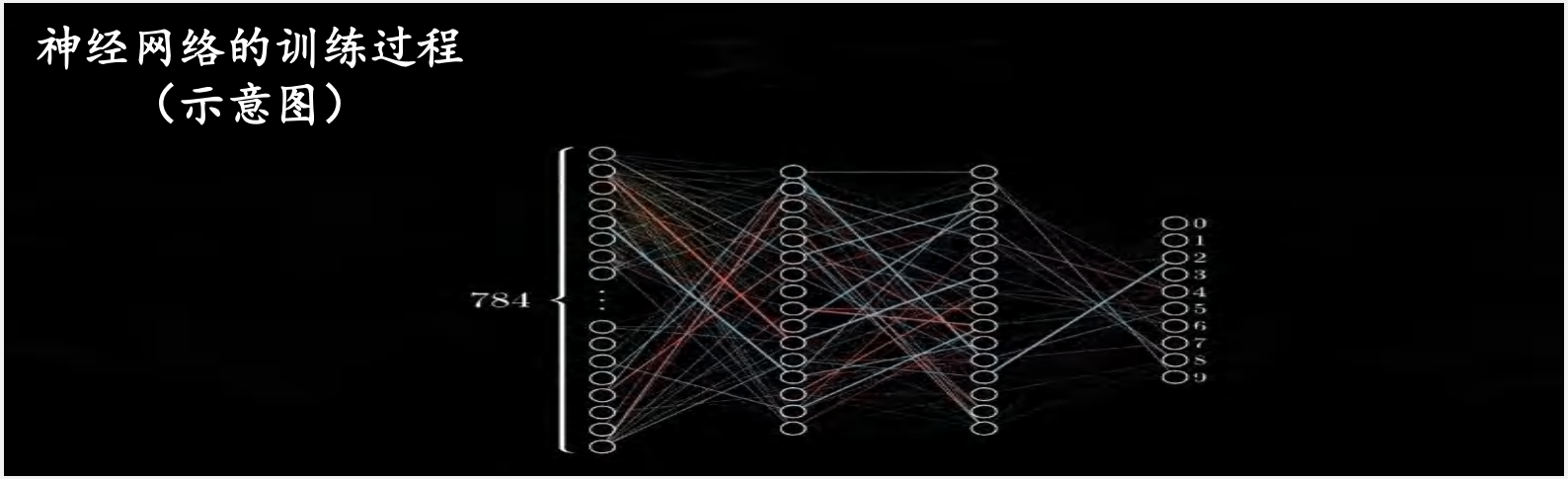


BP神经网络的训练流程

1. 初始各边权重；
2. 网络前向传播，计算损失函数值；
3. 网络反向传播，计算各结点的误差信号；
4. 按δ 学习规则更新各边的权重（参数）；
5. 重复2-4，直到训练误差满足要求；

1次前向传播+1次反向传播称为一次迭代 **Iteration**

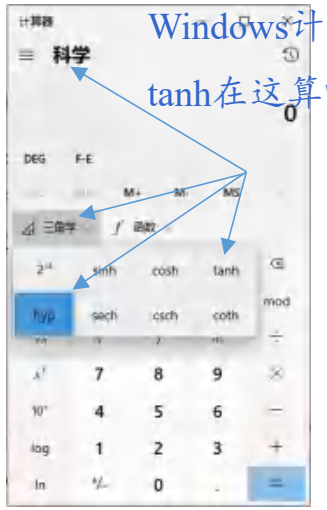
神经网络的训练过程
（示意图）



（图片来自网络）

➢ BP神经网络的反向传播算法的几个要点：

- 要能够计算网络输出与标记值之间的损失；
- 要能够计算梯度（即损失函数可导、激活函数可导）
- 由学习率η控制参数更新的速度



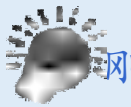
作业题：

对前例网络，将神经元激活函数改为Tanh()函数，其它超参数及初始权重不变，样本不变，试做一次迭代，更新参数w_4、w_8和w_{01}

* $$\frac{d}{dx}\tanh(x) = 1 - \tanh^2(x)$$

3 随机梯度下降和批量梯度下降

- 随机梯度下降法（Stochastic Gradient Descent, SGD）是指在样本中随机挑选一个来计算损失，然后以该损失更新参数



刚才的例子就是使用SGD来更新参数。其中：

$$Loss = \frac{1}{2}(y_{i1} - out_{o_1})^2 + \frac{1}{2}(y_{i2} - out_{o_2})^2$$

平方误差损失

优点：

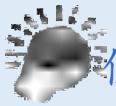
由于不是在全部训练数据上的损失函数，而是在每轮迭代中，随机优化某一条训练数据上的损失函数，这样每一轮参数更新的迭代速度大大加快

缺点：

单个样本并不能代表全体样本的趋势，可能会收敛到局部最优；

不易于并行实现；

- 批量梯度下降法（Batch Gradient Descent, BGD）是指使用m个全部样本来计算损失，然后以该损失更新参数



使用BGD来定义前述网络的损失:

$$Loss = \frac{1}{2m}\sum_{i=1}^{m}\left[\left(y_{i1} - out_{x_i,o_1}\right)^2 + \left(y_{i2} - out_{x_i,o_2}\right)^2\right]$$

MSE损失

优点：

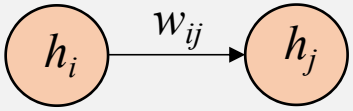
一次迭代是对所有样本进行计算，此时利用矩阵进行操作可实现并行；

由全数据集确定的方向能够更好地代表样本总体，从而更准确地朝向极值所在的方向。当目标函数为凸函数时，BGD一定能够得到全局最优

缺点：

当样本数目 m 很大时，每迭代一步都需要对所有样本计算，训练过程会很慢；每次参数更新都需要重新计算所有样本的梯度，即使某些样本对参数更新的贡献很小

➢ 对比随机梯度下降和批量梯度下降的参数更新方法：

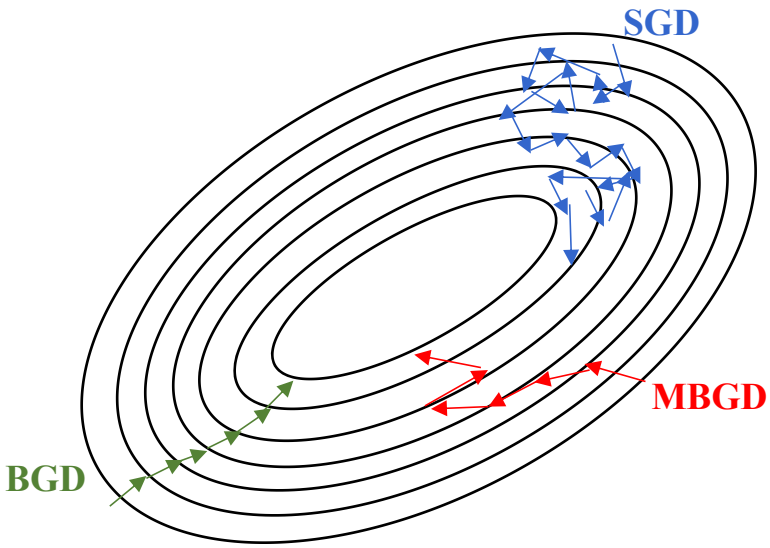


- 随机梯度下降SGD：$w_{ij}^{(t+1)} = w_{ij}^{(t)} - \eta \cdot \frac{\partial Loss}{\partial w_{ij}^{(t)}}$

m个样本的平均损失

- 批量梯度下降BGD：$w_{ij}^{(t+1)} = w_{ij}^{(t)} - \eta \cdot \frac{\partial \left(\frac{1}{m} \sum_{k=1}^{m} Loss_k \right)}{\partial w_{ij}^{(t)}} = w_{ij}^{(t)} - \eta \cdot \frac{1}{m} \sum_{k=1}^{m} \frac{\partial Loss_k}{\partial w_{ij}^{(t)}}$

- 小批量梯度下降法（Mini-Batch Gradient Descent, MBGD）是指每次迭代使用Batch Size个样本来计算损失，然后以该损失更新参数



SGD、BGD、MBGD的梯度搜索过程

注：有关Batch Size的大小设置是神经网络训练里很体现工程师经验的方面

Batch Size要怎么设置

批量大小
Batch Size

每次迭代使用Batch Size个样本来训练

训练轮数
Epoch[ˈepək]

注意读音

所有样本在网络中都进行了1次训练

迭代次数
Iteration

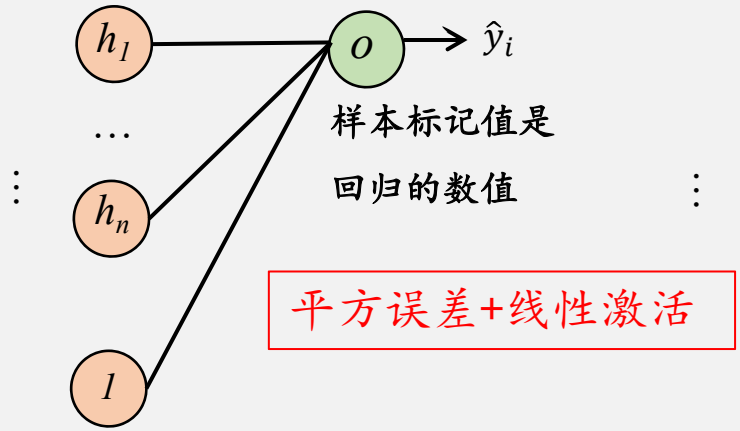
正向+反向传播，更新1次参数为1次迭代

4 输出层的设计

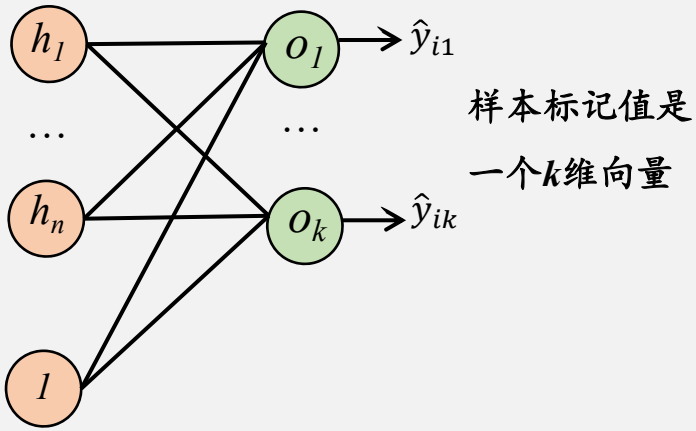
- 神经网络的输出层设计是与损失函数密切相关的一个问题。根据误差信号δ的定义：损失函数对输出层净输入的梯度（导数）是输出层结点的δ，也是反向误差传播的第一层δ
- 因此，输出层的δ与**损失函数导数**以及**输出神经元激活函数的导数**形式密切相关
- 以下针对**均方误差损失、二分类交叉熵损失和softmax损失**，观察其损失函数设计和误差信号δ的求解过程：

BP神经网络

➢ 平方误差及均方误差损失：（用于回归）



平方误差+线性激活



平方误差 $Loss = \frac{1}{2}(\hat{y}_i - y_i)^2$ $Loss = \frac{1}{2}\|\hat{\boldsymbol{y}}_i - \boldsymbol{y}_i\|_2^2$

均方误差 $Loss = \frac{1}{2m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2$ $Loss = \frac{1}{2m}\sum_{i=1}^{m}\|\hat{\boldsymbol{y}}_i - \boldsymbol{y}_i\|_2^2$

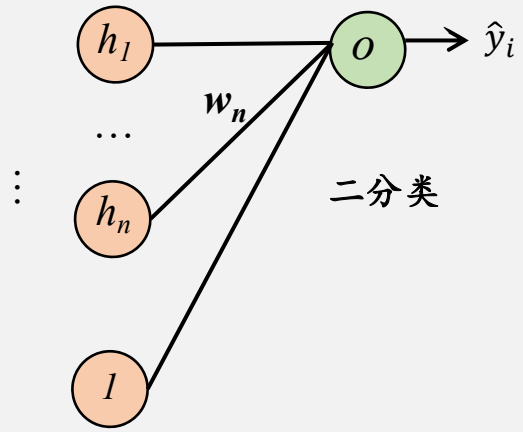
以上损失函数的导函数你们自己求↑

对于回归任务的神经网络**输出层**，一般选择**线性激活函数**

➢ 二分类交叉熵及平均交叉熵损失：(用于二分类)

（1）用ln方便求导计算

（2）这是单样本的交叉熵，若批量样本的平均交叉熵损失要加上$\frac{1}{m}\sum_i$~



- 交叉熵损失：$Loss = -y_i \cdot \ln\hat{y}_i - (1-y_i)\cdot\ln(1-\hat{y}_i)$

  - 在二分类交叉熵的输出结点中，$\hat{y}_i$是样本属于正类的概率，标记值$y_i \in \{1,0\}$，输出神经元的激活函数是Sigmoid函数 → 第四讲

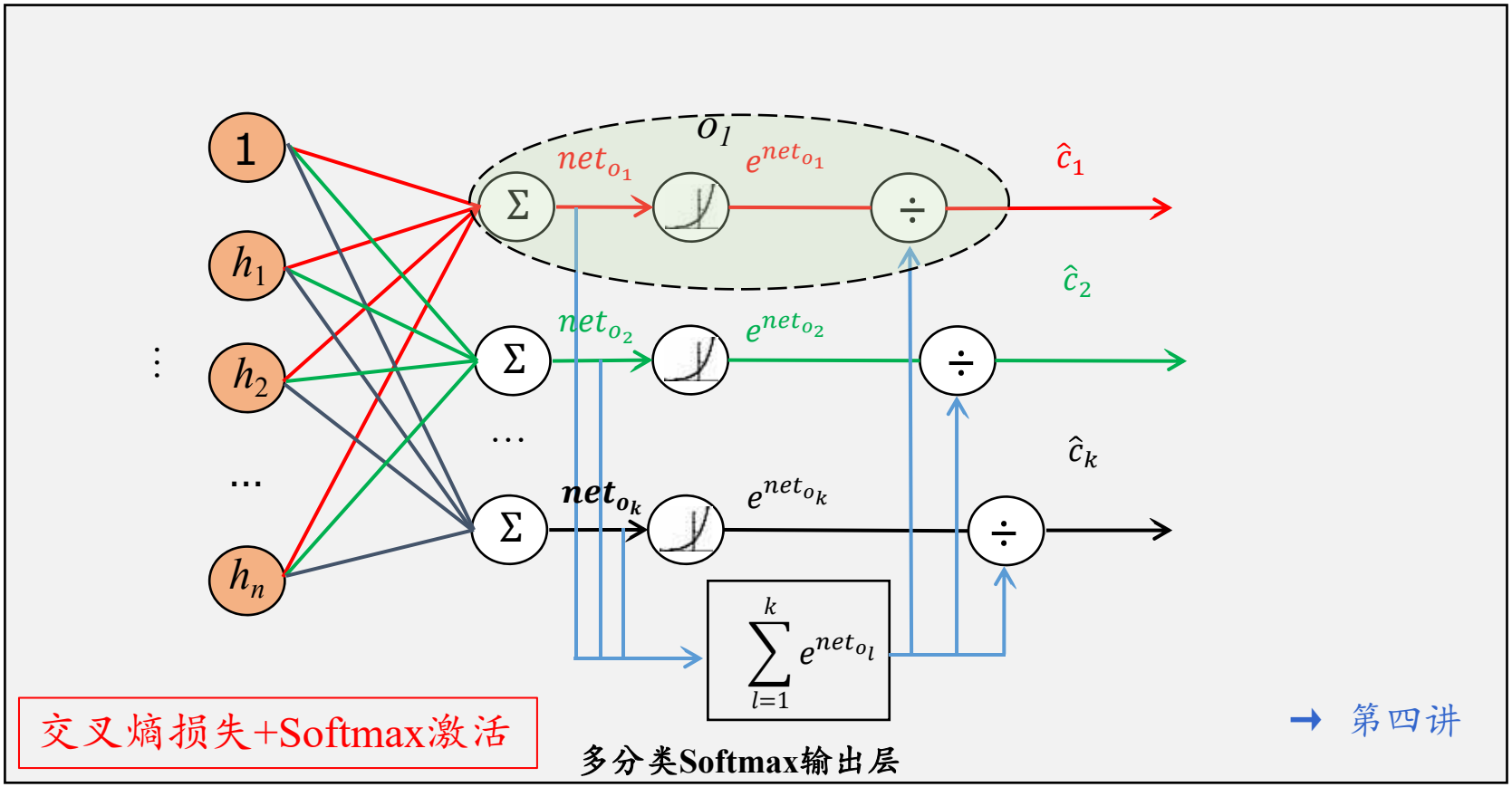
$$\delta_o = \frac{\partial Loss}{\partial \hat{y}_i} \times \frac{\partial \hat{y}_i}{\partial net_o} = \left[\frac{-y_i}{\hat{y}_i} + \frac{1-y_i}{(1-\hat{y}_i)}\right] \times \hat{y}_i(1-\hat{y}_i)$$

$$= (\hat{y}_i - y_i)$$

交叉熵损失+Sigmoid激活

- 交叉熵损失更新最后一层参数：$\frac{\partial Loss}{\partial w_n} = \delta_o out_{h_n}$

➢ 多分类Softmax损失：（用于多分类）



BP神经网络

- $\hat{c}_j = \frac{e^{net_{o_j}}}{\sum_k e^{net_{o_k}}}$ 是样本属于第j类的概率
- 对于Softmax输出，样本标记$\boldsymbol{y_i}$是形如$(0,0,0,1,0,0\ldots,0)^T$的$k\times 1$向量。输出为$(\hat{c}_1, \hat{c}_2, \hat{c}_3,\ldots, \hat{c}_k)^T$的$k\times 1$向量
- Softmax损失函数： $Loss = -\sum_{j=1}^{k} y_{ij}\cdot \ln \hat{c}_j$
- 输出结点o_j的误差信号： $\delta_{o_j} = \frac{\partial Loss}{\partial net_{o_j}} = \sum_{l=1}^{k} \frac{\partial Loss}{\partial \hat{c}_l} \times \frac{\partial \hat{c}_l}{\partial net_{o_j}}$

→ 第四讲：交叉熵的例题

$$\frac{\partial L}{\partial net_{oj}} = \frac{\partial}{\partial net_{oj}}\left(-\sum_i y_i \log(\hat{c}_i)\right)$$

$$\frac{\partial L}{\partial net_{oj}} = -\sum_i y_i \frac{1}{\hat{c}_i}\frac{\partial \hat{c}_i}{\partial net_{oj}}$$

注意：每一个$\hat{c}_l$里都含有net_{o_j}(在分母里)，所以这里是$\hat{c}_l$，且$\hat{c}_l$依赖于net_{o_j}

机器学习面试经典问题：为什么Softmax层每个结点的误差信号$\frac{\partial Loss}{\partial net_{o_j}} \neq \frac{\partial Loss}{\partial \hat{c}_j} \times \frac{\partial \hat{c}_j}{\partial net_{o_j}}$

BP神经网络

- 分别求$\frac{\partial Loss}{\partial \hat{c}_l}$和$\frac{\partial \hat{c}_l}{\partial net_{o_j}}$

① $$\frac{\partial Loss}{\partial \hat{c}_l}=\frac{\partial(-\sum_{j=1}^{k} y_{ij}\cdot \ln \hat{c}_j)}{\partial \hat{c}_l}=-\frac{y_{il}}{\hat{c}_l}$$

② $$\frac{\partial \hat{c}_l}{\partial net_{o_j}}=\begin{cases}\frac{\partial\left(\frac{e^{net_{o_j}}}{\sum_k e^{net_{o_k}}}\right)}{\partial net_{o_j}}=\frac{e^{net_{o_j}}\left(\sum_{k\neq l} e^{net_{o_k}}\right)}{\left(\sum_k e^{net_{o_k}}\right)^2}=\frac{e^{net_{o_j}}}{\sum_k e^{net_{o_k}}}\cdot\frac{\sum_{k\neq l} e^{net_{o_k}}}{\sum_k e^{net_{o_k}}}=\hat{c}_j(1-\hat{c}_j) & \\ \frac{\partial\left(\frac{e^{net_{o_l}}}{\sum_k e^{net_{o_k}}}\right)}{\partial net_{o_j}}=\frac{-e^{net_{o_j}}e^{net_{o_l}}}{\left(\sum_k e^{net_{o_k}}\right)^2}=-\frac{e^{net_{o_j}}}{\sum_k e^{net_{o_k}}}\cdot\frac{e^{net_{o_l}}}{\sum_k e^{net_{o_k}}}=-\hat{c}_j\hat{c}_l & \end{cases}$$

（注意下标！）

类似于求 $\left(\frac{e^x}{e^x+Const}\right)'$

（若$l=j$，$\hat{c}_j$ 的分子$e^{net_{o_j}}$ 与 net_{o_j}直接相关）

类似于求 $\left(\frac{1}{e^x+Const}\right)'$

当 $j\neq l$ 时，$\hat{c}_j$ 不依赖于 net_{ol}，因此 $\frac{\partial net_{oj}}{\partial net_{ol}}=0$。
但是，$\hat{c}_j$ 仍然会受到 net_{ol} 的影响。

（若l≠j，$\hat{c}_l$ 的分子$e^{net_{o_l}}$与 net_{o_j}无关，
但分母 $\sum_k e^{net_{o_k}}$包含 $e^{net_{o_j}}$）

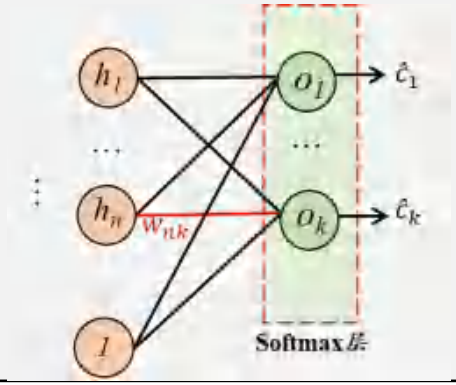
BP神经网络

- 输出结点o_j的误差信号：$\delta_{o_j} = \frac{\partial Loss}{\partial net_{o_j}} = \sum_{l=1}^{k} \frac{\partial Loss}{\partial \hat{c}_l} \times \frac{\partial \hat{c}_l}{\partial net_{o_j}}$

$$\delta_{o_j} = -\frac{y_{ij}}{\hat{c}_j} \cdot \hat{c}_j(1-\hat{c}_j) + \sum_{l \neq j} \frac{y_{il}}{\hat{c}_l} \hat{c}_j \hat{c}_l = -y_{ij} + y_{ij}\hat{c}_j + \sum_{l \neq j} y_{il}\hat{c}_j = -y_{ij} + \hat{c}_j \sum_{l=1}^{k} y_{il} = 1$$

注意：$\boldsymbol{y_i}$是形如$(0,0,0,1,0,0\ldots,0)^T$的$k \times 1$向量

- 所以 $\boldsymbol{\delta_{o_j} = \hat{c}_j - y_{ij}}$ 直接反映了预测与真实标记的偏离程度



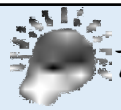
- Softmax损失更新最后一层参数：

$$\frac{\partial Loss}{\partial w_{nk}} = \delta_{o_k} out_{h_n}$$

5 梯度爆炸与梯度消失

- BP神经网络的反向传播过程中，各神经元**激活函数的导数会逐层连乘起来**。若该导数>1，则随着反向传播，靠前隐层神经元的误差信号会越来越大，发生梯度爆炸（exploding gradient）；反正如果各神经元激活函数导数<1，则随着反向传播，靠前隐层神经元的误差信号会越来越小，发生梯度弥散（vanishing gradient）或称梯度消失

$$\delta_{h_j^{(i)}} = \left(\sum_{l=1}^{n} \delta_{h_l^{(i+1)}} \cdot w_{jl}\right) \cdot \frac{\partial out_{h_j^{(i)}}}{\partial net_{h_j^{(i)}}}$$

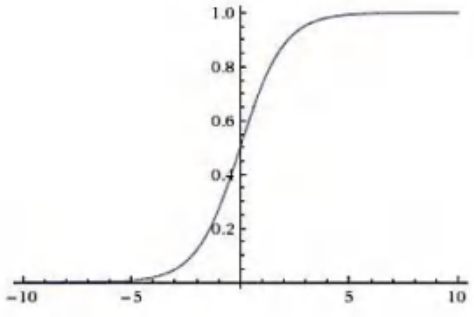


前面例子中，w_1~ w_4的更新幅度比w_5~ w_8明显要小

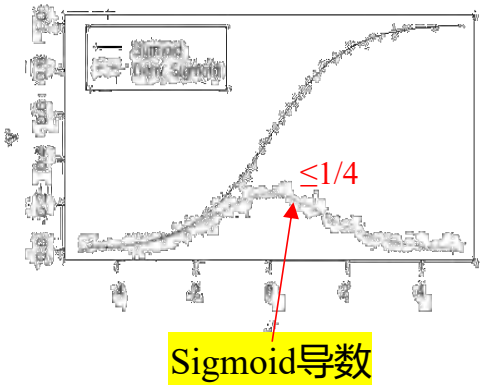
BP神经网络

Sigmoid函数

$$\text{Sigmoid} = \frac{1}{1+e^{-x}}$$



饱和激活函数



Sigmoid函数也叫Logistic函数

- 用于隐藏层的输出，输出在(0,1)之间，它可以将一个实数映射到(0,1)的范围内，可以用来做二分类；
- 常用于：在特征相差比较复杂或是相差不是特别大的时候效果比较好；
- 该函数将大的负数转换成0，将大的正数转换为1。

Sigmoid函数的缺点:

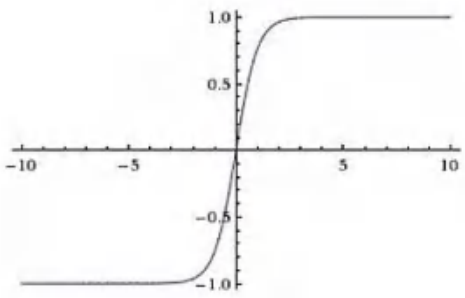
- 梯度消失：Sigmoid 函数趋近 0 和 1 的时候变化率会变得平坦，其梯度趋近于 0。

  神经网络使用Sigmoid激活函数进行反向传播时，输出接近0或1的神经元其梯度趋近于0。这些神经元叫作**饱和神经元**。因此，这些神经元的权重不会更新。此外，与此类神经元相连的神经元的权重也更新得很慢。该问题叫作**梯度消失（梯度弥散）**。因此，想象一下，如果一个大型神经网络包含Sigmoid神经元，而其中很多个都处于饱和状态，那么该网络无法执行反向传播。
- 不以零为中心：Sigmoid 输出不以零为中心的。（梯度消失或爆炸；神经元输出始终为正，多层累积后，导致学习过程难优化）
- 计算成本高昂：e^x函数与其他非线性激活函数相比，计算成本高昂。

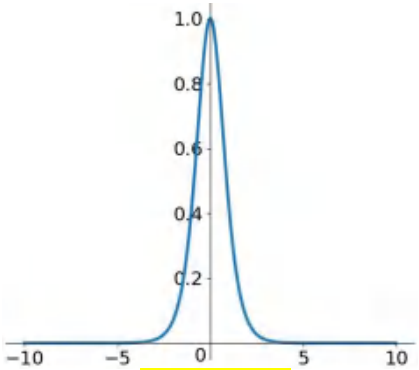
BP神经网络

Tanh函数

$$\text{Tanh} = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$



饱和激活函数



Tanh导数

- Tanh函数同Sigmoid也使用真值；
- 与 Sigmoid 不同，Tanh 函数的输出以零为中心，因为区间在-1~1之间；
- 在实践中，Tanh 函数的使用优先性高于 Sigmoid 函数，负数输入被当作负值，零输入值的映射接近零，正数输入被当作正值。

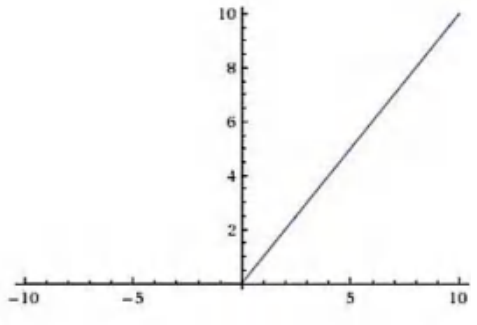
- **优点：** 它解决了Sigmoid函数的不是以0为中心的zero-centered输出问题。

- **缺点：** 梯度消失（gradient vanishing）的问题和e^{x}幂运算的问题仍然存在。

- 为了解决梯度消失问题， 另一个非线性激活函数——修正线性单元（Rectified Linear Unit，ReLU）被提出，该函数明显优于前面两个函数，是现在使用最广泛的函数。

ReLU函数

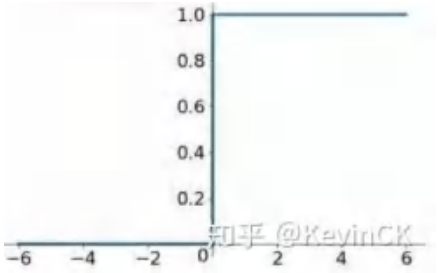
$Relu = max(0, x)$



修正线性单元
（Rectified Linear Unit，ReLU）

非饱和激活函数

ReLU-最常用的激活函数 (activation function)



ReLU导数

ReLU激活函数的解析式：

$$ReLU = \max(0, x)$$

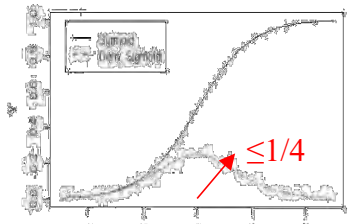
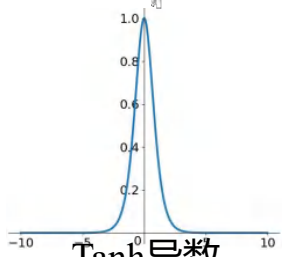
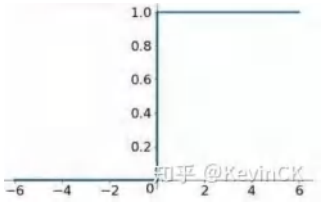
Relu激活函数优点：

- 当输入 $x < 0$ 时，输出为 0，当 $x > 0$ 时，输出为 x，该激活函数使网络更快速地收敛
- 它不会饱和，可以对抗梯度消失问题，至少在正区域（$x > 0$ 时）可以这样，因此神经元至少在一半区域中不会把所有零进行反向传播
- 由于使用了简单的阈值化（thresholding），ReLU 计算效率很高

Relu激活函数缺点：

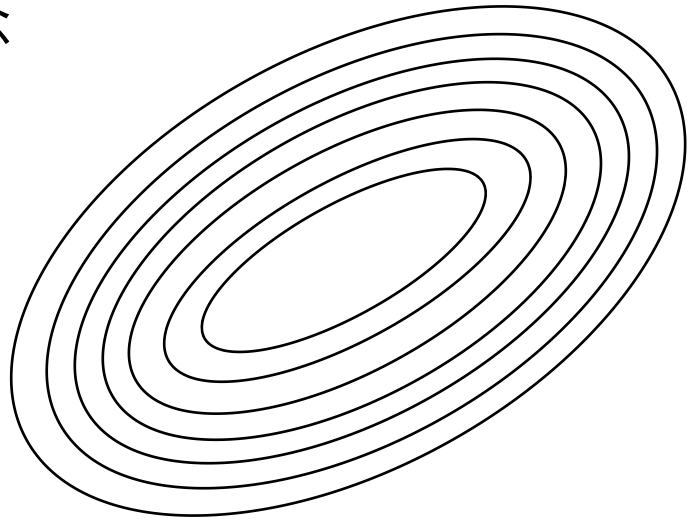
- 不以零为中心
- 前向传导（forward pass）过程中，如果 $x < 0$，则神经元保持非激活状态，且在后向传导（backward pass）中出现“梯度死亡”，这样权重无法得到更新，网络无法学习
- 当 $x = 0$ 时，该点的梯度未定义，但是这个问题在实现中得到了解决，通过采用左侧或右侧的梯度的方式

➢ 梯度弥散将造成网络参数更新无法进行，影响训练

 Sigmoid导数	• 反向传播求误差梯度时，求导涉及幂运算，计算量大 • Sigmoid 的输出不是0均值的。容易出现梯度弥散甚至梯度消失的情况，从而无法完成深层网络的训练
 Tanh导数	• 是0中心的，在实际中tanh会比sigmoid收敛速度快 • 仍然存在梯度弥散的问题
 RelU的导数	• 正方向上，梯度不变，解决了梯度消失问题 • 计算速度非常快，只需要判断神经元输出是否大于0。网络收敛速度非常快 • 会造成神经元“死亡”

6 自适应学习率

- 假设损失如图，在长轴方向上梯度变化相对较小，所以设置学习率的时候就不能太大，防止在该方向上参数更新震荡；而损失在短轴方向上的梯度非常大，学习率小又导致了该方向上参数更新过于缓慢。同时，当梯度趋于0的时候参数更新会无法进行。为此，引入**动量**的概念



7 防止神经网络过拟合

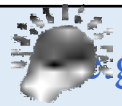
- **早停（early stopping）**：将数据分成训练集和验证集。训练集用来计算梯度、更新连接权和阈值。验证集用来估计误差。若训练集误差降低但验证集误差升高，则停止训练；同时返回具有最小验证集误差的参数（从训练角度）
- **正则化（regularization）**：在损失函数中增加限制网络复杂度的部分，如L1或L2正则化，将通过对“权重和”的约束，使得一部分神经元停止工作。类似的做法还有以一定概率“灭活”一部分神经元，称为Dropout（从模型角度）

- 此外在当前神经网络训练中，尤其是机器视觉和自然语言处理等任务时，还会使用**标签平滑（Lable Smoothing）**策略提高网络的泛化能力（从数据角度正则化）

- 对于一个k分类问题，样本标记$\boldsymbol{y}_i$是形如$(0,0,0,1,0,0\ldots,0)^T$的$k\times 1$独热编码
- 用更新的标签向量$\boldsymbol{y}'_i$来替换独热编码（one-hot）的标签向量，对于每个分量j有：

$$\boldsymbol{y}'_{ij} = (1-\varepsilon)\boldsymbol{y}_{ij} + \frac{\varepsilon}{k}$$

其中ε取一个较小的数，一般为0.1~0.3



g 经过标签平滑后的样本标记值、预测值和交叉熵（ε取0.3）

	轿车	卡车	自行车	行人	路牌	红绿灯
原始标记	0	1	0	0	0	0
平滑标记	0.05	0.75	0.05	0.05	0.05	0.05
预测概率	0.3	0.6	0.05	0.01	0.03	0.01

- 标签平滑后的分布就相当于往真实分布中加入了噪声，避免模型对于正确标签过于自信，使得预测正负样本的概率差别不那么大，从而避免过拟合，提高模型的泛化能力

注：请大家自行计算预测概率与平滑前后标签的交叉熵

- **批量归一化（Batchnorm）**：是一种用于神经网络的正则化技术，旨在通过调整每层网络的数据分布来加速训练过程并提高模型性能。它是在每个小批量（Batch）数据上进行操作，通过对该批数据的均值和方差进行计算，然后将数据调整为标准正态分布（通常是零均值和单位方差），最后通过引入两个可学习的参数（缩放因子γ和平移因子β）对规范化后的数据进行调整，以保留网络的学习能力。Batchnorm降低了数据之间的绝对差异，使得模型对初始权重和大学习率不再敏感（从数据角度正则化）

BatchNorm的原理与作用

第十讲　深度学习

1 深度学习的发展历程

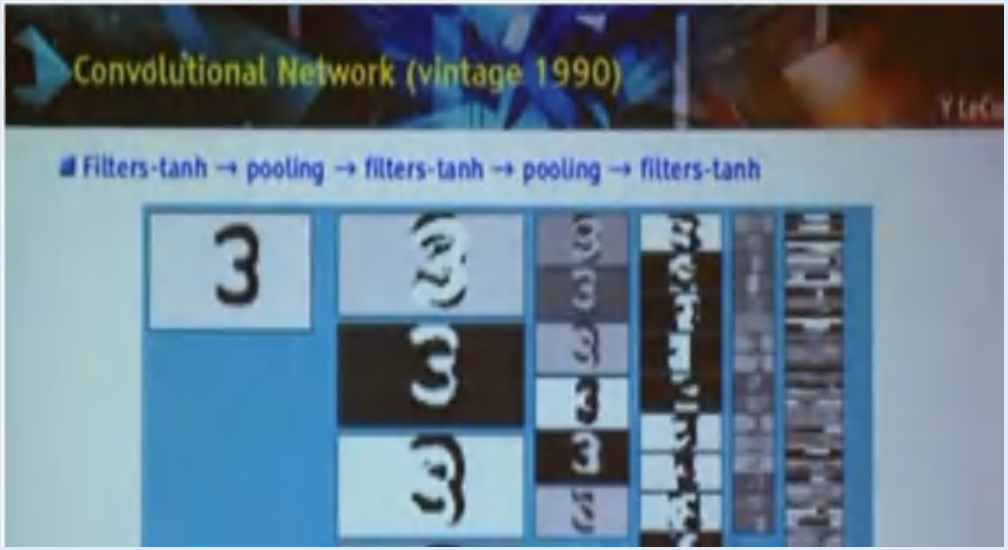
2 卷积神经网络

3 循环神经网络*

卷积神经网络

- 卷积神经网络（Convolutional Neural Networks, CNN）是一种有监督的深度学习模型，已成为当前语音分析、图像分析领域最有力的工具之一，是推动本轮人工智能热潮的主要技术方法之一

1990年，LeCun在贝尔实验室建立的首个“真实”卷积神经网络模型。模型包含带有滑动步长的卷积，以及紧密相连的池化过程，通过反向误差传播训练。经过7300次训练，可以很好地识别手写字母



卷积神经网络

- 在机器学习中，通过图像卷积的空域滤波仅是为了提取特征。因此为了减少计算量，“卷”（翻转180°）的操作一般都省去。卷积计算被替代为**互相关**计算，习惯上仍称“卷积”

$$\boldsymbol{g = h \otimes f}$$ （注意符号）

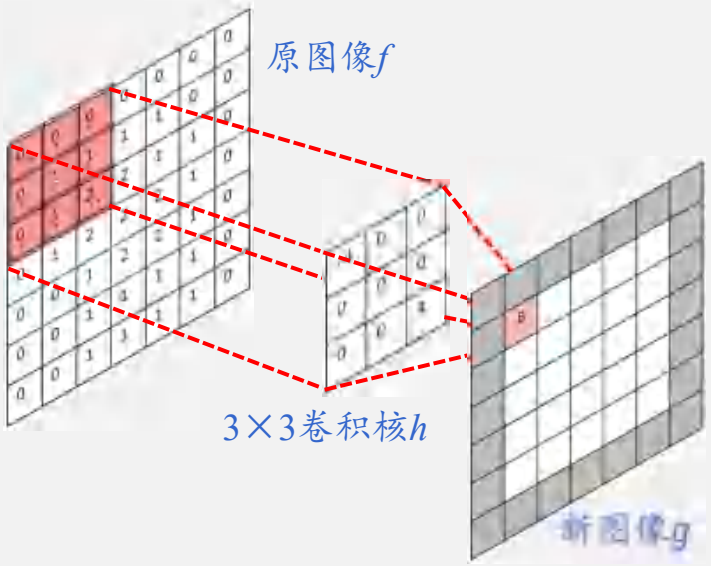
$$g(i,j) = \sum_k \sum_l h(k,l) f(i+k-1, j+l-1)$$

e.g $g(1,1) = h(1,1)f(1,1) + h(1,2)f(1,2) + h(1,3)f(1,3)$

$+h(2,1)f(2,1) + h(2,2)f(2,2) + h(2,3)f(2,3)$

$+h(3,1)f(3,1) + h(3,2)f(3,2) + h(3,3)f(3,3)$

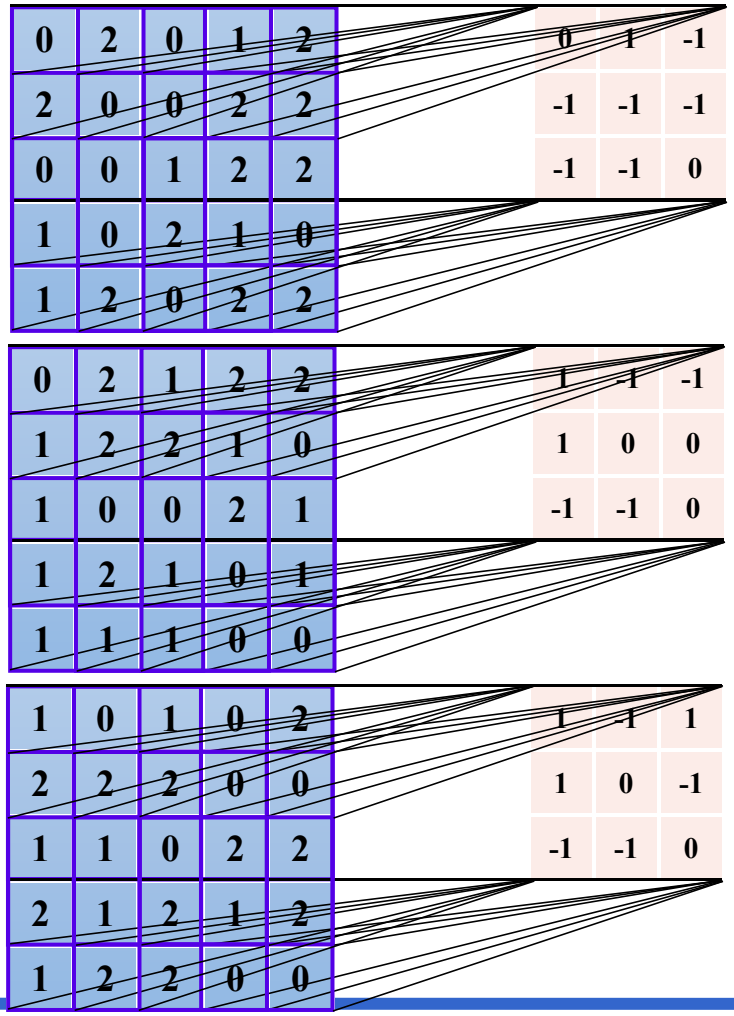
注意：g的下标从（1,1）开始

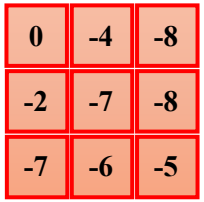


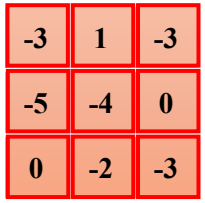
注：后面说的卷积、卷积滤波都指“互相关”

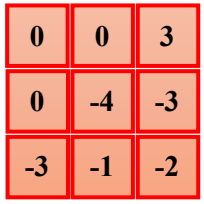
卷积神经网络

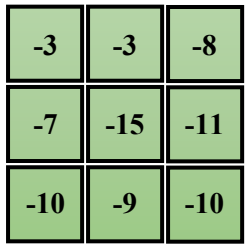
2D卷积用于多通道影像



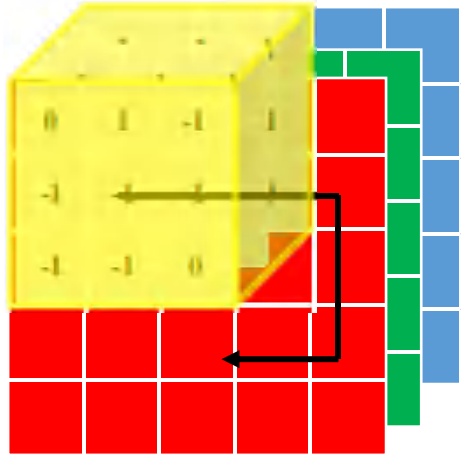






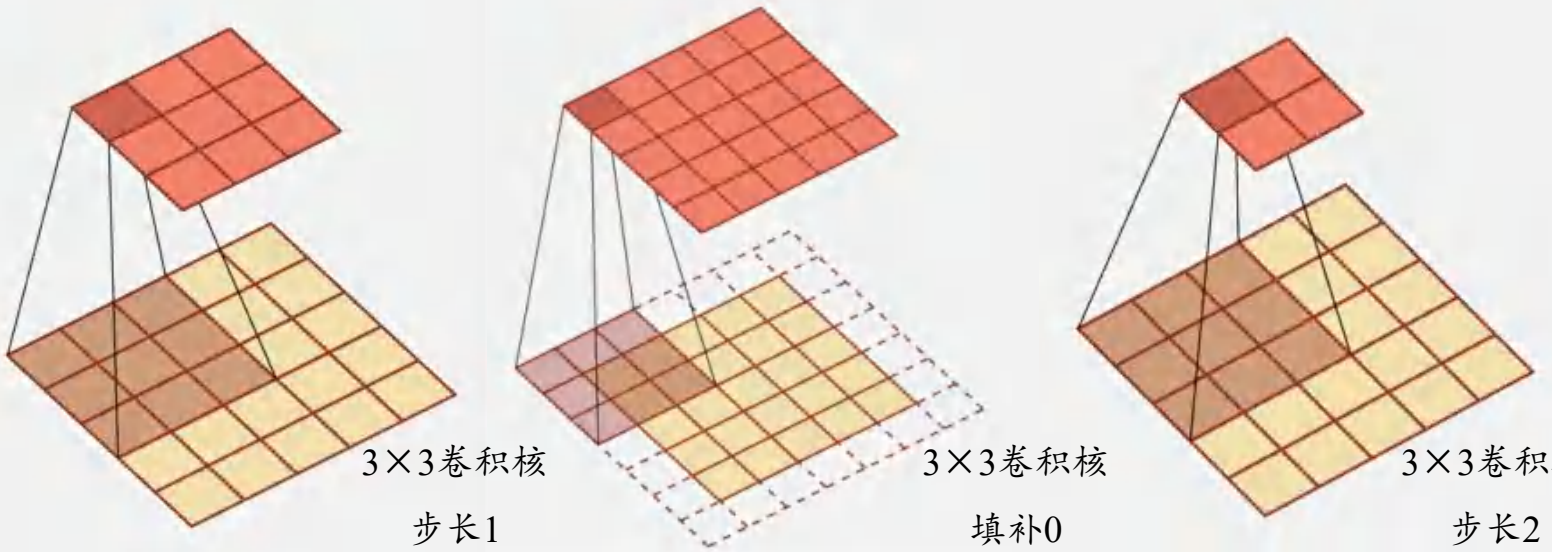


$(-5)+(-3)+(-2)=-10$



- 图像卷积滤波：根据卷积核大小以及滑动步长（Stride）逐个像素计算

（若保持图像大小不变，应对原图像四周填补0）

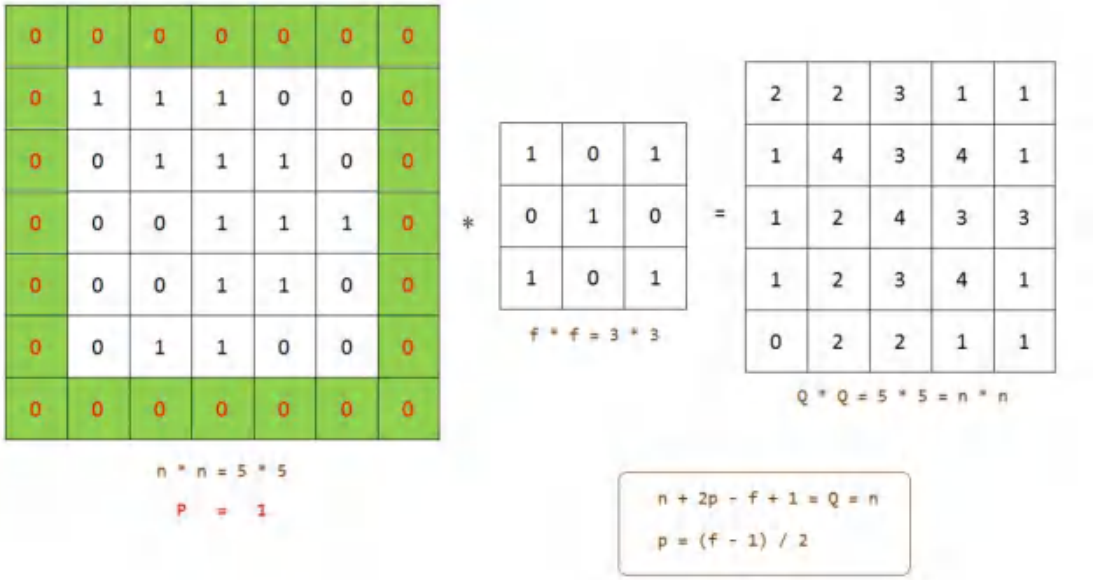


- 图像卷积滤波：根据卷积核大小以及滑动步长逐个像素计算

卷积过程中存在两个缺点：

1. 原图每经过依次卷积之后，像素就会缩小，因此卷积很多次之后图像就会变得非常小。
2. 角落或边缘的像素点信息容易丢失。

零填充(Zero-Padding)：在输入矩阵的边缘使用零值进行填充，这样就可以对输入图像矩阵的边缘进行滤波。零填充的一大好处是可以控制特征图的大小。使用零填充的也叫做泛卷积，不适用零填充的叫做严格卷积。



卷积神经网络

$\frac{1}{44}$ *

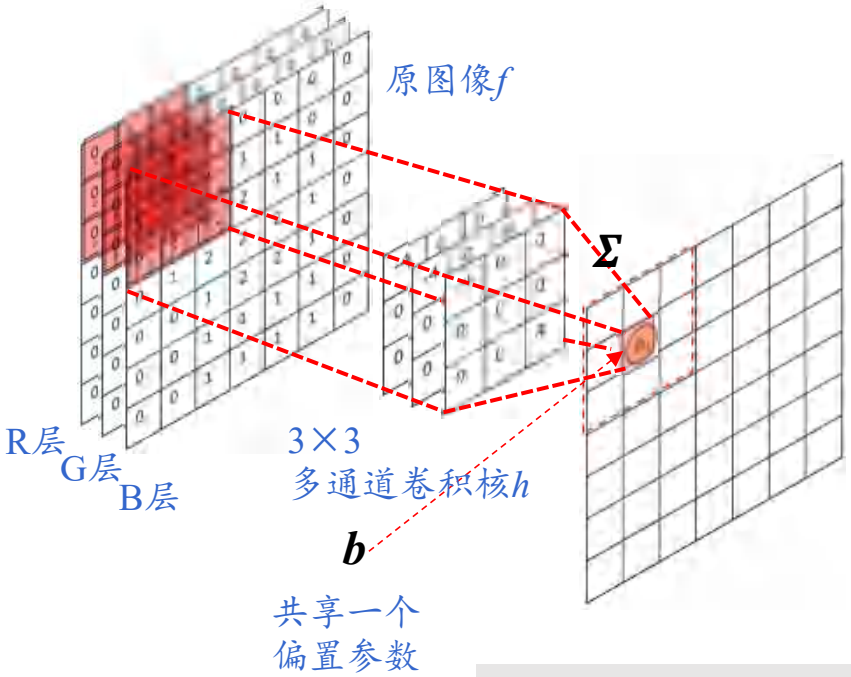
1	**1**	**2**	**1**	**1**
1	2	2	2	1
2	2	8	2	2
1	2	2	2	1
1	1	2	1	1

$\frac{1}{25}$ *

1	**1**	**1**	**1**	**1**
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

提问：这些卷积核可用来提取图像的什么特征？

- 对于RGB多个颜色通道的彩色图像，还可以使用多通道（channels）卷积对各个颜色分量进行滤波，然后组合为特征图

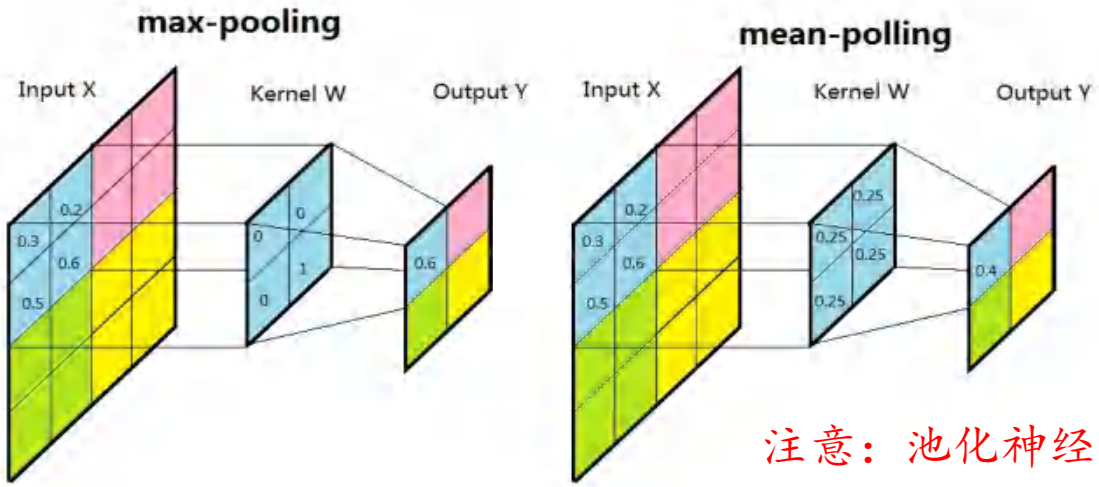


多通道卷积神经网络连接（示意图）

用各通道的卷积核（可不相同）分别与各颜色分量进行卷积滤波，将三个滤波结果叠加（相加）得到特征图

4 池化（下采样）

- CNN中的数据降维通过下采样实现，也称为“池化”。池化后的统计属性仍能够描述图像，而且由于降低了数据维度，有效地避免了过拟合。在实际应用中，下采样常用最大值下采样（Max-Pooling）或平均值下采样（Mean-Pooling）

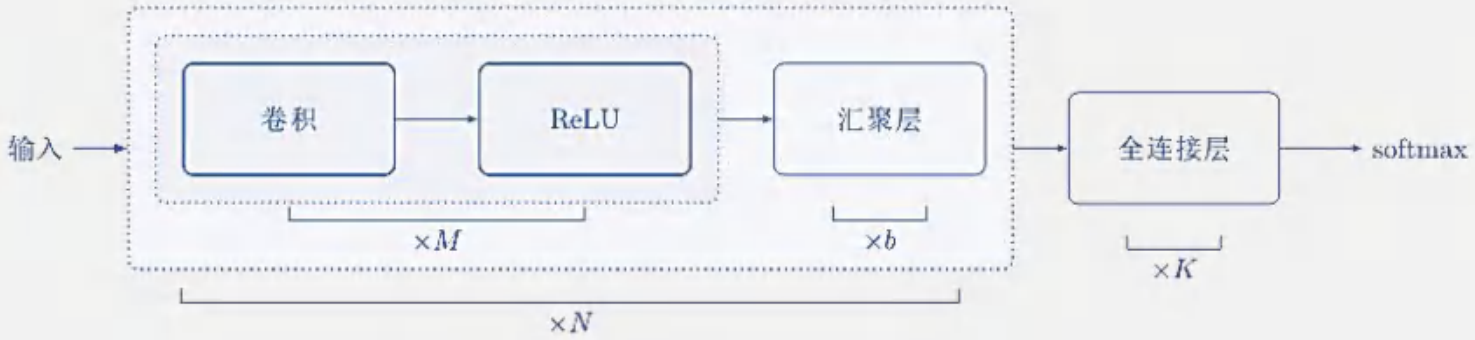


步长=2　池化层连接（示意图）

注意：池化神经元也要激活函数激活才能输出，但在实际工程中一般不加激活函数

5 卷积神经网络的基本结构

- 一个典型的CNN卷积块包括M个卷积层和b个池化层（汇聚层）。一个CNN网络可以堆叠N个连续的卷积块，再接k个全连接层（Fully Connected，FC），最后通过Softmax层输出



- 卷积层：发现图像特征
- 汇聚层：降低特征维度

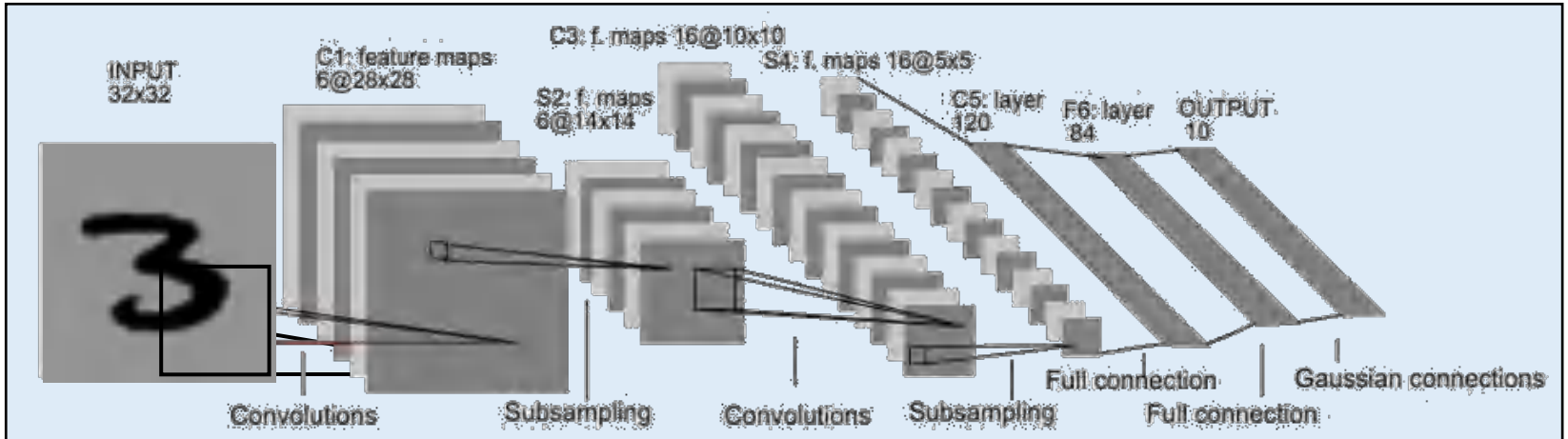
} 特征提取

- 全连接层：建立特征空间与标记空间的映射
- Softmax:分类输出

卷积神经网络

提问：一个卷积神经网络的输入为32×32×3的图像，第一个隐藏层卷积核大小为5×5，共10个卷积核。请问这一层总共含有多少参数（含偏置参数）

LeNet-5：识别手写英文字母、数字的CNN

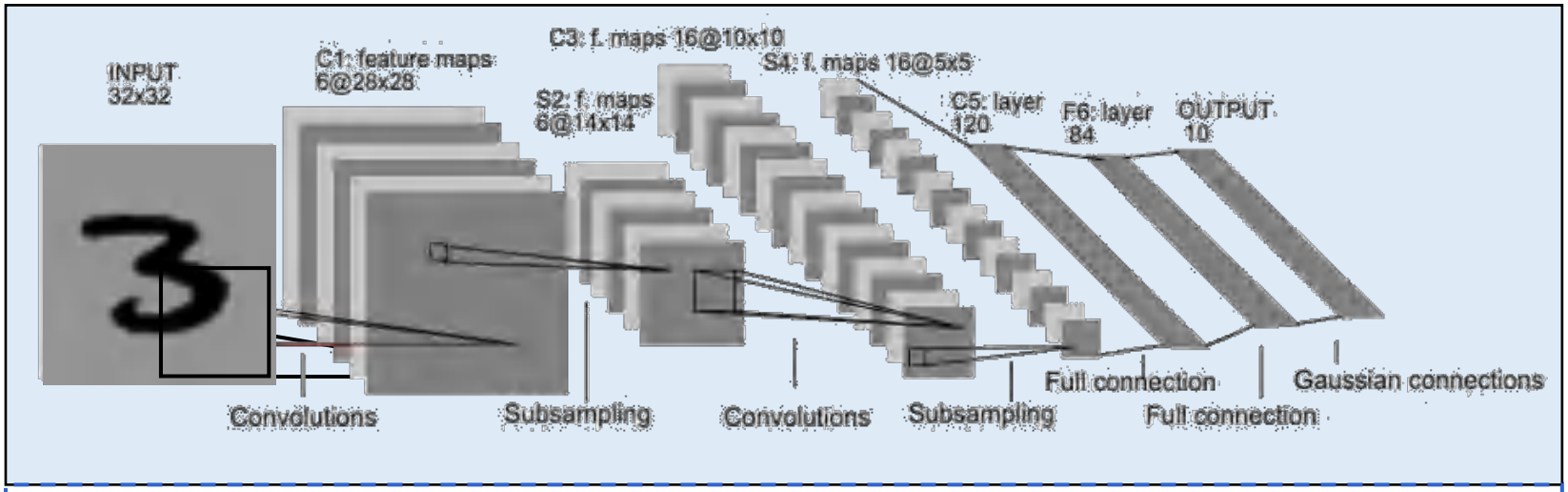


C1层：卷积层

- 卷积核大小：5×5
- 卷积核种类：6
- 输出feature map大小：28×28　　（32-5+1=28）
- 可训练参数：（5×5×1+1）× 6（每个滤波器5×5=25个unit参数和一个bias参数，一共6个滤波器）

$$\text{输出大小} = \frac{\text{输入大小} - \text{卷积核大小} + 2 \times \text{填充大小}}{\text{步长}} + 1$$

LeNet-5：识别手写英文字母、数字的CNN

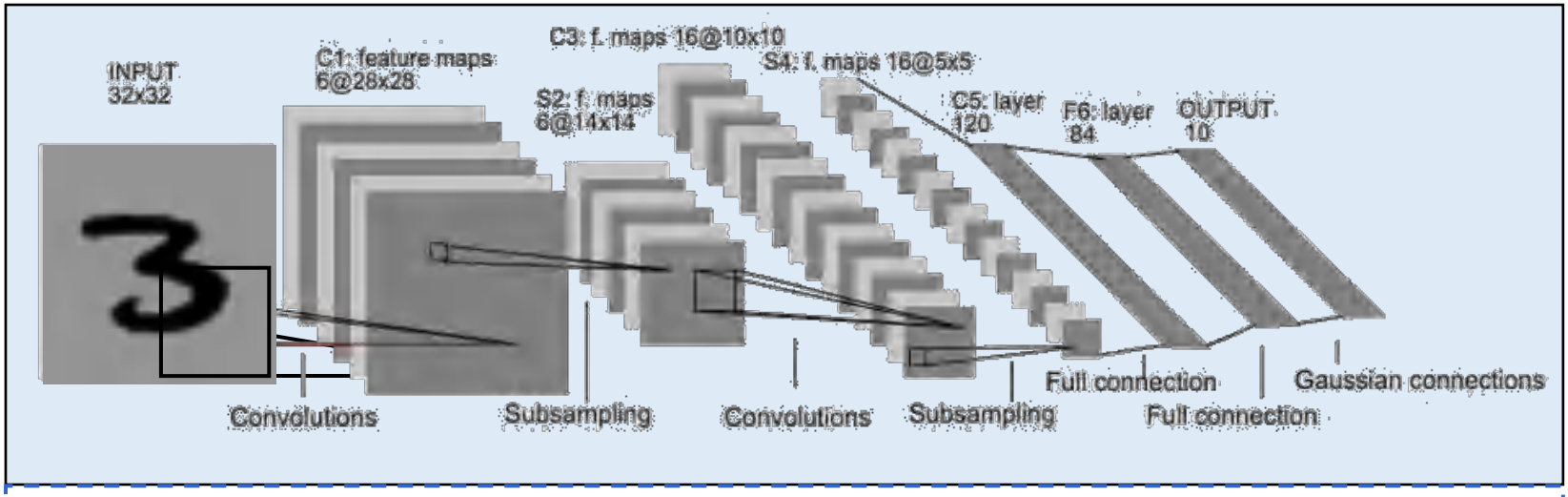


S2层：池化层

- 采样区域：2×2
- 采样方式：4邻输入相加，乘以一个权值系数再加上一个可训练偏置。结果通过sigmoid输出
- 输出feature map大小：14×14（28/2） 可训练参数：6×(1+1)=12

（1＋1）表示每个特征图有一个权重和一个偏置

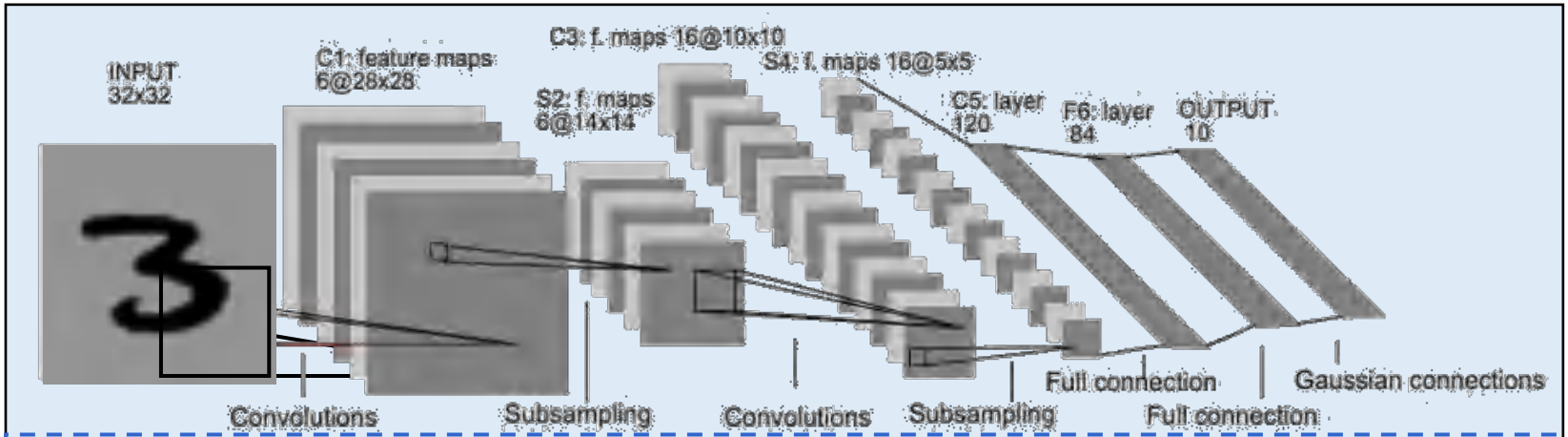
LeNet-5：识别手写英文字母、数字的CNN



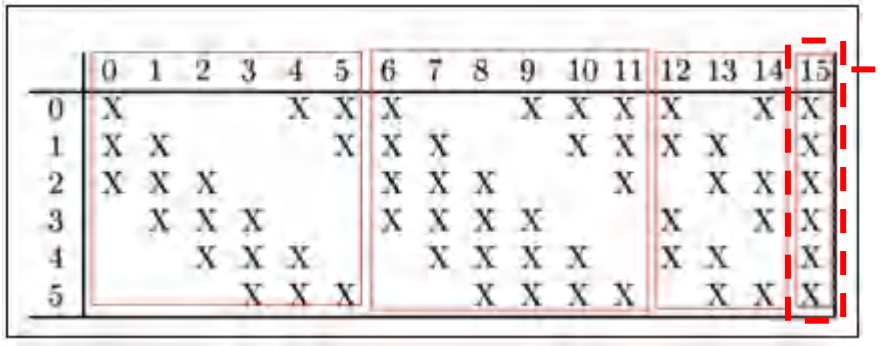
C3层：卷积层

- 卷积核大小：5×5　卷积核种类：16
- 输出feature map大小：10×10　　(14-5+1=10)
- C3中的每个特征图与S2中的连接关系见连接表。表示本层的特征图是上一层S2提取到的特征图的不同组合

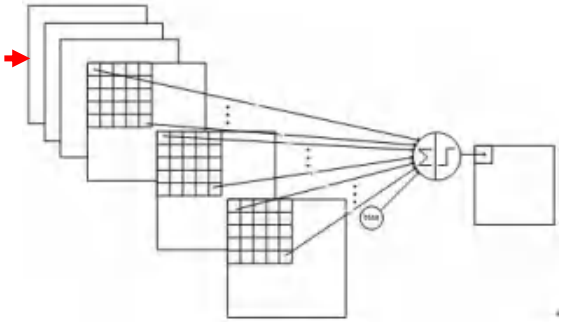
LeNet-5：识别手写英文字母、数字的CNN



C3层：卷积层

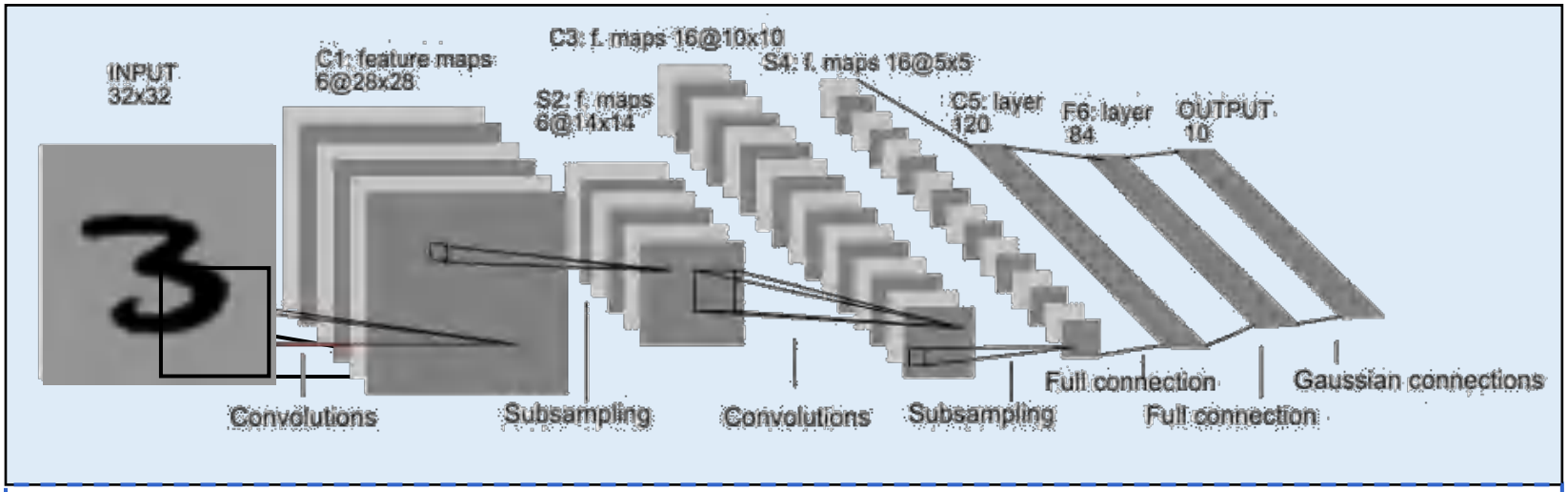


	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	X				X	X	X			X	X	X	X		X	X
1	X	X				X	X	X			X	X	X	X		X
2	X	X	X				X	X	X			X		X	X	X
3		X	X	X			X	X	X	X			X		X	X
4			X	X	X			X	X	X	X		X	X		X
5				X	X	X			X	X	X	X		X	X	X



卷积核大小为5×5，所以总共有6×(3×5×5+1)+6×(4×5×5+1)+3×(4×5×5+1)+1× (6×5×5+1)=1516个参数

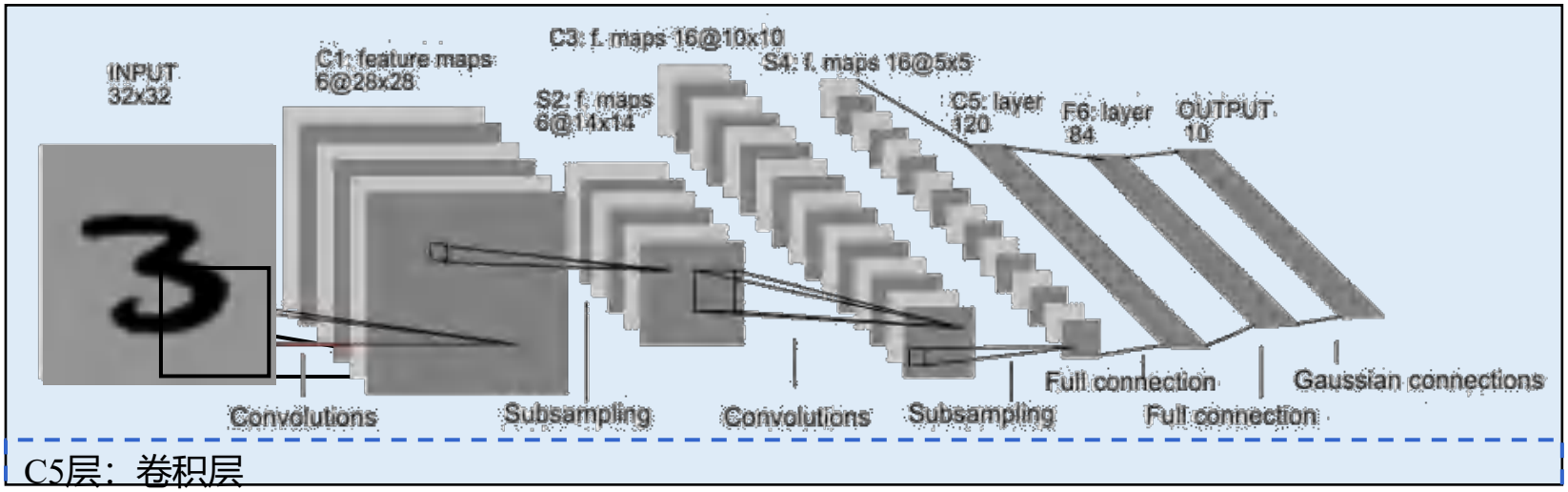
LeNet-5：识别手写英文字母、数字的CNN



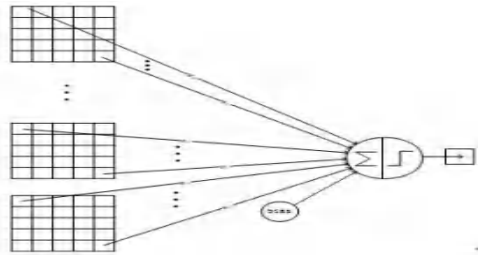
S4层：池化层

- 采样区域：2×2
- 采样方式：4邻输入相加，乘以一个权值系数再加上一个可训练偏置。结果通过sigmoid输出
- 输出feature map大小：5×5（10/2）　　可训练参数：16×(1+1)=32

LeNet-5：识别手写英文字母、数字的CNN

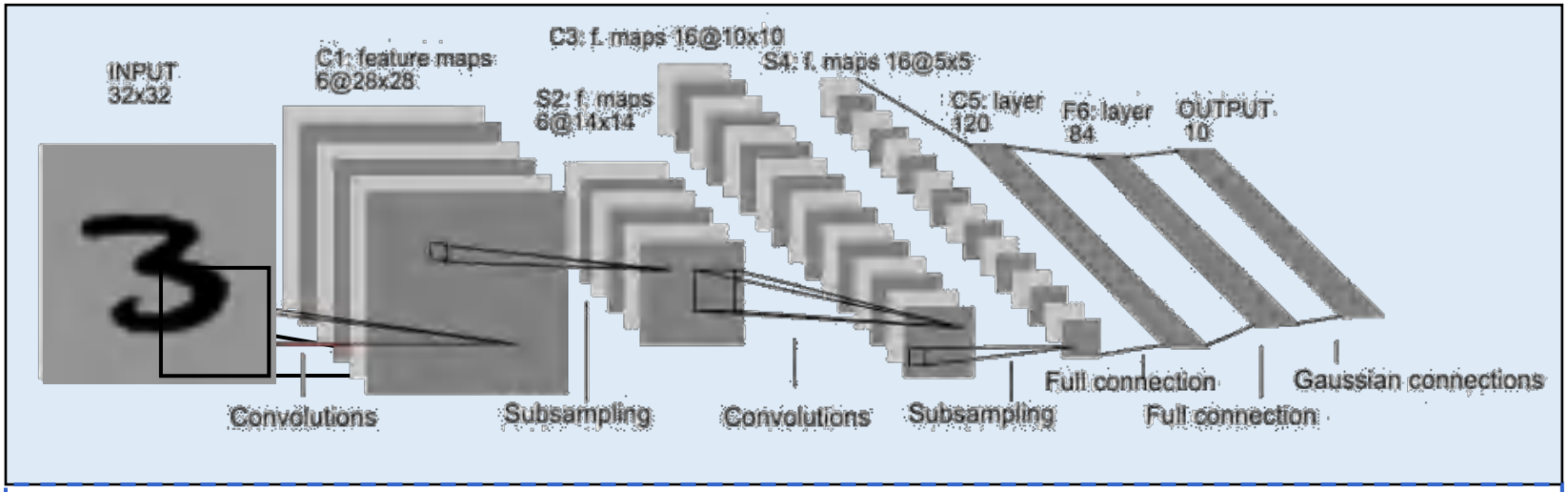


C5层：卷积层



- 卷积核大小：5×5
- 卷积核种类：120
- 输出feature map大小：1×1　　（5-5+1=1）
- 由于S4层的16个图的大小为5×5，卷积后形成的特征map大小为1×1。形成的120个卷积结果每个都与上一层的16个图相连。所以共有(5×5×16+1) ×120 = 48120个参数

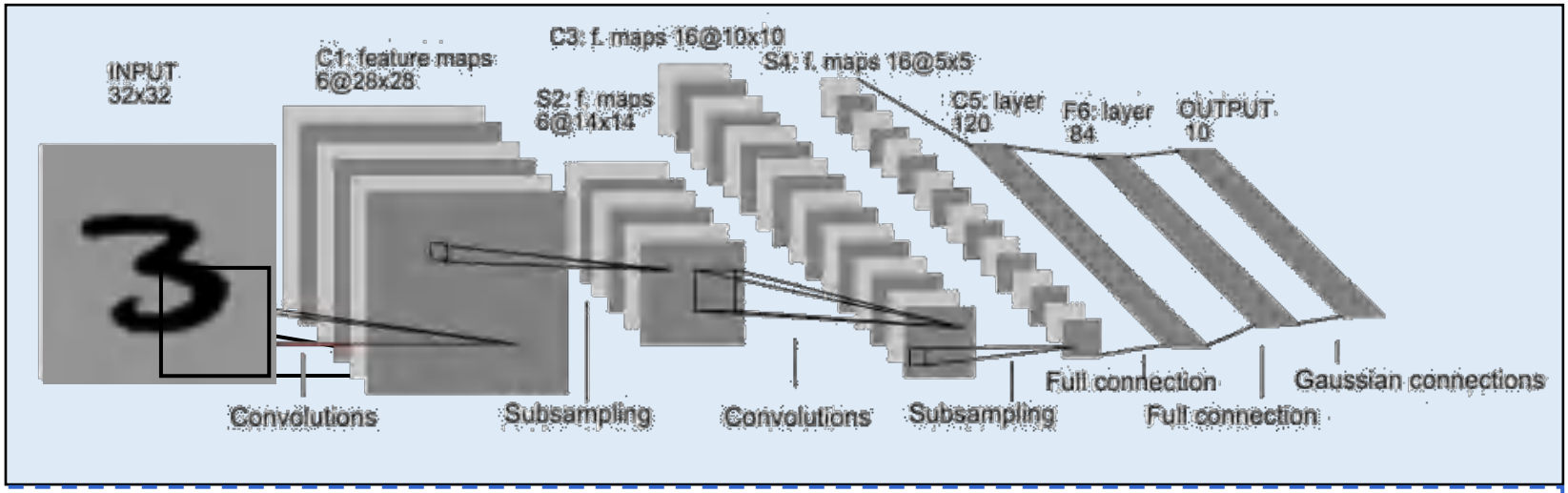
LeNet-5：识别手写英文字母、数字的CNN



F6层：全连接层

- 输入：c5 120维向量
- 计算方式：输入向量和权重向量之间的点积，再加上一个偏置，结果通过sigmoid函数输出。
- 可训练参数:84× (120+1)=10164

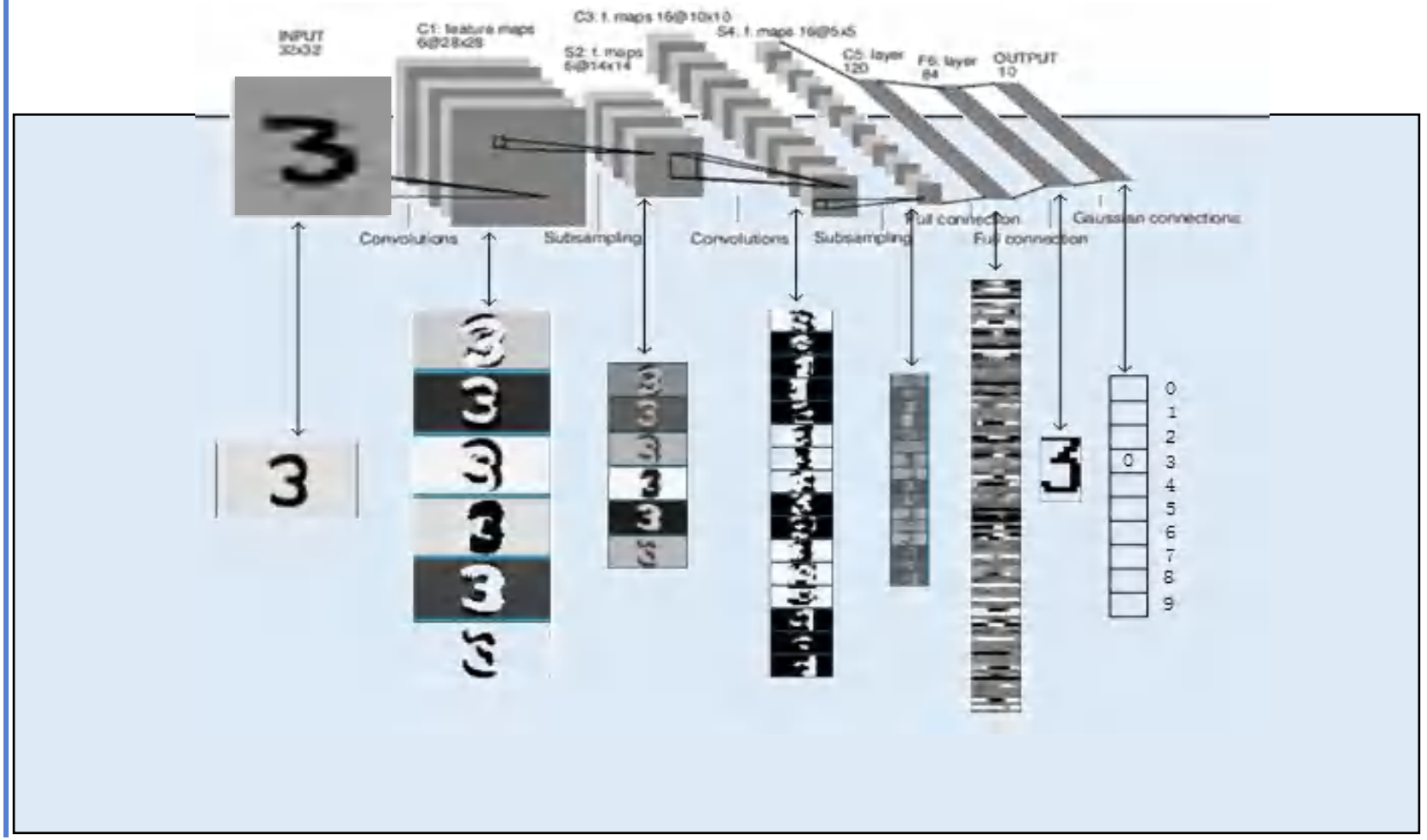
LeNet-5：识别手写英文字母、数字的CNN



OUTPUT层：输出层

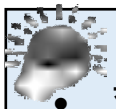
- Output层也是全连接层，共有10个节点，分别代表数字0到9，且如果节点i的值为0，则网络识别的结果是数字i。采用的是径向基函数（RBF）的网络连接方式。假设x是上一层的输入，y是RBF的输出，则RBF输出的计算方式是 $y_i = \sum_j (x_j - w_{ij})^2$
- 现在复现一般都用Softmax输出

LeNet-5：识别手写英文字母、数字的CNN



1 大脑的注意力

- 人脑每个时刻接收包括来源于视觉、听觉、触觉的各种各样的信息，通过注意力来解决信息超载问题

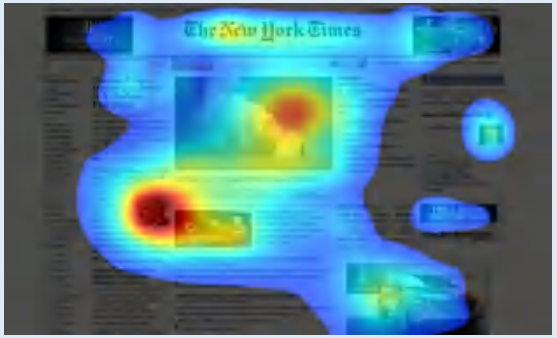


- 鸡尾酒会效应：

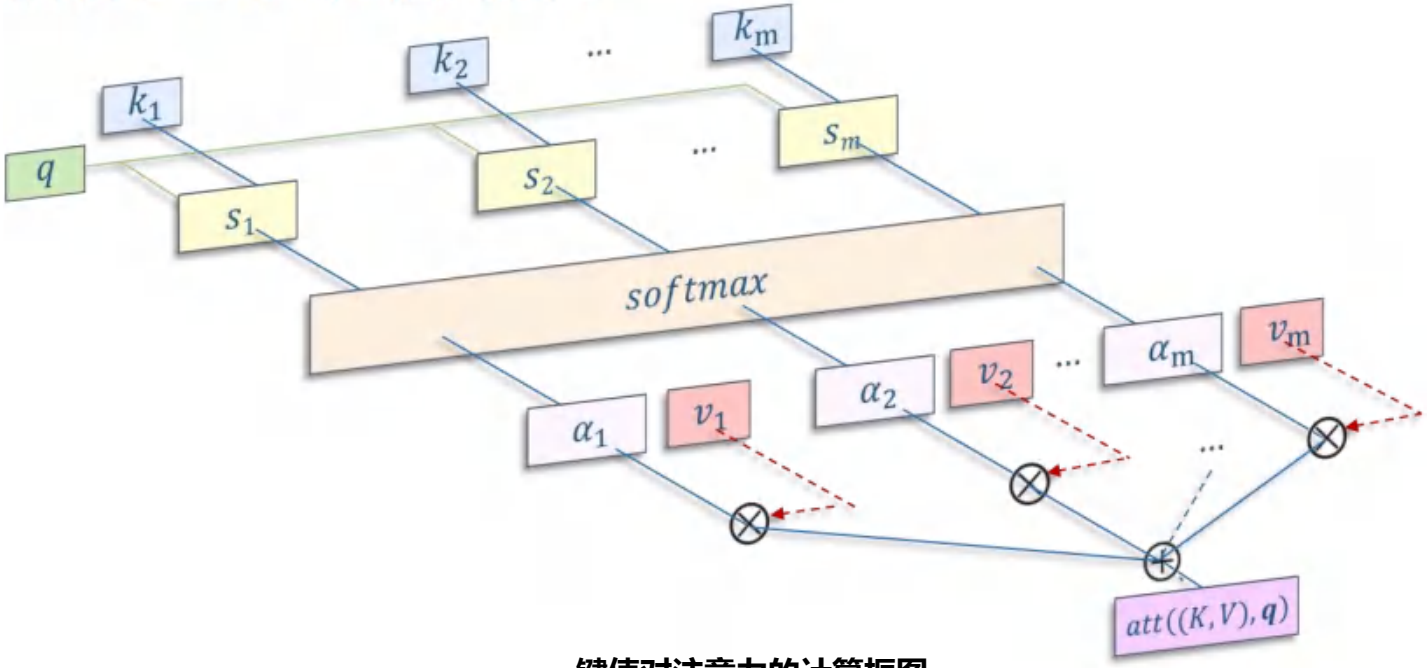
  当在吵闹的鸡尾酒会上和朋友聊天时，尽管周围噪音很大，人们还是可以听到朋友的谈话内容，而忽略其他人的声音。同时，如果听到其他人喊自己的名字，也会马上注意到

- 视觉注意力效应：

  如右侧图片人们读报纸更容易注意到的位置：



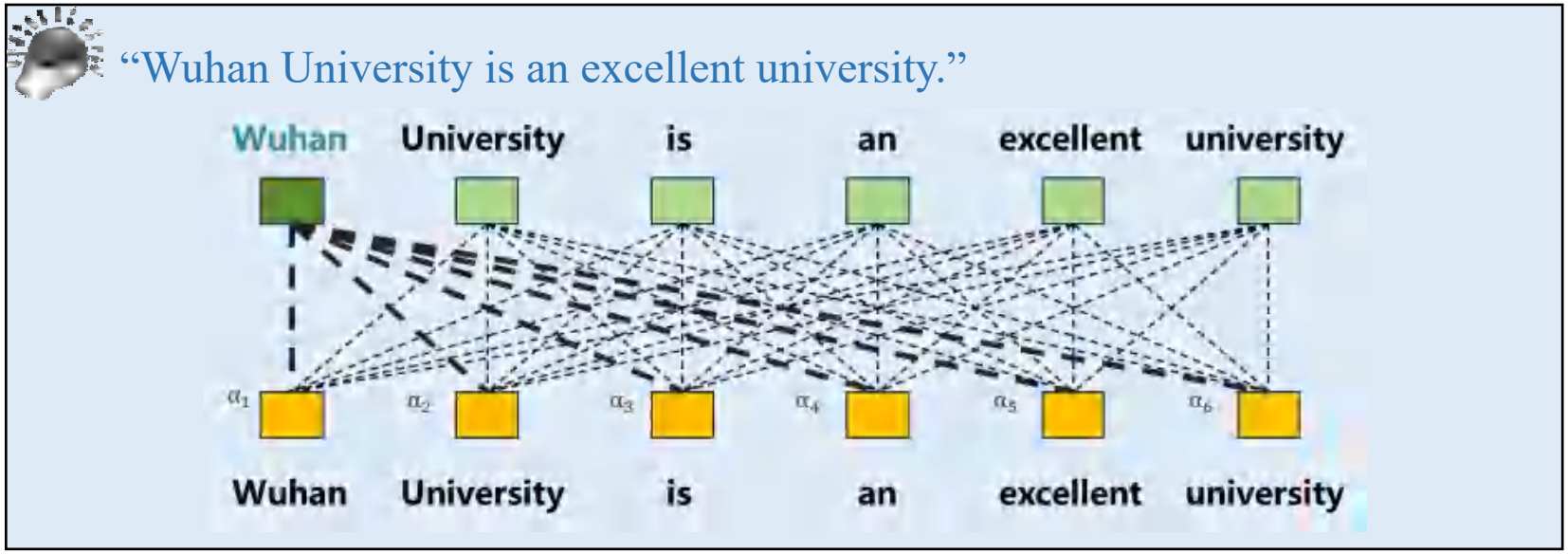
- 实践中更为常用的软注意力是**键值对注意力（Key-Value Attention）**。其核心思想是将输入数据分为键K（Key）和值V（Value）两部分，并通过查询向量q（Query）来索引这些键值对，以获取与查询最为相关的值



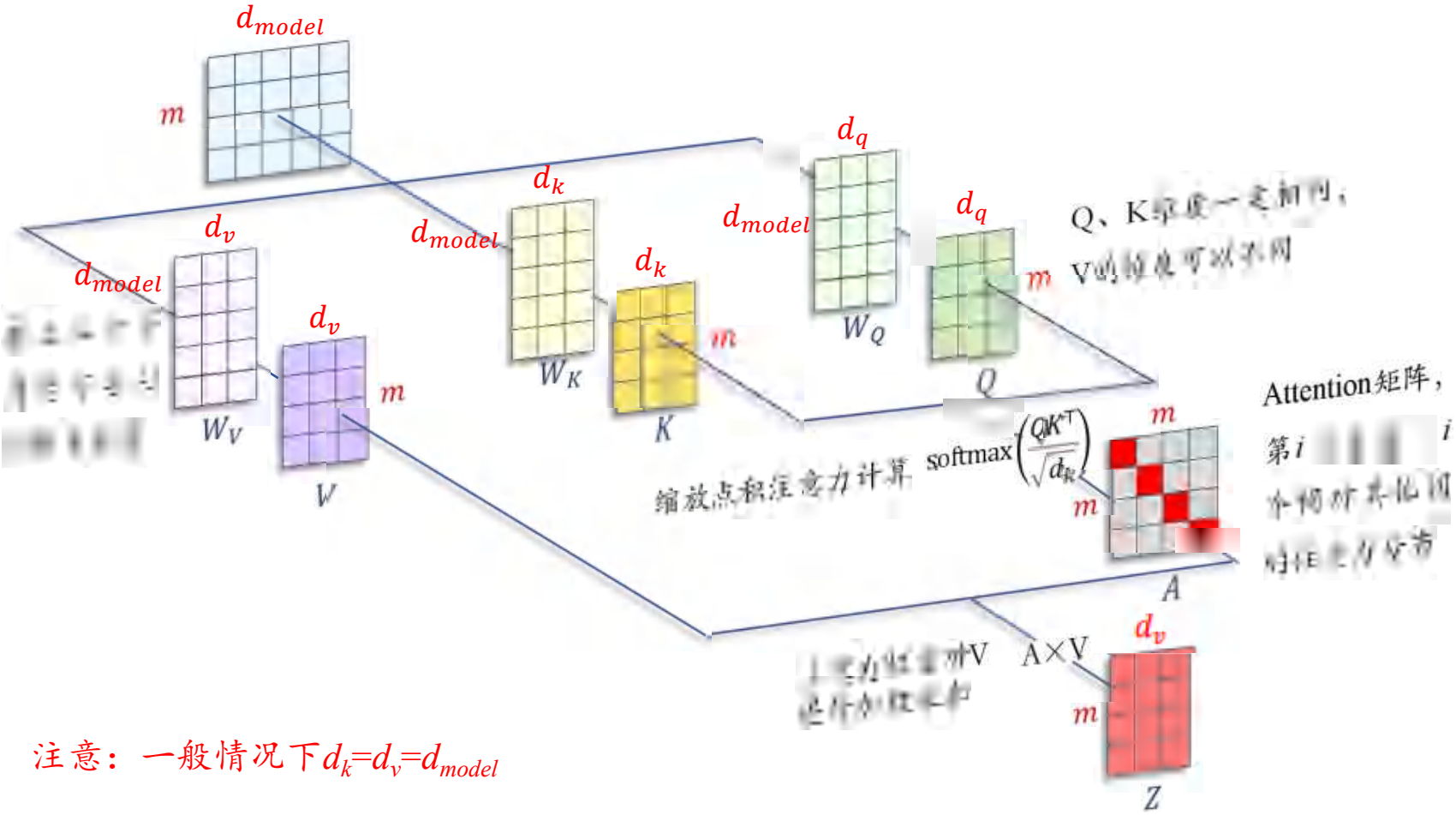
键值对注意力的计算框图

4 自注意力机制

- 如果Q，K和V均来自同一个输入序列，这种软注意机制称为**自注意力（Self Attention）**。自注意力计算输入序列中的每个元素与其他所有元素的关系，从而捕获上下文尤其是远程的依赖关系



- 自注意力机制的**计算流程**：



注意：一般情况下d_k=d_v=d_{model}

自注意力的计算过程

注意力机制

- W_Q、W_K、W_V三个权重矩阵使得注意力计算变为“可学习的”
- Q和K使用不同的权重矩阵生成，可以提供更灵活的模型表示能力以及捕捉数据中的复杂依赖关系
  - 使用不同的权重矩阵为Q和K提供了捕捉不同依赖关系的能力，增强了模型对输入数据的理解
  - 不同的权重矩阵意味着Q和K可以在不同的语义空间中进行表达，这有助于模型捕捉更丰富的语义信息

5 多头注意力机制

- **多头注意力（Multi-Head Attention）**是注意力机制的一种扩展形式。核心思想是并行计算多组注意力，使模型能够同时关注输入的不同子空间信息，从而提升模型的表达能力
- 均匀分割输入序列的$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$得到多个头$(head_1, \cdots, head_h)$，每个头计算不同的注意力特征，将结果拼接后并用$\boldsymbol{W_O}$投影回原维度

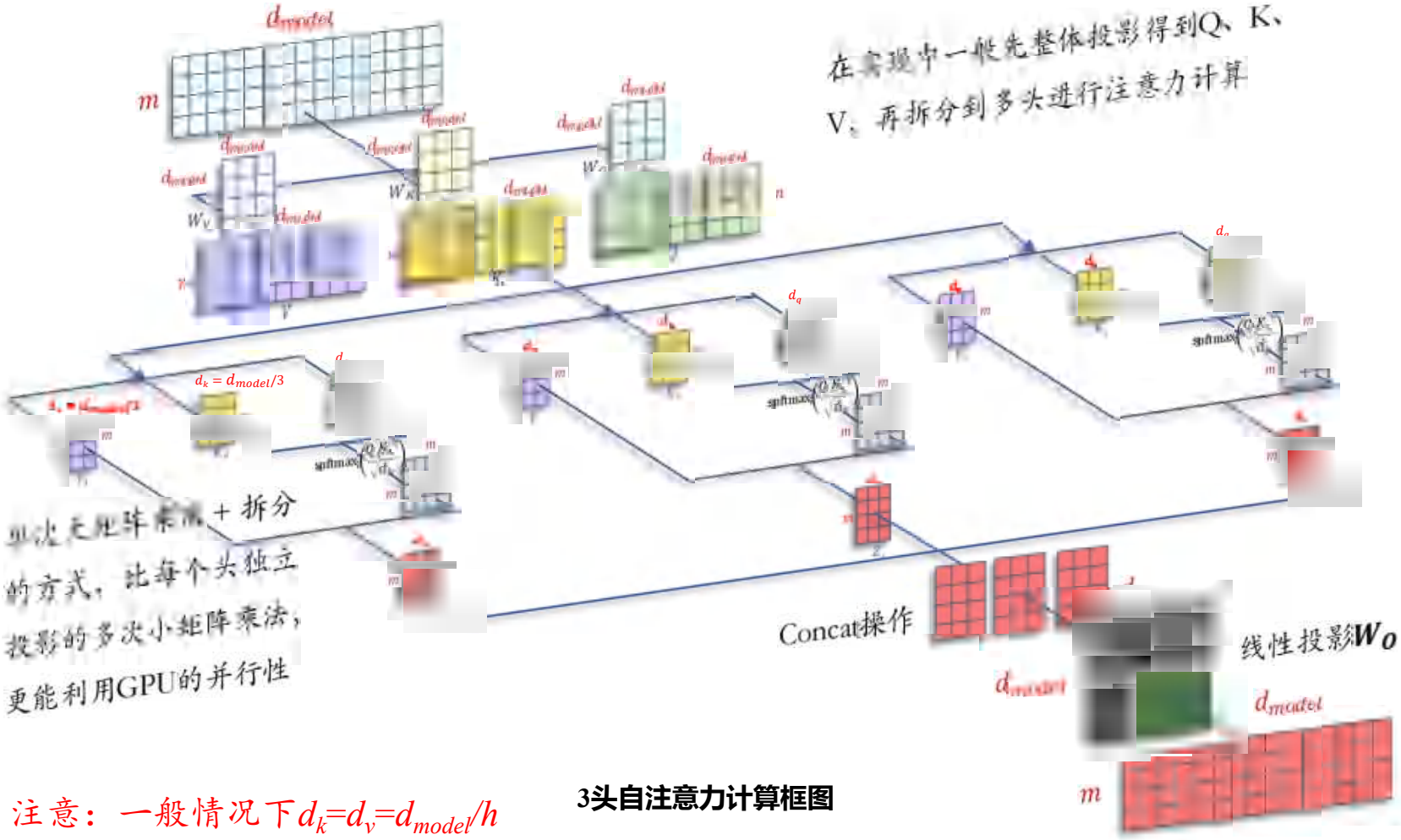
$$\text{MultiHead}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{Concat}(head_1, \cdots, head_h)\boldsymbol{W_O}$$

其中每个头的计算为：

$$head_i = \text{Attention}(\boldsymbol{Q_i}, \boldsymbol{K_i}, \boldsymbol{V_i})$$

- 多头的“头”的提法来自传统磁盘读取的“磁头”

- 多头自注意力机制的**计算流程**：

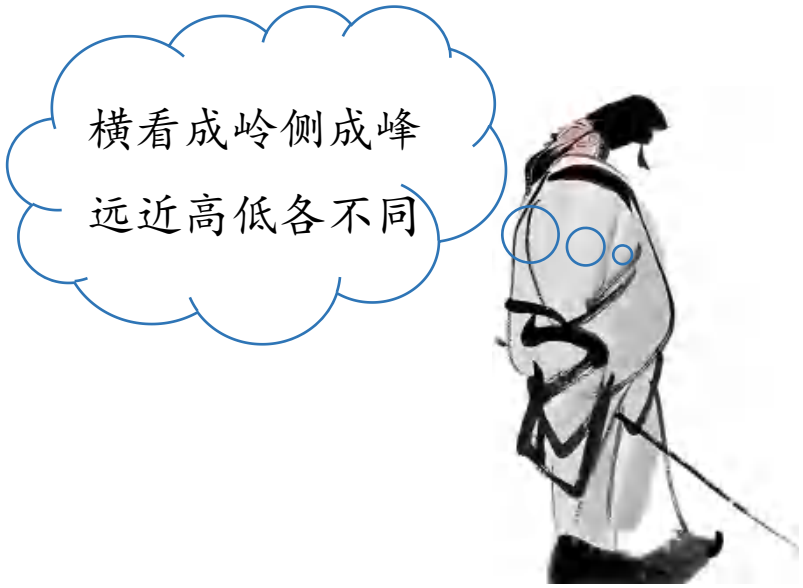


3头自注意力计算框图

注意：一般情况下$d_k=d_v=d_{model}/h$

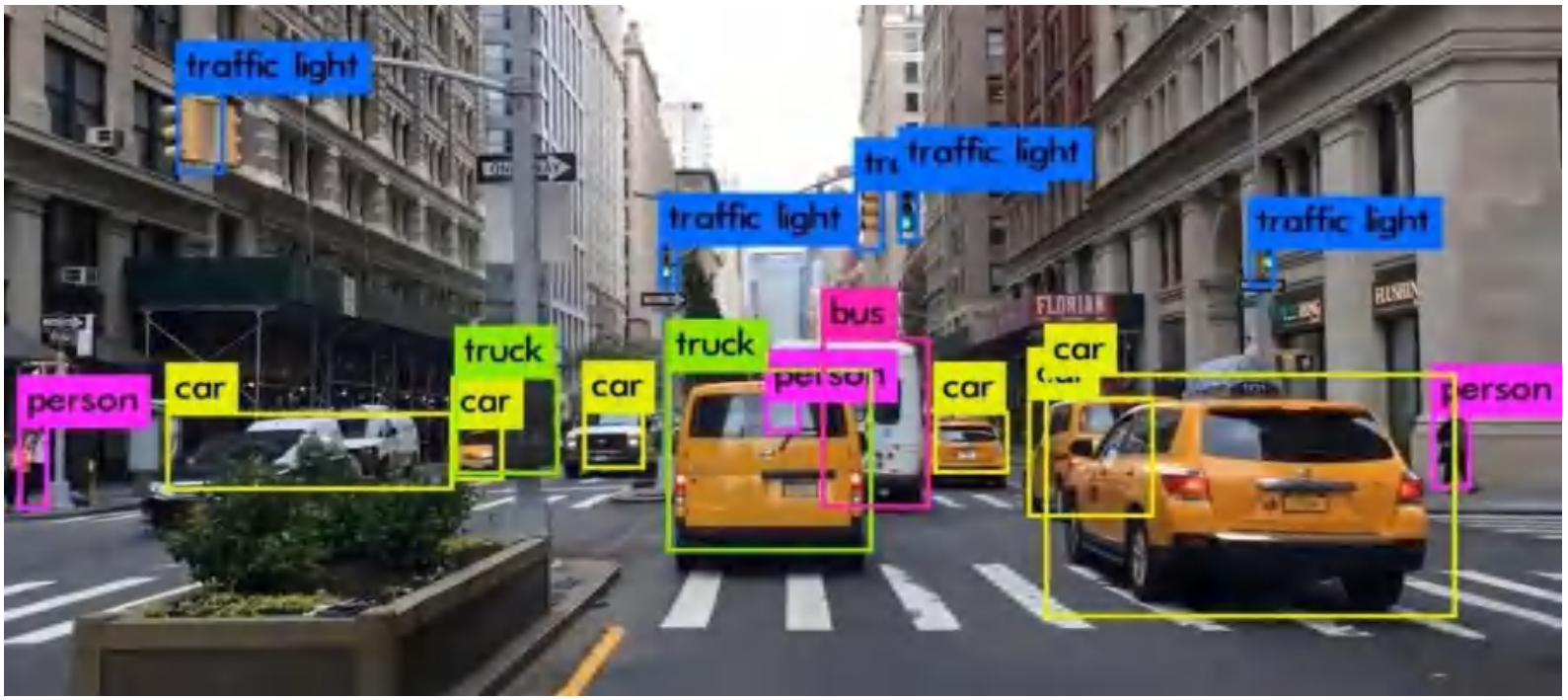
➢ 多头注意力机制的优势：

- **捕捉多种依赖关系**：不同的注意力头可以学习到序列中不同位置之间的不同依赖关系。一个头可能专注于捕捉语法依赖，另一个头可能专注于语义依赖，这样模型就能够更全面地理解输入数据
- **提高模型容量**：增加了模型的容量，使得模型能够学习到更复杂的表示
- **更好的泛化能力**：从多个角度分析输入数据，模型的泛化能力得到提升



- 机器视觉包含很多任务。其基本任务包括：**目标检测**、**图像分割**和**目标追踪**。此外三维重建、图像生成、图像超分辨率修复、图像风格迁移、人（物）体动作捕捉、视觉对话等也都在机器视觉的范畴

目标检测（Object Detection）



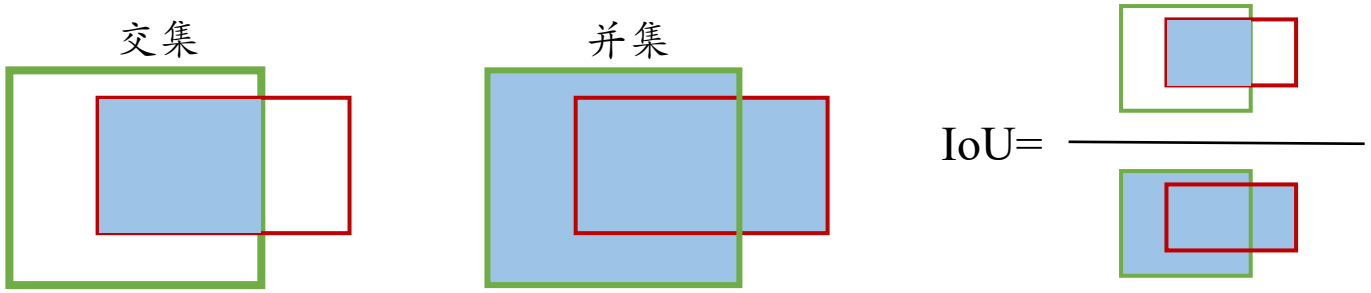
（图片来自网络）

交并比IoU
（Intersection over Union, IoU）

用于衡量两个区域的重叠程度，为两个区域重叠部分面积占两者总面积的比例。目标检测经常使用

平均交并比mIoU
（Mean IoU）

多个类别IoU的平均值，用于评估模型在所有类别上的整体表现。图像分割经常使用



（IoU表示真值边界框与预测边界框的交集和并集之比，IoU越高，模型表现越好）

注意：IoU是局部指标，mIoU是全局指标；前者适用于检测，后者适用于分割

精度/查准率Precision | 分类为正例的样本中实际为正例的比例$TP/(TP+FP)$

召回率/查全率Recall | 正例中被分类为正例的比例 $TP/(TP+FN)$

- TP（True Positive）：被判定为正样本，实际上是正样本的预测边界框的数量，即与真值的IoU≥threshold的预测边界框数量
- FP（False Positive）：被判定为正样本，实际上是负样本的预测边界框的数量，即与真值的IoU＜threshold的预测边界框数量
- FN（False Negative）：被判定为负样本，实际上是正样本的预测边界框的数量

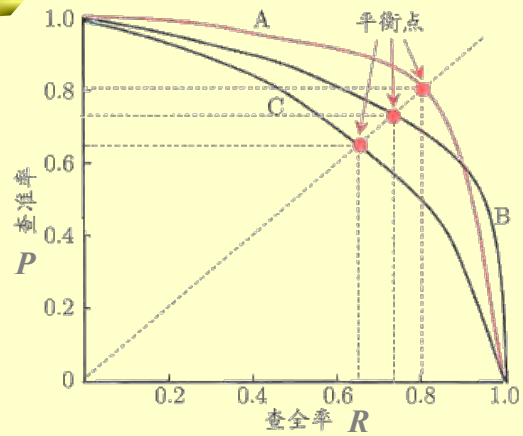
→第二讲

平均精度AP
（Average Precision）

$\mathrm{AP} = \int_0^1 p(r)dr$ ，r是召回率，$p(r)$是召回率对应的精确率，即P-R曲线与坐标轴围成的面积

多类平均mAP
（Mean AP）

多个类别AP的均值



- P-R曲线：精度（Precision）与召回率（Recall）的曲线。以召回率作为横坐标轴，精度作为纵坐标轴绘制而成

→第二讲

第十一讲 计算机视觉基础

1 计算机视觉概述

2 目标检测

3 图像分割

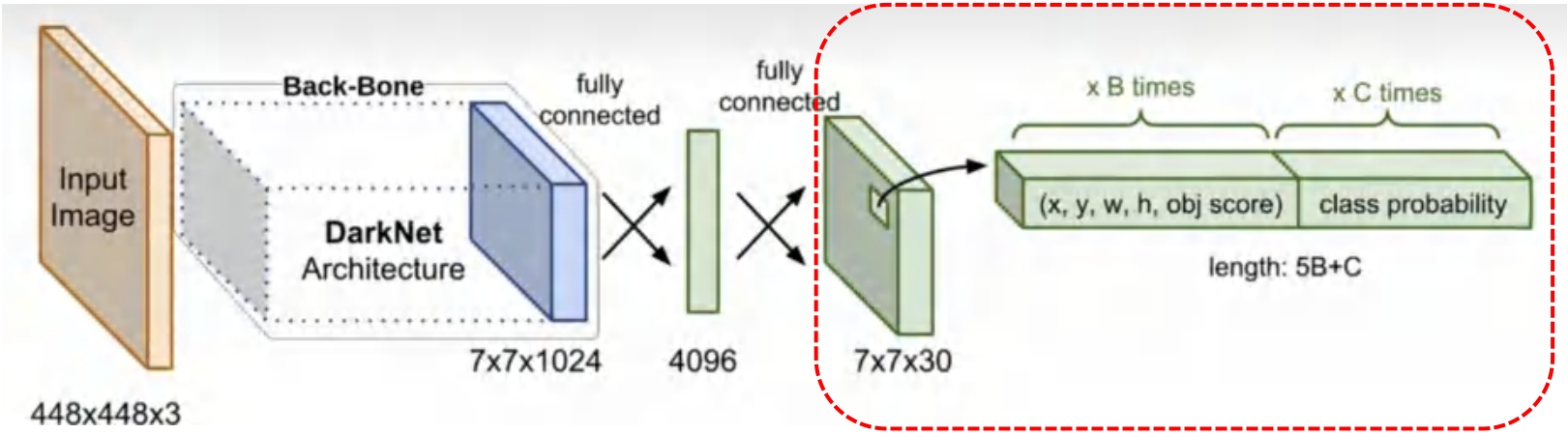
4 计算机视觉新进展 (自学)

目标检测

- 目标检测（Object Detection）是对图像中所有感兴趣的对象（目标）进行**识别**和**定位**。识别是指识别出图像中**目标的类别**，定位是指用一个**边界框（bounding box）**定位目标在图像中的**位置**

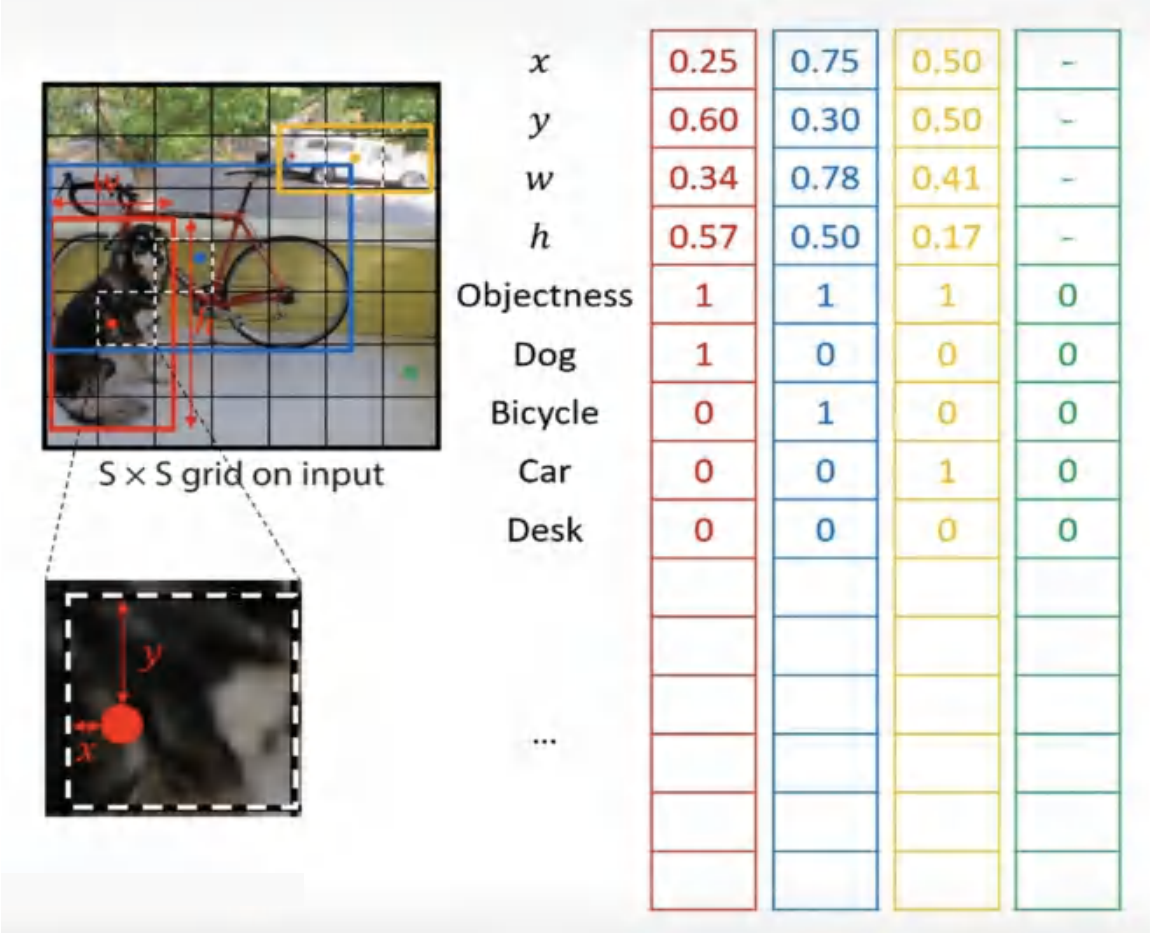
• YOLO v1模型

➢ YOLO v1默认的图像输入尺寸是448×448×3，骨干网络包含24 个卷积层和2个全连接层，输出得到$S\times S\times(B\times 5+C)$的张量（默认维度为7×7×30，$S$=7, B=2, C=20）



➢ Redmon J, Divvala S, Girshick R, et al. You only look once: Unified, real-time object detection[C]//***Proceedings of the IEEE conference on computer vision and pattern recognition*** (***CVPR***). 2016: 779-788

- YOLO v1模型的类别标签



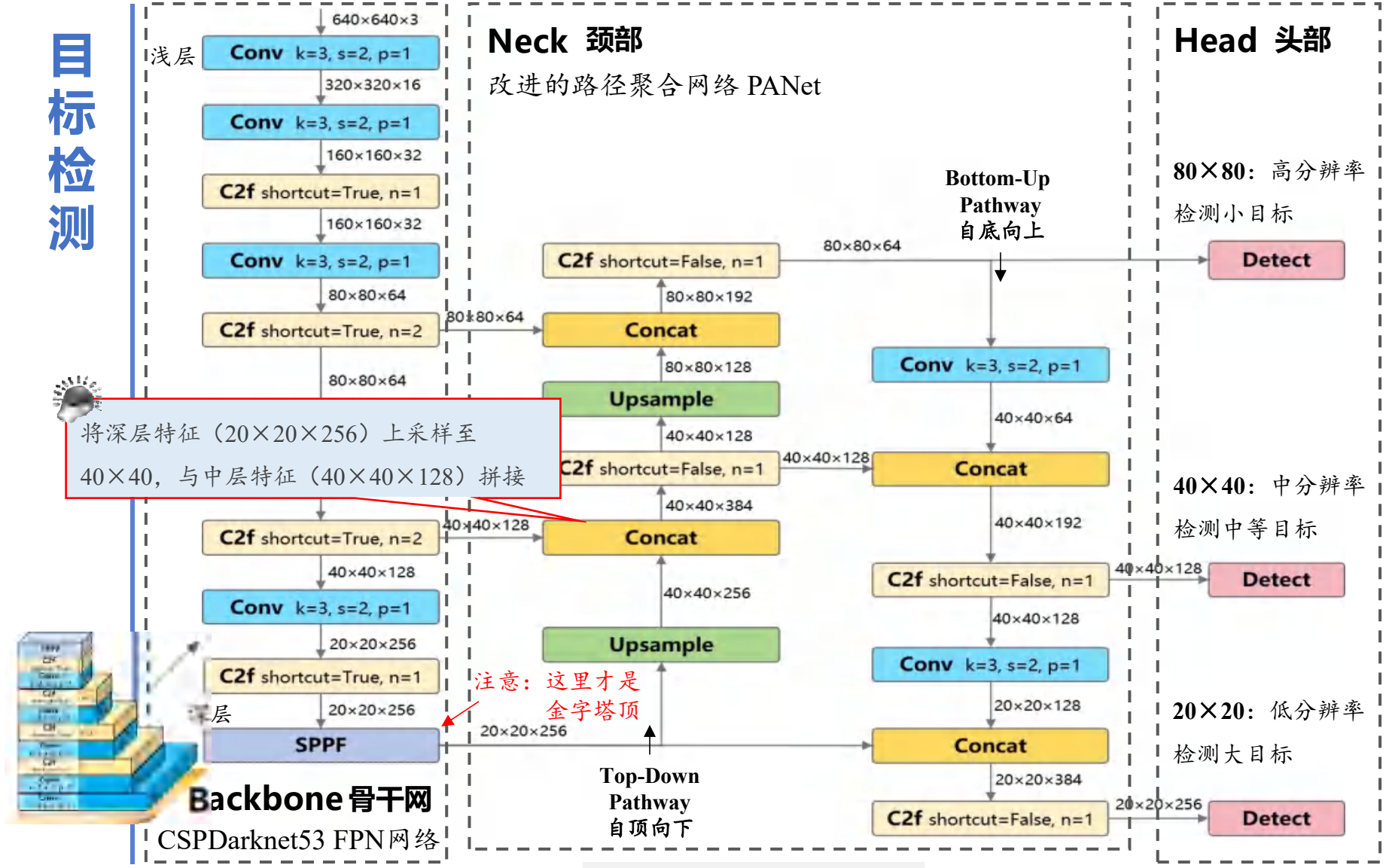
• Ultralytics的YOLO系列模型

- Ultralytics是一家专注于深度学习工具的AI公司，提出了YOLO v5、v8和v11模型，在YOLO系列模型中具有主导地位。其中YOLO v5和v8模型的很多设计方法已经成为目标检测的经典方法
- 模型均采用“骨干网+颈部+头部”的设计思路构建：
  - **骨干网（Backbone）**：提取图像特征，决定模型的基础感知能力
  - **颈部（Neck）**：对各个尺度特征进行融合，增强多尺度目标的适应性
  - **头部（Head）**：实现检测、分割等具体任务，输出任务结果

注：本课程以YOLO v8-n为主进行讲授

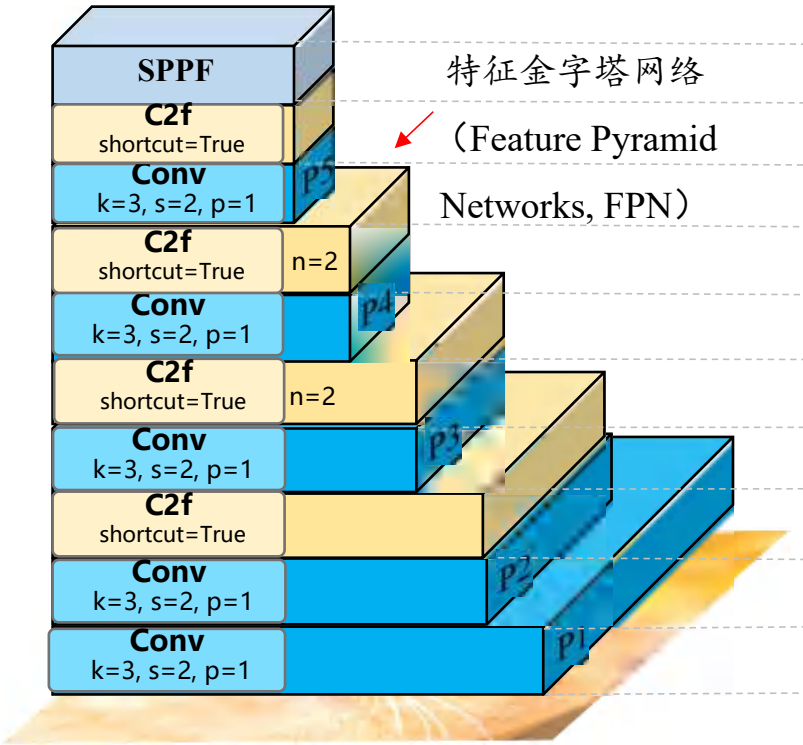
- Ultralytics公司中文主页：https://www.ultralytics.com/zh

目标检测



YOLO v8-n 的完整架构

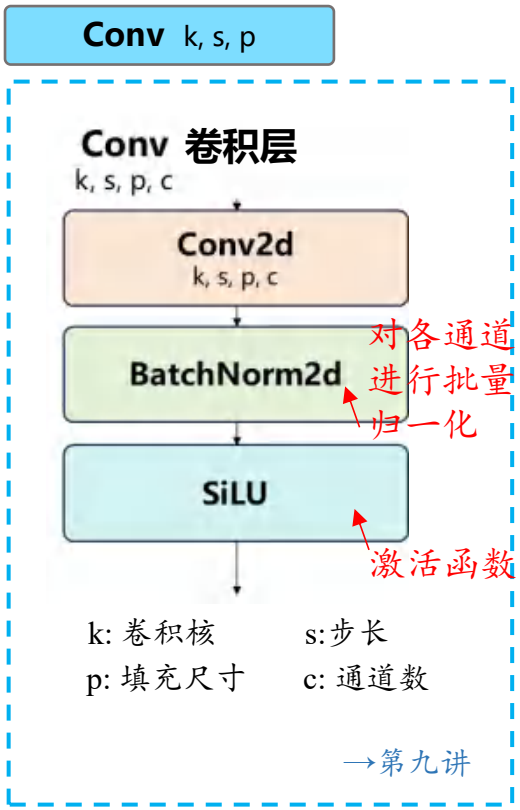
- YOLO大都采用深度的卷积神经网络（又称特征金字塔网络）作为其骨干网（Backbone）。YOLO v8在Darknet53的基础上，引入“跨阶段部分融合”模块，构成了其骨干网络CSPDarknet53



特征金字塔网络（Feature Pyramid Networks, FPN）

	20×20×256
	20×20×256
	20×20×256
	40×40×128
	40×40×128
	80×80×64
	80×80×64
	160×160×32
	160×160×32
	320×320×16
	640×640×3

YOLO v8-n 的骨干网

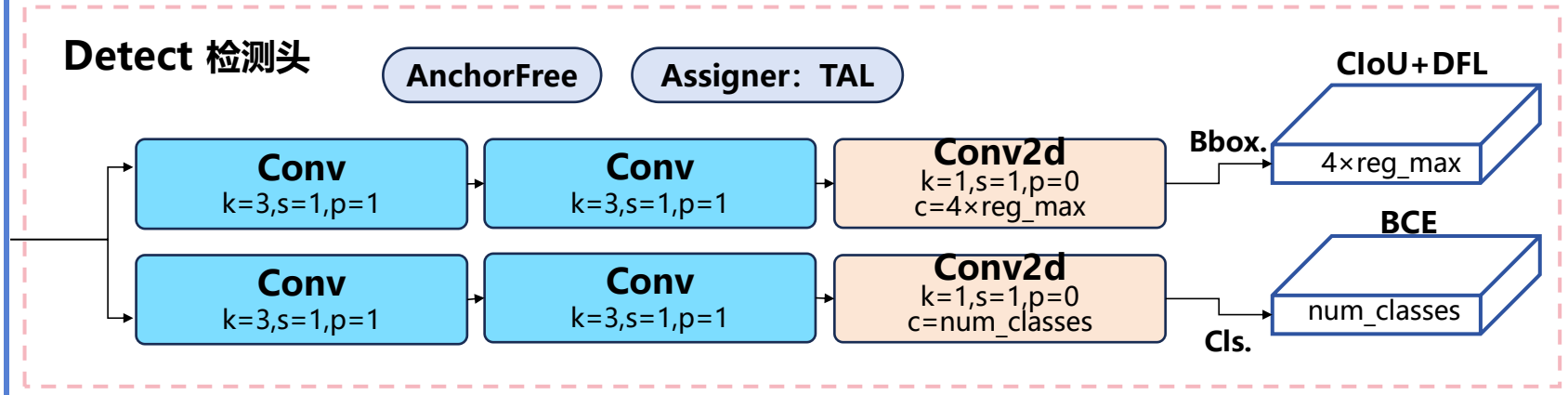


注：Darknet53 最早在 YOLO v3 中采用，包含 53个卷积层而得名。但在后续版本中卷积层数并非固定为53

- 路径聚合网络（Path Aggregation Network， PANet ） 是一种用于目标检测和实例分割任务的特征金字塔增强结构，由中国科学院和微软亚洲研究院于2018年提出。其核心目标是优化特征金字塔中不同层级的信息流动，通过双向（自顶向下+自底向上）特征融合，显著提升模型对多尺度目标的检测能力
  - **自顶向下路径（Top-Down Pathway）**通过上采样和 C2f 融合语义信息（如 FPN），增强对大目标的检测；将深层的语义特征通过上采样传递到浅层，与浅层的高分辨率特征拼接
  - **自底向上路径（Bottom-Up Pathway）**通过下采样和 C2f 补充细节信息（如 PAN），提升小目标检测能力，将浅层的高分辨率特征通过下采样传递到深层，补充深层特征的定位细节
- YOLO v8的PANet也融入了C2f模块，在多尺度目标检测任务中实现了速度与精度的平衡。其设计充分优化了信息流动的路径，使模型在复杂场景下仍保持高鲁棒性

Detect

- YOLOv8的检测头（Detection Head）是其目标检测架构中的最终预测模块，负责将骨干网络和颈部网络提取的多尺度特征转换为具体的检测结果（边界框、类别概率）
- YOLOv8的检测头采用解耦（Decoupled）头和无锚框（Anchor-Free）的设计，对应三个特征尺度网格（如80×80、40×40、20×20），每个网格负责预测1个边界框



目标检测（Object Detection）

YOLOv8 的优势

- 多尺度特征融合（PAFPN）：融合高、中、低分辨率特征（P3/P4/P5），可同时检测大目标（右下角黄色出租车，P5特征捕捉全局）和小目标（左侧白色SUV，P3特征保留细节），避免YOLOv1 的单一尺度限制。

